A Structural Approach to Interpretable Deep Learning

From DC Decompositions to Two-Player Games

Thesis in partial fulfillment of the requirements for the degree
Master of Science (M.Sc.)

submitted by
Jakob Zimmermann

Under the scientific supervision of
Prof. Dr. Wojciech Samek

Under the scientific guidance of
Georg Loho

Berlin, November 2026



Technische Universität Berlin
Fakultät IV – Elektrotechnik und Informatik
Fraunhofer Heinrich-Hertz-Institut
Department of Artificial Intelligence

Affidavit

Deutsch

1) Hiermit versichere ich, dass ich die vorliegende Arbeit ohne Hilfe Dritter und ausschließlich unter Verwendung der aufgeführten Quellen und Hilfsmittel angefertigt habe. Alle Stellen, die den benutzten Quellen und Hilfsmitteln unverändert oder inhaltlich entnommen sind, habe ich als solche kenntlich gemacht.
2) Sofern generative KI-Tools verwendet wurden, habe ich Produktnamen, Hersteller, die jeweils verwendete Softwareversion und die jeweiligen Einsatzzwecke (z.B. sprachliche Überprüfung und Verbesserung der Texte, systematische Recherche) benannt. Ich verantworte die Auswahl, die Übernahme und sämtliche Ergebnisse des von mir verwendeten KI-generierten Outputs vollumfänglich selbst.
3) Die Satzung zur Sicherung guter wissenschaftlicher Praxis an der TU Berlin vom 15. Februar 2023[1] habe ich zur Kenntnis genommen.
4) Ich erkläre weiterhin, dass ich die Arbeit in gleicher oder ähnlicher Form noch keiner anderen Prüfungsbehörde vorgelegt habe.

Jakob Zimmermann
Berlin, 11. November 2026

English

1) I hereby declare that I have written this thesis without the help of third parties and exclusively using the sources and aids listed. I have marked as such all passages that are taken from the sources and aids used, either unchanged or in terms of content.
2) Where generative AI tools were used, I have stated the product name, manufacturer, the software version used and the respective purposes (e.g. linguistic review and improvement of texts, systematic research). I am fully responsible for the selection, adoption and all results of the AI-generated output I used.
3) I have taken note of the Statutes for Safeguarding Good Scientific Practice at TU Berlin dated February 15, 2023[2].
4) I further declare that I have not yet submitted the work in the same or a similar form to any other examination authority.

Jakob Zimmermann
Berlin, November 11, 2026

[1] https://www.static.tu.berlin/fileadmin/www/10002457/K3-AMBl/Amtsblatt_2023/Amtliches_Mitteilungsblatt_Nr._16_vom_30.05.2023.pdf
[2] https://www.static.tu.berlin/fileadmin/www/10002457/K3-AMBl/Amtsblatt_2023/Amtliches_Mitteilungsblatt_Nr._16_vom_30.05.2023.pdf

Acknowledgements

I thank Gerrit Holzbach for giving me the opportunity to work on machine learning and gain a practical perspective. I am grateful to Georg Loho for opening my eyes to the theory of deep learning and polyhedral geometry.

I thank Grégoire Montavon for taking the time to discuss the idea of DC decomposition with me and for pointing me toward the Quantus framework and the intricacies of evaluating attribution methods. I am grateful to Wojciech Samek for his Responsible AI lecture, which made me aware of the problems in this area and strategies for addressing them. I also thank him for encouraging me to work on transformers and more modern machine learning models and paradigms, for asking me to study parameter randomization, for our conceptual discussions, and for his organizational support.

I thank Klaus Robert Müller for the interesting discussions of these results. I am also grateful to Shinichi Nakajima for the interesting discussions and for comparing my trajectory distribution view with the Normalized Relevance Measure framework and its signed-measure perspective on layer-wise relevance propagation and concept relevance propagation.

Abstract

Deutsch

Interpretierbarkeit dient zunehmend der Modellverbesserung und -steuerung neben der nachträglichen Analyse einzelner Vorhersagen. Wenn Attributionen die Korrektur unerwünschter Merkmale, das Training oder die Inferenz beeinflussen, muss ihre Berechnung verstanden und überprüft werden. Diese Arbeit verbindet dafür strukturierte Darstellungen interner Beiträge mit einer mathematischen Beschreibung ihrer Rückwärtspropagation.

Ein trainiertes neuronales Netz zu verstehen heißt, nachzuvollziehen, wie gegenläufige Einflüsse Eingaben in Ausgaben überführen. Diese Arbeit untersucht Interpretierbarkeit durch *Zerlegung*, die vorhandene interpretierbare Struktur eines trainierten ReLU-Netzes ohne zusätzliche Interpretierbarkeitsbeschränkungen offenlegt. Eine *Netzzerlegung* hebt die Berechnung in ein Paar koordinierter, nichtnegativer Ströme an, deren Differenz das ursprüngliche Netz exakt reproduziert. Dieser gemeinsame Begriff vereinheitlicht mehrere Strukturen und erhält jede Vorhersage.

Die erste ist die Differenz-konvexer-Funktionen-Zerlegung (DC-Zerlegung). Monotonie fördert Interpretierbarkeit und beschränkt Ausdrucksstärke. Wir zerlegen deshalb ein trainiertes ReLU-Netz — ein MLP oder CNN — in Teilnetze g und h mit $f = g - h$, die in verdoppelten Eingabekoordinaten konvex und monoton sind und positive und negative Evidenz auf getrennten Pfaden führen. Wir stabilisieren die ausgewertete ReLU-Zerlegung und konstruieren aus absoluter Krümmung exakte Paare für glatte Aktivierungen (GELU, Swish, Tanh, Sigmoid, Softplus). Explizite Softmax-, Produkt- und LayerNorm-Paare mit positivem ε ergeben eine DC-Zerlegung deterministischer Transformer beliebiger endlicher Tiefe; daraus folgt keine Garantie numerisch stabiler Auswertung. Auf der ReLU-Zerlegung bauen wir die Saliency-Methoden SplitCAM und SplitLRP auf: Sie zeigen auf VGG-16 und ResNet-18 über ImageNet-S konfigurationsabhängige Verbesserungen bei Lokalisierung und Selectivity sowie Zielkonflikte bei Pixel Flipping und Robustheit. Zudem trainieren wir Differenzen konvexer oder monotoner Netze zur MNIST-Klassifikation und beobachten klassenabhängige Stromgradientenmuster als qualitative Illustration der Darstellung.

Die zweite ist die Stopp-Zerlegung: Sie teilt Gewichte nach Vorzeichen auf, $W = W^+ - W^-$, trennt exzitatorische und inhibitorische Pfadbeiträge und verbindet Rückwärtspropagation mit Spieltheorie. Aufbauend auf dem ReLU-Netz-Spiel von Gaubert und Vlassopoulos stellen wir Rückwärts-Attribution als Zwei-Spieler-Nullsummen-Stoppspiel auf einem erweiterten Netzgraphen dar. Gradienten und die vollständige $\alpha\beta$-LRP-Familie ergeben sich unter bestimmten Gleichgewichten als Integrale über Spieltrajektorien; Attributionskarten sind Projektionen der zugrunde liegenden Trajektorienverteilung. Erwünschte Lokalisierungsschärfe, Robustheit gegenüber Eingangsrauschen und stabiles Attention-Routing lassen sich durch Politik-Regularisierung, Risikoaversion und erweiterte Aktionsräume formulieren und unmittelbar in neue Rückwärtsregeln übersetzen. Auf ViT-B/16 übertrifft eine solche Anpassung von $\alpha\beta$-LRP alle betrachteten Vergleichsmethoden, einschließlich transformer-spezifischer Methoden, bei der Attributionslokalisierung. Die revidierte Evaluation wählt Lokalisierungs- und Faithfulness-Konfigurationen unabhängig auf Validierungsdaten aus und berichtet Konfidenzintervalle und gepaarte Tests; andere Metriken zeigen Zielkonflikte.

Ein Rahmen erschließt somit mehrere Lesarten derselben trainierten Gewichte. Unsere Trajektorienverteilungs-Sicht vergleicht das Routing im Rückwärtsdurchlauf durch eine Hellinger-Distanz zwischen normalisierten Routing-Verteilungen, bevor signierte, diskontierte Belegungsmaße modellspezifisch auf Heatmaps projiziert werden.

Schlüsselwörter: *DC-Zerlegung, Interpretierbarkeit, Neuronale Netze, Spieltheorie, Attribution*

English

Interpretability increasingly supports model improvement and control alongside post hoc analysis of individual predictions. When attributions guide spurious-feature correction, training, or inference, their calculation must be understood and checked. This thesis connects structured representations of internal contributions with a mathematical account of their backward propagation.

Understanding a trained neural network means tracing how competing influences turn inputs into outputs. This thesis studies interpretability through *decomposition*, exposing interpretable structure already present in a trained ReLU network without imposing interpretability constraints. A *network decomposition* lifts the computation into a pair of coordinated, non-negative streams whose difference exactly reproduces the original network. This single notion unifies several structures while preserving every prediction.

The first is the difference-of-convex (DC) decomposition. Monotonicity aids interpretability while restricting expressivity. We therefore split a trained ReLU MLP or CNN into subnetworks g and h with $f = g - h$, convex and monotone in doubled input coordinates and carrying positive and negative evidence on separate pathways. We stabilize the evaluated ReLU split and construct exact smooth-activation pairs (GELU, Swish, tanh, sigmoid, softplus) from absolute curvature. Explicit softmax, product, and positive-epsilon LayerNorm pairs yield a DC decomposition of deterministic Transformers of arbitrary finite depth; this does not guarantee numerically stable evaluation. We build the saliency methods SplitCAM and SplitLRP on the ReLU representation, showing configuration-dependent gains in localization and Selectivity on VGG-16 and ResNet-18 over ImageNet-S, with trade-offs in Pixel Flipping and robustness. We also train differences of convex or monotone networks for MNIST classification and observe class-dependent stream-gradient patterns, qualitatively illustrating the representation.

The second is the stopping decomposition, which splits weights by sign, $W = W^+ - W^-$, separating excitatory and inhibitory path contributions and linking backward propagation to game theory. Building on Gaubert and Vlassopoulos' ReLU Net Game, we represent backward attribution as a two-player zero-sum stopping game on an extended network graph. Gradients and the full $\alpha\beta$-LRP family arise as integrals over game trajectories under specific equilibria; attribution maps project the underlying trajectory distribution. Desired localization focus, robustness to input noise, and stable attention routing can be formulated through policy regularization, risk aversion, and extended action sets, translating directly into new backward rules. On ViT-B/16, one such adaptation of $\alpha\beta$-LRP outperforms all evaluated baselines, including transformer-specific methods, in attribution localization. The revised evaluation selects localization and faithfulness configurations independently on validation data and reports confidence intervals and paired tests; other metrics expose trade-offs.

One framework thus offers several readings of the same trained weights. Our trajectory-distribution view compares backward routing through a Hellinger distance between normalized routing laws before the model-dependent projection of signed discounted occupation measures onto heatmaps.

Keywords: *DC Decomposition, Interpretability, Neural Networks, Game Theory, Attribution*

Contents

List of Figures i

List of Tables ii

List of Symbols iii

List of Abbreviations iv

1 Introduction 1
- 1.1 Motivation and Problem Statement . . . 1
- 1.2 Contributions of This Thesis . . . 4
- 1.3 Positioning within the Literature . . . 5

2 Mathematical Foundations 7
- 2.1 Neural Network Notation and PWL Mappings . . . 7
- 2.2 Convexity and Monotonicity in Neural Networks . . . 9
- 2.3 Game Theory and Markov Decision Processes . . . 10

3 Network Decompositions and DC Formulations 15
- 3.1 Generalized Network Decomposition . . . 15
- 3.2 Convex and Monotone DC Decompositions . . . 17
- 3.3 The Mixing Decomposition family . . . 26
- 3.4 The Stopping Decomposition . . . 27
- 3.5 Regularized Decompositions . . . 29
- 3.6 Numerical Stabilization of Split Passes . . . 31
- 3.7 Generalization to Convolutional Neural Networks (CNNs) . . . 34

4 Game-Theoretic Framework 37
- 4.1 Related Work . . . 37
- 4.2 Backward Attribution . . . 38
- 4.3 Stopping Game . . . 39
- 4.4 The Routing Game . . . 44
- 4.5 Signed affine output logits . . . 48
- 4.6 Game-Theoretic Modes . . . 49
- 4.7 Softplus Variant of the Stopping Game . . . 52

5 Extensions: Attention and Transformers 54
- 5.1 Related Work: Interpretability in Transformers . . . 54
- 5.2 Decomposing Multiplicative Operations . . . 54
- 5.3 A Constructive DC Decomposition of Transformers . . . 56
- 5.4 Attention via a detached-logit oracle . . . 60

6 Saliency and Attribution Methods 65
- 6.1 Split-Based and Difference-of-Convex Attribution . . . 65
- 6.2 Game-Based Attribution: Modes and Technicalities . . . 73

7 Experiments and Evaluation 75
- 7.1 Experimental Setup . . . 75

7.2 Split Methods: VGG-16 . . . 75
7.3 Split Methods: ResNet-18 . . . 76
7.4 Split-Method Sanity and Runtime Checks . . . 76
7.5 Split-Method Stabilization Ablations . . . 79
7.6 Game-Based Case Study . . . 79

8 The Trajectory Distribution View **83**
8.1 Comparison with Normalized Relevance Measures . . . 83
8.2 Trajectory Sanity Check . . . 84
8.3 Hellinger Distance and Disagreement Map . . . 84
8.4 Randomization Sanity Checks . . . 91

9 Conclusion and Future Work **98**
9.1 Summary of Contributions . . . 98
9.2 Limitations . . . 99
9.3 Broader Impact . . . 99
9.4 Future Research Directions . . . 100

A Experimental Protocol and Reproducibility **101**
A.1 Split-Method Evaluation . . . 101
A.2 Evaluation Protocol . . . 101
A.3 Implementation and Evaluation Details . . . 104
A.4 Formal Verification and Mathematical Audit . . . 107

B Extended Experimental Results **109**
B.1 Full Evaluation Results for the Split Methods . . . 109
B.2 Stabilization Ablations for the Split Methods . . . 111
B.3 Additional Results . . . 112

C Publication Provenance and Tool Use **125**
C.1 Author's Contribution . . . 125
C.2 Disclosure in the Revised Game Paper . . . 125
C.3 AI Assistance in the Thesis Revision . . . 125

List of References **126**

List of Figures

1.1 The backward pass as a two-player game 3

3.1 The network splitting procedure 15
3.2 Monotone, convex split streams and a standard ReLU layer 20

4.1 Stopping Game: trajectories and local stopping decisions 41
4.2 Routing Game: a local routing subgame 46

6.1 Stream gradients of the inverted-input DIC model 73
6.2 Second example of inverted-input DIC stream gradients 73
6.3 Input-gradient visualizations for ten MNIST digits 74

7.1 VGG-16 saliency map comparison 76

8.1 Hellinger trajectory distance under parameter randomization 84
8.2 Input-noise Hellinger trajectory distance 91

B.1 Additional VGG-16 saliency map comparisons 112
B.2 ViT-B/16 qualitative comparison on six ImageNet-S examples 123

List of Tables

3.1 Closed scalar profiles for smooth activations . . . 21
3.2 Linear CNN chunks . . . 35

4.1 Game modes: defaults, effects and costs . . . 50

7.1 Randomization sanity check for VGG-16 split methods . . . 77
7.2 Mean attribution and model-splitting times . . . 78
7.3 Scale versus shift stabilization for SplitLRP . . . 79
7.4 ViT-B/16: full ImageNet-S results with confidence intervals . . . 80
7.5 ViT-B/16 Routing Game attention rule ablation . . . 82

8.1 Reference baselines for the input-noise sweep . . . 92
8.2 Parameter randomization sanity check . . . 95
8.3 Hellinger trajectory distance at the double-random baseline . . . 96
8.4 Per-image Spearman rank correlations . . . 97

A.1 Per-method validation grids . . . 106

B.1 Comprehensive interpretability evaluation: VGG-16 . . . 109
B.2 Comprehensive interpretability evaluation: ResNet-18 . . . 110
B.3 Alpha ablation for SplitGrad and SplitCAM . . . 113
B.4 Alpha ablation without forward stabilization . . . 114
B.5 Alpha ablation with shift normalization . . . 114
B.6 Validation selection on ViT-B/16, VGG-16 and ResNet-50 . . . 115
B.7 VGG-16: full ImageNet-S results with confidence intervals . . . 116
B.8 ResNet-50: full ImageNet-S results with confidence intervals . . . 116
B.9 ViT-B/16 on the Pascal VOC 2012 test pool . . . 118
B.10 ViT-B/16: one-factor ablations on ImageNet-S . . . 119
B.11 VGG-16: one-factor ablations on ImageNet-S . . . 120
B.12 ResNet-50: one-factor ablations on ImageNet-S . . . 122
B.13 Pointing game: Quantus tie rule and matched granularity . . . 124

List of Symbols

Greek Symbols

Symbol	Description	Definition
α	Backward pass shift parameter	
δ	Network gradient (sensitivity)	
δ^g, δ^h	Split stream gradients	
γ	Discount factor / LRP emphasis parameter	
π, σ	Policies of Max and Min players	
τ	Temperature / entropy regularization parameter	
ε	Numerical stability constant	

Latin Symbols and Indices

c	Channel index
i,j	Neuron indices
l	Layer index
$a^{(l)}$	Post-activation at layer l
$a^{(l,+)}, a^{(l,-)}$	Positive and negative activations of split network
$d^{(l)}$	Width of layer l
f	Network function
g	Positive split stream (monotone, convex)
h	Negative split stream (monotone, convex)
L	Number of hidden layers
$P^l_{i\to j}$	Transition probability in game
$Q[l]$	Auxiliary network with min and absolute weights from layer l
Q^l	Q-function (action-value function)
$R^{(l)}$	LRP relevance score at layer l
$V^l_{i,\pm}$	Game value function at state (l,i,$\pm$)
$W^{(l)}$	Weight matrix of layer l
$W^{(l,+)}, W^{(l,-)}$	Positive and negative parts of weight matrix
x	Network input
$z^{(l)}$	Pre-activation at layer l

Operators

∇	Gradient operator	$\left[\frac{\partial}{\partial x_1}, \ldots, \frac{\partial}{\partial x_n}\right]^T$
$\odot$	Hadamard (element-wise) product	
ReLU	Rectified linear unit	$\max\{0, x\}$

List of Abbreviations

Methods

ADF	Assumed Density Filtering
AL	Attribution Localization
CAM	**C**lass **A**ctivation **M**apping
CDF	Cumulative Distribution Function
CPWL	**C**ontinuous **P**iece**w**ise **L**inear
DC	**D**ifference of **C**onvex
GAE	Generic Attention Explainability
ICNN	**I**nput-**C**onvex **N**eural **N**etwork
LMP	Layered Markov Process
LRP	**L**ayer-wise **R**elevance **P**ropagation
MDP	**M**arkov **D**ecision **P**rocess
NRM	**N**ormalized **R**elevance **M**easure
PG	Pointing Game
RG	**R**outing **G**ame
SG	**S**topping **G**ame
TIBAV	Transformer Interpretability Beyond Attention Visualization

Architectures

CNN	**C**onvolutional **N**eural **N**etwork
GELU	Gaussian Error Linear Unit
LLM	**L**arge **L**anguage **M**odel
MLP	**M**ulti**l**ayer **P**erceptron
ReLU	**Re**ctified **L**inear **U**nit
ViT	**Vi**sion **T**ransformer

1 Introduction

1.1 Motivation and Problem Statement

Interpretability is moving from post hoc inspection toward practical model improvement and control. As interpretations guide model correction, training, and inference (Anders et al., 2022; Komorowski et al., 2026; Pahde et al., 2023; Ross et al., 2017), we ask whether predictions are accurate, whether models use task-appropriate evidence, and whether we can understand, question, and correct that use. For example, attribution can investigate whether a language model draws on supplied context or knowledge acquired during training (Kahardipraja et al., 2025), and guide decoding toward supplied instructions or evidence (Komorowski et al., 2026).

For vision, this concerns shared visual grounding. When we ask a machine what it sees, seek its help, or entrust it with a task, we expect some alignment between its use of visual information and our understanding of the scene. A robot asked to grasp a cup should recognize its shape and position. A radiology assistant should attend to signs of disease, rather than clues about where the image was acquired—a documented shortcut (Zech et al., 2018). We seek decisions grounded in task-relevant objects, features, and relations that we can understand and question. Muttenthaler et al. (2025) show that human similarity judgments can guide visual representations toward closer agreement with human perception. Attribution can investigate this alignment, providing evidence for communication, justified trust, and correction.

Heatmaps make this concrete: does a prediction rely on relevant features or incidental artifacts? Attribution has exposed "Clever Hans" strategies (Lapuschkin et al., 2019) that can undermine generalization when correlations change (Geirhos et al., 2020), posing safety concerns when vision systems guide physical actions. Physical attacks on road-sign classifiers demonstrate additional security risks from deliberate visual manipulations (Eykholt et al., 2018). Heatmaps can identify suspicious dependencies for further testing; visual inspection alone establishes neither robustness nor safety. Such scrutiny extends to spurious behavior in anomaly detection and explanations of one-class models (Kauffmann et al., 2020a,b).

Diagnosed dependencies on spurious artifacts can guide targeted corrections. Input-gradient constraints can enforce which inputs should be irrelevant (Ross et al., 2017) or encourage adversarial robustness (Ross and Doshi-Velez, 2018); Class Artifact Compensation suppresses identified artifact directions (Anders et al., 2022), while Reveal to Revise iterates detection and correction during model development (Pahde et al., 2023).

This evidence requires scrutiny. Randomization sanity checks show that some attribution methods produce similar maps after model parameters are randomized (Adebayo et al., 2018): apparently informative heatmaps may reflect input structure or architecture more than learned computation. Input dependence, perturbation fragility, and misleading modified-gradient structure raise related concerns (Ghorbani et al., 2019; Kindermans et al., 2018; Nie et al., 2018; Sixt et al., 2020). Beyond inspecting maps, we must understand their calculation and its relationship to the trained network.

Evaluation requires clarity about what metrics measure. Lipton (2018) distinguishes objectives grouped under interpretability; Doshi-Velez & Kim (2017) argue that evaluation must match the application and audience. Murdoch et al. (2019) separate predictive accuracy, descriptive accuracy, and user relevance. We study trained network computations and the backward rules attributing their outputs. We use Quantus (Hedström et al., 2023) to select faithfulness, localization, and robustness

metrics, complemented by numerical-stability and parameter-randomization checks; a convincing heatmap alone cannot establish reliance on intended evidence.

The network's arithmetic provides a structural starting point. Signed weights combine reinforcing and opposing contributions into a pre-activation; activation gates determine what passes onward. The resulting activation records their balance without showing how they accumulated. This suggests a *debate* perspective: which contributions support or oppose a class score, and how does their interaction determine it? The $\alpha\beta$ rule of ***L**ayer-wise **R**elevance **P**ropagation* (LRP) already separates positive and negative pre-activation contributions, normalizes them separately, and weights their backward redistribution through α and β (Bach et al., 2015). Separating support and opposition connects how the network forms its output with how attribution traces it backward.

This thesis develops a unified mathematical framework connecting decompositions of trained network computations to backward attribution. Structured representations expose opposing contributions; games on the computational graph characterize backward attribution. Together, they make explicit what information attribution retains, how it propagates that information, and how its rules can be compared and systematically modified. Reliable visual inspection motivates the work; the relationship between network structure and attribution computation is its central research problem.

The first obstacle concerns the *representation* being interpreted. Monotonicity provides useful structure (Amos et al., 2017; Sill, 1997): with non-negative weights and monotone activations, increasing an input cannot decrease the downstream response. Within a non-negative stream, contributions accumulate without cancellation between positive and negative weighted terms (Ayinde and Zurada, 2018; Lee and Seung, 1999). Monotonicity constraints can encode chosen domain-specific fairness requirements (Wang and Gupta, 2020); these constraints alone do not guarantee fairness. For medical treatment or parole decisions, Rudin (2019) argues for inherently interpretable models with directly inspectable reasoning. Changing a trained, unconstrained network's weights to impose monotonicity generally changes its predictions.

The ***D**ifference of **C**onvex* (DC) decomposition structures the representation while preserving the learned predictor. Without requiring the network to be monotone or convex, we express it as $f = g - h$, constructing two stream functions layer by layer from its trained weights. The streams are convex and monotone in the paired, sign-separated input coordinates; their difference recovers the original network (Zimmermann and Loho, 2026). This realizes the debate perspective: one stream contributes positively to the class score, the other negatively. They separate contributions throughout the computation, distinct from the final score's positive and negative parts. These decomposition-defined roles alone do not establish a unique separation into human-interpretable concepts.

Using this representation at depth presents a computational challenge: rapidly growing streams can have a moderate difference, causing severe floating-point cancellation. *Hidden Monotonicity* stabilizes the forward and backward split passes to expose their signals without retraining (Zimmermann and Loho, 2026). Reconstructing the original computation does not ensure that numerical transformations preserve the exact DC representation's convexity and monotonicity in each stream.

To investigate the mechanisms of opposing streams exposed by DC decomposition of pretrained models, the MNIST experiments (Section 6.1.4) jointly train two small convex or monotone networks as a difference model. With complemented input $\mathbf{1} - x$ to the h-stream, the displayed gradients show class-dependent sensitivities on present strokes and candidate missing-stroke locations (Figure 6.1). These visual counterparts of debate motivate a factual/counterfactual interpretation requiring tests through complete-model interventions.

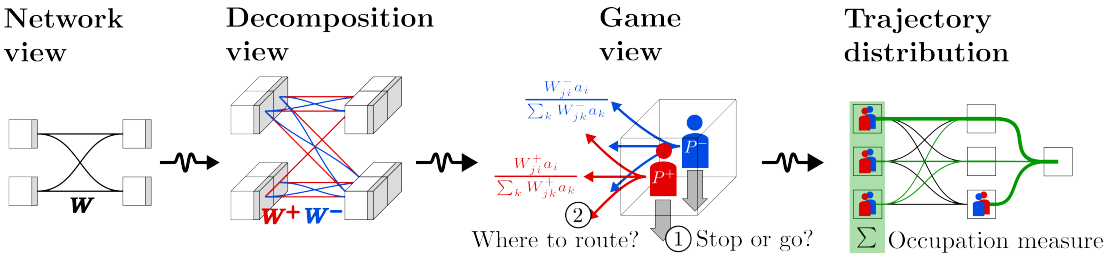


Fig. 1.1: The backward pass as a two-player game on a sign-separated network graph. Positive weights preserve the active player's role; negative weights switch it to the opponent. Activation states involve stopping decisions; in the routing game, additional decisions determine how play proceeds to predecessor states. Equilibrium policies induce a distribution over backward trajectories. With the prescribed signs, discounts, and output scaling, occupation measures recover the corresponding gradient or relevance attribution at the anchor settings.

The second obstacle is the zoo of backward attribution rules: local gradient sensitivities, LRP redistribution, local Deep Taylor decomposition, and transformer-specific extensions (Zimmermann et al., 2026). We unify gradients and the full $\alpha\beta$-LRP family through games of backward attribution. The stopping decomposition preserves their activation gates; the DC split retains opposing contributions even at inactive neurons.

Building on the ReLU Net Game of Gaubert & Vlassopoulos (2025), which represents forward evaluation as the value of a game played backward through the network, *Playing the Network Backward* develops this path structure into an attribution account (Zimmermann et al., 2026). Positive weights preserve the active player's role, negative weights switch it, and activation gates become stopping decisions. The stopping game represents gradients through differences of discounted occupation measures over backward trajectories. At its unit-temperature anchor, the entropy-regularized routing game recovers the full $\alpha\beta$-LRP family under the stated propagation conventions. Player roles, routing policies, and trajectory weights represent the positive and negative redistribution branches of $\alpha\beta$-LRP, embedding their separation in a two-player framework for comparison with gradients (Figure 1.1).

These representations support complementary ways to develop attribution methods. SplitCAM, SplitLRP, and SplitGrad use the DC representation; games expose stopping decisions, sign switches, and routing policies for principled modification. Policy regularization, risk aversion, and extended action sets translate desired properties into explicit changes of backward rules. Multiplicative and attention extensions reach beyond ReLU networks while distinguishing exact representations from conditioned attribution rules.

Trajectory analysis also addresses randomization sanity checks. Maps aggregate signed, discounted contributions from many backward trajectories and can obscure their routes through the model. Hellinger diagnostics compare normalized routing laws, complementing map comparisons: map comparisons test whether the presented explanation changes, and Hellinger diagnostics whether internal routing changes. Routing sensitivity alone cannot establish map faithfulness; recovering attribution also requires signs and model-dependent discounts. Experiments assess these distinct properties and trade-offs. Whether these methods improve oversight, correction, or deployment safety requires further empirical study.

Three research questions organize this account. *What structure can be exposed without changing the network's function?* We separate contributions into monotone convex streams and group paths by the sign of their weight product. *What does a backward attribution actually calculate?* Derivative identities and game occupation measures relate local rules to explicit global objects. *How can these*

representations guide the design and evaluation of attribution methods for visual inspection? Stable implementations, principled backward-rule modifications, and complementary map- and trajectory-level diagnostics provide a novel basis for assessing attribution's information and limitations.

Publication basis. This thesis brings together and extends two research works on which the thesis author is first author:

- *Hidden Monotonicity: Explaining Deep Neural Networks via their DC Decomposition*, coauthored with Georg Loho, was published at **CVPR 2026 and selected as a Highlight** (Zimmermann and Loho, 2026). [1]
- *Playing the Network Backward: A Game-Theoretic Attribution Framework*, coauthored with Jim Berend, Georg Loho, Sebastian Lapuschkin, and Wojciech Samek, is a **preprint**; this thesis uses arXiv:2605.06212v2, revised for AISTATS (Zimmermann et al., 2026).

Beyond the two papers, the thesis adds exact DC splits for smooth activations (GELU, Swish, sigmoid, and softplus), softmax, and LayerNorm, and combines them into a constructive Transformer decomposition for arbitrary finite depth. It connects the papers' decomposition and game frameworks, adds a partial Lean formalization, and reuses their experimental outputs. It corrects the uncertainty intervals in the VOC evaluation by resampling whole images. Appendix C records publication provenance and tool use.

Author's contribution. My principal contributions to both papers were developing the theory, developing and conducting the experiments, and writing all appendices and most text outside the introductions, in collaboration with the listed coauthors. Section C.1 distinguishes these contributions from the subsequent thesis integration and formalization.

1.2 Contributions of This Thesis

The thesis makes five contributions.

1) **A general framework of network decompositions.** We place DC, stopping, mixing, and regularized decompositions in a common framework and extend the DC construction to smooth activations, including GELU, Swish, sigmoid, and softplus.
2) **DC decompositions for ReLU networks, CNNs, and Transformers.** We stabilize the split forward and backward passes of pretrained ReLU networks and CNNs without changing their predictions. We also give a first exact Transformer DC decomposition algorithm within this framework, addressing the extension left open in *Hidden Monotonicity* through splits of softmax, LayerNorm, and non-piecewise-linear GELU (Section 5.3).
3) **A game-theoretic account of backward attribution.** We represent gradients and $\alpha\beta$-LRP through two-player games, providing a common basis for comparing and modifying their propagation rules and diagnosing changes in backward routing.
4) **New attribution methods with improved faithfulness and localization.** SplitCAM achieves the best Selectivity, a faithfulness metric, among the evaluated VGG-16 and ResNet-18 methods; we also introduce SplitLRP and SplitGrad. Game-derived attribution achieves the highest Attribution Localization among the evaluated ViT-B/16 methods on ImageNet-S (Gao et al., 2023), with this lead transferring to Pascal VOC without re-tuning.

[1] Official CVPR listing: https://cvpr.thecvf.com/virtual/2026/poster/36564.

5) **Directly trained difference models.** We train differences of convex or monotone networks and inspect their stream gradients on MNIST, as a counterpart to decomposing pretrained models.

1.3 Positioning within the Literature

This brief section positions the thesis along its three research questions; detailed chapter-opening discussions cover games and propagation rules (Section 4.1) and transformer interpretability (Section 5.1).

What structure is hidden inside a trained network? The use of monotonicity and convexity to structure interpretable models runs from monotonic networks (Daniels and Velikova, 2010; Sill, 1997) through certified and lattice-based monotone models (Gupta et al., 2016; Liu et al., 2020; Nguyen et al., 2023; Runje and Shankaranarayana, 2023; Sivaraman et al., 2020) to ***I**nput-**C**onvex **N**eural **N**etwork*s (ICNNs) (Amos et al., 2017) and convex network calculus (Bach, 2017; Calafiore et al., 2020; Sivaprasad et al., 2021). DC decompositions bridge this to unconstrained networks: classical DC programming (Le Thi and Pham Dinh, 2018; Sriperumbudur and Lanckriet, 2009; Tao and An, 1997) studies optimization with such decompositions, while the representation theory of piecewise-linear functions as differences of convex ones (Brandenburg et al., 2025; Griewank and Walther, 2020; Kazda and Li, 2024; Koutschan et al., 2025; Kripfganz and Schulze, 1987; Melzer, 1986; Schlüter and Schulze Darup, 2020; Zalgaller, 2000) guarantees the split exists; Awasthi et al. (2026) and Siahkamari et al. (2020) construct essentially the split used here, for optimization and regression, and CDiNN-type architectures make the difference structure trainable (Chen et al., 2025; Sankaranarayanan and Rengaswamy, 2022). This thesis studies the split's numerical instability in deep networks and its use for explanation. The geometric reading rests on the identification of ReLU networks with ***C**ontinuous **P**iece**w**ise **L**inear* (CPWL) maps (Arora et al., 2018; Balestriero and Baraniuk, 2021; Huchette et al., 2026) and their tropical representations (Zhang et al., 2018b); related piecewise-linear geometry informs functional identifiability (Phuong and Lampert, 2020).

When can explanations be trusted? The attribution toolbox spans gradient saliency (Simonyan et al., 2014), deconvolution and guided backpropagation (Springenberg et al., 2015; Zeiler and Fergus, 2014), LRP and Deep Taylor decomposition (Bach et al., 2015; Montavon et al., 2017, 2018), ***C**lass **A**ctivation **M**apping* (CAM) variants (Chattopadhay et al., 2018; Jiang et al., 2021; Selvaraju et al., 2017), axiomatic methods (Shrikumar et al., 2017; Sundararajan et al., 2017), noise averaging (Smilkov et al., 2017), and perturbation- or surrogate-based explanations (Fel et al., 2023; Fong and Vedaldi, 2017; Kolek et al., 2023; Lundberg and Lee, 2017; Ribeiro et al., 2016). A wave of critiques sharpened the trust question: sanity checks reveal parameter insensitivity (Adebayo et al., 2018), explanations can be fragile or manipulable (Alvarez-Melis and Jaakkola, 2018; Dombrowski et al., 2019; Ghorbani et al., 2019), modified backward rules can produce structure unrelated to the model's class decision (Nie et al., 2018; Sixt et al., 2020), and input dependence confounds attribution (Kindermans et al., 2018). In response, evaluation frameworks such as Quantus (Hedström et al., 2023) operationalize faithfulness, localization (Zhang et al., 2018a), robustness (Yeh et al., 2019), and complexity (Bhatt et al., 2020) as measurable desiderata — the standard this thesis holds its methods to in Chapter 7. The thesis' answer is structural rather than post hoc: attribution is computed on a representation (the split-pair) in which evidence cannot cancel invisibly, or derived from a game whose trajectory distribution records normalized backward routing and whose signed discounted occupation measures recover attribution, so sanity-check failures can be studied at both the trajectory and pixel levels

(Chapter 8); their different responses to randomization do not by themselves establish explanation faithfulness.

How can attribution be made stable? The closest precursor to the game framework is the ReLU Net Game of Gaubert & Vlassopoulos (2025), which identifies forward evaluation with the value of a zero-sum stopping game but connects it neither to the backward pass nor to attribution. Related stochastic views of relevance flow include Excitation Backprop's absorbing Markov chain (Zhang et al., 2018a), which the ***S**topping **G**ame* (SG) generalizes by adding an opponent, and the ***N**ormalized **R**elevance **M**easure* (NRM) framework (Xiong et al., 2026), a signed-measure calculus over walks compared with our trajectory distributions in Section 8.1. Model-agnostic Shapley-value methods such as KernelSHAP (Lundberg and Lee, 2017) play a *different*, cooperative game over input coalitions in a black-box setting, complementary to the white-box backward game here. Stability itself has been approached via curvature reduction with softplus activations (Dombrowski et al., 2022) — recovered here as entropy regularization of the stop decision. Risk-sensitive control (Fei et al., 2020) provides a separate motivation for the risk-averse game modes. On transformers, attention rollout and flow (Abnar and Zuidema, 2020), transformer relevance propagation (Achtibat et al., 2024; Ali et al., 2022; Chefer et al., 2021a,b), and artifact-aware constructions (Darcet et al., 2024; Wróbel et al., 2026) define the state of the art Chapter 5 engages with; Section 4.1 and Section 5.1 give the full accounts. The growing use of attribution as a control mechanism — model correction (Pahde et al., 2023), head identification (Kahardipraja et al., 2025; Voita et al., 2019), guided decoding (Komorowski et al., 2026), circuit localization (Hanna et al., 2024; Kramár et al., 2024; Syed et al., 2024) — raises the stakes: unstable attributions become failure modes of downstream procedures, not merely misleading pictures.

Outline. Chapter 2 assembles the foundations: network notation, CPWL mappings, convexity and monotonicity, ***M**arkov **D**ecision **P**rocesses* (MDPs) and zero-sum games, including the ReLU Net Game of Gaubert and Vlassopoulos. Chapter 3 develops the decomposition framework, numerical stabilization, and the CNN extension; Chapter 4 the two-player stopping and routing games, with gradients and $\alpha\beta$-LRP recovered as equilibrium occupation measures; Chapter 5 the extension and implementation on multiplicative operations and ***Vi**sion **T**ransformer*s (ViTs); and Chapter 6 the split methods SplitCAM, SplitLRP, and SplitGrad, inherently interpretable difference-of-convex architectures, and the attribution modes. Chapter 7 reports the evaluation on ImageNet-S and Pascal VOC across VGG-16, ResNet, and ViT-B/16; Chapter 8 develops the trajectory-distribution view and its Hellinger diagnostics; and Chapter 9 concludes with future directions from ***L**arge **L**anguage **M**odel*s (LLMs) to hardware-level acceleration.

2 Mathematical Foundations

We establish network notation and the ReLU–CPWL equivalence (Section 2.1); monotone networks, ICNNs, and Maxout units (Section 2.2); and MDPs, zero-sum games, and the ReLU Net Game of Gaubert and Vlassopoulos (Section 2.3). These underpin Chapters 3 and 4; the absolute-curvature construction for smooth activations accompanies the decomposition theorem in Section 3.2.2.

2.1 Neural Network Notation and PWL Mappings

The standard feedforward architecture serves as the reference model throughout; convolutional and attention-based architectures are treated in Section 3.7 and Chapter 5 as structured special cases and extensions.

Definition 2.1 (Feedforward ReLU network). A *feedforward ReLU network* of depth $L \in \mathbb{N}$ has *layer widths* $d^{(0)},\dots,d^{(L)} \in \mathbb{N}$, weights $W^{(l)} \in \mathbb{R}^{d^{(l)}\times d^{(l-1)}}$, and biases $b^{(l)} \in \mathbb{R}^{d^{(l)}}$. For input $a^{(0)} = x \in \mathbb{R}^{d^{(0)}}$, its *pre-activations* $z^{(l)}$, *activations* $a^{(l)}$, and function $f : \mathbb{R}^{d^{(0)}} \to \mathbb{R}^{d^{(L)}}$ satisfy

$$z^{(l)} \coloneqq W^{(l)}a^{(l-1)} + b^{(l)}, \qquad l = 1,\dots,L, \tag{2.1}$$

$$a^{(l)} \coloneqq \mathrm{ReLU}\big(z^{(l)}\big), \qquad l = 1,\dots,L-1,$$

$$f(x) = a^{(L)} \coloneqq z^{(L)}. \tag{2.2}$$

Here $\mathrm{ReLU}(z) = \max\{z,0\}$ acts componentwise on the hidden layers $1,\dots,L-1$; the output is affine. We use $\mathrm{ReLU}'(0) = 0$, so gates $\mathbf{1}\{z_i^{(l)} > 0\}$ are closed at zero.

Write $f^{(l)} : \mathbb{R}^{d^{(l)}} \to \mathbb{R}^{d^{(L)}}$ for the tail after layer l: thus $f^{(0)} = f$, $f^{(L-1)}$ is the affine output map, and $f^{(L)}$ is the identity. Appending a constant *intercept input* 1 absorbs $b^{(l)}$ into a column of $W^{(l)}$; some proofs use this bias-free formulation implicitly.

Chapter 3 splits parameters and inputs by sign using the following notation.

Definition 2.2 (Positive and negative parts). For a scalar $u \in \mathbb{R}$ we write $u^+ \coloneqq \max\{u,0\}$ and $u^- \coloneqq \max\{-u,0\}$, so that $u = u^+ - u^-$ and $|u| = u^+ + u^-$, with $u^+u^- = 0$. For a matrix or vector W and a sign label $\sigma \in \{+,-\}$, the *entrywise part* $[W]^\sigma$ is obtained by applying $u \mapsto u^\sigma$ to every entry. For the layer weights and biases of Definition 2.1 we abbreviate

$$W^{(l,\sigma)} \coloneqq [W^{(l)}]^\sigma, \qquad b^{(l,\sigma)} \coloneqq [b^{(l)}]^\sigma, \qquad \sigma \in \{+,-\}, \tag{2.3}$$

so that $W^{(l)} = W^{(l,+)} - W^{(l,-)}$ and $|W^{(l)}| = W^{(l,+)} + W^{(l,-)}$ entrywise, and both $W^{(l,\sigma)}$ are entrywise non-negative.

Remark 2.1 (Pair notation for decomposed activations). The activation labels $a^{(l,\sigma)}$, $\sigma \in \{+,-\}$, denote the two non-negative *streams* defined in Section 3.1, distinct from the entrywise parts $[a^{(l)}]^\sigma$. The streams carry strictly more information than the sign pattern of $a^{(l)}$, supplying the sub-threshold signals exploited in Chapter 6.

Continuous piecewise linear functions

The function class computed by ReLU networks admits an exact geometric description.

Definition 2.3 (CPWL function). A function $f\colon \mathbb{R}^d \to \mathbb{R}^m$ is ***C**ontinuous **P**iece**w**ise **L**inear* (CPWL) if it is continuous and there exist finitely many closed polyhedra $Q_1, \dots, Q_N \subseteq \mathbb{R}^d$ with $\bigcup_{r=1}^{N} Q_r = \mathbb{R}^d$ such that the restriction of f to each Q_r is an affine map. The full-dimensional polyhedra on which f is affine, taken maximal with respect to inclusion, are called the *linear regions* of f.

For example, $|\cdot|\colon \mathbb{R} \to \mathbb{R}$ is CPWL with the two regions $(-\infty, 0]$ and $[0, \infty)$, and $x \mapsto \max\{x_1, \dots, x_d\}$ is CPWL with d regions.

Observation 2.1 (`Lean_1`). Every feedforward ReLU network represents a CPWL function.

Proof. Affine maps and componentwise ReLU are CPWL. Composition preserves this class: if f and g are affine on polyhedral covers $Q_1, \dots, Q_N$ and $P_1, \dots, P_M$ of their respective domains, then $g \circ f$ is affine on the cover of at most NM polyhedra $Q_r \cap f^{-1}(P_s)$. These are polyhedra because f is affine on each Q_r; composition also preserves continuity. Induction along (2.1) proves the claim. □

Conversely, every CPWL function has an exact ReLU representation with a dimension-dependent depth bound (Arora et al., 2018). This equivalence underlies the difference-of-convex analysis of Chapter 3.

Theorem 2.1 (CPWL–ReLU equivalence, (Arora et al., 2018), `Lean_2`). *A function $f\colon \mathbb{R}^d \to \mathbb{R}$ is CPWL if and only if it is representable by a feedforward ReLU network. Moreover, every CPWL function on $\mathbb{R}^d$ is representable by a ReLU network of depth at most $\lceil \log_2(d+1) \rceil + 1$.*

Proof. Observation 2.1 proves the "if" direction. For the converse, every CPWL function f admits a *difference-of-convex* representation

$$f = g - h, \qquad g = \max_{r \in [N_g]} \ell_r, \qquad h = \max_{s \in [N_h]} m_s, \tag{2.4}$$

with finitely many affine functions $\ell_r, m_s\colon \mathbb{R}^d \to \mathbb{R}$ (Kripfganz and Schulze, 1987; Melzer, 1986); their finite maxima g,h are convex. The maximum of two scalars is a two-layer ReLU computation,

$$\max\{u, v\} = \frac{u+v}{2} + \frac{|u-v|}{2} = \frac{u+v}{2} + \frac{\mathrm{ReLU}(u-v) + \mathrm{ReLU}(v-u)}{2}, \tag{2.5}$$

absorbing the affine pre- and post-combinations into surrounding weight matrices. A balanced binary tree of pairwise maxima realizes g at depth $\lceil \log_2 N_g \rceil + 1$, and likewise h; a final affine layer forms (2.4), proving representability. For the bound independent of the number of pieces, Arora et al. (2018) use the Wang–Sun representation of every CPWL function on $\mathbb{R}^d$ as a signed sum of maxima of at most $d+1$ affine functions each. These need at most $\lceil \log_2(d+1) \rceil$ hidden layers and a final affine output layer. □

Remark 2.2 (Expressivity and linear regions). Theorem 2.1 identifies the function *class*, not the *cost* of representation: deep networks can represent CPWL functions with exponentially many linear regions in the number of neurons, an interplay of depth, width, and region counts surveyed by Huchette et al. (2026). The representation (2.4) also foreshadows the central theme of this thesis: the convex CPWL

streams g and h of the DC split can be extracted *constructively and locally* from a trained network, the subject of Section 3.2.3. For a stream $g(x) = \max_r(a_r^\top x + b_r)$, a uniquely active affine piece gives $\nabla g(x) = a_r$: the stream gradient records the slope of its current linear region.

2.2 Convexity and Monotonicity in Neural Networks

Chapter 3 splits general networks into sign-constrained components. We define the two classes and prove their monotonicity and convexity, respectively.

We order $\mathbb{R}^d$ coordinatewise: $x \le y$ if and only if $x_i \le y_i$ for all $i \in [d]$. A function $f\colon \mathbb{R}^d \to \mathbb{R}^m$ is *monotone* (non-decreasing) if $x \le y$ implies $f(x) \le f(y)$.

Definition 2.4 (Monotone neural network). A feedforward network with recursion (2.1) and monotone activation functions is called a *monotone neural network* if all its weight matrices are entrywise non-negative, $W^{(l)} \ge 0$ for all $l = 1, \dots, L$.

Weight-sign constraints are an established device for building monotone models (Daniels and Velikova, 2010; Sill, 1997) and remain central to modern monotone architectures (Runje and Shankaranarayana, 2023); monotonicity can also be certified without imposing such constraints (Liu et al., 2020).

Proposition 2.2 (Monotone networks represent monotone functions, `Lean_3`). *A monotone neural network represents a monotone function; each coordinate function is non-decreasing in every input variable.*

Proof. Induct from the monotone input $a^{(0)} = x$. Non-negative weights preserve coordinatewise order: $a \le a'$ implies $W^{(l)}a + b^{(l)} \le W^{(l)}a' + b^{(l)}$. Thus monotonicity of $a^{(l-1)}$ implies that of $z^{(l)}$; componentwise monotone activation gives monotonicity of $a^{(l)}$. The same order argument covers the affine output. □

Definition 2.5 (Input-Convex Neural Network, (Amos et al., 2017)). A feedforward network with recursion (2.1) whose activation functions are convex *and* monotone is called an ***I**nput-**C**onvex **N**eural **N**etwork* (ICNN) if all weight matrices except possibly the first are entrywise non-negative, $W^{(l)} \ge 0$ for all $l = 2, \dots, L$, while $W^{(1)}$ is unconstrained.

Definition 2.5 covers the plain feedforward core of the architecture of Amos et al. (2017); their sign-unconstrained skip ("passthrough") connections do not affect the statement below, since they enter each pre-activation affinely in x.

Proposition 2.3 (ICNNs represent convex functions, `Lean_4`). *An ICNN represents a convex function; each coordinate of every activation $a^{(l)}$, $l \ge 1$, is a convex function of the input x.*

Proof. Non-negative linear combinations plus constants preserve convexity. So does composition with a convex non-decreasing scalar function φ: for convex u and $\lambda \in [0,1]$,

$$\varphi\big(u(\lambda x + (1-\lambda)y)\big) \le \varphi\big(\lambda u(x) + (1-\lambda)u(y)\big) \le \lambda\varphi(u(x)) + (1-\lambda)\varphi(u(y)).$$

The first inequality uses convexity of u and monotonicity of φ; the second uses convexity of φ. Starting from the affine $z^{(1)}$, these two closure rules give convex coordinates at every layer by induction, including the affine output. □

Remark 2.3 (Monotonicity and convexity combine). With $W^{(1)} \geq 0$, an ICNN is both monotone and convex by Propositions 2.2 and 2.3. ReLU and Maxout meet both activation conditions. Section 3.2.3 therefore constructs g,h as monotone networks on a signed input embedding, convex in the original input. Non-negative-weight ReLU networks represent only convex functions; broader monotone architectures add concave activation counterparts (Runje and Shankaranarayana, 2023). Taking differences instead recovers every CPWL function by Theorem 2.1 and (2.4).

Maxout units

Definition 2.6 (Maxout unit and Maxout network, (Goodfellow et al., 2013)). A *rank-k Maxout unit* with weights $w_1, \dots, w_k \in \mathbb{R}^d$ and biases $\beta_1, \dots, \beta_k \in \mathbb{R}$ computes

$$x \longmapsto \operatorname{Maxout}(x) := \max_{j \in [k]} \big(w_j^\top x + \beta_j \big). \tag{2.6}$$

A *rank-k Maxout network* is a feedforward network whose neurons are rank-k Maxout units; equivalently, layer l carries k weight matrices $W_1^{(l)}, \dots, W_k^{(l)}$ and biases $b_1^{(l)}, \dots, b_k^{(l)}$, and

$$a_i^{(l)} := \max_{j \in [k]} \big(W_j^{(l)} a^{(l-1)} + b_j^{(l)} \big)_i. \tag{2.7}$$

Observation 2.2 (ReLU and max-pooling as Maxout, `Lean_5`*).* The ReLU activation is the rank-2 Maxout unit with parameters $(w_1, \beta_1) = (w, \beta)$ and $(w_2, \beta_2) = (0, 0)$, since $\operatorname{ReLU}(w^\top x + \beta) = \max\{w^\top x + \beta, 0\}$. Max-pooling over a group J of neurons is the rank-$|J|$ Maxout unit whose j-th argument is the coordinate projection onto the j-th element of J. Every rank-k Maxout unit is convex in its input and non-decreasing in each pre-activation argument $w_j^\top x + \beta_j$; hence Maxout qualifies as an admissible activation in Definitions 2.4 and 2.5.

Proof. The first two claims follow from (2.6). A maximum of affine functions is convex. For argument-wise monotonicity, set $u_j = w_j^\top x + \beta_j$: $u_j \leq u'_j$ for all j implies $\max_j u_j \leq \max_j u'_j$. □

Remark 2.4. Maxout networks are the natural piecewise-linear generalization of ReLU: a rank-k unit expresses a maximum of k affine functions in one neuron, matching the building blocks of the DC representation (2.4). The DC splitting of Chapter 3 is developed for ReLU networks and extended to Maxout units—and to the multiplicative interactions needed for attention—in Chapter 5, using the exchange identity $\max_{j \in [k]}(u_j - v_j) = \max_{j \in [k]} \{ u_j + \sum_{i \neq j} v_i \} - \sum_{i \in [k]} v_i$ for pairs of non-negative streams.

2.3 Game Theory and Markov Decision Processes

Chapter 4 interprets backward propagation as a two-player zero-sum game on the decomposed network graph. We introduce MDPs, turn-based games, and their optimality equations, restricting throughout to finite state and action spaces.

Definition 2.7 (Markov Decision Process). A (finite) ***Markov Decision Process*** (MDP) (Puterman, 1994; Sutton and Barto, 2018) is a tuple $\mathscr{M} = (S, A, \mathbb{P}, R, \gamma)$ consisting of a finite state set S, a finite nonempty action set $A(s)$ for each $s \in S$, a *transition kernel* $\mathbb{P}(s' \mid s, a) \geq 0$ with $\sum_{s'} \mathbb{P}(s' \mid s, a) = 1$, a bounded *reward function* $R(s, a) \in \mathbb{R}$, and a *discount factor* $\gamma \in [0, 1)$. A *(stationary) policy* is a map π assigning to each state s a probability distribution $\pi(\cdot \mid s)$ over $A(s)$;

π is *pure* (deterministic) if each $\pi(\cdot \mid s)$ is a point mass. A policy induces, for every start state, a Markov chain on S and hence a distribution over trajectories $(s_0, a_0, s_1, a_1, \ldots)$; the *value function* of π is the expected discounted return

$$V^\pi(s) \coloneqq \mathbb{E}^\pi\left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) \,\middle|\, s_0 = s\right], \tag{2.8}$$

and the *optimal value function* is $V^\star(s) \coloneqq \max_\pi V^\pi(s)$.

Fixing an enumeration of S, a pure policy π turns the kernel into a row-stochastic *transition matrix* $P^\pi \in \mathbb{R}^{S\times S}$ with $P^\pi_{ss'} = \mathbb{P}(s' \mid s, \pi(s))$ and the reward into a vector R^π, and (2.8) becomes the Neumann series $V^\pi = \sum_{t\geq 0} \gamma^t (P^\pi)^t R^\pi$. This matrix picture recurs in Chapter 4, where the (sub)stochastic matrices are built from the normalized weight parts $W^{(l,\sigma)}$ of Definition 2.2.

Proposition 2.4 (Bellman equations, `Lean_6`). *For a finite MDP with $\gamma \in [0,1)$, define*

$$Q_V(s,a) \coloneqq R(s,a) + \gamma \sum_{s'} \mathbb{P}(s' \mid s,a) V(s').$$

Then V^π and $V^\star$ uniquely solve, respectively,

$$V(s) = \sum_{a\in A(s)} \pi(a \mid s) Q_V(s,a), \qquad s \in S, \tag{2.9}$$

$$V(s) = \max_{a\in A(s)} Q_V(s,a), \qquad s \in S. \tag{2.10}$$

The first is a linear system; the second is the Bellman optimality equation. *Any pure policy $\pi^\star$ selecting a maximizer of $Q_{V^\star}$ in each state is optimal: $V^{\pi^\star} = V^\star$.*

Proof. Both right-hand sides are γ-contractions in $\|\cdot\|_\infty$, hence have unique fixed points. Conditioning (2.8) on the first transition identifies them as V^π and $V^\star$; a policy greedy for $V^\star$ attains its value (Puterman, 1994; Sutton and Barto, 2018). Acyclic games instead admit backward induction (Remark 2.5). □

Remark 2.5 (State-dependent structural discounts). Our games use *state-dependent* discounts $\gamma(s) \geq 0$ that may exceed 1. These are *structural* normalizers: for neuron i in layer l, the absolute weight mass

$$\gamma_i^{(l)} \coloneqq \sum_j \left(W^{(l,+)} + W^{(l,-)}\right)_{ij} = \sum_j |W_{ij}^{(l)}|, \tag{2.11}$$

restores the mass after normalizing row i of $|W^{(l)}|$ into transition probabilities. Although contraction is unavailable, *layered acyclic* graphs ensure finite trajectories and value sums. Backward induction gives a unique value and optimal policies, possibly nonunique at ties, as the two-player theorem below shows.

Zero-sum turn-based games

Two-player games enter because a decomposed network carries two interacting streams: a player P^+ who collects positive evidence and a player P^- who collects negative evidence. We use the following conventions throughout: $p, q, r \in \{+, -\}$ denote player, turn, or sign labels; p' denotes the opponent

$(+' = -,\ -' = +)$; the *sign-XOR* $p \oplus q$ is $+$ if $p = q$ and $-$ otherwise; and $\xi_p \in \{+1, -1\}$ is the numerical sign of the label p, so that $\xi_p\, \xi_q = \xi_{p \oplus q}$.

Definition 2.8 (Zero-sum turn-based game). A (finite) *zero-sum turn-based game* is a tuple $\mathscr{G} = (S_+, S_-, S_{\text{ter}}, A, \mathbb{P}, R, \gamma)$ where the state set $S = S_+ \uplus S_- \uplus S_{\text{ter}}$ is partitioned into states controlled by player P^+, states controlled by player P^-, and terminal states; the action sets $A(s)$, the transition kernel $\mathbb{P}$, and the reward R are as in Definition 2.7 on non-terminal states; $\gamma \colon S \to \mathbb{R}_{\geq 0}$ is a (possibly state-dependent) discount; and each terminal state $s \in S_{\text{ter}}$ carries a fixed one-time payoff $R(s)$. A *policy pair* $\pi = (\pi_+, \pi_-)$ consists of a policy π_p for player P^p on S_p. Play generates a trajectory as in an MDP, with the player in turn choosing the action; the expected total payoff $V^\pi(s)$, defined as in (2.8) with the discount accumulated multiplicatively along the trajectory (γ^t replaced by $\prod_{m=0}^{t-1} \gamma(s_m)$), is paid *by* P^- *to* P^+: player P^+ maximizes V^π and player P^- minimizes it.

Theorem 2.5 (Shapley–Bellman optimality on layered games, `Lean_7`). *Let $\mathscr{G}$ be a finite zero-sum turn-based game with acyclic transition graph. Use Q_V from Proposition 2.4, replacing γ by $\gamma(s) \geq 0$. Its unique* value $V^\star : S \to \mathbb{R}$ *satisfies the* Shapley–Bellman equations

$$V^\star(s) = \begin{cases} \max_{a \in A(s)} Q_{V^\star}(s,a), & s \in S_+, \\ \min_{a \in A(s)} Q_{V^\star}(s,a), & s \in S_-, \\ R(s), & s \in S_{\text{ter}}. \end{cases} \tag{2.12}$$

and there exist pure stationary policies $\pi^\star = (\pi^\star_+, \pi^\star_-)$, obtained by selecting maximizing respectively minimizing actions in (2.12), such that for all $s \in S$ and all policies π_+, π_-,

$$V^{(\pi^\star_+, \pi_-)}(s) \ \geq\ V^{\pi^\star}(s) = V^\star(s) \ \geq\ V^{(\pi_+, \pi^\star_-)}(s). \tag{2.13}$$

In particular, $\pi^\star$ is a Nash equilibrium of every subgame, i.e. a subgame perfect equilibrium.

Proof. Backward induction on the finite acyclic graph starts from $V^\star(s) = R(s)$ at terminal states. At each remaining state, the known successor values determine $Q_{V^\star}$; its finite nonempty action set has an attained maximum or minimum, uniquely determining the value and a pure optimal choice.

For the saddle inequalities, induct in the same order. At $s \in S_+$, playing the maximizing action against any π_- gives at least $V^\star(s)$ by the successors' left inequality and $\gamma(s) \geq 0$. Conversely, any π_+ against $\pi^\star_-$ gives at most the average of $Q_{V^\star}(s,\cdot)$, hence at most its maximum $V^\star(s)$, by the successors' right inequality. At $s \in S_-$ the argument exchanges maximum and minimum. With both players optimal, the inequalities give $V^{\pi^\star} = V^\star$. Since this holds at every state, the equilibrium is subgame perfect. □

Remark 2.6. For general discounted stochastic games with cycles, existence of the value and of optimal stationary policies is Shapley's theorem (Shapley, 1953), proved by the contraction argument of Proposition 2.4 applied to the *Shapley operator*—the min-max analogue of the Bellman operator (2.12). Gaubert & Vlassopoulos (2025) formulate their network games in this operator language. The layered acyclic case of Theorem 2.5 is all we need, and it additionally delivers subgame perfection, on which Chapter 8 relies.

2.3.1 The ReLU Net Game of Gaubert and Vlassopoulos

Gaubert & Vlassopoulos (2025) represent the forward pass of a ReLU network as the value of a zero-sum turn-based stopping game played backward on its graph. We give their full construction in our notation; Chapter 4 retains its states, transitions, and stopping structure with revised payoff bookkeeping. Fix a network from Definition 2.1 with scalar output ($d^{(L)} = 1$), an input $x \in \mathbb{R}^{d^{(0)}}$, and nonzero weight rows, ensuring strictly positive discounts (2.11).

Read the identity $a_i^{(l)} = \max\{0, z_i^{(l)}\}$ as a decision: a player at neuron i may *stop*, securing payoff 0, or *continue*, collecting the bias $b_i^{(l)}$ plus the weighted contributions of the previous layer. Continuing propagates the decision backward through the network; a negative weight flips the sign of its contribution, i.e. hands the turn to an adversary facing the mirrored problem. The optimal value at neuron i is exactly the activation $a_i^{(l)}$.

Definition 2.9 (ReLU Net Game, following (Gaubert and Vlassopoulos, 2025)). The *ReLU Net Game* on the network f at input x is the zero-sum turn-based game with the following components.

- **States.** Each hidden neuron $i \in [d^{(l)}]$, $l \in [L-1]$, has activation states $s_{i,q}^{(l)}$ controlled by P^q, $q \in \{+,-\}$. The affine output state $s^{(L)}$ is assigned to P^+ with no choice of action. Terminal states are $s_{k,q}^{(0)}$ for $k \in [d^{(0)}]$, $q \in \{+,-\}$, and a zero-payoff *cemetery* $\perp$; play ends upon reaching any terminal state.
- **Actions.** At every activation state the player in turn chooses an action from $\{\text{stop}, \text{cont}\}$. The output state has the single action cont.
- **Transitions.** Stopping moves deterministically to the cemetery, $\mathbb{P}(\perp \mid s_{i,q}^{(l)}, \text{stop}) = 1$. Continuation at $s_{i,q}^{(l)}$ moves one layer backward, to neuron j of layer $l-1$ with probability proportional to the absolute weight, keeping the turn on positive weights and passing it to the opponent on negative weights:

$$\mathbb{P}\big(s_{j,r}^{(l-1)} \mid s_{i,q}^{(l)}, \text{cont}\big) := \frac{W_{ij}^{(l,q\oplus r)}}{\gamma_i^{(l)}}, \qquad \gamma_i^{(l)} = \sum_j \big(W^{(l,+)} + W^{(l,-)}\big)_{ij}, \tag{2.14}$$

  where for $l = 1$ the successor $s_{j,r}^{(0)}$ is the corresponding terminal state. The output state transitions by the same rule (2.14) with $l = L$ and turn label $q = +$.
- **Payoffs and discounts.** Continuation at $s_{i,q}^{(l)}$ pays the signed bias $\xi_q\, b_i^{(l)}$; stopping pays 0. A terminal state pays the signed input coordinate once, $R\big(s_{k,q}^{(0)}\big) := \xi_q\, x_k$, and the game ends. The output state pays $b^{(L)}$ upon its forced continuation. The discount at every activation state and at the output state is the structural discount $\gamma\big(s_{i,q}^{(l)}\big) := \gamma_i^{(l)}$ of (2.11).

Player P^+ maximizes and P^- minimizes the expected discounted total payoff, as in Definition 2.8.

Each transition decreases the layer index or terminates, so Theorem 2.5 applies for arbitrary positive structural discounts. Multiplying (2.14) by $\gamma_i^{(l)}$ restores the sign-selected weight $W_{ij}^{(l,q\oplus r)}$; summing signed successor values recovers the forward affine calculation.

Theorem 2.6 (Forward pass as game value, (Gaubert and Vlassopoulos, 2025), `Lean_8`). *The ReLU Net Game of Definition 2.9 admits the subgame perfect equilibrium $\pi^\star = (\pi_+^\star, \pi_-^\star)$ in which*

both players use the same pure stopping rule,

$$\pi_q^\star\big(s_{i,q}^{(l)}\big) = \text{cont} \quad \Longleftrightarrow \quad z_i^{(l)} > 0, \tag{2.15}$$

and the value function of the game evaluates the network:

$$V^\star\big(s_{i,q}^{(l)}\big) = \xi_q\, a_i^{(l)} \quad \text{for } l \in \{0,1,\dots,L-1\}, \qquad V^\star\big(s^{(L)}\big) = f(x), \tag{2.16}$$

where for $l = 0$ we read $a_k^{(0)} = x_k$. The equilibrium is unique up to tie-breaking at neurons with $z_i^{(l)} = 0$, where both actions are optimal; the convention (2.15) stops at ties, matching $ReLU'(0) = 0$.

Proof. Induct from the terminal values $R(s_{k,q}^{(0)}) = \xi_q x_k$. If (2.16) holds at layer $l-1$, continuation at either turn q has value

$$Q_{V^\star}\big(s_{i,q}^{(l)},\text{cont}\big) = \xi_q b_i^{(l)} + \sum_{j,r} W_{ij}^{(l,q\oplus r)} \xi_r a_j^{(l-1)} = \xi_q z_i^{(l)}, \tag{2.17}$$

since $\sum_r \xi_r W_{ij}^{(l,q\oplus r)} = \xi_q W_{ij}^{(l)}$. Stopping yields zero. Thus P^+ obtains $\max\{0, z_i^{(l)}\} = a_i^{(l)}$, while P^- obtains $\min\{0, -z_i^{(l)}\} = -a_i^{(l)}$. Both continue precisely for positive pre-activation, with either action optimal at zero and our convention choosing stop. This proves the value and uniqueness up to ties by Theorem 2.5. Forced continuation at the affine output gives $V^\star(s^{(L)}) = z^{(L)} = f(x)$ by the same calculation with $l = L$, $q = +$. □

Remark 2.7 (Softplus networks from entropy regularization). Gaubert & Vlassopoulos (2025) also treat softplus networks, by *regularizing* the stop/continue decision. Replace the hard choice at a positive activation state by a mixed policy $\nu \in \Delta(\{\text{stop}, \text{cont}\})$ rewarded with a Shannon-entropy bonus at temperature $\theta > 0$. At a negative state, the entropy term enters with the opposite sign in the payoff to P^+. By the Legendre–Fenchel identity for the entropy (Boyd and Vandenberghe, 2004),

$$\max_{\nu\in\Delta_n}\Big\{\textstyle\sum_{i=1}^n \nu_i r_i + \theta\,\mathscr{H}(\nu)\Big\} = \theta \log \textstyle\sum_{i=1}^n e^{r_i/\theta}, \qquad \mathscr{H}(\nu) = -\textstyle\sum_i \nu_i \log \nu_i, \tag{2.18}$$

with the Gibbs distribution $\nu_i^\star \propto e^{r_i/\theta}$ as unique maximizer, the regularized value at a positive state with continuation value $z_i^{(l)}$ (computed as in (2.17)) becomes $\theta \log\big(1 + e^{z_i^{(l)}/\theta}\big)$—the softplus activation at temperature θ—and the equilibrium policy continues with the sigmoid probability $\big(1 + e^{-z_i^{(l)}/\theta}\big)^{-1}$. The hard game of Definition 2.9 is recovered in the limit $\theta \to 0$. Chapter 4 re-derives this mechanism from the regularized decompositions of Section 3.5 and reuses it at the routing states of the ***R**outing **G**ame* (RG), with the two temperatures θ (activation gates) and τ (routing) kept distinct.

Remark 2.8 (What Chapter 4 inherits and what it changes). The SG retains Definition 2.9's per-neuron ± states, stop/continue actions, bias and terminal input payoffs, weight-part kernel (2.14), cemetery, and structural discounts $\gamma_i^{(l)}$. Chapter 4 adds *non-negative player payoffs*: the values $V_p(s)$, $p \in \{+,-\}$, equal the Stopping Decomposition streams (Section 3.4), whose difference recovers the original signed value (2.16). Replacing signed terminal payoffs $\xi_q x_k$ by (x_k^+, x_k^-) (Definition 2.2) keeps all player-specific quantities non-negative and exposes backward-trajectory parity. The chapter also adds skip connections, max-pooling, risk aversion, and regularized policies, absent from the original game.

3 Network Decompositions and DC Formulations

Network decompositions separate positive and negative contributions while preserving their difference. DC streams provide convex, monotone functions for Chapter 6; stopping streams preserve the gates underlying Chapter 4. Mixing and regularization connect these choices. We compare their retained information and backward calculations, then stabilize them for deep networks.

3.1 Generalized Network Decomposition

Replacing each signed activation by a non-negative pair exposes accumulated supporting and opposing contributions while preserving the original value as their difference. Figure 3.1 illustrates the DC instance (Section 3.2.3).

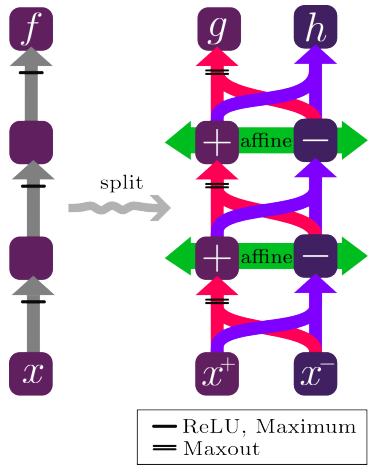


Fig. 3.1: The network splitting procedure: a ReLU network f is decomposed into two monotone, convex streams g and h with $f = g - h$ (Section 3.2.3).

Consider an L-layer feedforward network with weights $W^{(l)}$, biases $b^{(l)}$, and entrywise activation $\varrho\colon \mathbb{R} \to \mathbb{R}$ (e.g. ReLU or softplus):

$$a^{(0)} = x, \qquad z^{(l)} = W^{(l)} a^{(l-1)} + b^{(l)}, \qquad a^{(l)} = \varrho_l\big(z^{(l)}\big), \qquad l = 1, \dots, L, \tag{3.1}$$

where $\varrho_l = \varrho$ for $1 \le l < L$ and $\varrho_L = \mathrm{id}$ gives the ungated affine readout $f(x) = a^{(L)} \in \mathbb{R}$. For multi-class networks, fix one target logit. We use the notation of Section 2.1, reserving g,h for the split streams of Section 3.2.3.

Definition 3.1 (Decomposition). A *decomposition* of the L-layer network (3.1) propagates non-negative activation pairs $\big(a_i^{(l,+)}, a_i^{(l,-)}\big) \in \mathbb{R}^2_{\ge 0}$ from input to output using only

1) **non-negative linear maps**—matrices with non-negative entries together with non-negative bias vectors—and
2) **pairwise activation functions** $\phi\colon \mathbb{R}^2_{\ge 0} \to \mathbb{R}^2_{\ge 0}$, $(z^+, z^-) \mapsto (a^+, a^-) = \phi(z^+, z^-)$, applied at every hidden neuron; the final affine readout uses the identity on the pair.

Activations are indexed by $l \in \{0,\dots,L\}$, affine parameters by $l \in \{1,\dots,L\}$. The layer-0 pair is any non-negative split of the input,

$$\big(a_k^{(0,+)}, a_k^{(0,-)}\big) := \big(x_k^+, x_k^-\big), \qquad x_k^+ - x_k^- = x_k, \qquad x_k^+, x_k^- \geq 0, \tag{3.2}$$

and each affine layer l maps the preceding activation pair to a non-negative pre-activation pair $\big(z_i^{(l,+)}, z_i^{(l,-)}\big)$. The pairwise activation applies for $l < L$; at $l = L$, $a^{(L,\pm)} = z^{(L,\pm)}$. We call $\big(a_i^{(l,+)}, a_i^{(l,-)}\big)$ the *decomposed activation* and $\big(z_i^{(l,+)}, z_i^{(l,-)}\big)$ the *decomposed pre-activation*.

Canonical non-negative linear map. We use the entrywise positive/negative split of the original weights and biases,

$$W^{(l,+)} := [W^{(l)}]^+, \qquad W^{(l,-)} := [W^{(l)}]^-, \qquad b^{(l,+)} := [b^{(l)}]^+, \qquad b^{(l,-)} := [b^{(l)}]^-, \tag{3.3}$$

where $[\cdot]^\pm$ are the entrywise positive and negative parts, so $W^{(l)} = W^{(l,+)} - W^{(l,-)}$ with $W^{(l,\pm)} \geq 0$. The resulting pre-activation pair at neuron i is

$$z_i^{(l,+)} := b_i^{(l,+)} + \sum_j W_{ij}^{(l,+)}\, a_j^{(l-1,+)} + \sum_j W_{ij}^{(l,-)}\, a_j^{(l-1,-)}, \tag{3.4}$$

$$z_i^{(l,-)} := b_i^{(l,-)} + \sum_j W_{ij}^{(l,+)}\, a_j^{(l-1,-)} + \sum_j W_{ij}^{(l,-)}\, a_j^{(l-1,+)}. \tag{3.5}$$

Positive weights preserve streams; negative weights swap them, turning excitatory contributions into inhibitory ones and vice versa. The following definition connects the structure of Definition 3.1 to the original network.

Definition 3.2 (Valid decomposition). A decomposition (Definition 3.1) is *valid* for the network (3.1) with activation function ϱ if the propagated activation pairs satisfy

$$a_i^{(l,+)} - a_i^{(l,-)} = \varrho_l\big(z_i^{(l)}\big) = a_i^{(l)} \tag{3.6}$$

for every layer $l \in \{0,\dots,L\}$ and every neuron i, where at $l = 0$ condition (3.6) is read as $a_k^{(0,+)} - a_k^{(0,-)} = x_k$.

Validity propagates through the canonical linear map, recovering the network output at the final layer.

Proposition 3.1 (Validity recovers the network, `Lean_21`). *Let a decomposition use the canonical non-negative linear map (3.4)–(3.5). If the validity condition (3.6) holds at layer $l-1$, then the decomposed pre-activations at layer l satisfy*

$$z_i^{(l,+)} - z_i^{(l,-)} = z_i^{(l)} \qquad \text{for every neuron } i. \tag{3.7}$$

Consequently, every valid decomposition recovers the original network output as the difference of the final decomposed activations,

$$f(x) = a^{(L,+)} - a^{(L,-)}. \tag{3.8}$$

Proof. Subtracting (3.5) from (3.4) gives $z^{(l,+)} - z^{(l,-)} = W^{(l)}(a^{(l-1,+)} - a^{(l-1,-)}) + b^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)} = z^{(l)}$. Validity at the ungated output then gives $a^{(L,+)} - a^{(L,-)} = z^{(L)} = f(x)$. □

A valid pairwise activation must recover $\varrho(z^+ - z^-)$ and preserve non-negativity on the input quadrant. We first construct convex, coordinatewise non-decreasing pairs from absolute curvature, obtaining ReLU and smooth-activation formulas (Section 3.2); mixing, stopping, and regularization retain the same invariant with different structure (Sections 3.3–3.5).

3.2 Convex and Monotone DC Decompositions

The required activation-level object is a pair of functions $F_+,F_-\colon \mathbb{R}^2 \to \mathbb{R}$ satisfying

$$\begin{gathered}\varrho(x-y) = F_+(x,y) - F_-(x,y),\\ F_+,F_- \text{ convex and coordinatewise non-decreasing.}\end{gathered} \tag{3.9}$$

Convexity and coordinatewise monotonicity together ensure convex composition. Neither implies non-negative values, which we require on the visited quadrant $\mathbb{R}^2_{\geq 0}$. Global convexity and monotonicity on $\mathbb{R}^2$ also allow signed stream inputs.

3.2.1 The profile criterion and network composition

Adding a common affine correction lifts a scalar DC decomposition $\varrho = p - q$ to such a pair. Here L is a slope bound, independent of network depth. The criterion also covers ReLU's kink.

Proposition 3.2 (Criterion for a convex, monotone pair, `Lean_22`). *Let $L \geq 0$ and $C\colon \mathbb{R} \to \mathbb{R}$, and define*

$$T_{C,L}(x,y) \coloneqq C(x-y) + Ly. \tag{3.10}$$

Then $T_{C,L}$ is convex on $\mathbb{R}^2$ if and only if C is convex. It is coordinatewise non-decreasing on $\mathbb{R}^2$ if and only if

$$0 \leq C(v) - C(u) \leq L(v-u) \qquad (u \leq v). \tag{3.11}$$

If C is differentiable, condition (3.11) is equivalent to $0 \leq C' \leq L$. Consequently, given $p - q = \varrho$, the pair

$$F_+(x,y) = p(x-y) + Ly, \qquad F_-(x,y) = q(x-y) + Ly \tag{3.12}$$

is convex and coordinatewise non-decreasing exactly when both profiles are convex and satisfy (3.11). Under these conditions it maps $\mathbb{R}^2_{\geq 0}$ to itself exactly when $p(0),q(0) \geq 0$.

Proof. Affine precomposition and affine addition preserve convexity; restriction to $y = 0$ proves the converse. Increasing x gives the lower increment bound; increasing y gives the upper bound. These conditions also suffice. Difference quotients and the mean value theorem give the derivative criterion. The common Ly cancels, and monotonicity makes the origin values the minima on the non-negative quadrant. □

Theorem 3.3 (Network DC decomposition from activation pairs, `Lean_23`). *Consider the network (3.1) with hidden activation ϱ. Choose a pair F_+,F_- satisfying (3.9) on all of $\mathbb{R}^2$ and $F_\pm(0,0) \geq 0$. Use this pair entrywise after the canonical affine split (3.4)–(3.5), and use the identity pair at the final affine readout. Then every decomposed activation, and each of the two output streams G,H, is convex and coordinatewise non-decreasing as a function of the independent doubled input coordinates (x^+,x^-). Moreover,*

$$G(x^+,x^-) - H(x^+,x^-) = f(x^+ - x^-), \tag{3.13}$$

and both streams are non-negative when $x^+,x^- \geq 0$. These conclusions also hold for every tail network starting at an intermediate layer.

If each component of an input split $x \mapsto (x^+(x),x^-(x))$ is convex, then $x \mapsto G(x^+(x),x^-(x))$ and $x \mapsto H(x^+(x),x^-(x))$ are convex. In particular, the usual split $x^\pm = [x]^\pm$ gives a DC representation on the original input space.

Proof. Non-negative affine combinations and convex coordinatewise non-decreasing activation components preserve convexity and monotonicity; they also preserve non-negativity on the quadrant by origin normalization. Induction from the doubled input coordinates proves these properties for every layer and tail. Applying the same composition rule to a convex input split proves convexity of its pullbacks (Amos et al., 2017).

Proposition 3.1 recovers each affine difference, and (3.9) recovers each hidden activation. The final identity pair preserves the ungated difference, proving (3.13), including negative logits. □

Monotonicity refers to independent pair coordinates. Under $x^\pm = [x]^\pm$, convexity survives, while monotonicity in x generally fails: $x^- = \max\{-x,0\}$ decreases with x. The theorem therefore gives monotone networks on doubled space and a DC representation on the original input space.

3.2.2 Construction from absolute second derivatives

Finite positive and negative curvature gives bounded profile slopes. We construct them from $|f''|$, with f now denoting a scalar activation.

Theorem 3.4 (Absolute-curvature construction, `Lean_24`). *Let $f\colon \mathbb{R} \to \mathbb{R}$ be differentiable, let f' be locally absolutely continuous, and assume $f'' \in L^1(\mathbb{R})$. Set*

$$\begin{aligned} s_- &:= \lim_{t\to-\infty} f'(t), \qquad V(t) := \int_{-\infty}^{t} |f''(u)|\, du, \\ D(z) &:= |f(0)| + |s_-|z + \int_0^z V(t)\, dt, \\ p &:= \frac{D+f}{2}, \qquad q := \frac{D-f}{2}. \end{aligned} \tag{3.14}$$

The limits s_- and $s_+ := \lim_{t\to+\infty} f'(t)$ exist. With

$$L := \frac{\|f''\|_{L^1(\mathbb{R})} + |s_-| + |s_+|}{2}, \tag{3.15}$$

the profiles p,q are convex, satisfy $p - q = f$ and $0 \leq p',q' \leq L$, and have $p(0) = \max\{f(0),0\}$, $q(0) = \max\{-f(0),0\}$. Therefore (3.12) is a convex, coordinatewise non-decreasing pair on $\mathbb{R}^2$, is non-negative on $\mathbb{R}^2_{\geq 0}$, and recovers $f(x-y)$ exactly.

Proof. Local absolute continuity gives $|f'(v) - f'(u)| \leq \int_u^v |f''(t)|\, \mathrm{d}t$ for $u \leq v$. Integrability makes f' Cauchy at both ends of the real line, so its finite limits exist. Write

$$P = \frac{|f''| + f''}{2}, \qquad N = \frac{|f''| - f''}{2}. \tag{3.16}$$

Differentiating (3.14) yields

$$p'(z) = \max\{s_-,0\} + \int_{-\infty}^{z} P(u)\,\mathrm{d}u, \qquad q'(z) = \max\{-s_-,0\} + \int_{-\infty}^{z} N(u)\,\mathrm{d}u. \tag{3.17}$$

Both derivatives are non-negative, non-decreasing, and bounded, and $p'' = P$, $q'' = N$ almost everywhere. Their upper limits are $A = \max\{s_-,0\} + \int_{\mathbb{R}} P$ and $B = \max\{-s_-,0\} + \int_{\mathbb{R}} N$. Since $A + B = |s_-| + \|f''\|_1$ and $A - B = s_+$, their maximum is precisely (3.15). The difference and origin values follow directly from the definition. Proposition 3.2 now proves all pair properties. □

The curvature split is pointwise minimal (Proposition 3.13). Moreover, any globally convex non-decreasing profiles with difference f have limiting slopes at $-\infty$ at least $\max\{s_-,0\}$ and $\max\{-s_-,0\}$, since those slopes are non-negative and differ by s_-. Their curvature measures dominate $P\,\mathrm{d}z$,$N\,\mathrm{d}z$, so their upper slopes are at least A,B. Thus (3.15) is the least common slope bound within (3.12). This claim concerns the profile-and-affine lift, not arbitrary bivariate pairs or network stream sizes.

Remark 3.1 (Why the global domain matters). Under the stated regularity of f', integrable f'' is necessary for finite-valued convex, coordinatewise non-decreasing pairs on all of $\mathbb{R}^2$ recovering $f(x-y)$. Restrict each component to $(t,0)$ for $t \le 0$ and $(0,-t)$ for $t \ge 0$. These convex restrictions are respectively non-decreasing and non-increasing, with slopes bounded by 0 and finite one-sided slopes at the origin. Both therefore have finite curvature mass. Their differences equal $f(t)$, so the sum of their positive curvature measures bounds the total variation of its curvature measure, giving $\int_{\mathbb{R}} |f''| < \infty$. The quadrant alone permits non-integrable curvature: $(x-y)^2 = (2x^2+2y^2) - (x+y)^2$, whose two right-hand terms are convex and coordinatewise non-decreasing there.

Corollary 3.5 (Exact splitting on a bounded activation interval). *Let $a < b$ and let $f \in C^1([a,b])$ have absolutely continuous derivative. There exist finite-valued convex, coordinatewise non-decreasing functions $F_+,F_-\colon \mathbb{R}^2 \to \mathbb{R}$, non-negative on $\mathbb{R}^2_{\ge 0}$, such that $F_+(x,y) - F_-(x,y) = f(x-y)$ whenever $x - y \in [a,b]$.*

Proof. Extend f by its endpoint tangents. The extension is C^1, has locally absolutely continuous derivative, and curvature supported in $[a,b]$. Apply Theorem 3.4. □

This local statement applies, for example, to exponentiation on a fixed bounded score interval. It does not assert a global monotone convex split for e^z on all of $\mathbb{R}$, whose curvature is not integrable.

3.2.3 ReLU: the maximum and identity pair

For $f(z) = \mathrm{ReLU}(z)$ choose $p(z) = \mathrm{ReLU}(z)$, $q(z) = 0$, and $L = 1$. Both profiles are convex and satisfy (3.11), even though p is not differentiable at the origin. The profile lift gives the exact pair

$$\begin{aligned} F_+(a,b) = \mathrm{ReLU}(a-b) + b = \max\{a,b\}, \qquad F_-(a,b) = b, \\ \mathrm{ReLU}(a-b) = \max\{a,b\} - b. \end{aligned} \tag{3.18}$$

This specializes the general criterion to a positive Maxout component (Goodfellow et al., 2013) and a negative identity component. Replacing $f''\,\mathrm{d}z$ by δ_0 in the absolute-curvature construction gives the same pair (Remark 3.5). This classical ReLU-network DC split (Awasthi et al., 2026; Siahkamari

et al., 2020; Zhang et al., 2018b; Zimmermann and Loho, 2026) underlies our stabilized passes, CNN extension, and saliency methods.

The hidden-layer update is therefore

$$a^{(l,+)} = \max\{z^{(l,+)}, z^{(l,-)}\}, \qquad a^{(l,-)} = z^{(l,-)}, \qquad 1 \le l < L, \tag{3.19}$$

with $a^{(L,\pm)} = z^{(L,\pm)}$ at the final affine readout. Theorem 3.3 gives validity and convexity. Section 3.3 identifies this as the mixing family's $\eta = 1$ endpoint.

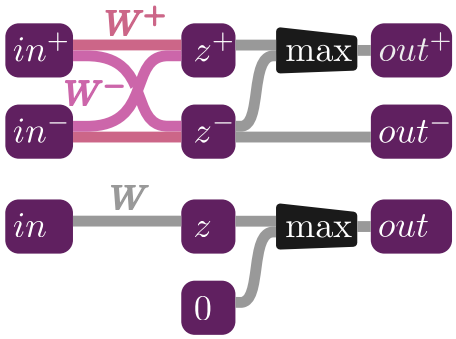


Fig. 3.2: Monotone, convex split streams (top) and a standard ReLU layer (bottom). Negative weights swap positive and negative input streams into opposite pre-activation streams; positive weights preserve them.

The stream functions g and h. The DC *split pair* consists of two monotone *split streams*, non-negative on the input quadrant (Figure 3.2). Like the tail network $f^{(l)}\colon \mathbb{R}^{d^{(l)}} \to \mathbb{R}$ on layers $l+1, \dots, L$, define the restricted streams

$$g^{(l)}\colon \mathbb{R}^{d^{(l)}} \times \mathbb{R}^{d^{(l)}} \to \mathbb{R}, \qquad h^{(l)}\colon \mathbb{R}^{d^{(l)}} \times \mathbb{R}^{d^{(l)}} \to \mathbb{R}, \tag{3.20}$$

mapping an activation pair at layer l to the $+$ and $-$ components of the decomposed output; we abbreviate $g := g^{(0)}$, $h := h^{(0)}$. On the output layer $g^{(L)}(x^+, x^-) = x^+$, $h^{(L)}(x^+, x^-) = x^-$, and for $0 \le l < L$,

$$g^{(l)}(x^+, x^-) = g^{(l+1)}\big(a^{(l+1,+)}, a^{(l+1,-)}\big), \qquad h^{(l)}(x^+, x^-) = h^{(l+1)}\big(a^{(l+1,+)}, a^{(l+1,-)}\big), \tag{3.21}$$

where (3.4)–(3.5) and (3.19) compute $\big(a^{(l+1,+)}, a^{(l+1,-)}\big)$ from (x^+, x^-). These full tail functions, rather than individual layer activations, define $f = g - h$ and underpin Chapter 6.

Theorem 3.6 (Structure and correctness of the split streams, `Lean_25`, `Lean_26`). *For every $0 \le l \le L$, the streams $g^{(l)}, h^{(l)}$ of (3.20) are convex and coordinatewise non-decreasing on $\mathbb{R}^{d^{(l)}} \times \mathbb{R}^{d^{(l)}}$. For all $x^+, x^- \in \mathbb{R}^{d^{(l)}}_{\ge 0}$,*

$$f^{(l)}\big(x^+ - x^-\big) = \big(g^{(l)} - h^{(l)}\big)(x^+, x^-). \tag{3.22}$$

Proof. Apply Theorem 3.3 to each ReLU tail, using (3.18) and the identity output pair, including the identity tail at $l = L$. The convex-monotone composition principle is recalled in Section 2.2 (Amos et al., 2017). □

Remark 3.2 (Preservation of sub-threshold signals). At an inactive neuron, $z_i^{(l)} \le 0$, ReLU outputs $a_i^{(l)} = 0$, discarding the losing pre-activation's magnitude. Downstream computation cannot distinguish degrees of suppression. The split pair $\big(a_i^{(l,+)},a_i^{(l,-)}\big) = \big(z_i^{(l,-)},z_i^{(l,-)}\big)$ has the required zero difference while retaining the cancelled mass $z_i^{(l,-)}$ in both components. These sub-threshold contributions give the streams more information than the original activations, which Chapter 6 exploits. Retaining them also causes the numerical growth of Section 3.6: without subtractive cancellation, the streams accumulate absolute mass $z_i^{(l,+)} + z_i^{(l,-)}$ instead of the signed residual.

Remark 3.3 (Flexibility of the input split). Theorem 3.6 allows *any* non-negative representation $x = x^+ - x^-$, including $x^\pm = [x]^\pm$ and $x^\pm = [x]^\pm + c$ with $c \ge 0$. Numerical stabilization and Chapter 6's attribution baselines exploit this freedom. The half–half split $x^\pm = \pm x/2$ (Section 3.6) instead has signed components: it serves only as a backward chain-rule device along the unified input direction.

3.2.4 Closed formulas for smooth activations

We give both minimal-curvature profiles and shorter smooth majorants. Write $\sigma(z) = (1+e^{-z})^{-1}$, let φ,Φ be the standard Gaussian density and distribution function, and set $s_\beta(z) = \beta^{-1}\log(1+e^{\beta z})$ for $\beta > 0$.

Corollary 3.7 (Explicit convex, monotone activation pairs, `Lean_27`). *For $f(z) = z\Phi(z)$ (GELU), $f(z) = z\sigma(\beta z)$ (Swish), $f = \sigma$ (sigmoid), and $f = s_{1/\theta}$ (softplus), choose the corresponding row of Table 3.1. Then the functions $F_+(x,y) = p(x-y) + Ly$ and $F_-(x,y) = q(x-y) + Ly$ are convex and coordinatewise non-decreasing on $\mathbb{R}^2$, map $\mathbb{R}^2_{\ge 0}$ to itself, and satisfy $F_+ - F_- = f(x-y)$.*

Activation	$p(z)$	$q(z)$	L
GELU	$2z\Phi(z) + 2\varphi(z)$	$z\Phi(z) + 2\varphi(z)$	2
Swish	$2s_\beta(z)$	$2s_\beta(z) - z\sigma(\beta z)$	2
Sigmoid	$\frac{1}{2}(s_1(z) + \sigma(z))$	$\frac{1}{2}(s_1(z) - \sigma(z))$	$\frac{1}{2}$
Softplus	$s_{1/\theta}(z)$	0	1

Tab. 3.1: Closed profiles for (3.12), $\beta,\theta > 0$. GELU, Swish, and sigmoid trade additional curvature for shorter smooth formulas. GELU denotes the exact Gaussian activation, excluding its tanh approximation; SiLU is Swish with $\beta = 1$.

Proof. In every row $p - q = f$. For GELU (Hendrycks and Gimpel, 2016), direct differentiation gives

$$p' = 2\Phi, \quad p'' = 2\varphi, \qquad q' = \Phi - z\varphi, \quad q'' = z^2\varphi.$$

Hence both profiles are convex; p' increases from 0 to 2 and q' from 0 to 1. For Swish (Ramachandran et al., 2018), writing $u = \beta z$ gives

$$p' = 2\sigma(u), \quad p'' = 2\beta\sigma'(u), \qquad q' = \sigma(u) - u\sigma'(u), \quad q'' = \beta u\big(2\sigma(u) - 1\big)\sigma'(u) \ge 0.$$

The last inequality uses that u and $2\sigma(u) - 1$ have the same sign. The slopes again have limits 0,2 for p' and 0,1 for q', so they lie in [0,2]. For the sigmoid profiles,

$$p' = \tfrac{1}{2}\sigma(2-\sigma), \quad q' = \tfrac{1}{2}\sigma^2, \qquad p'' = \sigma'(1-\sigma) \ge 0, \quad q'' = \sigma'\sigma \ge 0,$$

with slopes in [0,1/2]. Softplus has $p' = \sigma(z/\theta) \in [0,1]$ and $p'' = \sigma'(z/\theta)/\theta \geq 0$, while $q = 0$. All profile values at zero are non-negative (for sigmoid, $s_1(0) = \log 2 > 1/2$). Proposition 3.2 proves the claim. □

The lifted pairs require no quadrature or root solve. With $d = x - y$, their values at (x,y) are

$$\text{Softplus:} \quad (F_+,F_-) = \big(\theta \log(e^{x/\theta} + e^{y/\theta}), y\big), \tag{3.23}$$

$$\text{Sigmoid:} \quad F_\pm = \frac{\log(e^x + e^y) \pm \sigma(d)}{2}, \tag{3.24}$$

$$\text{GELU:} \quad (F_+,F_-) = \big(2(d\Phi(d) + \varphi(d) + y), d\Phi(d) + 2\varphi(d) + 2y\big), \tag{3.25}$$

$$\text{Swish:} \quad (F_+,F_-) = \left(\frac{2}{\beta} \log(e^{\beta x} + e^{\beta y}), F_+ - d\sigma(\beta d)\right). \tag{3.26}$$

The softplus pair tends to $(\max\{x,y\},y)$ as $\theta \downarrow 0$. Stable implementations of the exponential formulas should evaluate log-sum-exp through its usual shifted form.

Closed forms for the canonical absolute-curvature split. The smaller coefficients in Theorem 3.4 can also be attained explicitly. For sigmoid,

$$p(z) = \begin{cases} \sigma(z), & z \leq 0, \\ \frac{1}{2} + \frac{1}{4}z, & z \geq 0, \end{cases} \qquad q(z) = p(z) - \sigma(z), \qquad L = \tfrac{1}{4}. \tag{3.27}$$

For both GELU and Swish, let $a > 0$ be the positive zero of f'', and define $m = f'(a) - 1$, $c = af'(a) - f(a)$ and $L = 1 + 2m$. Then

$$p(z) = \begin{cases} -c, & z \leq -a, \\ f(z) + mz, & -a \leq z \leq a, \\ Lz - c, & z \geq a, \end{cases} \qquad q(z) = p(z) - f(z). \tag{3.28}$$

The constants are

$$\begin{aligned} &\text{GELU:} \quad a = \sqrt{2}, \quad m = \Phi(\sqrt{2}) + \sqrt{2}\varphi(\sqrt{2}) - 1, \quad c = 2\varphi(\sqrt{2}), \\ &\text{Swish:} \quad a = r/\beta, \quad m = (r-2)/4, \quad c = (r^2-4)/(4\beta), \quad L = r/2, \end{aligned} \tag{3.29}$$

where r uniquely solves $r \tanh(r/2) = 2$ on $r > 0$ ($r \approx 2.39936$): the left-hand side increases strictly from 0 to infinity. Fixing r gives a pointwise Swish formula without quadrature. Softplus already uses the canonical profiles in Table 3.1.

For (3.28), both activations satisfy $f(-z) = f(z) - z$, $f'(-\infty) = 0$, $f'(+\infty) = 1$, and have positive curvature precisely on $(-a,a)$. Piecewise differentiation gives $p'' = \max\{f'',0\}$, $q'' = \max\{-f'',0\}$, with matching values and first derivatives at $\pm a$. Since $p(0) = q(0) = 0$, these are Theorem 3.4's normalized profiles. Sigmoid's curvature changes sign at zero, giving (3.27) analogously. Canonical GELU and Swish profiles can be negative on the negative half-line; origin normalization and coordinatewise monotonicity ensure their lifted pairs are non-negative on the required quadrant.

Corollary 3.8 (Modern activation functions, `Lean_28`). *Theorems 3.4 and 3.12 apply to:*

1) GELU, $f(x) = x\,\Phi(x)$, *with* $f''(x) = (2 - x^2)\,\varphi(x)$; *the curvature is positive on* $(-\sqrt{2}, \sqrt{2})$ *and negative outside.*

2) Swish/SiLU, $f(x) = x\,\sigma(\beta x)$ *with* $\beta > 0$, *with* $f''(x) = \beta\,\sigma'(\beta x)\big(2 + \beta x\,(1 - 2\sigma(\beta x))\big)$; *the curvature is positive on a bounded interval around the origin and negative in both tails.*
3) Mish, $f(x) = x\tanh\big(\log(1 + e^x)\big)$.
4) Hyperbolic tangent, $f(x) = \tanh(x)$, *with* $f''(x) = -2\tanh(x)\operatorname{sech}^2(x)$, *so* P *is supported on* $(-\infty, 0]$ *and* N *on* $[0, \infty)$.
5) Logistic sigmoid, $f = \sigma$, *with* $f'' = \sigma'\,(1 - 2\sigma)$.
6) Softplus at temperature $\theta > 0$, $f(x) = \theta\log(1 + e^{x/\theta})$, *with* $f''(x) = \sigma'(x/\theta)/\theta \geq 0$; *here* $N = 0$, $C_2 = 0$, *and the decomposition degenerates to the statement that softplus itself is convex.*

In each case $f(0) \geq 0$, *and both constructions preserve non-negativity on the input quadrant (Remark 3.6).*

Proof. Direct differentiation gives the stated second derivatives. Each f'' is continuous and decays at least exponentially as $|x| \to \infty$ (Gaussian for GELU). For Swish and Mish, exponential decay of $\sigma'(\beta x)$ and the derivatives of $\tanh \circ \operatorname{softplus}$ dominates the polynomial factor x. Thus $f'' \in L^1(\mathbb{R})$ and f' is locally absolutely continuous. Both constructions apply, with non-negative $f(0) \in \{0, \frac{1}{2}, \theta\log 2\}$. □

Corollary 3.9 (Networks with smooth activations, `Lean_29`). *Every finite feedforward network with GELU, Swish/SiLU, sigmoid, or softplus hidden activations, arbitrary signed weights and biases, and an affine output admits an exact decomposition into two convex, coordinatewise non-decreasing streams on doubled input space. Either the explicit profiles of Corollary 3.7 or the canonical profiles (3.27)–(3.28) and (3.23) may be used at every hidden neuron. With the usual non-negative input split, the streams remain non-negative and their pullbacks are convex functions of the original signed input. The same existence result holds for Mish and tanh by Theorem 3.4.*

Proof. Apply Theorem 3.3 with the activation pairs from Corollaries 3.7 and 3.8, and the identity output pair. □

The induction also permits different qualifying activations or profiles at each hidden layer. The theorem states the uniform-activation case used by the formal network interface.

Remark 3.4 (Scope of the smooth extension). These exact activation and feedforward results establish structure. Empirical attribution quality and extensions of the ReLU hard-gate games and stabilization formulas require separate analysis. The new smooth formulas add no experimental results beyond the later stated evaluation protocols.

3.2.5 Multivariate activations and softmax

For activations coupling coordinates, convexity means componentwise convexity of each vector-valued pair. This differentiable lift suffices for softmax.

Proposition 3.10 (Multivariate profile lift, `Lean_30`). *Let* $C \in C^1(\mathbb{R}^n)$ *and* $\ell \in \mathbb{R}^n_{\geq 0}$. *Define*

$$T_{C,\ell}(x,y) = C(x - y) + \langle \ell, y\rangle, \qquad x,y \in \mathbb{R}^n. \tag{3.30}$$

Then $T_{C,\ell}$ *is convex on* $\mathbb{R}^{2n}$ *if and only if* C *is convex, and is coordinatewise non-decreasing if and only if* $0 \leq \partial_j C(z) \leq \ell_j$ *for all* z *and* j. *Consequently, if* $C_+ - C_- = f$, *both profiles are convex, and both satisfy these derivative bounds, the common lift gives a convex, coordinatewise non-decreasing pair with difference* $f(x - y)$. *If* $C_+(0), C_-(0) \geq 0$, *the lifted pair is non-negative on* $\mathbb{R}^{2n}_{\geq 0}$.

Proof. Use the scalar proof of Proposition 3.2, replacing its coordinate derivatives by $\partial_{x_j} T = \partial_j C$ and $\partial_{y_j} T = \ell_j - \partial_j C$. Restriction to $y = 0$, cancellation, and the origin-minimum argument are unchanged. □

Softmax admits a short pair because the log-sum-exp Hessian dominates its curvature. This uses multivariate curvature comparison, not the scalar integrability theorem.

Theorem 3.11 (Explicit convex, monotone softmax pair, `Lean_31`). *Let $n \geq 1$, and define*

$$H(z) = \log\left(\sum_{j=1}^{n} e^{z_j}\right), \qquad \pi_i(z) = \frac{e^{z_i}}{\sum_{j=1}^{n} e^{z_j}}, \qquad c_n = \frac{\log n - 1/n}{2}.$$

For each output coordinate i, set

$$F_{i,\pm}(x,y) = \frac{H(x-y) \pm \pi_i(x-y) + \sum_{j=1}^{n} y_j}{2} - c_n. \tag{3.31}$$

Both components are finite, convex, and coordinatewise non-decreasing on all of $\mathbb{R}^{2n}$. They are non-negative on $\mathbb{R}^{2n}_{\geq 0}$ and satisfy $F_{i,+}(x,y) - F_{i,-}(x,y) = \pi_i(x-y)$.

Proof. Fix z and a direction $v \in \mathbb{R}^n$, and abbreviate $\mu = \sum_j \pi_j(z) v_j$ and $V = \sum_j \pi_j(z)(v_j - \mu)^2$. Differentiation along $z + tv$ gives

$$\begin{aligned} D^2 H(z)[v,v] &= V, \\ D^2 \pi_i(z)[v,v] &= \sum_j \pi_j(z)\big(\mathbf{1}[j=i] - \pi_i(z)\big)(v_j - \mu)^2. \end{aligned} \tag{3.32}$$

Since $|\mathbf{1}[j=i] - \pi_i| \leq 1$, it follows that $|D^2 \pi_i(z)[v,v]| \leq V$. Thus $H + \pi_i$ and $H - \pi_i$ have positive semidefinite Hessians and are convex. For the profiles $C_{i,\pm} = (H \pm \pi_i)/2 - c_n$, their partial derivatives are

	$\partial_j C_{i,+}$	$\partial_j C_{i,-}$
$j = i$	$\pi_i - \pi_i^2/2$	$\pi_i^2/2$
$j \neq i$	$\pi_j(1-\pi_i)/2$	$\pi_j(1+\pi_i)/2.$

(3.33)

Every entry lies in $[0,1/2]$; for the last entry, use $\pi_j \leq 1 - \pi_i$. Proposition 3.10 with $\ell_j = 1/2$ gives global convexity and monotonicity. At the origin, $F_{i,+}(0,0) = 1/n$ and $F_{i,-}(0,0) = 0$, proving non-negativity on the orthant. Subtraction gives exact recovery. □

For fixed masked softmax, apply the theorem to each row's nonempty allowed set and assign masked outputs $(0,0)$. Fully masked rows require a separate convention and lie outside this theorem. Fixed positive temperatures and fixed additive score biases enter the preceding affine map. Section 5.3 combines this pair with convex product and normalization pairs to split the full Transformer.

3.2.6 The symmetric-tilt variant and curvature minimality

The symmetric tilt $M(a+b)$ yields different streams with the same difference. Its fixed-profile minimality complements the asymmetric lift, whose normalization recovers the literal ReLU pair $(\max\{a,b\},b)$.

Theorem 3.12 (DC–monotone splitting under integrable curvature, Lean_32). *Let $f\colon \mathbb{R} \to \mathbb{R}$ be differentiable with locally absolutely continuous derivative f' and $f'' \in L^1(\mathbb{R})$. Then the limit $s_- := \lim_{u\to-\infty} f'(u)$ exists and f' is bounded. Write $P := \max\{f'',0\}$ and $N := \max\{-f'',0\}$, so that $f'' = P - N$ with $P, N \geq 0$, and define*

$$C_1(x) := f(0) + \int_0^x \Big(s_- + \int_{-\infty}^t P(u)\ du\Big)\ dt, \qquad C_2(x) := \int_0^x \int_{-\infty}^t N(u)\ du\ dt, \tag{3.34}$$

as well as $M := \max\{\sup_{x\in\mathbb{R}}|C_1'(x)|, \sup_{x\in\mathbb{R}}|C_2'(x)|\} < \infty$. Then the functions

$$g(a,b) := C_1(a-b) + M(a+b), \qquad h(a,b) := C_2(a-b) + M(a+b) \tag{3.35}$$

are convex on $\mathbb{R}^2$, non-decreasing in each argument, and satisfy

$$f(a-b) = g(a,b) - h(a,b) \qquad \text{for all } (a,b) \in \mathbb{R}^2. \tag{3.36}$$

Proof. Theorem 3.4 establishes s_-, bounded f', and $P,N \in L^1(\mathbb{R})$. Differentiating (3.34) gives locally absolutely continuous, bounded, non-decreasing derivatives

$$C_1'(x) = s_- + \int_{-\infty}^x P(u)\ \mathrm{d}u, \qquad C_2'(x) = \int_{-\infty}^x N(u)\ \mathrm{d}u.$$

Thus C_1,C_2 are convex and $M < \infty$. Since $C_1' - C_2' = f'$ and $C_1(0) - C_2(0) = f(0)$, integration gives $C_1 - C_2 = f$. Affine precomposition and the common tilt preserve convexity and the difference; their coordinate derivatives $M \pm C_j'(a-b)$ are non-negative by definition of M. If f'' is continuous, the Hessian of $C_1(a-b)$ equivalently has eigenvalues 0 and $2P(a-b) \geq 0$. □

The following minimality concerns the fixed profiles (3.34) and symmetric tilts, excluding comparisons with asymmetric corrections or other normalizations.

Proposition 3.13 (Minimality of the curvature split, Lean_33). *Let f be as in Theorem 3.12 and suppose $f = c_1 - c_2$ for convex functions $c_1, c_2\colon \mathbb{R} \to \mathbb{R}$ whose second distributional derivatives are locally integrable functions. Then $c_1'' \geq P$ and $c_2'' \geq N$ almost everywhere. In particular, (C_1, C_2) is, up to a common affine summand, the unique pair of convex functions with $C_1 - C_2 = f$ of pointwise minimal curvature, and among all tilts of the form $(a,b) \mapsto \lambda(a+b)$, $\lambda \geq 0$, the constant M in Theorem 3.12 is the smallest one rendering both components of (3.35) non-decreasing in each argument.*

Proof. Convexity gives $c_1'' = f'' + c_2'' \geq f''$ and $c_1'' \geq 0$, hence $c_1'' \geq \max\{f'',0\} = P$ almost everywhere; symmetrically, $c_2'' \geq N$. Equality a.e. in both bounds makes $c_1 - C_1$ and $c_2 - C_2$ affine. They coincide because $c_1 - c_2 = f = C_1 - C_2$. Monotonicity in both arguments requires $\lambda \geq \pm C_1'(x)$ for all x, equivalently $\lambda \geq \sup_x|C_1'(x)|$, and likewise for C_2'. The least admissible λ is M. □

Remark 3.5 (Bounded variation and the piecewise-linear case). The construction extends to locally absolutely continuous f whose weak derivative has a bounded-variation representative, with finite signed Radon curvature measure μ. Substitute $|\mu|$ for $|f''(u)|\,\mathrm{d}u$ in Theorem 3.4, or Jordan parts μ_+,μ_- for $P(u)\,\mathrm{d}u, N(u)\,\mathrm{d}u$. The resulting profiles are convex with bounded slopes; criterion (3.11) handles kinks. ReLU has $\mu = \delta_0$, $s_- = 0$, giving $p(z) = \max\{z,0\}$, $q = 0$, $L = 1$, hence exactly

$(\max\{a,b\},b)$ under the asymmetric lift. The symmetric variant gives $(\mathrm{ReLU}(a-b)+a+b, a+b)$. Leaky ReLU with slope α has curvature $(1-\alpha)\delta_0$; standard ELU with unit negative-branch scale has curvature $e^z \mathbb{1}_{z<0}\,\mathrm{d}z$. Both fit this finite-measure construction, unifying piecewise-linear and smooth activations through bounded variation.

Remark 3.6 (Compatibility with valid decompositions). Both constructions fit Definition 3.1. The asymmetric pair has non-negative origin values $F_+(0,0) = \max\{f(0),0\}$, $F_-(0,0) = \max\{-f(0),0\}$; monotonicity gives non-negativity on $\mathbb{R}^2_{\geq 0}$. The symmetric pair (3.35) has $g(0,0) = f(0)$, $h(0,0) = 0$, so this holds whenever $f(0) \geq 0$, including Corollary 3.8. If $f(0) < 0$, adding $|f(0)|$ to both components restores non-negativity and preserves their difference. Exactness gives activation-level validity; Theorem 3.3 gives network recovery, convexity, and monotonicity.

3.3 The Mixing Decomposition family

ReLU networks compute CPWL functions (Arora et al., 2018) admitting DC decompositions (Kripfganz and Schulze, 1987; Melzer, 1986; Zimmermann and Loho, 2026). The ReLU pair extends to a one-parameter interpolation between two arithmetic extremes.

Definition 3.3 (Mixing decomposition function, `Lean_34`). For $\eta \in [0,1]$, the *mixing decomposition function* is

$$f^{\eta}_{\mathrm{mix}}(z^+, z^-) := \big(\eta \max\{z^+, z^-\} + (1-\eta)\, z^+,\ \ \eta\, z^- + (1-\eta)\min\{z^+, z^-\}\big). \tag{3.37}$$

The *Mixing Decomposition* uses f^{η}_{mix} at every hidden neuron, the identity pair at the affine output, the canonical map (3.4)–(3.5), and input convention (3.2). We tag its activations $a^{(l,\pm,\eta)}_{\mathrm{mix}}$ when comparing decompositions.

Convex combinations and maxima/minima preserve non-negativity on $\mathbb{R}^2_{\geq 0}$, so f^{η}_{mix} is a pairwise activation (Definition 3.1). It linearly interpolates between $\big(\max\{z^+, z^-\}, z^-\big)$ at $\eta = 1$ and $\big(z^+, \min\{z^+, z^-\}\big)$ at $\eta = 0$, giving a valid ReLU decomposition for every parameter.

Theorem 3.14 (Layer-wise recovery of the Mixing Decomposition, `Lean_35`). *Let the activation pairs be propagated by the canonical non-negative linear map (3.4)–(3.5), initialised by (3.2), and updated by f^{η}_{mix} for some $\eta \in [0,1]$. Then for every layer l and neuron i,*

$$a^{(l,+,\eta)}_{\mathrm{mix},i} - a^{(l,-,\eta)}_{\mathrm{mix},i} = \varrho_l\big(z^{(l)}_i\big) = a^{(l)}_i, \tag{3.38}$$

i.e. the Mixing Decomposition is a valid decomposition (Definition 3.2) of the ReLU network for every $\eta \in [0,1]$.

Proof. For any u,v, both $\max\{u,v\} - v$ and $u - \min\{u,v\}$ equal $\mathrm{ReLU}(u-v)$, so their convex combination in (3.37) does too. Starting from (3.2), Proposition 3.1 propagates this identity inductively through the hidden layers and the final identity pair. □

The maximum in the $+$ stream at $\eta = 1$ and minimum in the $-$ stream at $\eta = 0$ recover the same difference. The former gives convex activations throughout the network.

Theorem 3.15 ($\eta = 1$: the convex DC endpoint, Lean_36, Lean_37). *Setting $\eta = 1$ in Definition 3.3 gives the valid DC split, with $a^{(l,\pm)} := a^{(l,\pm,1)}_{\text{mix}}$: canonical affine maps (3.4)–(3.5), input (3.2), hidden update (3.19), and final identity $a^{(L,\pm)} = z^{(L,\pm)}$. If every input component $x^{\pm}_k$ is convex in x, as for $x^{\pm} = [x]^{\pm}$ or any admissible affine split, every activation is convex, including the final non-negative affine combinations. Thus $a^{(l)}_i = a^{(l,+,1)}_{\text{mix},i} - a^{(l,-,1)}_{\text{mix},i}$ is a DC representation of the network's CPWL computation (Arora et al., 2018; Zimmermann and Loho, 2026).*

Proof. At $\eta = 1$, (3.37) is the maximum/identity pair (3.18). Proposition 3.2 and Theorem 3.3 give validity and convexity, including the convex input pullback and affine readout. □

Remark 3.7 (On the $\eta = 0$ extreme). The dual DC representation writes the output as $u(x) - v(x)$ with v *concave*, and f^0_{mix} places $a^{(l,-,0)}_{\text{mix},i} = \min\{z^{(l,+)}_i, z^{(l,-)}_i\}$ in the role of v. This is concave only when both decomposed pre-activations are concave (or affine): the minimum of two concave functions is concave, but of two convex functions is in general neither. Under the input convention (3.2), the first pre-activations are convex in x, but later $\eta = 0$ streams and pre-activations need not remain convex or concave. A canonical scalar example is $W^{(1)} = 1$, $b^{(1)} = 1$, giving the negative stream $v(x) = \min\{\max(x,0)+1, \max(-x,0)\}$. It satisfies $v(-2) = v(-1) = 1$, $v(0) = v(1) = 0$: the midpoint tests at -1 and 0 respectively exclude convexity and concavity. By Theorem 3.14, f^0_{mix} still yields a valid arithmetic decomposition, and the network-level concave representation is recoverable with an unsplit affine input convention, at the cost of the $\pm$ structure.

3.4 The Stopping Decomposition

The mixing family uses per-stream maxima and minima. The Stopping Decomposition applies the original network's hard gate as one *shared* decision to both streams.

Definition 3.4 (Stopping decomposition function, Lean_38). The *stopping decomposition function* is

$$f_{\text{stop}}(z^+, z^-) := \big(\mathbf{1}[z^+ - z^- > 0]\, z^+,\ \ \mathbf{1}[z^+ - z^- > 0]\, z^-\big). \tag{3.39}$$

The *Stopping Decomposition* uses f_{stop} at every hidden neuron, the identity pair at the affine output, the canonical map (3.4)–(3.5), and input convention (3.2). We tag its activations $a^{(l,\pm)}_{\text{stop}}$ when comparing decompositions.

Multiplying non-negative inputs by indicators makes f_{stop} a pairwise activation $\mathbb{R}^2_{\geq 0} \to \mathbb{R}^2_{\geq 0}$ (Definition 3.1). Its gate becomes a stop/continue decision in Chapter 4.

Theorem 3.16 (Validity of the Stopping Decomposition, Lean_39). *Let the activation pairs be propagated by the canonical non-negative linear map (3.4)–(3.5), initialised by (3.2), and updated by f_{stop}. Then for every layer l and neuron i,*

$$a^{(l,+)}_{\text{stop},i} - a^{(l,-)}_{\text{stop},i} = \varrho_l\big(z^{(l)}_i\big) = a^{(l)}_i, \tag{3.40}$$

i.e. the Stopping Decomposition is a valid decomposition (Definition 3.2) of the ReLU network.

Proof. The shared gate in (3.39) gives the local difference $\mathbf{1}[z^+ - z^- > 0](z^+ - z^-) = \text{ReLU}(z^+ - z^-)$. Proposition 3.1 propagates this identity from (3.2) through every hidden layer and the final affine readout. □

Remark 3.8 (Shared gate versus per-stream maxima). At an inactive neuron, $z_i^{(l)} \le 0$, the DC split of Theorem 3.15 outputs $\big(z_i^{(l,-)}, z_i^{(l,-)}\big)$, preserving sub-threshold magnitude (Remark 3.2) and propagating cancelled mass. The Stopping Decomposition outputs $(0,0)$, annihilating both streams exactly where the original ReLU vanishes. Thus the convex mixing streams ($\eta = 1$) retain dead-neuron contributions in downstream pairs, while stopping mass follows only active neurons. The discontinuous stopping gate sacrifices global convexity for exact forward-support agreement and retains regularity on each linear region.

Lemma 3.17 (Monotonicity of the Stopping Decomposition on linear regions, `Lean_40`). *On each linear region of the ReLU network—a maximal subset of input space on which all gate indicators $\mathbf{1}\big[z_j^{(l)} > 0\big]$ are constant—each stopping decomposed activation $a_{\mathrm{stop},i}^{(l,\pm)}$ is a non-negative-linear (i.e. affine with non-negative coefficients) function of the input pair $(x^+, x^-) \in \mathbb{R}^{2d^{(0)}}_{\ge 0}$, hence monotone non-decreasing in each coordinate x_k^+ and x_k^-.*

Proof. The input pair consists of coordinate functions. On a fixed linear region, every hidden gate is a constant 0 or 1; composing these gates with the canonical affine maps (3.4)–(3.5), whose weights and biases are non-negative, preserves non-negative affine coefficients through the final ungated readout. □

Excitatory and inhibitory path isolation. The shared gate never mixes streams, allowing exact accounting of *which input mass reaches which stream.* Each decomposed pre-activation sums backward paths; the parity of their negative weights determines the excitatory (+) or inhibitory (−) destination.

Lemma 3.18 (Parity path view on the Stopping Decomposition, `Lean_41`, `Lean_56`). *Fix a ReLU network with weights $W^{(l)}$ and, for clarity, vanishing biases. For a neuron i in layer $l \ge 1$, a* backward path *to the input is a list $P = (P_l, \dots, P_0)$ with $P_l = i$ and P_t a neuron in layer t, using weights $W^{(t)}_{P_t,P_{t-1}}$. Define the absolute weight product, parity sign, and gate factor*

$$
\begin{aligned}
|w|(P) &:= \prod_{t=1}^{l} \big|W^{(t)}_{P_t,P_{t-1}}\big|, \\
\operatorname{sgn}(P) = + \;&\Longleftrightarrow\; \#\big\{t : W^{(t)}_{P_t,P_{t-1}} < 0\big\} \text{ is even}, \\
G_{<l}(P) &:= \prod_{t=1}^{l-1} \mathbf{1}\big[z^{(t)}_{P_t} > 0\big].
\end{aligned}
$$

Then the decomposed pre-activations of the Stopping Decomposition admit the path-sum representation

$$
z_i^{(l,+)} = \sum_{P:\, P_l = i} G_{<l}(P)\, |w|(P)\, x_{P_0}^{\operatorname{sgn}(P)}, \qquad z_i^{(l,-)} = \sum_{P:\, P_l = i} G_{<l}(P)\, |w|(P)\, x_{P_0}^{\operatorname{sgn}(P)'}, \tag{3.41}
$$

where $\operatorname{sgn}(P)'$ denotes the opposite sign of $\operatorname{sgn}(P)$.

Proof. Expand (3.4)–(3.5) recursively, substituting $a_{\mathrm{stop},j}^{(t,\pm)} = \mathbf{1}[z_j^{(t)} > 0] z_j^{(t,\pm)}$ at each hidden predecessor. Each term selects a unique path P and accumulates the factor $G_{<l}(P)|w|(P)$; for $l = 1$, the gate product is empty. Positive edges preserve stream membership, negative edges swap it, and zero-weight edges contribute zero. Thus the positive output stream takes $x_{P_0}^{\operatorname{sgn}(P)}$, and the negative stream takes

the opposite input sign, proving (3.41). Biases add stub paths ending at intermediate layers with the same parity rule. □

Subtracting the path sums using $x_k^+ - x_k^- = x_k$ recovers $z_i^{(l)} = \sum_P G_{<l}(P)\, w(P)\, x_{P_0}$ with signed weight products $w(P)$. The split sorts these active paths by parity. Chapter 4 turns this representation into a two-player zero-sum stopping game, with (3.39) as the optimal stop/continue decision.

3.5 Regularized Decompositions

The hard gate of the Stopping Decomposition solves an optimization problem: for any $\Delta \in \mathbb{R}$,

$$\mathrm{ReLU}(\Delta) = \max_{\pi \in [0,1]} \pi\Delta, \qquad \mathbf{1}[\Delta > 0] \in \mathrm{argmax}_{\pi \in [0,1]}\, \pi\Delta, \tag{3.42}$$

so the indicator in (3.39) is the *unregularized* stop ($\pi = 0$)/continue ($\pi = 1$) argmax for $\Delta = z^+ - z^-$. A convex regularization penalty can smooth this gate, replacing the recovered ReLU by the regularizer's Fenchel conjugate.

Definition 3.5 (Regularized decomposition function, `Lean_42`). Let $\Omega\colon [0,1] \to \mathbb{R}_{\le 0}$ be a proper, closed, convex regularizer with $\Omega \le 0$. Define the *regularized gate* and the *conjugate activation*

$$\pi_\Omega(\Delta) \in \mathrm{argmax}_{\pi \in [0,1]} \big\{\pi\Delta - \Omega(\pi)\big\}, \qquad \Omega^*(\Delta) \coloneqq \max_{\pi \in [0,1]} \big\{\pi\Delta - \Omega(\pi)\big\}, \tag{3.43}$$

where an arbitrary maximizer is fixed if several exist. The *regularized decomposition function* is

$$f_{\mathrm{reg}}^{\Omega}(z^+, z^-) \coloneqq \Big(\pi_\Omega(\Delta)\, z^+ - \Omega\big(\pi_\Omega(\Delta)\big),\ \ \pi_\Omega(\Delta)\, z^- \Big), \qquad \Delta \coloneqq z^+ - z^-. \tag{3.44}$$

Compactness of [0,1] and upper semicontinuity of the objective ensure attainment in (3.43); Ω^* is the Fenchel conjugate of Ω extended by $+\infty$ outside [0,1]. Since $\pi_\Omega(\Delta) \in [0,1]$, $z^\pm \ge 0$, and $-\Omega \ge 0$, both components of (3.44) are non-negative, making $f_{\mathrm{reg}}^{\Omega}$ a pairwise activation (Definition 3.1). All maximizers recover $\Omega^*(\Delta)$, but their components can differ: for $\Omega = 0$, $z^+ = z^- = 1$, the choices $\pi = 0{,}1$ produce (0,0),(1,1). Hence Definition 3.5 fixes a selection. Here the untagged pair $\big(a_i^{(l,+)}, a_i^{(l,-)}\big)$ denotes regularized-family activations, replacing the DC ($\eta = 1$) meaning of Theorem 3.15.

Theorem 3.19 (Validity of regularized decompositions, `Lean_43`). *Let Ω be as in Definition 3.5, and let the activation pairs be propagated by the canonical non-negative linear map (3.4)–(3.5), initialised by (3.2), and updated by $f_{\mathrm{reg}}^{\Omega}$ at hidden layers, with the identity pair at the final affine readout. Then for every hidden layer l and neuron i,*

$$a_i^{(l,+)} - a_i^{(l,-)} = \Omega^*\big(z_i^{(l)}\big), \tag{3.45}$$

At the final affine layer, the difference is $z^{(L)} = a^{(L)}$. Thus $f_{\mathrm{reg}}^{\Omega}$ is a valid decomposition of the network (3.1) with activation function $\varrho = \Omega^*$. Moreover, Ω^* is convex, non-decreasing, and 1-Lipschitz, and wherever Ω^* is differentiable the gate is its derivative, $\pi_\Omega(\Delta) = (\Omega^*)'(\Delta) \in [0,1]$.

Proof. Recovery. At a hidden neuron, subtraction in (3.44) gives $\pi_\Omega(\Delta)\Delta - \Omega(\pi_\Omega(\Delta)) = \Omega^*(\Delta)$ by optimality in (3.43) (Fenchel–Young). Proposition 3.1 propagates this difference identity from (3.2) through the hidden layers and final affine readout.

Properties. The supremum of affine functions with slopes $\pi \in [0,1]$ is convex with subgradients in $[0,1]$, hence non-decreasing and 1-Lipschitz. Fenchel–Young makes every maximizer $\pi_\Omega(\Delta)$ a subgradient; wherever Ω^* is differentiable, that subgradient equals $(\Omega^*)'(\Delta)$. □

Corollary 3.20 (Unregularized case, `Lean_44`). *For $\Omega \equiv 0$ on $[0,1]$, Definition 3.5 with the maximizer $\pi_\Omega(\Delta) = \mathbf{1}[\Delta > 0]$ recovers the stopping decomposition function, $f^0_{\mathrm{reg}} = f_{\mathrm{stop}}$, and Theorem 3.19 specialises to Theorem 3.16 with $\Omega^*(\Delta) = \max_{\pi\in[0,1]} \pi\Delta = ReLU(\Delta)$.*

Proof. For $\Delta \neq 0$, $\pi\Delta$ uniquely maximizes at $\mathbf{1}[\Delta > 0]$ on $[0,1]$; for $\Delta = 0$, every π maximizes, including the selected $\mathbf{1}[0 > 0] = 0$. Thus (3.44) with $\Omega = 0$ equals (3.39), with value $\mathrm{ReLU}(\Delta)$. □

3.5.1 The Softplus Decomposition as the entropic subcase

The canonical smooth regularizer is the (negative) binary Shannon entropy at temperature $\theta > 0$,

$$\Omega_\theta(\pi) := -\theta H(\pi), \qquad H(\pi) := -\pi \log \pi - (1-\pi)\log(1-\pi), \tag{3.46}$$

which is strictly convex on $[0,1]$ and satisfies $\Omega_\theta \leq 0$.

Corollary 3.21 (Softplus Decomposition, `Lean_45`). *For $\Omega = \Omega_\theta$ as in (3.46), the regularized gate and conjugate activation are*

$$\pi_{\Omega_\theta}(\Delta) = \sigma'_\theta(\Delta) := \frac{1}{1 + e^{-\Delta/\theta}}, \qquad \Omega^*_\theta(\Delta) = \sigma_\theta(\Delta) := \theta \log\bigl(1 + e^{\Delta/\theta}\bigr), \tag{3.47}$$

the sigmoid gate and the softplus function at temperature θ. The resulting decomposition function is the softplus decomposition function

$$f^\theta_{\mathrm{soft}}(z^+, z^-) = \Bigl(\sigma'_\theta(\Delta)\, z^+ + \theta\, H\bigl(\sigma'_\theta(\Delta)\bigr),\ \ \sigma'_\theta(\Delta)\, z^-\Bigr), \qquad \Delta = z^+ - z^-, \tag{3.48}$$

and by Theorem 3.19 it yields a valid decomposition—the Softplus Decomposition*—of the softplus network with parameter $\beta = 1/\theta$. In the limit $\theta \to 0$, $f^\theta_{\mathrm{soft}} \to f_{\mathrm{stop}}$ pointwise on $\{\Delta \neq 0\}$.*

Proof. The objective $\pi\Delta + \theta H(\pi)$ is strictly concave on $(0,1)$ with derivative $\Delta - \theta \log \frac{\pi}{1-\pi}$; the first-order condition gives the unique maximizer $\pi = \sigma'_\theta(\Delta)$. Substituting back, with $\log \sigma'_\theta(\Delta) = \Delta/\theta - \log(1 + e^{\Delta/\theta})$ and $\log(1 - \sigma'_\theta(\Delta)) = -\log(1 + e^{\Delta/\theta})$, yields the optimal value

$$\sigma'_\theta(\Delta)\,\Delta + \theta H\bigl(\sigma'_\theta(\Delta)\bigr) = \theta \log\bigl(1 + e^{\Delta/\theta}\bigr) = \sigma_\theta(\Delta), \tag{3.49}$$

the Legendre–Fenchel form of binary log-sum-exp over $\{0,\Delta\}$ at temperature θ. Substituting $-\Omega_\theta = \theta H$ into (3.44) gives (3.48) and validity by Theorem 3.19. For $\Delta \neq 0$, as $\theta \to 0$, the gate tends to $\mathbf{1}[\Delta > 0]$ and its bonus $\theta H(\sigma'_\theta(\Delta)) = \sigma_\theta(\Delta) - \sigma'_\theta(\Delta)\Delta$ tends to zero. □

The $+$ stream carries the entire entropy bonus $\theta H(\sigma'_\theta(\Delta))$ in (3.48), leaving the $-$ stream purely gated. In Chapter 4, this bonus is the active player's surplus from mixing.

Remark 3.9 (Loss of stream monotonicity). In the negative stream $a^{(l,-)} = \sigma'_\theta(\Delta) z^-$, increasing z^- at fixed z^+ raises one factor while lowering the gate through $\Delta = z^+ - z^-$. Its derivative $\partial a^{(l,-)}/\partial z^- = \sigma'_\theta(\Delta)\bigl(1 - z^- \sigma''_\theta(\Delta)/\sigma'_\theta(\Delta)\bigr)$ is negative when $z^- > \theta/(1 - \sigma'_\theta(\Delta))$. Thus the soft-gated

family loses the per-region monotonicity of Lemma 3.17 and need not inherit the hard-gated or DC streams' monotone/convex structure; difference recovery still holds by Theorem 3.19.

Remark 3.10 (Probit regularizer and GELU). The framework is not tied to the entropy. For the probit regularizer $\Omega_\Phi(\pi) \coloneqq -\phi\big(\Phi^{-1}(\pi)\big)$, with ϕ, Φ the standard Gaussian density and distribution function, one has $\frac{\mathrm{d}}{\mathrm{d}\pi}\big(-\Omega_\Phi\big)(\pi) = -\Phi^{-1}(\pi)$, so the first-order condition for (3.43) gives the gate $\pi_{\Omega_\Phi}(\Delta) = \Phi(\Delta)$ and the conjugate activation

$$\Omega_\Phi^*(\Delta) = \Delta\,\Phi(\Delta) + \phi(\Delta) = \mathbb{E}_{\varepsilon\sim\mathcal{N}(0,1)}\big[\max(\Delta+\varepsilon, 0)\big],$$

the Gaussian smoothing of ReLU, differing from GELU $\Delta\Phi(\Delta)$ (Hendrycks and Gimpel, 2016) by $\phi(\Delta)$. The gate $\Phi(\Delta) = \mathbb{E}_\varepsilon[\mathbf{1}[\Delta+\varepsilon > 0]]$ averages the hard stopping gate over Gaussian threshold shifts, matching GELU's stochastic-gate motivation. Theorem 3.19 makes the recovered Ω_Φ^* convex, non-decreasing and 1-Lipschitz. GELU itself is non-monotone, dipping below zero for $\Delta < 0$, so it cannot be such a conjugate activation: this regularized decomposition recovers the $+\phi(\Delta)$ variant exactly and GELU networks only approximately.

Remark 3.11 (Connection to the integrable-curvature splitting). Suppose Ω is strictly convex and π_Ω locally absolutely continuous on $\mathbb{R}$. Then $(\Omega^*)' = \pi_\Omega$ is non-decreasing with limits 0,1 at $-\infty, +\infty$, so the weak curvature $(\Omega^*)'' \geq 0$ lies in $L^1(\mathbb{R})$ with integral 1. A sufficient condition is $\Omega \in C^2((0,1))$, $\Omega'' > 0$: the gate inverts Ω' on its range and extends by 0,1 beyond finite endpoint slopes. Strict convexity and smoothness alone allow singular distributional curvature and need not give an absolutely continuous gate. Under the stated regularity, Theorem 3.12 applies with $N = C_2 = 0$, $C_1 = \Omega^*$, as for softplus in Corollary 3.8(6). Gate regularization and the curvature-based split of Section 3.2.2 therefore give compatible valid decompositions of these networks (Remark 3.6).

3.6 Numerical Stabilization of Split Passes

Both streams accumulate *absolute* mass (Remark 3.2). In deep networks, $z^{(l,+)} + z^{(l,-)}$ grows roughly geometrically with absolute-weight products and can exceed the representable range while $a^{(l,+)} - a^{(l,-)} = a^{(l)}$ stays moderate. Following Zimmermann & Loho (2026), we shift forward activations and backward sensitivities. Absorb biases into weights and define

$$W_{\mathrm{abs}}^{[l,\to]} \coloneqq \prod_{j=L,\dots,l+1} |W^{(j)}| = \prod_{j=L,\dots,l+1} \big(W^{(j,+)} + W^{(j,-)}\big), \tag{3.50}$$

with the empty product at $l = L$ read as the identity.

3.6.1 Shifting the forward pass

A *common* shift at any layer translates both stream outputs equally, preserving their difference and hence the network value (Theorem 3.6).

Theorem 3.22 (Shift equivariance of the split streams, `Lean_46`). *For $0 \leq l \leq L$, $s \in \{g,h\}$, and $c, a^{(+)}, a^{(-)} \in \mathbb{R}^{d^{(l)}}$ with $a^{(\pm)}, a^{(\pm)} + c \geq 0$,*

$$s^{(l)}\big(a^{(+)} + c, a^{(-)} + c\big) = s^{(l)}\big(a^{(+)}, a^{(-)}\big) + W_{\mathrm{abs}}^{[l,\to]} c. \tag{3.51}$$

In particular, $\big(g^{(l)} - h^{(l)}\big)\big(a^{(+)} + c, a^{(-)} + c\big) = \big(g^{(l)} - h^{(l)}\big)\big(a^{(+)}, a^{(-)}\big)$.

Proof. A common shift c at layer l shifts both pre-activations at $l+1$ by $(W^{(l+1,+)} + W^{(l+1,-)})c = |W^{(l+1)}|c$ (Figure 3.2). The identity $\max\{u+d,v+d\} = \max\{u,v\} + d$ propagates this shift through the hidden Maxout/identity pair; the ungated affine readout propagates it by linearity. Backward induction from the output projections, where $W_{\text{abs}}^{[L,\to]} = I$, therefore gives the common output shift $W_{\text{abs}}^{[l,\to]}c$. Subtraction proves the difference statement. □

Theorem 3.22 permits function-preserving *re-centering* by $c^{(l)} = -\min\{a^{(l,+)},a^{(l,-)}\}$ entrywise, zeroing at least one component of each pair. A fixed shift preserves head Jacobians because Maxout branch comparisons are unchanged. Input-dependent shifts must be detached for these sensitivities: differentiating through them adds $W_{\text{abs}}^{[l,\to]}Dc$ to both head derivatives, preserving their difference but potentially changing each derivative. The paper reports double-precision intermediates; the released evaluation command requires an explicit precision setting. The forward implementation offers several magnitude-reduction modes. Common shifts preserve the difference (Theorem 3.22); common scaling generally changes it. With cached original activations, the implementation then applies the symmetric correction

$$e^{(l)} = a^{(l)} - \big(a^{(l,+)} - a^{(l,-)}\big), \qquad a^{(l,+)} \leftarrow a^{(l,+)} + \tfrac{1}{2}e^{(l)}, \qquad a^{(l,-)} \leftarrow a^{(l,-)} - \tfrac{1}{2}e^{(l)}.$$

This restores the **forward invariant** in exact arithmetic. The default backward pass detaches the correction; differentiating through it changes stream sensitivities. Scaling modes, correction gradients, and convexity and monotonicity of stabilized streams require justification beyond the common-shift theorem. Floating-point reconstruction error still requires measurement.

3.6.2 Shifting the backward pass

Classical derivatives below require strict gate inequalities in the relevant head; ties use a fixed backpropagated branch selection without asserting differentiability. The non-negative matrices in the head gradients cause growth like that of forward activations. Define the *local sensitivities*

$$\delta^{(l,\sigma,s)} := \frac{\partial s^{(l)}}{\partial a^{(l,\sigma)}}, \qquad \sigma \in \{+,-\}, \quad s \in \{g,h\}. \tag{3.52}$$

Theorem 3.22 determines their pairwise structure.

Theorem 3.23 (Sensitivity complementarity, `Lean_47`**).** *For every $0 \le l \le L$ and $s \in \{g,h\}$,*

$$\delta^{(l,+,s)} = W_{\text{abs}}^{[l,\to]} - \delta^{(l,-,s)}. \tag{3.53}$$

Proof. For each coordinate j, differentiate (3.51) along $c = te_j$ at $t = 0^+$, keeping shifted pairs non-negative. The chain rule gives $\delta^{(l,+,s)} + \delta^{(l,-,s)} = W_{\text{abs}}^{[l,\to]}$ for $s \in \{g,h\}$, which rearranges to (3.53). □

The chain rule gives four block Jacobians at hidden layers with strict gate patterns:

$$\begin{aligned} J^{(l+1)} &:= \frac{\partial(a^{(l+1,+)},a^{(l+1,-)})}{\partial(a^{(l,+)},a^{(l,-)})} \\ &= \begin{pmatrix} M^{(l+1,+)}W^{(l+1,+)} + M^{(l+1,-)}W^{(l+1,-)} & M^{(l+1,+)}W^{(l+1,-)} + M^{(l+1,-)}W^{(l+1,+)} \\ W^{(l+1,-)} & W^{(l+1,+)} \end{pmatrix}. \end{aligned} \tag{3.54}$$

Here $M^{(l,\pm)}$ select complementary Maxout branches, defined below; the ungated readout uses $M^{(L,+)} = I$, $M^{(L,-)} = 0$. Fixed biases have zero derivatives with respect to preceding activations and preserve convexity, monotonicity, and piecewise-affine structure.

By (3.53), each sensitivity pair sums to the exploding matrix $W_{\text{abs}}^{[l,\to]}$, making at least one component large. Stabilization subtracts an $\alpha^{(l)}$-multiple of this bulk at each layer. Starting from $\delta_{\text{shift}}^{(L,\pm,g)} := \delta^{(L,\pm,g)} - \alpha^{(L)}\big(\delta^{(L,+,g)} + \delta^{(L,-,g)}\big)$ and analogously for h, the *stabilized backward pass*, for $s \in \{g,h\}$, iterates for $l = L-1,\dots,0$: first the ordinary gradient recursion on the already shifted quantities,

$$\big(\delta_{\text{int}}^{(l,+,s)},\delta_{\text{int}}^{(l,-,s)}\big) = \big(\delta_{\text{shift}}^{(l+1,+,s)},\delta_{\text{shift}}^{(l+1,-,s)}\big)J^{(l+1)}, \tag{3.55}$$

and then the uniform shift

$$\delta_{\text{shift}}^{(l,\pm,s)} := \delta_{\text{int}}^{(l,\pm,s)} - \alpha^{(l)}\big(\delta_{\text{int}}^{(l,+,s)} + \delta_{\text{int}}^{(l,-,s)}\big), \tag{3.56}$$

The Maxout gates are $M^{(l,+)} := \operatorname{diag}(\mathbf{1}[z^{(l)} \geq 0])$ and $M^{(l,-)} := \operatorname{diag}(\mathbf{1}[z^{(l)} < 0])$. Differentiating (3.4)–(3.5) and (3.19) gives (3.54), hence the chain-rule recursion (3.55). For $\alpha^{(l)} \equiv 0$ the scheme reproduces the plain split-pair sensitivities. In the implementation, the gate patterns $M^{(l,\pm)}$ are taken from the cached original activations rather than the drifted split pre-activations, and cached original gradients give small corrections enforcing the **backward invariant** $\frac{1}{2}\big[(\delta^{(l,+,g)} - \delta^{(l,-,g)}) - (\delta^{(l,+,h)} - \delta^{(l,-,h)})\big] = \partial f^{(l)}/\partial a^{(l)}$.

For $s \in \{g,h\}$ define the head on the half–half split by $\tilde{s}^{(l)}(a^{(l)}) := s^{(l)}(a^{(l)}/2, -a^{(l)}/2)$. The chain rule gives the unified-input gradient

$$\delta^{(l,s)} := \frac{\partial \tilde{s}^{(l)}}{\partial a^{(l)}} = \frac{\delta^{(l,+,s)} - \delta^{(l,-,s)}}{2}. \tag{3.57}$$

Define $\delta_{\text{shift}}^{(l,s)} := (\delta_{\text{shift}}^{(l,+,s)} - \delta_{\text{shift}}^{(l,-,s)})/2$ and $\delta_{\text{shift}}^{s} := \delta_{\text{shift}}^{(0,s)}$. For the final affine readout we use the uniform-notation convention $M^{(L,+)} = I$, $M^{(L,-)} = 0$; these are algebraic placeholders, not an additional gate. Standard backpropagation gives $\delta^{(l)} := \partial f^{(l)}/\partial a^{(l)} = \prod_{j=L,\dots,l+1} W^{(j,x)}$ with gated matrices $W^{(l,x)} := M^{(l,+)}W^{(l)}$.

Theorem 3.24 (Closed formulas for the stabilized split gradients, `Lean_48`). *With $\overline{\alpha}_j := \prod_{j\leq i\leq L}(1-2\alpha^{(i)})$, $\varepsilon_g = 1$, and $\varepsilon_h = -1$, the pass (3.55)–(3.56) computes, for $0 \leq l \leq L$ and $s \in \{g,h\}$,*

$$\delta_{\text{shift}}^{(l,s)} = \frac{1}{2}\Big[\varepsilon_s\delta^{(l)} - \sum_{l\leq j<L} \overline{\alpha}_{j+1}\, W_{\text{abs}}^{[j+1,\to]}\big(M^{(j+1,-)}W^{(j+1)}\big)\prod_{i=(j-1),\dots,l} W^{(i+1,x)}\Big]. \tag{3.58}$$

The $j = L-1$ correction term is zero because $M^{(L,-)} = 0$: no minimum or auxiliary Q source is inserted at the affine readout. In particular, $\delta_{\text{shift}}^{(l,g)} - \delta_{\text{shift}}^{(l,h)} = \delta^{(l)}$ for all l; for $\alpha^{(i)} \equiv 0$ (i.e. $\overline{\alpha}_j = 1$) the formulas describe the plain split-pair gradients, and for $\alpha^{(i)} \equiv 1/2$ all correction terms vanish and the stabilized quantities are $\pm\delta^{(l)}/2$, halves of the original gradient.

Proof. Claim 1 (bulk of the shifted sensitivities). For every l,

$$\delta_{\text{shift}}^{(l,+,g)} + \delta_{\text{shift}}^{(l,-,g)} = \prod_{l\leq j\leq L}\big(1-2\alpha^{(j)}\big)\, W_{\text{abs}}^{[l,\to]} = \overline{\alpha}_l\, W_{\text{abs}}^{[l,\to]}. \tag{3.59}$$

At $l = L$, the unshifted sum is I. Each shift multiplies its pair sum by $1 - 2\alpha^{(l)}$. Adding the two components of (3.55) multiplies the next layer's sum by $|W^{(l+1)}|$, since $M^{(l+1,+)} + M^{(l+1,-)} = I$. Backward induction proves (3.59), equally for h.

Claim 2 (recursion of the stabilized difference). The common shift cancels when subtracting the two components of (3.55). Using $W^{(l+1)} = W^{(l+1,+)} - W^{(l+1,-)}$ gives

$$\begin{aligned} 2\,\delta_{\text{shift}}^{(l,g)} &= \big(\delta_{\text{shift}}^{(l+1,+,g)} - \delta_{\text{shift}}^{(l+1,-,g)}\big) M^{(l+1,+)} W^{(l+1)} - \big(\delta_{\text{shift}}^{(l+1,+,g)} + \delta_{\text{shift}}^{(l+1,-,g)}\big) M^{(l+1,-)} W^{(l+1)} \\ &= 2\,\delta_{\text{shift}}^{(l+1,g)}\, W^{(l+1,x)} - \overline{\alpha}_{l+1}\, W_{\text{abs}}^{[l+1,\to]} M^{(l+1,-)} W^{(l+1)}, \end{aligned} \tag{3.60}$$

The first equality groups terms using $M^{(l+1,+)} + M^{(l+1,-)} = I$; the second uses Claim 1 at $l+1$.

Induction. At $l = L$, the initial common shift cancels, giving $\delta_{\text{shift}}^{(L,g)} = I/2$ and $\delta_{\text{shift}}^{(L,h)} = -I/2$. Unroll (3.60) backward: multiplying the hypothesis at $l+1$ by $W^{(l+1,x)}$ extends every prefix product and gives $\delta^{(l+1)} W^{(l+1,x)} = \delta^{(l)}$; the remaining term supplies $j = l$. This proves (3.58) for $s = g$; the same recurrence with initial value $-I/2$ proves the $s = h$ case. Subtracting cancels the common correction sum and gives the backward invariant. The endpoint cases follow from $\overline{\alpha}_j$. □

Remark 3.12 (Interpretation of the correction terms). The correction summands in (3.58) are gradients of an auxiliary network: define $Q[j](x) \coloneqq W_{\text{abs}}^{[j,\to]} \min\{z^{(j)}(x), 0\}$, with the original prefix through layer j followed by a linear absolute-weight tail without biases or activation gates. In a bias-free tail this agrees in value with subsequent minimum activations, but the explicit definition also handles arbitrary prefix biases. Then the j-th summand equals $\overline{\alpha}_{j+1}\, \partial Q[j+1]/\partial x$ (at $l = 0$), capturing the mass that switches streams at layer $j+1$. The factors $\overline{\alpha}_j$ counter their rapid growth toward the input and control each layer's stream-switching contribution. In practice $\alpha^{(l)} = \alpha$, whose intermediate values control the additional split signal between the theorem's two endpoints. Chapter 6 uses these stabilized gradients for split saliency.

3.7 Generalization to Convolutional Neural Networks (CNNs)

Definitions 3.1 and 3.2 extend to standard CNNs (He et al., 2016; Simonyan and Zisserman, 2015) through *chunk modules* $(u^+, u^-) \mapsto (v^+, v^-)$. Separate linear and ReLU chunks allow several linear operations between activations; a PyTorch converter (Paszke et al., 2019) splits supported computation graphs. Chunks preserve non-negativity, the difference $v^+ - v^- = \mathrm{op}(u^+ - u^-)$, and, where applicable, monotonicity and convexity (Theorem 3.6). Their composition gives Theorem 3.27. Split convolution and gated max-pooling need specific rules; one linear-chunk lemma covers the remaining operations.

Convolution. A convolutional layer with kernel K and bias b computes $u \mapsto K * u + b$, with $*$ the (multi-channel) cross-correlation. Splitting the parameters entrywise, $K = [K]^+ - [K]^-$ and $b = [b]^+ - [b]^-$, we define the split chunk

$$v^+ \coloneqq [K]^+ * u^+ + [K]^- * u^- + [b]^+, \qquad v^- \coloneqq [K]^- * u^+ + [K]^+ * u^- + [b]^-. \tag{3.61}$$

Lemma 3.25 (Affine chunks and convolution, `Lean_50`, `Lean_49`). *For any real affine map $u \mapsto Au + c$, the canonical parameter split (3.4)–(3.5), with $W = A$ and $b = c$, maps non-negative pairs to non-negative pairs. Each component is affine with non-negative coefficients, hence convex and coordinatewise non-decreasing, and $v^+ - v^- = A(u^+ - u^-) + c$. In particular, (3.61) recovers $K * (u^+ - u^-) + b$.*

Proof. Non-negative split coefficients give the structural properties; subtraction gives recovery as in Proposition 3.1. Convolution is a sparse, weight-shared affine map, so the same argument applies. Composing it with (3.19) also recovers convolution followed by ReLU. □

Split convolution (3.61) is the instance $A = K * (\cdot)$; the remaining linear chunks are listed in Table 3.2. Residual connections combine the identity chunk ($A = I$) with an already-split branch by streamwise addition $v^{\pm} = u^{\pm} + F^{\pm}(u^{+}, u^{-})$, preserving all three invariants whenever the branch does.

Tab. 3.2: Linear CNN chunks as instances of Lemma 3.25.

Chunk	Affine map	Notes
Average pooling	$A = \frac{1}{q}\sum_{j\in[q]}(\cdot)_j$, $c = 0$	non-negative A, applied streamwise
Batch norm (eval)	$A = \mathrm{diag}(c_{\mathrm{bn}})$, $c = d$	scale c_{bn} may be signed; 1×1 diagonal conv
Dropout (train)	$A = \mathrm{diag}(m)$, $c = 0$	mask $m \in \{0, \frac{1}{1-p}\}^d$ from the original pass
Residual add	$A = I$ plus split branch	streamwise sum with branch (F^{+}, F^{-})

Evaluation-mode batch normalization has a per-channel scale $c_{\mathrm{bn}} = \gamma/\sqrt{\sigma^2 + \epsilon}$ that may carry either sign (Guillemot et al., 2020; Hui and Binder, 2019); in practice we merge it into an adjacent convolution before splitting. Training-mode batch normalization, whose statistics depend on the batch, is genuinely multiplicative and requires the techniques of Chapter 5. Training-mode dropout uses the *same* realised mask m on both streams and on $u = u^{+} - u^{-}$, so the split network drops exactly the units the original does.

Gated max-pooling. Max-pooling over a window $[q]$ computes $\max_{j\in[q]} u_j$ per channel, which is neither linear nor a shared gate on a single pair. It is a CPWL function of the window and admits its own convexity-preserving splitting identity.

Proposition 3.26 (Correctness of gated max-pooling, `Lean_51`). *For a pooling window $[q]$, define the split chunk*

$$v^{+} := \max_{j\in[q]}\Big\{u_j^{+} + \sum_{i\in[q],\, i\neq j} u_i^{-}\Big\}, \qquad v^{-} := \sum_{i\in[q]} u_i^{-}. \tag{3.62}$$

Then $v^{\pm} \geq 0$ for $u^{\pm} \geq 0$, both components are convex and non-decreasing in each coordinate of (u^{+}, u^{-}), and

$$v^{+} - v^{-} = \max_{j\in[q]}\big(u_j^{+} - u_j^{-}\big). \tag{3.63}$$

Proof. Each candidate and the sum have non-negative coefficients, proving non-negativity; the linear combinations and maximum preserve convexity and monotonicity. Each candidate equals $(u_j^{+} - u_j^{-}) + v^{-}$. Maximizing and subtracting the common v^{-} proves (3.63). □

Identity (3.63) generalizes (3.18), the case $q = 2$ with the pair $\big(u^{+} - u^{-}, 0\big)$. Optionally, (3.62) can be replaced by a *winner-takes-it-all* rule forwarding, in both streams, the entries where the *difference* is maximal; this preserves the difference invariant but sacrifices monotonicity and convexity, mirroring the shared-gate trade-off of Remark 3.8.

Implementation and caches. The pinned implementation uses PyTorch FX to trace supported operations into decomposition chunks. One cache accumulates inflowing activations and gradients per chunk; another mirrors the original model for layer-wise error analysis, optional correction, and reuse of its positivity patterns during backpropagation instead of drifted split maxima. Arithmetic is configurable as `float16`, `float32` (default), or `float64`; the original paper reports double precision. Conversion covers operations supported by the chunk implementations.

Alternative nonlinearities and exactness. The source implementation discussion proposes approximating nonlinearities such as ELU (Clevert et al., 2016) or Swish (Ramachandran et al., 2018) by small ReLU subnetworks before splitting. Finite ReLU networks are piecewise affine and cannot exactly reproduce these non-piecewise-affine activations. The split preserves the approximating network; its error relative to the original activation remains separate. The pinned repository's scalar approximation utility includes Swish, does not list ELU, and is unused by the inspected hybrid conversion path. Section 3.2.2 provides exact curvature-based scalar decompositions under its stated hypotheses.

The CNN split pass. The split forward pass processes the chunks of a computation graph $\mathscr{G}$ in topological order, applying to each the split rule of its type—Lemma 3.25 for the linear chunks, the Maxout gate (3.19) or max-pool split (3.62) for the non-linear ones—with optional entrywise re-centering $v^{\pm} \leftarrow v^{\pm} - \min\{v^+, v^-\}$ after each chunk. The stabilization of Section 3.6 applies unchanged, since Theorem 3.22 uses only non-negative linearity and the Maxout gate structure.

Theorem 3.27 (Correctness of the CNN split pass, `Lean_52`). *Let $\mathscr{G}$ be a feedforward computation graph composed of the chunk types above, computing the function $x \mapsto f(x)$. Then for every input split $x = x^+ - x^-$ with $x^{\pm} \geq 0$, the split pass produces non-negative pairs satisfying $v_m^+ - v_m^- = v_m$ at every chunk C_m, where v_m is the original activation; hence $f(x) = v_M^+ - v_M^-$. If neither winner-takes-it-all pooling nor input-dependent re-centering is used, every $v_m^{\pm}$ is convex in (x^+, x^-) and non-decreasing in each coordinate.*

Proof. Induct in topological order from the non-negative affine input coordinates. Lemma 3.25, Proposition 3.26, and Theorem 3.15 with (3.18) give non-negativity and recovery for each chunk. Convexity and monotonicity propagate by composition of convex coordinatewise non-decreasing maps (Amos et al., 2017), as in Theorem 3.3. Re-centering preserves non-negativity and the difference (Theorem 3.22), but can destroy monotonicity: $(\max\{p,n\},n)$ becomes $(\mathrm{ReLU}(p-n),0)$, whose first component decreases with n. Hence re-centering is excluded from the structural conclusion, along with winner-takes-it-all pooling. The invariant at C_M proves output recovery. □

These valid decompositions—the mixing family with its convex DC endpoint, curvature-based smooth splits (Section 3.2.2), and hard/regularized gates—now have stabilized passes and CNN extensions. Chapter 4 builds games on the Stopping Decomposition; Chapter 6 derives attribution from g,h.

4 Game-Theoretic Framework

The sign-separated streams of Chapter 3 expose accumulated opposing contributions. This chapter formalizes the backward-game picture of Figure 1.1, tracing backward rules through these contributions. The stopping decomposition supplies a common state graph for two games whose policies represent gradients and relevance propagation. Their trajectory distributions support method design (Chapter 6) and diagnostics (Chapter 8). Exact recovery concerns the anchor rules; myopic temperature changes and risk-sensitive policies deform these rules.

4.1 Related Work

Ancona et al. (2018) unify several gradient-based rules as modified gradients, a map-level construction excluding the $\alpha\beta$ rule. Our signed trajectory-space representation unifies gradients and the $\alpha\beta$ rule through their routing laws. Together with the split representation, it distinguishes which internal signals attribution retains and how it propagates them backward.

Game and trajectory views. The closest mathematical precursor, the ReLU Net Game of Gaubert & Vlassopoulos (2025), identifies forward ReLU-network evaluation with a backward Stopping Game's value, without connecting it to the backward pass or attribution. We reformulate it through a novel Stopping Decomposition whose weight split relates to the DC decomposition used directly for attribution by Zimmermann & Loho (2026). Our SG and RG use this weight split to recover backward attribution rules rather than the pure DC decomposition. The NRM framework of Xiong et al. (2026) assigns normalized signed relevance to network walks. Section 8.1 compares its treatment of LRP and concept relevance propagation with our trajectory probabilities and discounted occupations.

Relevance propagation and stochastic explanations. LRP, Deep Taylor, and Excitation Backprop form the core relevance propagation family (Bach et al., 2015; Montavon et al., 2017; Zhang et al., 2018a); variants modify redistribution for contrast, stability, or signed relevance (Binder et al., 2016; Gu et al., 2019; Kohlbrenner et al., 2020; Nam et al., 2020; Vukadin et al., 2024). Our common trajectory-space semantics recovers gradients through SG and the generalized $\alpha\beta$ rule through RG, whose analytical modes give principled modifications of backward rules. Excitation Backprop (Zhang et al., 2018a) is an absorbing Markov chain whose redistribution on non-negative activations gives the unstabilized z^+-rule with biases omitted: the $(\alpha,\beta) = (1,0)$ case of $\alpha\beta$-LRP. Our game adds an opponent for negative-weight influence and replaces input-dependent transitions with soft Bellman equations. Deep Taylor (Montavon et al., 2017) justifies z^+ on non-negative inputs; RG constructs a game for the generalized $\alpha\beta$ rule.

Shapley-value attribution methods. KernelSHAP explains marginal input-feature contributions relative to a baseline through a cooperative *black-box* game over feature coalitions (Lundberg and Lee, 2017). Our complementary *white-box* game traces relevance through network internals.

Robust explanations via Softplus. Dombrowski et al. (2022) propose Softplus replacement as a curvature-based defense against gradient-explanation manipulation. Section 4.7 shows that entropy-regularizing the activation-state stop/continue choice in SG recovers Softplus-network gradients, situating their construction within our framework.

Interpretability in Transformers and ViTs. Attention Rollout / Flow use attention as a routing graph (Abnar and Zuidema, 2020); transformer-LRP propagates through attention and skip connections (Achtibat et al., 2024; Ali et al., 2022; Chefer et al., 2021b); DAVE addresses ViT tokenization artifacts through conditioned operators and equivariant averaging (Wróbel et al., 2026). Our game

semantics represents conditioned operators through a QK-softmax oracle reference policy and Value-routing attribution trajectories. Its RG+Eq mode uses Reynolds averaging to suppress DAVE's targeted tokenization-grid artifacts, with further control through the game modes.

4.2 Backward Attribution

Throughout the game, attention, and trajectory derivations, $W_{ij}^{(l)}$ denotes the edge from predecessor i to successor j: the adjacency matrices transpose the DC recurrences' output-by-input matrices while representing the same network. The tag stop distinguishes stopping from DC streams. Consider a ReLU network with layers $l = 1, \ldots, L$, input $a^{(0)} = x$, pre-activations $z_j^{(l)} = \sum_i a_i^{(l-1)} W_{ij}^{(l)} + b_j^{(l)}$, and activations $a_j^{(l)} = \mathrm{ReLU}(z_j^{(l)})$. We initially explain a scalar activation $f(x) = a_u^{(L)}$; Section 4.5 extends this to ungated, possibly negative affine logits. Write $[u]^+ = \max(u,0)$, $[u]^- = \max(-u,0)$, hence $u = [u]^+ - [u]^-$, and $W^{(l,\pm)} = [W^{(l)}]^\pm$ entrywise.

Layer-wise relevance propagation. Starting from $R_u^{(L)} = f(x)$, LRP (Bach et al., 2015) redistributes each neuron's relevance $R_j^{(l)}$ backward, layer by layer, in proportion to each input's pre-activation contribution $a_i^{(l-1)} W_{ij}^{(l)}$, with bias and stabilizer shares absorbed. Rules weight these contributions differently. The $\alpha\beta$-rule uses $\alpha,\beta \geq 0$; $\alpha - \beta = 1$ gives conservation when both participating sign streams are nonempty and bias, stabilizer, and empty-stream absorption are accounted for. With $\varepsilon \geq 0$ added to each sign denominator, as in common implementations (Anders et al., 2026), positive and negative fractions are normalized separately and scaled by α and β:

$$
\begin{aligned}
R_i^{(l-1)} &= \sum_j \Big(\alpha \, \frac{[a_i^{(l-1)} W_{ij}^{(l)}]^+}{\Sigma_j^{(l,+)} + \varepsilon} - \beta \, \frac{[a_i^{(l-1)} W_{ij}^{(l)}]^-}{\Sigma_j^{(l,-)} + \varepsilon} \Big) R_j^{(l)}, \\
\Sigma_j^{(l,\pm)} &= \sum_k [a_k^{(l-1)} W_{kj}^{(l)}]^\pm + [b_j^{(l)}]^\pm .
\end{aligned}
\tag{4.1}
$$

Relevance assigned to the bias and to ε is absorbed. For non-negative activations, $(\alpha,\beta) = (1,0)$ with $\varepsilon = 0$ and biases omitted from normalization gives the z^+-rule of Deep Taylor decomposition (Montavon et al., 2017) and Excitation Backprop (Zhang et al., 2018a); other rules (LRP-ε, LRP-γ) change the redistribution fractions.

Transformers. In a Vision Transformer (ViT), LayerNorm couples the features of a token through its input-dependent mean and variance, and the attention matrix $A = \mathrm{softmax}(QK^\top/\sqrt{d})$ depends on the input, which breaks conservation (Ali et al., 2022). Transformer-specific rules therefore *condition* on the forward pass: conservative propagation (Ali et al., 2022) and DAVE (Wróbel et al., 2026) hold A and the LayerNorm denominator fixed, retain linear mean-centering, and propagate attention relevance through the Value path only; AttnLRP (Achtibat et al., 2024) also propagates through the softmax and the QK product.

These rules route mass backward from each neuron to its inputs; the games below make that routing explicit.

4.2.1 Notation and boundary conventions

Players and turns are $p,q,r \in \{+,-\}$, with opponent p'. Write $\delta_{p,q} = \mathbf{1}[p = q]$, $\xi_{p,q} = 2\delta_{p,q} - 1$, and $p \oplus q = +$ for $p = q$, $-$ otherwise. This commutative, associative operation satisfies $p \oplus p = +$, $p \oplus + = p$, $p \oplus - = p'$, and $p' \oplus q = (p \oplus q)'$. We select $\mathrm{ReLU}'(0) = 0$ and the smallest-index

max-pooling winner; these are backward conventions, not classical differentiability claims at ties. A *trajectory* is a sequence of game states, a *path* a sequence of neuron indices. Other symbols are defined with their operators below and in the attention and trajectory chapters.

4.2.2 Attribution as a Game

We construct finite-horizon two-player games (Osborne and Rubinstein, 1994) played backward from unit mass at the positive output state. States record layer, neuron, player turn and stream sign; stopping enters an absorbing cemetery †. Skip connections randomly select summands, while max-pooling selects maximal-payoff states. For player p in turn and opponent p', the Stopping Game's discounted payoff advantage satisfies

$$V_p(s) = \max_{a \in \mathscr{A}_x(s)} \Big\{ r_p(s,a) - r_{p'}(s,a) \\ + \gamma(s) \textstyle\sum_{s'} \mathbb{P}_x(s' \mid s,a)\, V_p(s') \Big\}, \tag{4.2}$$

whereas the Routing Game entropy-regularizes mixed routing decisions. Policies induce trajectory laws Π_x; their discounted occupation measures Γ_x weight visits to each state by the cumulative game-specific discount (Definition 4.6).

4.3 Stopping Game

The Stopping Game realizes the Stopping Decomposition as non-negative player-specific payoffs; its discounted occupations recover gradients. We use the edge orientation of Section 4.2, $W_{ij}^{(l)}$ from predecessor i to successor j, and initially explain the scalar activation $f(x) = a_u^{(L)}$, with ReLU at every layer $l \geq 1$. Section 4.5 treats ungated signed logits.

4.3.1 Stopping Decomposition

Use the canonical weight and bias split (3.3), the affine recursions (3.4)–(3.5) with transposed matrix indices, and the shared gate f_{stop} of Definition 3.4. Here the input pair is the canonical positive/negative split, rather than an arbitrary non-negative pair:

$$a_{\text{stop},k}^{(0,\pm)} = x_k^{\pm} = [\pm x_k]^+. \tag{4.3}$$

Theorem 4.1 (Layer-wise recovery, `Lean_55`**).** *For every neuron and layer $l \geq 1$,*

$$a_{\text{stop},j}^{(l,+)} - a_{\text{stop},j}^{(l,-)} = a_j^{(l)} = ReLU(z_j^{(l)}), \tag{4.4}$$

with difference x_k at layer 0.

Proof. The induction of Theorem 3.16 also applies ReLU at the last layer: $z_j^{(l,+)} - z_j^{(l,-)} = z_j^{(l)}$ and the common gate gives $\mathbf{1}[z_j^{(l)} > 0](z_j^{(l,+)} - z_j^{(l,-)}) = \text{ReLU}(z_j^{(l)})$. □

ReLU networks are CPWL (Arora et al., 2018). Their DC split uses $(\max\{z^+,z^-\},z^-)$ and has convex streams (Zimmermann and Loho, 2026); stopping instead discards both components at closed gates. They remain affine with non-negative coefficients, hence monotone in the doubled input (x^+,x^-), on each fixed-gate region (Lemma 3.17); the streams need not be globally convex.

Parity paths. Lemma 3.18 gives the pre-activation path sums (3.41): each input path contributes its absolute weight product, multiplied by the gates strictly below its starting neuron, to x^+ or x^- according to its negative-edge parity. Zero-weight paths contribute zero. A bias at layer t adds a path terminating there, with the $l-t$ edges above it, gates at layers $t,\dots,l-1$, and the absolute bias value; parity includes the bias sign. Subtracting streams restores the signed active-path sum. The game implements this parity bookkeeping by switching players at negative weights.

4.3.2 Game Definition

The construction retains the ± states, sign-split transitions, bias continuation rewards, input payoffs, cemetery, and stop/continue choices of Gaubert & Vlassopoulos (2025). For attribution, we resolve their signed value $\pm a_j^{(l)}$ into non-negative player payoffs and replace terminal $\pm x_k$ by x_k^+,x_k^-, exposing the parity paths.

Definition 4.1 (Stopping Game State Space). The states are $s_{j,q}^{(l,\mathrm{act})}$, $1 \le l \le L$, for neuron j with player $q \in \{+,-\}$ in turn; input terminals $s_{k,q}^{(0)}$; and the absorbing cemetery †. In predecessor formulas, $s_{i,r}^{(0,\mathrm{act})}$ means $s_{i,r}^{(0)}$.

Definition 4.2 (Payoffs, transitions, and values). Fix x. At $s = s_{j,q}^{(l,\mathrm{act})}$, stopping pays zero and enters †; continuation pays player p the non-negative bias part $b_j^{(l,p\oplus q)}$. Input $s_{k,q}^{(0)}$ pays $x_k^{p\oplus q}$, and the cemetery pays zero. Thus positive biases and inputs pay the player in turn; negative ones pay their magnitudes to the opponent. The kernel is

$$\mathbb{P}(\dagger \mid s,\mathrm{stop}) = 1, \tag{4.5}$$

$$\mathbb{P}(s_{i,r}^{(l-1,\mathrm{act})} \mid s,\mathrm{cont}) = \frac{W_{ij}^{(l,q\oplus r)}}{\gamma_j^{(l)}}, \qquad \gamma_j^{(l)} = \sum_i |W_{ij}^{(l)}|. \tag{4.6}$$

Positive edges retain the player; negative edges switch it. If $\gamma_j^{(l)} = 0$, continuation pays the bias and enters †. The same $\gamma_j^{(l)}$ normalizes the kernel and discounts continuation payoffs and occupations.

A pure policy $\pi_q(s) \in \{0,1\}$ selects stop (0) or continue (1). Player p's discounted action value satisfies

$$\tilde{Q}_p^\pi(s,a) = r_p(s,a) + \gamma(s) \sum_{s'} \mathbb{P}(s' \mid s,a) \tilde{V}_p^\pi(s'). \tag{4.7}$$

Consequently,

$$\tilde{Q}_p^\pi(s,\mathrm{stop}) = 0, \tag{4.8}$$

$$\tilde{Q}_p^\pi(s,\mathrm{cont}) = b_j^{(l,p\oplus q)} + \sum_{i,r} W_{ij}^{(l,q\oplus r)} \tilde{V}_p^\pi(s_{i,r}^{(l-1,\mathrm{act})}), \tag{4.9}$$

$$\tilde{V}_p^\pi(s_{k,q}^{(0)}) = x_k^{p\oplus q}, \qquad \tilde{V}_p^\pi(\dagger) = 0, \tag{4.10}$$

$$\tilde{V}_p^\pi(s) = \pi_q(s) \tilde{Q}_p^\pi(s,\mathrm{cont}), \tag{4.11}$$

$$V_p^\pi(s) = \tilde{V}_p^\pi(s) - \tilde{V}_{p'}^\pi(s). \tag{4.12}$$

The active player q maximizes its advantage V_q, rather than its individual payoff $\tilde{V}_q$.

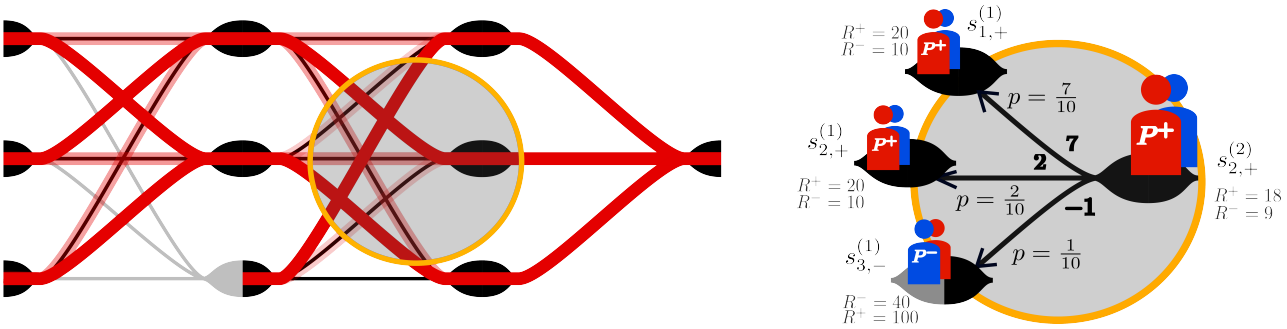


Fig. 4.1: Stopping Game: trajectories (left) and local decisions (right). Continuing from $s^{(2)}_{2,+}$ reaches $s^{(1)}_{1,+}$, $s^{(1)}_{2,+}$, and $s^{(1)}_{3,-}$ with probabilities 7/10, 2/10, and 1/10; only the negative edge switches players. Labels $R^{\pm}$ denote expected continuation payoffs, excluding discount $\gamma^{(2)}_2 = 10$ at $s^{(2)}_{2,+}$. At $s^{(1)}_{3,-}$, $R^- = 40$ and $R^+ = 100$ give advantage -60, so P^- stops and $V_- = 0$ (gray half-neuron). These trajectories terminate there; the cemetery is omitted. Their continuation payoff to both players is zero, leaving positive continuation advantage for P^+ at $s^{(2)}_{2,+}$.

Theorem 4.2 (Forward equivalence, `Lean_57`). *The Stopping Game has a subgame-perfect Nash equilibrium (SPNE)*

$$\pi^\star_q(s^{(l,\mathrm{act})}_{j,q}) = \mathbf{1}[z^{(l)}_j > 0], \tag{4.13}$$

with player-specific values and advantages

$$\tilde{V}^{\pi^\star}_p(s^{(l,\mathrm{act})}_{j,q}) = a^{(l,p\oplus q)}_{\mathrm{stop},j}, \qquad V^{\pi^\star}_p(s^{(l,\mathrm{act})}_{j,q}) = \xi_{p,q} a^{(l)}_j. \tag{4.14}$$

Proof. Induct from the input terminals, where the payoff is $x^{p\oplus q}_k$ and its difference from the opponent's payoff is $\xi_{p,q}x_k$. Assuming the identities at layer $l-1$, player q's continuation advantage is

$$b^{(l,+)}_j - b^{(l,-)}_j + \sum_{i,r} W^{(l,q\oplus r)}_{ij} \xi_{q,r} a^{(l-1)}_i = b^{(l)}_j + \sum_i W^{(l)}_{ij} a^{(l-1)}_i = z^{(l)}_j.$$

Stopping has advantage zero, so strict positivity selects continuation and we select stopping at equality. This extends each lower-layer equilibrium. Substitution in (4.11) gives $\tilde{V}^{\pi^\star}_p = \mathbf{1}[z^{(l)}_j > 0] z^{(l,p\oplus q)}_j = a^{(l,p\oplus q)}_{\mathrm{stop},j}$; subtracting the two values and applying Theorem 4.1 proves the advantage identity. The zero-weight case has the same argument with an empty weighted sum. □

Remark 4.1 (Ties and implementation convention). The equilibrium choice is unique at nonzero pre-activations. At $z^{(l)}_j = 0$, stopping and continuation have equal advantage; the theorem selects stopping. The pinned Stopping-Game attribution code instead continues at exact zero, while the Routing Game and Hellinger diagnostic use strict positivity. Original forward activations agree at ties, but the selected backward quantities can differ.

4.3.3 Skip Connections and Max Pooling

Definition 4.3 (Addition Oracle). For $z = x + y$, state $s^{\mathrm{add}}_{z,q}$ selects $s_{x,q}$ or $s_{y,q}$ with probability 1/2, retaining player q. Both players receive zero immediate payoff; the discount is $\gamma_{\mathrm{O}} = 2$.

Proposition 4.3 (Addition equivalence, Lean_58). *For every policy,* $\tilde{V}_p^{\pi}(s_{z,q}^{\mathrm{add}}) = \tilde{V}_p^{\pi}(s_{x,q}) + \tilde{V}_p^{\pi}(s_{y,q})$. *Hence operand values recovering the stopping streams give* $\tilde{V}_p^{\pi^\star}(s_{z,q}^{\mathrm{add}}) = x^{(p\oplus q)} + y^{(p\oplus q)} = z^{(p\oplus q)}$.

Proof. The Bellman equation has zero payoff and probability–discount product $2(1/2) = 1$ on each branch. □

The uniform Oracle is a convention, not an architectural constraint. Any $0 < \rho < 1$ with branch probabilities $(\rho, 1-\rho)$ and branch-dependent discounts $(1/\rho, 1/(1-\rho))$ preserves forward values and the gradient occupations of Theorem 4.5, since each probability–discount product is one. A single state discount requires $\rho\gamma_{\mathrm{O}} = (1-\rho)\gamma_{\mathrm{O}} = 1$, forcing $\rho = 1/2$. This uniform, maximum-entropy choice treats summands symmetrically. Changing the split changes Π_x and its Hellinger distance (Section 8.3), though not these Stopping-Game identities. For the Routing Game it also changes relevance allocation (Section 4.4.2).

Definition 4.4 (Max-Pooling Decision State). For $z = \max\{x_1, \ldots, x_m\}$, state $s_{z,q}^{\max}$ offers actions $k \in \{1, \ldots, m\}$, leading deterministically to $s_{x_k,q}$ with player q retained, zero immediate payoff, and discount one. At ties choose $k^\star = \min\{k : x_k = \max_l x_l\}$.

Proposition 4.4 (Max-pooling equivalence, Lean_59). *If* $x_k \geq 0$ *and* $\tilde{V}_p^{\pi^\star}(s_{x_k,q}) = x_k^{(p\oplus q)}$ *for every operand, the selected equilibrium gives* $\tilde{V}_p^{\pi^\star}(s_{z,q}^{\max}) = x_{k^\star}^{(p\oplus q)} = z^{(p\oplus q)}$.

Proof. Player q's advantage at operand k is $\xi_{q,q}x_k = x_k$, so it selects the stated maximizer. The deterministic, unit-discount Bellman recursion copies that operand's player-specific values. □

Tied maximizers have equal forward values but may induce different trajectories and occupations; all subsequent statements use the smallest-index convention.

4.3.4 Discounted Occupation Measures and Gradient Representations

Definition 4.5 (Trajectory law, Lean_60). For fixed x and root s, let $\mathscr{T}_s$ contain finite state sequences $\zeta = (\zeta_0, \ldots, \zeta_T)$ with $\zeta_0 = s$, following the kernels above until an input terminal or † is reached by stopping or zero-weight continuation. Combining these kernels with a possibly stochastic policy π induces $\Pi_x^{\pi}(\cdot \mid s)$. The ground set includes trajectories continuing through closed gates; these have zero probability under $\pi^\star$. Write $\Pi_x(\cdot \mid s) = \Pi_x^{\pi^\star}(\cdot \mid s)$ and $\Pi_x = \Pi_x(\cdot \mid s_0)$ for output root $s_0 = s_{u,+}^{(L,\mathrm{act})}$.

Definition 4.6 (Discounted occupation, Lean_61). Let $d_t(\zeta) = \prod_{m=0}^{t-1} \gamma(\zeta_m)$, with $d_0 = 1$. The factors are $\gamma_j^{(l)}$ at continuing activation states, 2 at addition states, and 1 at max-pooling states; a stopping move has unit discount. Define

$$\Gamma_x(v \mid s) = \mathbb{E}_{\zeta \sim \Pi_x(\cdot|s)}\left[\sum_t d_t(\zeta)\mathbf{1}\{\zeta_t = v\}\right], \qquad \Gamma_x(v) = \Gamma_x(v \mid s_0). \tag{4.15}$$

Theorem 4.5 (Gradient and occupation representation, Lean_62, Lean_53). *Fix $s = s_{j,q}^{(m,\mathrm{act})}$ and $l \le m$, interpreting layer-0 states as terminals. On open cells with nonzero ReLU pre-activations and unique pooling winners, the selected stop-at-tie-breaks equilibrium gives, also for networks with skip connections and max pooling,*

$$\frac{\partial a_{\mathrm{stop},j}^{(m,p\oplus q)}}{\partial a_{\mathrm{stop},i}^{(l,k)}} = \Gamma_x(s_{i,p\oplus k}^{(l,\mathrm{act})} \mid s), \tag{4.16}$$

$$\frac{\partial a_j^{(m)}}{\partial a_i^{(l)}} = \xi_{q,+}\big(\Gamma_x(s_{i,+}^{(l,\mathrm{act})} \mid s) - \Gamma_x(s_{i,-}^{(l,\mathrm{act})} \mid s)\big). \tag{4.17}$$

Thus $\partial_i f(x) = \Gamma_x(s_{i,+}^{(0)}) - \Gamma_x(s_{i,-}^{(0)})$ at the output root. At ties these formulas describe the selected backward quantities, not classical derivatives of individual stopping streams, which may be discontinuous. All occupations are non-negative.

Proof. Fix x and abbreviate Γ_x by Γ. For the first identity, induct along the finite computation graph. At the target layer, both the derivative and occupation are the indicator of the same neuron and sign: $\delta_{p\oplus q,k} = \mathbf{1}\{q = p \oplus k\}$. For an activation state, the stopping recursion is

$$a_{\mathrm{stop},j}^{(m,p\oplus q)} = \mathbf{1}[z_j^{(m)} > 0]\left[b_j^{(m,p\oplus q)} + \sum_{h,r} W_{hj}^{(m,q\oplus r)} a_{\mathrm{stop},h}^{(m-1,p\oplus r)}\right]. \tag{4.18}$$

On a fixed-gate cell its derivative has the same gated, weighted recursion as

$$\Gamma(v \mid s_{j,q}^{(m,\mathrm{act})}) = \mathbf{1}[z_j^{(m)} > 0] \sum_{h,r} W_{hj}^{(m,q\oplus r)} \Gamma(v \mid s_{h,r}^{(m-1,\mathrm{act})}),$$

where $v = s_{i,p\oplus k}^{(l,\mathrm{act})}$ lies below the starting state. Indeed the bias derivative is zero and kernel normalization cancels the continuation discount. For zero incoming weight the sum and derivative vanish; a closed gate stops before reaching v. At addition nodes, differentiation sums the two operand derivatives, matching occupations because $2(1/2) = 1$ on each branch. At pooling nodes, both recursions copy the same chosen operand, with unit probability and discount. This proves the first identity, including the prescribed backward conventions at ties.

For the second, vary the positive component of $a_i^{(l)} = a_{\mathrm{stop},i}^{(l,+)} - a_{\mathrm{stop},i}^{(l,-)}$ while fixing its negative component. Signed recovery gives

$$\frac{\partial a_j^{(m)}}{\partial a_i^{(l)}} = \frac{\partial a_{\mathrm{stop},j}^{(m,+)}}{\partial a_{\mathrm{stop},i}^{(l,+)}} - \frac{\partial a_{\mathrm{stop},j}^{(m,-)}}{\partial a_{\mathrm{stop},i}^{(l,+)}} = \Gamma(s_{i,q}^{(l,\mathrm{act})} \mid s) - \Gamma(s_{i,q'}^{(l,\mathrm{act})} \mid s).$$

This equals $\xi_{q,+}$ times the stated fixed-sign occupation difference. At s_0, $q = +$ and $a_u^{(L)} = f(x)$, giving the input-gradient formula. □

Modeling $\alpha\beta$-LRP or attention within SG would require activation-dependent transitions. The Routing Game instead puts this dependence in the players' policies.

4.4 The Routing Game

The Routing Game places activation-dependent routing in the players' policies. Its entropy-regularized log-space subgames have a Gibbs SPNE for every $\tau > 0$; at $\tau = 1$, their occupations recover $\alpha\beta$-LRP on affine/ReLU chains, with explicit stopping at inactive activations on general residual graphs. This global construction assumes non-negative activation outputs. The later GELU discussion concerns local gates conditioned on a fixed forward pass.

4.4.1 States, values, and equilibrium

For Shannon entropy $\mathscr{H}(\pi) = -\sum_i \pi(i) \log \pi(i)$, the Legendre–Fenchel identity (Boyd and Vandenberghe, 2004; Geist et al., 2019) gives, for $\tau > 0$, $n \geq 1$, and $r \in \mathbb{R}^n$,

$$\max_{\pi \in \Delta_n} \left\{ \sum_i \pi(i) r_i + \tau \mathscr{H}(\pi) \right\} = \tau \log \sum_i e^{r_i/\tau}, \qquad \pi^\star(i) \propto e^{r_i/\tau}. \tag{4.19}$$

Write $\ell(z) = \log z$ for $z > 0$ and $\ell(z) = -\infty$ otherwise, with $e^{-\infty} = 0$; thus $e^{\ell(z)} = \mathrm{ReLU}(z)$. In expectations, a zero-probability term contributes zero even at score $-\infty$; positive probability on such a score gives value $-\infty$.

Definition 4.7 (Entropy-Regularized Two-Player Routing Game). Fix x, $\varepsilon \geq 0$, and $\tau > 0$. A neuron has activation states $s_{j,p}^{(l,\mathrm{act})}$ and linear states $s_{j,q,\kappa}^{(l,\mathrm{lin})}$, separating the player turn $p,q \in \{+,-\}$ from the weight-stream sign $\kappa \in \{+,-\}$. At an activation state, player p stops at † with both payoffs $-\infty$, or continues uniformly to $s_+ = s_{j,p,+}^{(l,\mathrm{lin})}$ and $s_- = s_{j,p',-}^{(l,\mathrm{lin})}$. The negative branch switches players; structural value discount $\gamma_O = 2$ compensates the $1/2$ branching. At a linear state $s = s_{j,q,\kappa}^{(l,\mathrm{lin})}$, player q chooses a distribution ν over predecessors, a terminal bias node, and an absorbing outside option $\perp_{\mathrm{lin}}$. The action gains are

$$Q_r(s,i) = \begin{cases} \ell(W_{ij}^{(l,\kappa)}) + V_r(s_{i,q}^{(l-1,\mathrm{act})}), & r = q,\ l \geq 2, \\ \ell([x_i W_{ij}^{(1)}]^\kappa), & r = q,\ l = 1, \\ -\infty, & r = q', \end{cases} \tag{4.20}$$

with $Q_q(s,\mathrm{bias}) = \ell([b_j^{(l)}]^\kappa)$, $Q_q(s, \perp_{\mathrm{lin}}) = \ell(\varepsilon)$, and inactive-player gains $-\infty$. Bias and outside shares are absorbed. The entropy-regularized opponent-payoff differences reduce to

$$\begin{aligned} \bar{Q}_r(s;\nu) &= \sum_a \nu(a) \ell\big(e^{Q_r(s,a)} - e^{Q_{r'}(s,a)}\big) + \tau \mathscr{H}(\nu), \\ \bar{Q}_q(s;\nu) &= \sum_a \nu(a) Q_q(s,a) + \tau \mathscr{H}(\nu), \qquad \bar{Q}_{q'}(s;\nu) = -\infty. \end{aligned} \tag{4.21}$$

Consequently, the equilibrium linear values are

$$\begin{aligned} V_q^\star(s) &= \max_\nu \bar{Q}_q(s;\nu) = \tau \ell\left(\varepsilon^{1/\tau} + ([b_j^{(l)}]^\kappa)^{1/\tau} + \sum_i e^{Q_q(s,i)/\tau} \right), \\ V_{q'}^\star(s) &= -\infty. \end{aligned} \tag{4.22}$$

An identically zero weight/bias stream has value $\ell(\varepsilon)$. If all scores are $-\infty$ (including the outside score), every mixture is optimal; we select $\nu = \delta_{\perp_{\mathrm{lin}}}$ and set ordinary-successor fractions $0/0 := 0$. Otherwise, when $\varepsilon = 0$, the outside option has zero probability and the game reduces to predecessor/bias routing.

At an activation state, let $D_p = e^{V_p(s_+)} - e^{V_{p'}(s_-)}$. The uniform branch kernel and value discount give

$$Q_r(s_{j,p}^{(l,\mathrm{act})},\mathrm{continue}) = \ell\left(\sum_{s'\in\{s_+,s_-\}} \tfrac{1}{2}\gamma_O(e^{V_r(s')} - e^{V_{r'}(s')})\right) = \ell(\xi_{r,p}D_p). \tag{4.23}$$

The selected pure activation policy stops at every log-space tie:

$$\begin{aligned}\pi_p^\star(\mathrm{continue} \mid s_{j,p}^{(l,\mathrm{act})}) &= \mathbf{1}[D_p > 0],\\ V_p(s_{j,p}^{(l,\mathrm{act})}) &= \max\{-\infty,\ell(D_p)\}, \qquad V_{p'}(s_{j,p}^{(l,\mathrm{act})}) = -\infty.\end{aligned} \tag{4.24}$$

Thus only positive signed continuation mass passes the gate. An admissible profile chooses one pure activation action and one mixture at each linear state; replacing the maxima by these choices defines its values. Backward induction on the finite DAG is well-posed: first-layer gains use x directly, and subsequent linear states use only lower-layer activation values.

Theorem 4.6 (SPNE and Forward Equivalence of the Routing Game, `Lean_63`). *For every $\tau > 0$ and $\varepsilon \geq 0$, Definition 4.7 admits a canonical SPNE with the selected activation policy and, on nonempty finite-score support, Gibbs routing at $s = s_{j,q,\kappa}^{(l,\mathrm{lin})}$:*

$$\pi_q^\star(i \mid s) \propto e^{Q_q(s,i)/\tau} = \begin{cases}(W_{ij}^{(l,\kappa)})^{1/\tau} e^{V_q(s_{i,q}^{(l-1,\mathrm{act})})/\tau}, & l \geq 2,\\ ([x_i W_{ij}^{(1)}]^\kappa)^{1/\tau}, & l = 1.\end{cases} \tag{4.25}$$

Normalization includes the bias and outside weights $([b_j^{(l)}]^\kappa)^{1/\tau}$ and $\varepsilon^{1/\tau}$; zero or wrong-sign routes have zero mass. At $\tau = 1$, for every l,j,p,q,r,κ, the network quantities of (4.1) satisfy

$$V_r(s_{j,q,\kappa}^{(l,\mathrm{lin})}) = \ell\big(\delta_{r,q}(\Sigma_j^{(l,\kappa)} + \varepsilon)\big), \tag{4.26}$$

$$Q_r(s_{j,p}^{(l,\mathrm{act})},\mathrm{continue}) = \ell(\xi_{r,p} z_j^{(l)}), \tag{4.27}$$

$$V_r(s_{j,p}^{(l,\mathrm{act})}) = \ell(\delta_{r,p} a_j^{(l)}), \tag{4.28}$$

$$\pi_p^\star(\mathrm{continue} \mid s_{j,p}^{(l,\mathrm{act})}) = \mathbf{1}[z_j^{(l)} > 0]. \tag{4.29}$$

Proof. Backward induction solves each linear subgame by (4.19) on its finite-score simplex face, or by the selected absorbing action if that face is empty. The activation player continues exactly when its gain exceeds stopping's $-\infty$. These best responses compose to an SPNE on the finite DAG, including terminal bias/outside actions.

For the anchor, induct on l. The first layer uses the fused masses $[x_i W_{ij}^{(1)}]^\kappa$; subsequently, $a_i^{(l-1)} \geq 0$ gives $Q_q = \ell(a_i^{(l-1)} W_{ij}^{(l,\kappa)})$ by the induction hypothesis. The linear log-sum-exp is therefore $\ell(\Sigma_j^{(l,\kappa)} + \varepsilon)$. At activation states the two ε terms cancel, leaving $D_p = \Sigma_j^{(l,+)} - \Sigma_j^{(l,-)} = z_j^{(l)}$. The selected gate gives $a_j^{(l)} = \mathrm{ReLU}(z_j^{(l)})$; its opponent receives $-\infty$ whether the gate continues or stops. This proves all four identities. □

Remark 4.2 (Uniqueness). Strict concavity makes Gibbs routing unique when finite-score support is nonempty; otherwise every mixture is optimal and the selected outside action absorbs the trajectory. At an activation state, $D_p \le 0$ makes both active-player action values $-\infty$, so multiple best responses exist. Our *stop-at-tie-breaks* convention resolves these ties as well as empty streams. At $\tau = 1$ it selects $\mathbf{1}[z_j^{(l)} > 0]$.

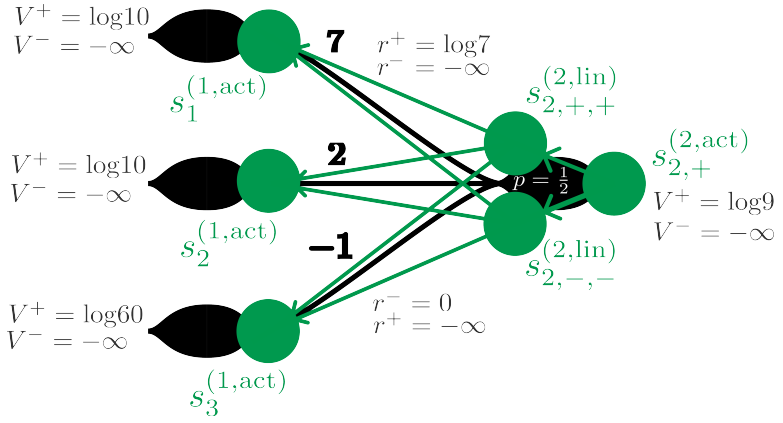


Fig. 4.2: Routing around $s_{2,+}^{(2,\mathrm{act})}$ in the toy network of Figure 4.1. Since $V^+ = \log 9 > -\infty$, P^+ continues, retaining the turn on the positive branch or switching to P^- on the negative branch, each with probability 1/2. Linear states use Gibbs routing. Along $s_{2,+,+}^{(2,\mathrm{lin})} \to s_{1,+}^{(1,\mathrm{act})}$ the active payoff is $\log W_{12}^{(2,+)} = \log 7$ and the opponent's is $-\infty$; zero-mass edges have $\log 0 = -\infty$. The layer-1 activation copies are collapsed, and the intermediate payoff $\log 2$ on the position-2 to 2 transition is omitted.

4.4.2 Addition and max pooling

Use Section 4.3.3's addition Oracle (uniform operands, player retained, value discount 2) and max-pooling decision (player retained, value discount 1, smallest-index winner). Their log values are

$$V_r(s_{z,p}^{\mathrm{add}}) = \ell(e^{V_r(s_{x,p})} + e^{V_r(s_{y,p})}), \qquad V_p(s_{z,p}^{\mathrm{max}}) = \max_k V_p(s_{x_k,p}). \tag{4.30}$$

At $\tau = 1$, for the active player and non-negative operand activations, these equal $\ell(a_x + a_y)$ and $\ell(\max_k a_{x_k})$, extending forward equivalence. Addition's *trajectory* discount is 1, so each operand receives half its incoming occupation. More generally, an Oracle with probabilities $(\rho, 1-\rho)$, $0 < \rho < 1$, and value discounts $(1/\rho, 1/(1-\rho))$ preserves forward values but splits relevance as $(\rho, 1-\rho)$: uniform splitting is a routing convention. The implementation instead sends full relevance to both operands (trajectory discount 2). Every input-reaching trajectory in the evaluated ResNet/ViT crosses each of its B addition nodes once, so this multiplies input occupations by 2^B and preserves normalized maps.

4.4.3 Occupation measures and relevance recovery

Let Π_x start at $s_{u,+}^{(L,\mathrm{act})}$ with unit mass. For $\alpha, \beta \ge 0$ with $\alpha - \beta = 1$, use Definition 4.6 with the *entered-state* trajectory discounts

$$\gamma(s) = \begin{cases} 2\alpha, & s = s_{j,q,+}^{(l,\mathrm{lin})}, \\ 2\beta, & s = s_{j,q,-}^{(l,\mathrm{lin})}, \\ 1, & \text{otherwise (activation, addition, max-pool).} \end{cases} \qquad d_t(\zeta) = \prod_{m=1}^{t} \gamma(\zeta_m), \quad d_0 = 1. \tag{4.31}$$

These discounts differ from those evaluating game values. They keep each occupation non-negative; the LRP sign comes from the difference

$$\Gamma_j^{(l)} = \Gamma_x(s_{j,+}^{(l,\mathrm{act})}) - \Gamma_x(s_{j,-}^{(l,\mathrm{act})}). \tag{4.32}$$

The attribution-side vector instead follows the deterministic recursion (4.1) from $R_j^{(L)} = r\delta_{j,u}$. Bias and outside shares are absorbed. This combines Bach et al.'s $\alpha\beta$ rule with a separate ε in each sign denominator: Bach et al. (2015) present $\alpha\beta$ and ε as alternative rules, while their combination is the convention of common implementations (Anders et al., 2026). Recovery below is exact for this convention on affine/ReLU chains; general residual graphs also apply the stated activation gates. At $\varepsilon = 0$, the affine rule is the original $\alpha\beta$ rule, with bias absorption retained.

Theorem 4.7 (Recovery of Multi-Path $\alpha\beta$-LRP, `Lean_64`, `Lean_54`). *The Gibbs policies and selected continuation decisions give the canonical SPNE of Theorem 4.6 for every $\tau > 0$. At $\tau = 1$, for every $\varepsilon \geq 0$ and input with $f(x) = a_u^{(L)} > 0$, using stop-at-log-ties, empty-stream absorption, and smallest-index pooling winners:*

1) On an affine/ReLU chain, or whenever incoming occupation is supported on positive activations, the projected occupations satisfy

$$\Gamma_i^{(l-1)} = \sum_j \left(\alpha \frac{[a_i^{(l-1)} W_{ij}^{(l)}]^+}{\Sigma_j^{(l,+)} + \varepsilon} - \beta \frac{[a_i^{(l-1)} W_{ij}^{(l)}]^-}{\Sigma_j^{(l,-)} + \varepsilon} \right) \Gamma_j^{(l)}. \tag{4.33}$$

Hence $R_j^{(l)} = r\Gamma_j^{(l)}$ at every layer. On a general residual graph, replace each summand by its product with $\mathbf{1}[z_j^{(l)} > 0]$; the identity then recovers the corresponding explicitly gated relevance rule. This includes signed inputs through the fused first-layer masses, with $0/0 = 0$. For $r = f(x)$ it gives the output-seeded LRP map; when $f(x) = 0$, the root stops and $R^{(l)} = f(x)\Gamma^{(l)} = 0$.

2) At addition $z = x_{\mathrm{op}} + y_{\mathrm{op}}$, each outgoing edge carries $\frac{1}{2}\Gamma_x(s_{z,p}^{\mathrm{add}})$ for either player p.

3) At max pooling, the winner edge carries $\Gamma_x(s_{z,p}^{\max})$ and all other outgoing edges carry zero.

Here edge occupation is $\Gamma_x(s \to s') = \mathbb{E}_{\Pi_x} \sum_t d_{t+1}(\zeta)\mathbf{1}\{\zeta_t = s, \zeta_{t+1} = s'\}$; a state's total occupation sums its incoming edges, including other parents.

Proof. At the root $\Gamma_j^{(L)} = \delta_{j,u}$. At $\tau = 1$, the linear Gibbs probabilities on a nonempty stream are

$$\pi_{j,\kappa}^\star(i) = \frac{[a_i^{(l-1)} W_{ij}^{(l)}]^\kappa}{\Sigma_j^{(l,\kappa)} + \varepsilon}, \quad \pi_{j,\kappa}^\star(\mathrm{bias}) = \frac{[b_j^{(l)}]^\kappa}{\Sigma_j^{(l,\kappa)} + \varepsilon}, \quad \pi_{j,\kappa}^\star(\perp_{\mathrm{lin}}) = \frac{\varepsilon}{\Sigma_j^{(l,\kappa)} + \varepsilon}. \tag{4.34}$$

Empty streams absorb. On a chain, zero hidden activations receive zero Gibbs arrival probability, so all occupied hidden activations and the positive root continue. Addition Oracles can reach zero operands; their closed activation gates absorb the incoming mass. Writing $g_j^{(l)} = \mathbf{1}[z_j^{(l)} > 0]$, the uniform branch probabilities cancel the factors 2 in the trajectory discounts, while negative branches switch players. Thus, for either player p,

$$\Gamma_x(s_{i,p}^{(l-1,\mathrm{act})}) = \sum_j g_j^{(l)} \left[\alpha \pi_{j,+}^\star(i) \Gamma_x(s_{j,p}^{(l,\mathrm{act})}) + \beta \pi_{j,-}^\star(i) \Gamma_x(s_{j,p'}^{(l,\mathrm{act})}) \right]. \tag{4.35}$$

Subtracting the player equations gives the gated recurrence; under the stated support condition, $g_j^{(l)}\Gamma_j^{(l)} = \Gamma_j^{(l)}$ yields (4.33). Multiplication by r proves the corresponding relevance recursion by backward induction. At $l = 1$ the same calculation uses $[x_i W_{ij}^{(1)}]^\kappa$. Absorbing bias/outside actions contribute no predecessor mass. Addition and pooling follow from their probabilities 1/2 and $\mathbf{1}[k = k^\star]$, respectively, and unit trajectory discounts; edge occupations avoid counting contributions from other parents. □

The support condition matters: for $a = \mathrm{ReLU}(x_1 - 2x_2)$ and $f = a + 1$ at $(x_1,x_2) = (1,1)$, a uniform addition Oracle sends half its unit mass to $a = 0$, where the game stops. Ungated $(\alpha,\beta,\varepsilon) = (2,1,0)$ redistribution would instead send $(1, -\frac{1}{2})$ to the inputs. Thus the residual extension represents the specified gated routing rule, not ungated LRP after arbitrary uniform splits.

Remark 4.3 (Starting player and signed-discount alternative). Starting at the negative output copy swaps the two occupations and negates every $\Gamma_j^{(l)}$; we use the positive copy. Equivalently, signed discounts 2α and -2β recover relevance from the *sum* of player-copy occupations. Our non-negative discounts retain genuine non-negative occupations and put the sign in their difference.

4.4.4 Temperature as a policy deformation

Solving every subgame at $\tau > 0$ yields an SPNE, with exact forward recovery and the relevance correspondence above at $\tau = 1$. The experiments instead use the *myopic* policy

$$\pi_{j,\kappa}^{(\tau)\star}(i) \propto \exp\big(Q_q(s_{j,q,\kappa}^{(l,\mathrm{lin})},i)/\tau\big), \qquad V(s_{i,q}^{(l-1,\mathrm{act})}) \text{ fixed at its } \tau = 1 \text{ value.} \tag{4.36}$$

It changes local routing while each player evaluates continuation using the network's actual forward activations, unaware of downstream temperature changes. Re-solving downstream at τ would change those values. For non-negative predecessor activations, ordinary route weights become $(a_i^{(l-1)} W_{ij}^{(l,\kappa)})^{1/\tau}$: $\tau < 1$ sharpens and $\tau > 1$ flattens routing. No exactness is claimed for this deformation at $\tau \neq 1$. The implementation exponentiates activations, weights, and bias masses by $1/\tau$, but leaves the outside weight at ε, not $\varepsilon^{1/\tau}$. On ResNet-50 it redistributes global-average-pool relevance uniformly and merges batch normalization into the preceding convolution with an ε-stabilized rule. These are additional implementation conventions beyond the addition and activation-gate rules above; VGG-16 (identity average pool at 224×224) and ViT-B/16 are unaffected.

4.5 Signed affine output logits

The activation-output formulation above extends to the signed logits of a classifier by an *ungated affine readout*. Let $h_j = a_j^{(L)} \geq 0$ be the last hidden activations and

$$f_u(x) = \sum_j c_{uj} h_j(x) + b_u. \tag{4.37}$$

This output has no ReLU or sign-dependent stopping gate.

Stopping Game. Add a root with forced continuation, immediate bias payoff difference b_u, and structural discount $\gamma_u = \sum_j |c_{uj}|$. When $\gamma_u > 0$, it moves to hidden neuron j with probability $|c_{uj}|/\gamma_u$, retaining the player for $c_{uj} \geq 0$ and switching players for $c_{uj} < 0$. Its value is therefore $b_u + \sum_j c_{uj} h_j = f_u(x)$, even when this value is negative. If $\gamma_u = 0$, the root pays b_u and moves to the cemetery; the input derivative is zero. On the strict gate cells of Theorem 4.5, let $\Gamma^{\mathrm{SG}}_{j,\pm}(i)$ denote the input-state occupations from a unit source at hidden neuron j with initial player $+$. The added transition gives

$$\partial_i f_u(x) = \sum_j c_{uj} \big(\Gamma^{\mathrm{SG}}_{j,+}(i) - \Gamma^{\mathrm{SG}}_{j,-}(i) \big). \tag{4.38}$$

This chain rule retains non-negative path occupations and the selected-backward convention at gate or pooling ties.

Routing Game. Append an unconditionally continuing, ungated source to the hidden Routing Game, with positive and negative branch masses including the bias: $\Sigma^{\kappa}_u = \sum_j [h_j c_{uj}]^{\kappa} + [b_u]^{\kappa}$. At $\tau = 1$, the sign-oracle probabilities $1/2$, trajectory discounts 2α and 2β, and the branchwise Gibbs policies give the coefficients

$$q^+_j = \alpha \frac{[h_j c_{uj}]^+}{\Sigma^+_u + \varepsilon}, \qquad q^-_j = \beta \frac{[h_j c_{uj}]^-}{\Sigma^-_u + \varepsilon}. \tag{4.39}$$

Bias and outside-option shares are absorbed. An empty branch is completed by a transition to the cemetery, and its ordinary-successor fractions are zero ($0/0 \coloneqq 0$). For unit-source hidden occupations $\Gamma^{\mathrm{RG}}_{j,\pm}(i)$, define

$$\widehat{\Gamma}_{\pm}(i) = \sum_j \big(q^+_j \Gamma^{\mathrm{RG}}_{j,\pm}(i) + q^-_j \Gamma^{\mathrm{RG}}_{j,\mp}(i) \big). \tag{4.40}$$

Since $h_j = 0$ implies $q^+_j = q^-_j = 0$, only positive hidden activations receive nonzero relevance. Subtracting the equations gives affine $\alpha\beta$ redistribution followed by Theorem 4.7's hidden-layer recursion, including its residual-graph gate convention. For any real initial relevance r_u,

$$R_i = r_u \big(\widehat{\Gamma}_+(i) - \widehat{\Gamma}_-(i) \big). \tag{4.41}$$

The trajectory law and both occupations remain non-negative; the signed seed scales their *difference*. Logarithms apply only to non-negative branch masses, never the signed logit. Classical LRP seeds $r_u = f_u(x)$; the released Routing-Game routine uses $r_u = 1$, so the former map is $f_u(x)$ times the latter, reversing sign for negative logits. At zero logits, output-seeded relevance vanishes while unit-seeded relevance can remain nonzero.

4.6 Game-Theoretic Modes

The exact SG and RG anchors recover gradients and $\alpha\beta$-LRP. Beyond the exact parameters $(\alpha,\beta,\varepsilon)$, temperature, risk aversion, gate smoothing, and equivariant averaging deform the backward rule. They recover the anchor at their defaults; the fan-in entropy mode below instead changes the explained network's forward pass. Section B.3.4 reports the empirical sweeps.

Tab. 4.1: Game modes: defaults, effects and measured costs. "Exact" stays within a represented rule; other modes are deformations, inactive at the anchor. σ^2 is input variance for *Assumed Density Filtering* (ADF); N counts averaged input transformations.

Mode	Default	Effect	Cost (s/image)
(α,β), ε	(2,1), 0.5	Stream discounts and outside-option absorption (exact)	none
τ	1	sharper ($\tau < 1$) or more diffuse ($\tau > 1$) routing	negligible (VGG-16 0.035 $\to$ 0.037, ResNet-50 0.089 $\to$ 0.094)
λ, σ^2	λ=0	lowers high-variance activation-state values	one *Assumed Density Filtering* (ADF) pass (ResNet-50 0.089 $\to$ 0.140)
λ_{sm}, σ^2	0	shifts attention logits by per-key uncertainty	one *Assumed Density Filtering* (ADF) pass (ViT-B/16 0.10 $\to$ 0.17)
$\tilde{\Phi}$	off	smooth stop/continue gate	one *Assumed Density Filtering* (ADF) pass for σ
N	off	equivariant averaging (token-grid artifacts)	$\times N$ (ViT-B/16, N=50: 4.93; with λ_{sm}: 8.66)
λ_{ent}	0	fan-in entropy penalty on the forward bias	negligible

4.6.1 Defaults, costs and order of use

Table 4.1 combines game effects, defaults, and measured costs (median seconds per image, one A100 GPU, batch size 1; Section A.3.3). Start at the anchor, vary τ at negligible cost, add risk aversion with input variance σ^2 to discount high-variance units (λ for CNNs, λ_{sm} for ViTs), then use RG+Eq averaging for ViT token-grid artifacts. Section B.3.1 gives the selected combinations by model and criterion.

Focus and exact modes. The myopic Gibbs temperature retains the $\tau = 1$ game values (Section 4.4.4): low τ focuses on dominant predecessors, high τ exposes broader context and architectural priors. The exact (α,β) parameters weight support and opposition; ε adds an outside option with log-score $\log \varepsilon$, absorbing weak-evidence routes while barely affecting strong ones (Theorem 4.7).

4.6.2 Activation-Side Risk Aversion λ via ADF Variance Propagation

An *Assumed Density Filtering* (ADF) pass (Gast and Roth, 2018), initialized with variance σ^2 at every input, propagates Gaussian pre-activation means and variances. We use the exact $\mathbb{E}[\text{ReLU}(X)]$ for $X \sim \mathcal{N}(\mu,\sigma^2)$; our implementation approximates its variance by $\sigma^2\Phi(\mu/\sigma)$. At activation states, risk aversion uses the lower-confidence value $V_\lambda = \mu + \lambda\sigma$, $\lambda \leq 0$, attenuating high-variance routes while leaving linear-state routing unchanged. This is the pessimistic counterpart of confidence-bound exploration (Auer et al., 2002; Fei et al., 2020). The attention variant shifts oracle logits by λ_{sm} times per-key uncertainty before softmax, downweighting high-variance keys (Section 5.4.5).

4.6.3 Noisy-Observation Gate $\tilde{\Phi}$

Replace the hard activation gate $\mathbf{1}[\mu/\sigma + \lambda > 0]$ by continuation probability

$$\tilde{\Phi}(z_i^{(l)}) = \Phi\big(\mu_i^{(l)}/\sigma_i^{(l)} + \lambda\big), \tag{4.42}$$

with stopping probability $1-\tilde{\Phi}$, where Φ is the standard normal *Cumulative Distribution Function* (CDF) and $\mu_i^{(l)}$,$\sigma_i^{(l)}$ are the ADF pre-activation mean and standard deviation. Thus input variance remains active whenever $\tilde{\Phi}$ is on. For $\mu = z$, $\sigma = 1$, $\lambda = 0$, the gate has a variational interpretation using the normal density ϕ and quantile Φ^{-1}:

$$\mathscr{R}_\Phi(\pi) = \phi(\Phi^{-1}(\pi)), \qquad \pi \in [0,1]. \tag{4.43}$$

As in Remark 3.10, its endpoint values are zero; symmetry gives its maximum $\phi(0) = 1/\sqrt{2\pi}$ at $\pi = 1/2$. Like Shannon entropy, it rewards mixing.

Lemma 4.8 (Probit dual of the soft gate). *For every $z \in \mathbb{R}$,*

$$\max_{\pi\in[0,1]} \{\pi z + \mathscr{R}_\Phi(\pi)\} \quad \textit{has unique maximizer } \pi^\star = \Phi(z), \tag{4.44}$$

$$\max_{\pi\in[0,1]} \{\pi z + \mathscr{R}_\Phi(\pi)\} = z\Phi(z) + \phi(z) = \mathbb{E}_{\eta\sim\mathscr{N}(0,1)}[ReLU(z+\eta)]. \tag{4.45}$$

Proof. The inverse-function rule and $\phi'(y) = -y\phi(y)$ give

$$\mathscr{R}'_\Phi(\pi) = -\Phi^{-1}(\pi), \qquad \mathscr{R}''_\Phi(\pi) = -1/\phi(\Phi^{-1}(\pi)) < 0. \tag{4.46}$$

Strict concavity and the first-order condition give $\pi^\star = \Phi(z)$; substitution and Gaussian integration give the displayed value. For $\sigma > 0$, replace z by $\mu + \lambda\sigma$ and $\mathscr{R}_\Phi$ by $\sigma\mathscr{R}_\Phi$ to obtain (4.42). □

Remark 4.4 (Optimal policy under perceived payoff noise). The same policy marginalizes "continue iff the perceived payoff $z+\eta$ is positive", with independent $\eta \sim \mathscr{N}(0,1)$ at each state:

$$\mathbb{P}(z + \eta > 0) = \Phi(z). \tag{4.47}$$

Only perception is noisy; continuation pays z, so the expected backward comparison is $z\Phi(z)$ rather than $z\mathbf{1}[z > 0]$. This threshold-noise view matches the stochastic-gate motivation for GELU (Hendrycks and Gimpel, 2016). On ReLU networks, applying $\tilde{\Phi}$ at the current layer retains the original forward activations and downstream game values; this optional smoothing mode, used in the VGG-16 SG row, carries no exactness claim.

GELU networks. The regularized value $z\Phi(z)+\phi(z)$ has derivative $\Phi(z)$. It is the Gaussian-smoothed ReLU, whereas $\mathrm{GELU}(z) = z\Phi(z)$ is nonconvex and cannot be a regularized stop/continue value $\max_\pi\{\pi z + \mathscr{R}(\pi)\}$. On GELU networks, using $\Phi(z)$ in the occupation recursion recovers the gradient of the gate-conditioned map $a = \Phi(\bar{z})z$, with $\bar{z}$ held at its forward value (Section 5.4.6); the full GELU derivative is $\Phi(z) + z\phi(z)$.

4.6.4 Fan-in entropy bias λ_{ent}

Fan-in entropy lowers the continuation bias (SG) or bias-node payoff (RG) for diffuse fan-in, using $b_j^{(l)} \mapsto b_j^{(l)} - \lambda_{\text{ent}}\tilde{\mathscr{H}}_j^{(l)}$, where $\tilde{\mathscr{H}}_j^{(l)} = -\sum_i p_{ij}\log p_{ij}/\log n_j$, $p_{ij} = |W_{ij}^{(l)}|/\gamma_j^{(l)}$, for fan-in $n_j > 1$ and $\gamma_j^{(l)} > 0$. This weight-only mode has negligible cost, changes the explained forward pass, and is inactive at $\lambda_{\text{ent}} = 0$.

4.7 Softplus Variant of the Stopping Game

Activation-state entropy regularization (Gaubert and Vlassopoulos, 2025) extends the Stopping Decomposition to Softplus networks: player values recover split activations, and occupation differences recover gradients.

Modified rule: entropy-regularized activation-state policy. The Shannon-entropy identity (4.19) applies with temperature $\theta > 0$ and has the unique Gibbs optimizer. For the binary stop/continue choice, it gives the Softplus conjugacy of (3.49). At an activation state $s = s_{j,q}^{(l,\mathrm{act})}$, write $\Delta(s) = \tilde{Q}_q^{\pi}(s,\mathrm{cont}) - \tilde{Q}_{q'}^{\pi}(s,\mathrm{cont})$; stopping pays both players zero. Entropy-regularize only the active player's binary decision:

$$V_q^{\pi}(s) = \max_{\rho\in[0,1]} \{\rho\Delta(s) + \theta\mathscr{H}(\rho)\}, \tag{4.48}$$

$$\pi_q^{\star}(\mathrm{cont} \mid s) = \mathrm{sp}'_{\theta}(\Delta(s)), \qquad V_q^{\pi}(s) = \mathrm{sp}_{\theta}(\Delta(s)), \tag{4.49}$$

where $\mathrm{sp}_{\theta}(z) = \theta\log(1 + e^{z/\theta})$ and $\mathrm{sp}'_{\theta}(z) = 1/(1 + e^{-z/\theta})$. All transition and immediate-payoff rules (4.5)–(4.9) remain; policies become mixed and the active player receives the entropy bonus $\theta\mathscr{H}(\pi_q(s))$ in the value recursion (4.11). This activation temperature θ differs from the Routing Game's linear-state temperature τ.

Theorem 4.9 (Softplus forward equivalence, `Lean_65`). *The entropy-regularized Stopping Game has an SPNE with policy (4.49), unique without max-pooling; pooling ties use Section 4.2.1's convention. At every activation state,*

$$\tilde{V}_p^{\pi^\star}(s_{j,q}^{(l,\mathrm{act})}) = a_{\mathrm{soft},j}^{(l,p\oplus q,\theta)}, \qquad V_q^{\pi^\star}(s_{j,q}^{(l,\mathrm{act})}) = \mathrm{sp}_{\theta}(z_j^{(l)}). \tag{4.50}$$

Here the Softplus split starts at $a_{\mathrm{soft},k}^{(0,\pm,\theta)} = x_k^{\pm}$, follows the affine recursion (3.4)–(3.5), and uses the entropy pair (3.48), with $\mathrm{sp}_{\theta} = \sigma_{\theta}$ and $\mathscr{H}_{\theta}(z) = \mathscr{H}(\mathrm{sp}'_{\theta}(z))$.

Proof. Induct from input values $x_k^{p\oplus q}$. If the preceding split recovers its activation, (4.9) gives continuation values $z_j^{(l,+)}$ and $z_j^{(l,-)}$, hence $\Delta = z_j^{(l)}$. At zero fan-in these are simply the bias parts. The binary entropy identity gives the sigmoid policy and Softplus advantage. Crediting its entropy bonus only to the active player gives exactly (3.48), whose difference is $\mathrm{sp}_{\theta}(z)$. This closes the induction; finite-horizon backward induction and strict concavity give the stated SPNE and uniqueness, with the prescribed pooling tie rule. □

Gradient recovery on Softplus networks. Let Π_x^{θ} be the trajectory law generated by the Gibbs continuation policies, and use it in Definition 4.6.

Theorem 4.10 (Softplus gradient recovery, `Lean_66`). *Under the preceding SPNE on a Softplus network, for $l \le m$ and $s = s_{j,q}^{(m,\mathrm{act})}$,*

$$\frac{\partial a_j^{(m)}}{\partial a_i^{(l)}} = \xi_{q,+}\big(\Gamma_x(s_{i,+}^{(l,\mathrm{act})} \mid s) - \Gamma_x(s_{i,-}^{(l,\mathrm{act})} \mid s)\big), \tag{4.51}$$

where occupations use Π_x^{θ}, with the pooling convention of Theorem 4.5 at ties.

Proof. Let $D_j^{(m)}(q)$ denote the theorem's right-hand side, including $\xi_{q,+}$. Multiplying transition probabilities by structural discounts cancels their normalization; negative edges switch players and reverse the sign. Inducting from $D_j^{(l)}(q) = \delta_{ij}$, the preceding-layer values are independent of q, so $D_j^{(m)}(q) = \mathrm{sp}'_\theta(z_j^{(m)}) \sum_h W_{hj}^{(m)} D_h^{(m-1)}$, the Softplus chain rule (zero at zero fan-in). Addition and selected max-pooling steps remain as in Theorem 4.5. Individual occupations need not differentiate the split heads because their smooth gate depends on them; the difference recovers the network derivative. □

Probit analog and GELU. Replace $\theta\mathscr{H}$ by $\mathscr{R}_\Phi$ in the binary game. Lemma 4.8 gives value $z\Phi(z) + \phi(z)$, continuation $\Phi(z)$, and bonus $\phi(z)$. The same proofs therefore recover Gaussian-smoothed-ReLU networks and their gradients. The GELU conditioning distinction of Section 4.6.3 remains: $\Phi(z)$ alone is its conditioned gate, not its full derivative.

Relation to the curvature-based defense of Dombrowski et al. Dombrowski et al. (2022) train Softplus networks from scratch against gradient-explanation manipulation. Their weight- and activation-dependent Hessian Frobenius-norm bound diverges in the ReLU limit; $|\mathrm{sp}''_\theta(z)| \le 1/(4\theta)$ gives a finite uniform bound for fixed weights, which bounds gradient-explanation changes under input perturbations. Theorems 4.9–4.10 realize this replacement by entropy-regularized stopping. Whether Hellinger responses of these trajectory laws (Section 8.3) yield sharper or differently targeted bounds remains open.

5 Extensions: Attention and Transformers

This chapter extends decomposition and routing games to multiplicative layers and Transformers: the sign-sorted monotone split of Zimmermann & Loho (2026), the exact convex split of the full deterministic Transformer developed here, and the conditioned routing rules evaluated experimentally. Their domains and preserved properties differ.

Transformers (Vaswani et al., 2017) introduce input-dependent products $QK^\top$, AV and coordinate-coupled softmax and LayerNorm. Section 5.2 preserves monotonicity on non-negative split inputs; Section 5.3 adds convexity using polarization, Theorem 3.11's softmax pair, and a reciprocal-square-root pair. The conditioned Value-routing game (Section 5.4) and its ViT-B/16 implementation (Section 5.4.6) supply Chapter 7's results; the full Transformer DC construction is analytical.

5.1 Related Work: Interpretability in Transformers

Early Transformer-head analyses combined attention patterns, LRP-based importance and pruning (Voita et al., 2019). To include propagation through depth, Abnar & Zuidema (2020) proposed *attention rollout*, multiplying residual-corrected attention matrices, and *attention flow*, a max-flow computation on the layered graph. Rollout uses these matrices as token-walk transition kernels; flow uses their entries as capacities. Both omit value magnitudes, ***Multilayer Perceptrons*** (MLPs) and projection signs, allowing rollout to highlight tokens whose values contribute nothing.

A second line adapts LRP (Bach et al., 2015). Chefer et al. (2021b) combined relevance with attention gradients for class-specific self-attention attribution, later extended to co-attention and encoder–decoder architectures (Chefer et al., 2021a). Ali et al. (2022) showed that naive gradients through attention and LayerNorm can *violate conservation*; detaching attention and the LayerNorm denominator while retaining linear centering gives conservative local rules. AttnLRP (Achtibat et al., 2024) introduced bilinear-product and non-linearity rules scaling to LLMs. Other variants yield concept-level (Achtibat et al., 2023) and magnitude-aware (Vukadin et al., 2024) explanations. DAVE (Wróbel et al., 2026) addresses patch-grid attribution artifacts in ViTs (Dosovitskiy et al., 2021) through conditioned operators and spatial averaging; Darcet et al. (2024) mitigated high-norm artifact tokens with registers. The ResLRP preprint addresses residual-branch cancellation: its residual γ-rule conserves relevance and bounds absolute relevance amplification per backward merge by $1 + 2/\gamma$, for nonzero residual output and $\gamma > 0$ (Berend et al., 2026). At LLM scale, gradient approximations of activation patching (Kramár et al., 2024; Syed et al., 2024) localize circuits against interventions (Hanna et al., 2024); attribution also supports head identification and guided decoding (Kahardipraja et al., 2025; Komorowski et al., 2026).

The structural question asks whether the full forward map, including attention generation, admits explicit convex streams (Sections 5.2 and 5.3). The attribution question concerns Value routing under fixed realized attention. Section 5.4's KL-regularized game recovers Ali et al. (2022)'s conservative attention-block rule, formalizing a routing calculation motivated by Abnar & Zuidema (2020) with explicit conditioning.

5.2 Decomposing Multiplicative Operations

Gating layers, squeeze-and-excitation blocks, and attention multiply activations entrywise or contract tensors. The sign-sorted construction of Zimmermann & Loho (2026) preserves the difference and

monotonicity of non-negative stream pairs. Section 5.3 gives a convexity-preserving product identity combining with Chapter 3's multivariate softmax pair.

The gated product rule. For $a = a^+ - a^-$ and $b = b^+ - b^-$ with non-negative streams, as guaranteed by Section 3.1, expand and sort the cross terms by sign:

$$(a^+ - a^-)(b^+ - b^-) \;=\; \underbrace{\big(a^+b^+ + a^-b^-\big)}_{=:\,(ab)^+} \;-\; \underbrace{\big(a^+b^- + a^-b^+\big)}_{=:\,(ab)^-}. \tag{5.1}$$

Both terms are non-negative and coordinatewise non-decreasing, preserving monotone pairs like the sign-sorted weights of Section 3.2.3. Entrywise application gives Hadamard layers; scalar contractions give matrix products. Convexity need not survive products. Streams are finite maxima of *posynomials*: non-negative-coefficient polynomials, closed under addition, multiplication and positive scaling and monotone on the non-negative orthant.

Maxout multiplication networks. We enlarge the architecture class for the splitting theorem. A rank-k Maxout unit (Goodfellow et al., 2013) takes the maximum of k affine forms; ReLU is the $k = 2$ case with one form zero.

Definition 5.1 (Maxout multiplication network). A *rank-k Maxout multiplication network* is a finite directed acyclic graph of *input neurons*, *addition neurons*, and *multiplication neurons*. An addition neuron v carries weight vectors $w^{uv} \in \mathbb{R}^k$ on its incoming arcs and computes the rank-k Maxout activation $a^v = \max_{i\in[k]} \sum_u w_i^{uv} a^u$; a multiplication neuron carries natural exponents $w^{uv} \in \mathbb{N}$ and computes the monomial $a^v = \prod_u (a^u)^{w^{uv}}$. The network is called *monotone* if all weights are non-negative, and *input-convex* if all weights are non-negative except possibly those on arcs leaving input neurons.

This includes linear/convolutional layers, biases via constant inputs, maxima/max-pooling, residuals, Hadamard products and matrix products. Natural exponents avoid division and fractional powers of negative quantities.

Proposition 5.1 (Representation by maxima of posynomials, `Lean_72`). *A monotone rank-k Maxout multiplication network with non-negative intermediate quantities represents the maximum of finitely many posynomials in its non-negative input variables, and is monotone on the non-negative orthant. This representation does not imply ordinary convexity: the positive-coefficient monomial xy is a counterexample. Allowing signed input-incident affine maps likewise supplies no general input-convexity guarantee in the presence of multiplication.*

Proof sketch. Induct in topological order on the non-negative orthant. Coordinate projections are posynomials; non-negative scaling, addition and maxima preserve finite maxima of posynomials. Products of non-negative factors distribute over finite maxima, covering every network operation. Convexity fails for $F(x,y) = xy$: $F(1,1) = 1$ exceeds $\frac{1}{2}F(2,0) + \frac{1}{2}F(0,2) = 0$. □

The splitting theorem. Two identities extend Section 3.2.3's split to this class. Maxout uses the maximum-shift identity

$$\max_{j\in[k]}\big(a_j^+ - a_j^-\big) \;=\; \max_{j\in[k]}\Big\{a_j^+ + \sum_{i\neq j} a_i^-\Big\} \;-\; \sum_{i\in[k]} a_i^-, \tag{5.2}$$

which generalizes both the ReLU splitting identity $\mathrm{ReLU}(a-b) = \max\{a,b\} - b$ of Section 3.2.3 and the gated max-pooling rule of Section 3.7; multiplication neurons are split by the multinomial generalization of (5.1), expanding $\prod_u (a^{u,+} - a^{u,-})^{w^{uv}}$ and sorting the resulting monomials by the parity of the number of $(-)$-stream factors.

Theorem 5.2 (Monotone split of multiplicative ReLU networks, `Lean_73`). *Let N be a (residual) ReLU network containing linear, convolutional, maximum, matrix-product, and Hadamard-product components. Then, on non-negative split-input variables, N has two* monotone *rank-2 Maxout multiplication networks N^+ and N^- satisfying*

$$N(x) = N^+(x^+,x^-) - N^-(x^+,x^-), \qquad x = x^+ - x^-, \quad x^\pm \geq 0,$$

Each component is monotone as a function of its non-negative split inputs and is representable there by a finite maximum of posynomials. This gives a difference-of-monotone representation; it does not establish a difference-of-convex representation in the original inputs. If the number of expanded monomials per multiplication neuron is uniformly bounded, the sizes of the split networks are linear in the size of N.

Proof sketch. Extend Hertrich & Loho (2026)'s argument in topological order. Supply non-negative inputs (x^+,x^-), e.g. $(\mathrm{ReLU}(x),\mathrm{ReLU}(-x))$, with outgoing-weight signs flipped toward the negative copy. Split Maxout weights by sign and use (5.2). For multiplication, create one auxiliary monomial per expanded summand and collect even/odd negative-stream parity into v^+ and v^-, respectively. These identities telescope to $N = N^+ - N^-$; non-negative weights and Proposition 5.1 give monotonicity and representation. Uniformly bounded monomial count gives linear size; bounded fan-in with unbounded exponents does not. Unsplit inputs likewise do not generally restore convexity with multiplication. □

Section 5.3 replaces the sign-sorted product pair to preserve convexity too.

Remark 5.1 (Smooth normalizers: softmax and sigmoid gates). Theorem 5.2 covers projections, bilinear products and residual additions, but excludes softmax. Section 3.2.4 gives exact scalar sigmoid and GELU pairs; Corollary 3.5 covers exponentiation exactly on a fixed bounded score interval. Splitting softmax's coupled denominator requires Theorem 3.11's multivariate row pair, combined next with convex products and LayerNorm.

Conditioned attribution instead fixes the realized softmax matrix A, making $X \mapsto AV(X)$ affine (Ali et al., 2022; Wróbel et al., 2026). The routing-game sections study this conditioned calculation; the full forward decomposition retains A's input dependence.

5.3 A Constructive DC Decomposition of Transformers

Replacing (5.1)'s sign-sorted product by a square identity preserves convexity and coordinatewise monotonicity on non-negative split inputs. With Theorem 3.11's softmax pair and a reciprocal-square-root extension, this yields a constructive DC decomposition of the full deterministic Transformer. This analytical extension is separate from the conditioned attribution implementation evaluated below.

Proposition 5.3 (Convex product pair by polarization, `Lean_74`). *For $A,B,C,D \geq 0$, let*

$$\begin{aligned} P(A,B,C,D) &= \tfrac{1}{2}\big((A+C)^2 + (B+D)^2\big), \\ Q(A,B,C,D) &= \tfrac{1}{2}\big((A+D)^2 + (B+C)^2\big). \end{aligned} \tag{5.3}$$

Then $P - Q = (A - B)(C - D)$. Both functions are convex, non-negative, and coordinatewise non-decreasing on $\mathbb{R}^4_{\geq 0}$. Hence substituting non-negative convex, coordinatewise non-decreasing incoming streams preserves these properties. In particular,

$$(A - B)^2 = 2(A^2 + B^2) - (A + B)^2 \tag{5.4}$$

is a valid convex, monotone square pair on $\mathbb{R}^2_{\geq 0}$.

Proof. Expand for recovery. Squares of affine forms are convex and non-negative; their derivatives on the orthant are non-negative, permitting convex monotone composition. Set $C = A$, $D = B$ for (5.4). □

Monotonicity requires the orthant: $\partial_A (A + C)^2 = 2(A + C)$ can be negative elsewhere. Thus Transformer streams use non-negative doubled inputs, although softmax permits a larger domain. Summing scalar pairs (5.3) splits matrix products.

5.3.1 Layer normalization with positive epsilon

For $\varepsilon > 0$ and $d \geq 1$, LayerNorm on $u \in \mathbb{R}^d$ uses

$$\begin{gathered} \bar{u} = \frac{1}{d}\sum_{j=1}^{d} u_j, \qquad c_i = u_i - \bar{u}, \qquad v = \frac{1}{d}\sum_{j=1}^{d} c_j^2 \geq 0, \\ \mathrm{LN}_\varepsilon(u)_i = \gamma_i c_i (\varepsilon + v)^{-1/2} + \beta_i. \end{gathered} \tag{5.5}$$

Centering, scale and bias are signed affine; variance is a positive sum of squares. Extending the remaining reciprocal square root to negative arguments permits independently varying variance streams.

Proposition 5.4 (Reciprocal-square-root pair, `Lean_75`). *Let $\varepsilon > 0$, put $M = (2\varepsilon^{3/2})^{-1}$, and define*

$$r_\varepsilon(t) = \begin{cases} \varepsilon^{-1/2} - Mt, & t \leq 0, \\ (\varepsilon + t)^{-1/2}, & t \geq 0. \end{cases} \tag{5.6}$$

Then

$$R_+(x,y) = r_\varepsilon(x - y) + Mx, \qquad R_-(x,y) = Mx \tag{5.7}$$

are convex and coordinatewise non-decreasing on $\mathbb{R}^2$, are non-negative on $\mathbb{R}^2_{\geq 0}$, and have difference $r_\varepsilon(x - y)$. Whenever $x - y \geq 0$, this difference is $(\varepsilon + x - y)^{-1/2}$.

Proof. The pieces meet with derivative $-M$; their second derivatives are 0 and $3/(4(\varepsilon + t)^{5/2}) > 0$. Thus r_ε is convex with $-M \leq r'_\varepsilon \leq 0$. Apply Proposition 3.2 to $p(t) = r_\varepsilon(t) + Mt$, $q(t) = Mt$, whose derivatives lie in $[0,M]$ and origin values are $\varepsilon^{-1/2}$,0. Cancellation gives the difference. □

To split (5.5), split affine centering, square each centered pair by (5.4), and average streams to obtain $v = v^+ - v^-$. Apply (5.7): recovered $v \geq 0$ makes its extension exactly the normalization factor. Multiply by each centered pair using polarization, then apply signed scale and bias. These operations preserve non-negative convex monotone streams on the input orthant; $\varepsilon > 0$ covers every input, including constant vectors.

5.3.2 An explicit pre-normalization architecture

Tokens $t \in \{1,\dots,T\}$ have features in $\mathbb{R}^d$, with $T,d \geq 1$. All matrices and biases below are fixed real parameters of either sign.

Definition 5.2 (Finite pre-normalization Transformer, `Lean_76`). A Transformer is specified by a positional bias $P \in \mathbb{R}^{T\times d}$, a finite list of blocks $B_1,\dots,B_K$, an optional final LayerNorm, and a signed affine scalar readout. In block k, first normalize each token,

$$U_t = \mathrm{LN}_{k,1}(X_t).$$

Each head h has positive query/key width $d_{q,h}$ and value width $d_{v,h}$, signed affine projections, a real score scale τ_h, and a fixed nonempty allowed-key set $I_{h,t} \subseteq \{1,\dots,T\}$ for every query token. It computes

$$\begin{aligned} Q_{h,t} &= W_{Q,h}U_t + b_{Q,h}, \qquad K_{h,t} = W_{K,h}U_t + b_{K,h},\\ V_{h,t} &= W_{V,h}U_t + b_{V,h}, \qquad S_{h,ts} = \tau_h\langle Q_{h,t},K_{h,s}\rangle,\\ A_{h,ts} &= \frac{e^{S_{h,ts}}}{\sum_{r\in I_{h,t}} e^{S_{h,tr}}} \quad (s\in I_{h,t}), \qquad O_{h,t} = \sum_{s\in I_{h,t}} A_{h,ts}V_{h,s}. \end{aligned} \tag{5.8}$$

Masked coefficients outside $I_{h,t}$ are zero. The usual scaled dot product is the choice $\tau_h = 1/\sqrt{d_{q,h}}$. Summing the per-head output projections is equivalent to concatenation followed by one output projection. The residual attention and pointwise feedforward updates are

$$\begin{aligned} Y_t &= X_t + \sum_h W_{O,h}O_{h,t} + b_O,\\ B_k(X)_t &= Y_t + W_2\varrho_k(W_1\,\mathrm{LN}_{k,2}(Y_t) + b_1) + b_2. \end{aligned} \tag{5.9}$$

Both LayerNorms have positive epsilon and arbitrary signed learned scale and bias as in (5.5). The activation ϱ_k is chosen from ReLU, exact Gaussian GELU, Swish/SiLU, sigmoid, and softplus, with the positive parameters required in Chapter 3. Head widths, feedforward widths, masks, and parameters may differ between blocks, while the residual width is d.

For any finite depth $K \geq 0$, define

$$\begin{aligned} X^{(0)} &= X + P, \qquad X^{(k)} = B_k(X^{(k-1)}) \quad (1\leq k\leq K),\\ \widehat{X}^{(K)} &= \begin{cases} \mathrm{LN}_{\mathrm{out}}(X^{(K)}), & \text{if a final norm is present,}\\ X^{(K)}, & \text{otherwise,} \end{cases}\\ N(X) &= b_{\mathrm{out}} + \sum_{t=1}^{T}\sum_{j=1}^{d} w_{tj}^{\mathrm{out}}\,\widehat{X}_{tj}^{(K)}. \end{aligned} \tag{5.10}$$

The optional final norm is tokenwise with positive epsilon. Depth zero retains only the positional map, optional norm and readout; zero readout weights outside one token give a class-token logit. Fixed coordinate scalings can represent a fixed dropout mask in this deterministic inference computation.

5.3.3 The recursive split algorithm

Expand Definition 5.2 into scalar expressions built from inputs, constants, sums, signed multiples, products, supported activations, softmax and LayerNorm coordinates. Their ordinary evaluation is (5.8)–(5.10), with no assumed convexity certificate.

Write $\mathscr{C}(e) = (e^+,e^-)$ for the recursively constructed pair, as a function of independent non-negative input streams (x,y). Pair addition is componentwise, and signed scalar multiplication is

$$\mathrm{Scale}_c(p,q) = \begin{cases} (cp,cq), & c \geq 0, \\ (-cq,-cp), & c < 0. \end{cases} \tag{5.11}$$

Use product pair (5.3), the activation pairs of Section 3.2.4, softmax Theorem 3.11, and the LayerNorm recipe above, retaining its input-dependent statistics.

Algorithm 5.1: Recursive DC evaluation of a finite Transformer expression

Input: A scalar output expression e; independent split inputs $x,y \geq 0$.
Output: $\mathscr{C}(e)(x,y)$ recovering the ordinary output at $x - y$.
Return (x_i,y_i) for input X_i and $(\max\{c,0\}, \max\{-c,0\})$ for constant c.
Otherwise compile all children recursively. Add pairs for sums; use Scale_c for signed multiples and (5.3) for products.
For an activation, softmax coordinate, or LayerNorm coordinate, apply its explicit pair to the compiled arguments (allowed scores for softmax; features with positive epsilon and signed scale/bias for LayerNorm).
Return the pair at the signed affine readout.

Translation follows the finite block list: the empty list preserves its input, and each block substitutes current token expressions into its formulas. This real-arithmetic recursion permits shared-expression reuse and requires no expansion or runtime bound for correctness.

5.3.4 The full Transformer construction

Theorem 5.5 (Exact DC decomposition of a deterministic Transformer, `Lean_77`). *Let $N\colon \mathbb{R}^{T\times d} \to \mathbb{R}$ be the Transformer of Definition 5.2, with any finite list of blocks, including the empty list, and identify its token-feature input with $\mathbb{R}^m$, where $m = Td$. Algorithm 5.1, applied to its output expression, terminates and exactly decomposes this forward map. It constructs functions $N_+,N_-\colon \mathbb{R}^{2m}_{\geq 0} \to \mathbb{R}_{\geq 0}$, convex and coordinatewise non-decreasing on that orthant, such that*

$$N(x-y) = N_+(x,y) - N_-(x,y) \qquad (x,y \in \mathbb{R}^m_{\geq 0}). \tag{5.12}$$

In particular, the functions

$$g(u) = N_+([u]^+,[u]^-), \qquad h(u) = N_-([u]^+,[u]^-) \tag{5.13}$$

are convex on all of $\mathbb{R}^m$, are non-negative, and satisfy $N = g - h$. The functions N_+,N_- are coordinatewise non-decreasing in the independent split inputs; g,h need not be coordinatewise non-decreasing in the original signed variable u.

Proof. The finite block list yields finite syntax, so compilation terminates. Structural induction gives exact recovery and non-negative convex, coordinatewise non-decreasing heads: projections and split constants start the induction; sums and signed scaling preserve it, swapping heads for negative scalars. Products use Proposition 5.3; activations use their scalar pairs; softmax uses Theorem 3.11 on nonempty allowed keys; LayerNorm follows its recipe with non-negative variance and positive epsilon. The convex monotone composition argument is the one in Theorem 3.3.

Block-list induction identifies ordinary evaluation with (5.10), including depth zero and the ungated signed affine readout, proving (5.12). Finally, the convex components $[u]^{\pm}$ take values in the orthant, so the same composition rule gives convex g,h with difference $N(u)$. □

This real-arithmetic DC representation retains the input dependence of Queries, Keys, Values, softmax and LayerNorm statistics. It guarantees neither stable floating-point evaluation nor attribution quality or practical runtime: $M = (2\varepsilon^{3/2})^{-1}$ and repeated products can amplify common stream mass and cause cancellation. Section A.4.1 documents the unstabilized float64 failures in a post-normalization block, whose placement differs from Definition 5.2. Experiments below evaluate conditioned attribution, not this construction.

5.4 Attention via a detached-logit oracle

We now derive the conditioned Value-routing rule used in the ViT experiments. Its LayerNorm rule fixes both mean and scale, giving a pointwise affine map; CP-LRP and DAVE retain linear centering while fixing the scale (Ali et al., 2022; Wróbel et al., 2026). The realized softmax row supplies a detached reference policy, and explanatory trajectories follow only the Value route. Theorem 5.8 identifies their signed discounted occupations with composite $\alpha\beta$-LRP. This attribution question conditions on attention, LayerNorm statistics and GELU gates; the full DC construction above retains their input dependence.

5.4.1 The conditioned map and target rule

Consider one head with output/query token t, key token $k \in \{1,\dots,S\}$, feature $d \in \{1,\dots,d_h\}$, and input embedding coordinate $e \in \{1,\dots,D\}$. For signed inputs X, shared per-token weights W_v and bias b_v, and detached row-stochastic attention $A \geq 0$,

$$v_{k,d} = \sum_e W_{v,e,d} X_{k,e} + b_{v,d}, \tag{5.14}$$

$$O_{t,d} = \sum_k A_{t,k} v_{k,d} = \sum_{k,e} A_{t,k} W_{v,e,d} X_{k,e} + b_{v,d}. \tag{5.15}$$

Mixing preserves feature d. In ViT-B/16, $v = XW_v$ uses one $D \times D$ projection reshaped into n_h heads, with $W_v^{(h)} \in \mathbb{R}^{D \times d_h}$ its head-h column block. Each $X_{k,e}$ feeds every head; per-head input relevances sum there.

The composite weights $W^{\mathrm{eff}}_{(k,e),(t,d)} = A_{t,k} W_{v,e,d}$ inherit the projection signs. For $\kappa \in \{+,-\}$, define the projection-stream and output masses

$$[W^{\mathrm{eff}}_{(k,e),(t,d)}]^{\kappa} = A_{t,k} W^{\kappa}_{v,e,d}, \tag{5.16}$$

$$\tilde{v}^{\kappa}_{k,d} = \sum_e [X_{k,e} W_{v,e,d}]^{\kappa} + [b_{v,d}]^{\kappa}, \qquad Z^{\kappa}_{t,d} = \sum_k A_{t,k} \tilde{v}^{\kappa}_{k,d} = \sum_{k,e} A_{t,k} [X_{k,e} W_{v,e,d}]^{\kappa} + [b_{v,d}]^{\kappa}. \tag{5.17}$$

These streams retain predecessor signs; netting $v_{k,d}$ before taking its positive or negative part agrees only for single-signed predecessors. Composite $\alpha\beta$-LRP applies (α,β) once to this split. Sequential $\alpha\beta$-LRP on $X \to v \to O$ instead splits netted Values and applies the factors twice, giving a different rule for mixed-sign predecessors. All reported ViT results use the composite rule. Conditioning

resembles CP-LRP (Ali et al., 2022) and DAVE (Wróbel et al., 2026); AttnLRP (Achtibat et al., 2024) also attributes the softmax and QK paths.

Zero mass and stabilization. Every contribution ratio uses

$$\rho(u,D) = \begin{cases} u/D, & D > 0, \\ 0, & D = 0, \end{cases} \qquad 0 \le u \le D. \tag{5.18}$$

A zero-mass state routes probability one to the cemetery. Zero $Z^{\kappa}_{t,d}$ forces every $A_{t,k}\tilde{v}^{\kappa}_{k,d}$ to vanish; zero $\tilde{v}^{\kappa}_{k,d}$ forces all matching input and bias contributions to vanish. The formulas below use $\varepsilon = 0$. The positive stabilizer of §4.4.3 transfers to the composite block by replacing $Z^{\kappa}_{t,d}$ with $Z^{\kappa}_{t,d} + \varepsilon$ and absorbing the remaining share into the outside option.

Target attribution. For incoming relevance R^O, the composite rule (4.1) factors through Value entries as

$$R^{\mathrm{val},\kappa}_{k,d} = \sum_t \frac{A_{t,k}\tilde{v}^{\kappa}_{k,d}}{Z^{\kappa}_{t,d}} R^O_{t,d}, \qquad R^{\mathrm{val}}_{k,d} = \alpha R^{\mathrm{val},+}_{k,d} - \beta R^{\mathrm{val},-}_{k,d}, \tag{5.19}$$

$$R^X_{k,e} = \sum_d \left(\alpha \frac{[X_{k,e}W_{v,e,d}]^+}{\tilde{v}^+_{k,d}} R^{\mathrm{val},+}_{k,d} - \beta \frac{[X_{k,e}W_{v,e,d}]^-}{\tilde{v}^-_{k,d}} R^{\mathrm{val},-}_{k,d} \right) \tag{5.20}$$

$$= \sum_{t,d} \left(\alpha \frac{A_{t,k}[X_{k,e}W_{v,e,d}]^+}{Z^+_{t,d}} - \beta \frac{A_{t,k}[X_{k,e}W_{v,e,d}]^-}{Z^-_{t,d}} \right) R^O_{t,d}. \tag{5.21}$$

The bias share is absorbed. Substitution cancels positive intermediate masses; when an intermediate or output mass is zero, both forms vanish by (5.18). Thus the factorization holds in every case.

5.4.2 KL routing and the attention game

Lemma 5.6 (Legendre–Fenchel transform of KL divergence, `Lean_78`). *For a finite set $\mathscr{X}$, full-support reference $\bar{\pi} \in \Delta(\mathscr{X})$ and real payoff r, let $Z = \sum_x \bar{\pi}(x)e^{r(x)}$. Then*

$$F(r;\bar{\pi}) = \max_{\pi \in \Delta(\mathscr{X})} \{\mathbb{E}_\pi[r] - \mathrm{KL}(\pi\|\bar{\pi})\}, \tag{5.22}$$

$$\pi^\star_r(x) = \frac{\bar{\pi}(x)e^{r(x)}}{Z}, \qquad F(r;\bar{\pi}) = \log Z. \tag{5.23}$$

Proof. The Gibbs identity (Boyd and Vandenberghe, 2004), $\mathbb{E}_\pi[r] - \mathrm{KL}(\pi\|\bar{\pi}) = \log Z - \mathrm{KL}(\pi\|\pi^\star_r)$, is maximized exactly at $\pi = \pi^\star_r$. □

For $u \ge 0$, $r = \ell(u)$ and positive weighted mass, the identity extends to

$$F(\ell(u);\bar{\pi}) = \ell\left(\sum_x \bar{\pi}(x)u(x)\right), \qquad \pi^\star_{\ell(u)}(x) = \frac{\bar{\pi}(x)u(x)}{\sum_{x'} \bar{\pi}(x')u(x')}. \tag{5.24}$$

Indeed, $\exp(-\infty) = 0$, and policies charging zero-u actions have payoff $-\infty$; the Gibbs identity applies on positive-mass support. For a reference with zeros, restrict admissible policies to its nonempty

support. A uniform reference gives $\mathrm{KL}(\pi\|\mathrm{Unif}_{\mathscr{X}}) = \log|\mathscr{X}| - \mathscr{H}(\pi)$, recovering (4.19)'s entropy bonus and Gibbs optimizer, with the value shifted by $-\log|\mathscr{X}|$.

State space. Use four state families in the notation $s^{(\mathrm{att,Side,type})}$ of §4.4: output activations O_{tdp}, output sign branches $L_{tdp\kappa}$, Value-projection linear states $V_{kdp\kappa}$, and input activations X_{kep}. Their respective (Side,type) are (O,act), (O,lin), (val,lin), and (X,act). The surrounding recursion enters at O and resumes at X. Player $p \in \{+,-\}$ is retained except at the sole sign split:

1) From O_{tdp}, an unobserved oracle chooses L_{tdp+} or $L_{tdp'-}$ with probability 1/2, discounts 2α or 2β, and p' the opponent.
2) From $L_{tdp\kappa}$, the active player chooses key k and moves to $V_{kdp\kappa}$ using the KL policy below; p,κ,d are preserved.
3) From $V_{kdp\kappa}$, the linear-state Gibbs rule routes to X_{kep} or the absorbed bias node. This preserves p and drops d.

The last two transitions have discount 1. The Value-projection state has active-player value $\ell(\tilde{v}^{\kappa}_{k,d})$.

Proposition 5.7 (Value-side optimal policy, `Lean_79`). *At $L_{tdp\kappa}$, use reference $\bar{\pi}_t = A_{t,\cdot}$, shared across features, and objective*

$$V_p(L_{tdp\kappa}) = \max_{\pi \in \Delta(\operatorname{supp}\bar{\pi}_t)} \left\{ \sum_k \pi(k)\ell(\tilde{v}^{\kappa}_{k,d}) - \mathrm{KL}(\pi\|\bar{\pi}_t) \right\}. \tag{5.25}$$

If $Z^{\kappa}_{t,d} > 0$, the unique maximizing policy and subsequent projection policy are

$$\pi^{\star}_{t,d,\kappa}(k) = \frac{\bar{\pi}_t(k)\tilde{v}^{\kappa}_{k,d}}{Z^{\kappa}_{t,d}}, \qquad \kappa \in \{+,-\}, \tag{5.26}$$

$$\pi^{\star}_{\mathrm{val},k,d,\kappa}(e) = \frac{[X_{k,e}W_{v,e,d}]^{\kappa}}{\tilde{v}^{\kappa}_{k,d}}, \quad \textit{bias share } \frac{[b_{v,d}]^{\kappa}}{\tilde{v}^{\kappa}_{k,d}}. \tag{5.27}$$

The active-player value at L is $\ell(Z^{\kappa}_{t,d})$; the inactive-player value is $-\infty$. At $Z^{\kappa}_{t,d} = 0$, every admissible key policy has value $-\infty$; the game uses cemetery completion. Projection states with zero mass use the same completion.

Proof. Apply (5.24) with $u(k) = \tilde{v}^{\kappa}_{k,d}$. Its payoff is the predecessor Value-state value; the reference weight gives the composite log-payoff $\ell(A_{t,k}\tilde{v}^{\kappa}_{k,d})$. Definition 4.7 gives the projection Gibbs rule and inactive-player value, as in (4.20). If $Z = 0$, every weighted mass vanishes, giving value $\ell(0) = -\infty$. □

Together with the surrounding equilibrium, these local maximizing policies form an SPNE by backward induction.

5.4.3 Occupation-measure recovery

For the discounted occupations Γ_x of §4.4.3, define player differences

$$\Gamma^{O}_{t,d} = \Gamma_x(O_{td+}) - \Gamma_x(O_{td-}), \tag{5.28}$$

$$\Gamma^{X}_{k,e} = \Gamma_x(X_{ke+}) - \Gamma_x(X_{ke-}). \tag{5.29}$$

These are the attention entry and exit instances of (4.32).

Theorem 5.8 ($\alpha\beta$-LRP recovery for attention, Lean_80). *For signed inputs X, the preceding SPNE with reference $A_{t,\cdot}$ and discounts (4.31) applied only at the sign oracle satisfies*

$$\Gamma^X_{k,e} = \sum_{t,d} \left(\alpha \frac{A_{t,k}[X_{k,e}W_{v,e,d}]^+}{Z^+_{t,d}} - \beta \frac{A_{t,k}[X_{k,e}W_{v,e,d}]^-}{Z^-_{t,d}} \right) \Gamma^O_{t,d}. \tag{5.30}$$

The surrounding recursion supplies $\Gamma^O = R^O$ for the implementation's unit seed, hence $\Gamma^X = R^X$ by (5.21). Aggregating input and bias contributions for each Value entry also gives

$$\Gamma^{\mathrm{val}}_{k,d} = R^{\mathrm{val}}_{k,d} \quad \textit{as defined in (5.19).} \tag{5.31}$$

Every real seed r_u scales these occupation differences by r_u, including the signed affine-logit seed of Section 4.5.

Proof. For each (t,d,k,e), the two routing probabilities multiply to $A_{t,k}[X_{k,e}W_{v,e,d}]^\kappa / Z^\kappa_{t,d}$: positive $\tilde{v}$ cancels; zero $\tilde{v}$ or Z makes both sides zero. The sign oracle's probability $1/2$ cancels the factor 2 in its discounts, retaining the player on $+$ and switching it on $-$. Thus

$$\Gamma_x(X_{ker}) = \sum_{t,d} \left[\alpha \frac{A_{t,k}[X_{k,e}W_{v,e,d}]^+}{Z^+_{t,d}} \Gamma_x(O_{tdr}) + \beta \frac{A_{t,k}[X_{k,e}W_{v,e,d}]^-}{Z^-_{t,d}} \Gamma_x(O_{tdr'}) \right]. \tag{5.32}$$

Subtracting the $r = -$ recursion from $r = +$ gives (5.30); with $\Gamma^O = R^O$, this is the composite target rule. Summing matching input and bias masses gives $\tilde{v}^\kappa_{k,d}$, proving the Value-entry equality. Linearity gives the arbitrary-seed claim. □

5.4.4 The reference as the policy of a QK oracle player

The cached reference $A_{t,\cdot}$ suffices for Value routing. Optionally, interpret it as the entropy-regularized best response to detached logits $e_{tk} = Q_t K_k^\top / \sqrt{d_h}$. At temperature 1, (4.19) gives

$$\begin{aligned} A_{t,\cdot} &= \mathrm{argmax}_{\pi \in \Delta(\{1,\dots,S\})} \left\{ \sum_k \pi(k) e_{tk} + \mathscr{H}(\pi) \right\}, \\ \max_\pi \left\{ \sum_k \pi(k) e_{tk} + \mathscr{H}(\pi) \right\} &= \log \sum_k e^{e_{tk}}. \end{aligned} \tag{5.33}$$

For masked attention, use the nonempty allowed-key set. This oracle acts only in the forward pass and supplies $\bar{\pi}_t = A_{t,\cdot}$; its states carry no occupation mass, so relevance does not traverse Q or K.

5.4.5 Risk-Averse Attention via ADF

With ADF variance propagation, the implementation uses $s_{tk} = \hat{\sigma}_k$, the square root of the feature-averaged variance of token k's query projection. This key-dependent score is a proxy, not the standard deviation of e_{tk}. Risk-averse attention tilts toward smaller proxy scores:

$$\tilde{e}_{tk} = e_{tk} + \lambda_{\mathrm{sm}} s_{tk}, \qquad \lambda_{\mathrm{sm}} \le 0, \tag{5.34}$$

$$\tilde{\pi}_t(k) = \frac{\exp(\tilde{e}_{tk})}{\sum_{k'} \exp(\tilde{e}_{tk'})} \propto A_{t,k} \exp(\lambda_{\mathrm{sm}} s_{tk}). \tag{5.35}$$

More-negative values penalize high scores more strongly: $\sum_k \tilde{\pi}_t(k) s_{tk}$ is non-decreasing in λ_{sm}, while entropy may rise or fall. The AV-backward hook shifts cached pre-softmax logits, leaving predictions unchanged. Unlike activation-side λ, which lowers values by $|\lambda|\sigma$, this risk aversion changes routing rather than utility curvature.

5.4.6 Implementation details

The RG on ViT-B/16 implements the composite rule with bias and stabilizer absorption. A full-gradient attention fallback reduces localization from 0.711 to 0.624 at the localization-selected configuration. Four operator conventions specify the conditioned pass:

- *LayerNorm detachment.* Standard gradients through data-dependent mean $m(x)$ and scale $s(x)$ couple features within each token and can violate LRP conservation (Ali et al., 2022). With $c = x.\text{detach}()$, use the pointwise affine map $x \mapsto w_{\text{LN}} \odot (x - m(c))/s(c) + b_{\text{LN}}$. Both mean and scale are fixed; CP-LRP and DAVE fix only the scale (Wróbel et al., 2026).
- *GELU gate detachment.* Use $x\Phi(c)$, with $c = x.\text{detach}()$ and standard normal CDF Φ. Its backward scaling by constant $\Phi(c)$ realizes the elementwise LRP identity rule in modified Gradient×Input implementations (Arras et al., 2025, Appendix D.3).
- *Softmax detachment.* Cache the realized row $A_{t,\cdot}$ as the fixed KL reference supplied by the optional QK oracle.
- *QK gradient detachment.* Softmax and QK-matmul hooks return zero. Value routing uses (5.26) and (5.27), applying (α,β) once per block. The released code's sole rule, `av_rule='merged'`, is checked against an independent reference for (5.21) by Section A.3.4's identity tests.

5.4.7 Equivariant Spatial Averaging

The 16×16 patch tokenization breaks translation equivariance. Following DAVE (Wróbel et al., 2026), RG+Eq explains transformed inputs, maps explanations back, and averages the aligned maps to suppress tokenization-grid artifacts:

$$\mathscr{M}_N(x) = \frac{1}{N} \sum_{n=1}^{N} T_n^{-1}[\mathscr{M}(T_n x)]. \tag{5.36}$$

Here $\mathscr{M}$ is the RG attribution, $N = 50$, and T_n are i.i.d. rotations (uniform in $\pm 10°$), translations (up to $\pm 5\%$ per image axis), and horizontal flips (probability 0.5), without input noise. The conditioned pass separates operator variation from the effective transformation, following DAVE's argument. These transforms do not form a group, so this Monte-Carlo Reynolds smoothing has no exactness guarantee.

6 Saliency and Attribution Methods

This chapter develops interpretability methods from the streams g,h with $f = g - h$ constructed in Section 3.2.3. Section 6.1 reads the streams separately to explain pretrained networks through SplitCAM, SplitLRP and SplitGrad (Zimmermann and Loho, 2026). It also studies directly trained difference-of-convex input neural networks (DCINNs), whose MNIST examples exhibit class-dependent stream sensitivities. Section 6.2 collects practical *modes* that turn the game equilibria of Chapter 4 into tunable attribution maps, and the *technicalities* of equivariant averaging that improve attribution quality.

6.1 Split-Based and Difference-of-Convex Attribution

Post-hoc attribution assigns each input feature a score for its influence on a fixed network output. Established families include gradient saliency (Simonyan et al., 2014), deconvolution and guided backpropagation (Springenberg et al., 2015; Zeiler and Fergus, 2014), LRP and deep Taylor decomposition (Bach et al., 2015; Montavon et al., 2017), class-activation mapping (Selvaraju et al., 2017; Zhou et al., 2016), baseline-based methods such as Integrated Gradients and DeepLIFT (Shrikumar et al., 2017; Sundararajan et al., 2017), noise-averaged gradients (Smilkov et al., 2017), and perturbation- or surrogate-based explanations (Fel et al., 2023; Fong and Vedaldi, 2017; Kolek et al., 2023; Lundberg and Lee, 2017; Ribeiro et al., 2016). These explain the original network. We identify signal loss in rules using ordinary local derivatives, then develop SplitCAM, SplitLRP and SplitGrad from the separate signals preserved by the split-pair of Section 3.2.3, closing with by-design DCINNs.

PRCA and DRSA optimize relevance over activation subspaces; their virtual layer preserves the decision function when a full orthogonal basis is retained (Chormai et al., 2024). Our split representation separates contributions using the original network's signs and gates.

Consider a feedforward ReLU network with layers $1,\dots,L$, weights $W^{(l)}$, pre-activations $z^{(l)} = W^{(l)}a^{(l-1)}$, hidden activations $a^{(l)} = \mathrm{ReLU}(z^{(l)})$ for $l < L$, and ungated readout $a^{(L)} = z^{(L)}$. We attribute a fixed scalar class logit: $f^{(l)}\colon \mathbb{R}^{d^{(l)}} \to \mathbb{R}$ denotes layers $l{+}1,\dots,L$ acting on layer-l activations. Biases are absorbed as in Section 3.2.3. The split streams $g^{(l)}$,$h^{(l)}\colon \mathbb{R}^{d^{(l)}} \times \mathbb{R}^{d^{(l)}} \to \mathbb{R}$ act on $a^{(l,+)}$,$a^{(l,-)}$ and satisfy

$$f^{(l)}\big(a^+ - a^-\big) = g^{(l)}(a^+, a^-) - h^{(l)}(a^+, a^-) \qquad \text{for all } a^+, a^- \in \mathbb{R}^{d^{(l)}}, \tag{6.1}$$

and we write $g \coloneqq g^{(0)}$, $h \coloneqq h^{(0)}$ with balanced input split $a^{(0,\pm)} = \pm x/2$. Derivatives are row vectors (scalar Jacobians); derivative statements hold at differentiability points, hence Lebesgue-almost everywhere because all functions involved are CPWL.

A ReLU gate transmits positive pre-activations and otherwise returns zero, erasing their negative magnitude and blocking the local gradient. This *subthreshold signal loss* hides the suppressed pre-activation from the output through that gate and from backward rules retaining its ordinary derivative. Ordinary input gradients can therefore exhibit exact regional blindness.

Proposition 6.1 (Regional blindness of the input gradient, `Lean_67`**).** *Let f be as above and x_0 an input with no zero pre-activations. If every path from input coordinate i to the output passes through a neuron inactive at x_0, then $\partial f/\partial x_i \equiv 0$ on some open neighborhood $U \ni x_0$, even if f depends on x_i elsewhere.*

Proof. Set $M^{(l,+)} := \operatorname{diag}\big(\mathbf{1}\{z^{(l)} > 0\}\big)$, $M^{(l,-)} := \operatorname{diag}\big(\mathbf{1}\{z^{(l)} \le 0\}\big)$ and $W^{(l,x)} := M^{(l,+)}W^{(l)}$ for hidden layers $1 \le l < L$. The affine logit readout is ungated and may be negative, so $W^{(L,x)} := W^{(L)} = \partial f/\partial a^{(L-1)}$. Continuity and nonzero hidden pre-activations at x_0 give an open $U \ni x_0$ with constant masks and affine f. Iterating the chain rule on U yields

$$\frac{\partial f}{\partial x} = W^{(L,x)}\, W^{(L-1,x)} \cdots W^{(1,x)}, \tag{6.2}$$

and its i-th entry sums weight products over paths from input i to the output, multiplied by indicators that every hidden neuron on the path is active; the output carries no mask. Every path contains an inactive hidden neuron, so every summand vanishes throughout U. □

The phenomenon already occurs in a single-hidden-neuron network.

Remark 6.1 (The canonical counterexample). Following Sundararajan et al. (2017), consider the one-dimensional ReLU network

$$f(x) = 1 - \mathrm{ReLU}(1 - x), \tag{6.3}$$

and compare input $x = 2$ with baseline $x' = 0$. The prediction changes by $f(2) - f(0) = 1$, making the sole feature relevant. Yet f is constant on $x \ge 1$: at $x = 2$, gradients, Input×Gradient and rules retaining local ReLU derivatives assign zero attribution. The hidden neuron is inactive, so Proposition 6.1 applies with $U = (1,\infty)$.

Sundararajan et al. (2017) axiomatize this requirement.

Definition 6.1 (Sensitivity(a)). An attribution method A, mapping an input x and a baseline x' to feature scores $A_i(x, x') \in \mathbb{R}$, satisfies *Sensitivity* if for every pair (x, x') that differs in exactly one feature i and satisfies $f(x) \ne f(x')$, the attribution of that feature is non-zero, $A_i(x, x') \ne 0$.

Plain gradients violate Sensitivity (Remark 6.1). Integrated Gradients restores it by integrating the gradient along the segment from x' to x, requiring many forward–backward passes and inheriting the baseline choice. The split-pair instead preserves subthreshold signals in the *representation* used for attribution.

Recall from Section 3.2.3 how the split-pair processes a ReLU gate: the pre-activation pair $z^{(l,+)}, z^{(l,-)}$ is mapped to

$$a^{(l,+)} = \max\big\{z^{(l,+)}, z^{(l,-)}\big\}, \qquad a^{(l,-)} = z^{(l,-)}, \tag{6.4}$$

a rank-2 Maxout unit in the g-stream and identity in the h-stream. Their difference reproduces $\mathrm{ReLU}(z^{(l)})$. At an inactive neuron, both activations equal $z^{(l,-)}$: the suppressed signal's magnitude remains in their common level. The backward pass likewise retains subthreshold information.

Definition 6.2 (Local sensitivities). Use the row sensitivities $\delta^{(l,\sigma,s)}$ of (3.52), for $s \in \{g,h\}$, $\sigma \in \{+,-\}$ and $0 \le l \le L$, and write $\delta^{(l)} := \partial f^{(l)}/\partial a^{(l)}$. Abbreviate the absolute-weight product (3.50) by $W^{[l]}_{\mathrm{abs}}$, with $W^{[L]}_{\mathrm{abs}} = \mathbf{1}^\top$. For our scalar output, its i-th entry sums absolute weight products over all paths from layer-l neuron i to the output, active or inactive.

The four local sensitivities obey coupled backward recursions, the split analogue of backpropagation underlying this chapter's methods.

Lemma 6.2 (Split backward recursions, Lean_68). *With the activation masks $M^{(l,\pm)}$ of Proposition 6.1, extended by $M^{(L,+)} = I$ and $M^{(L,-)} = 0$ for the affine readout, for every $0 \le l < L$ and $s \in \{g, h\}$,*

$$\delta^{(l,+,s)} = \delta^{(l+1,+,s)}\big(M^{(l+1,+)}W^{(l+1,+)} + M^{(l+1,-)}W^{(l+1,-)}\big) + \delta^{(l+1,-,s)}\,W^{(l+1,-)}, \tag{6.5}$$

$$\delta^{(l,-,s)} = \delta^{(l+1,+,s)}\big(M^{(l+1,+)}W^{(l+1,-)} + M^{(l+1,-)}W^{(l+1,+)}\big) + \delta^{(l+1,-,s)}\,W^{(l+1,+)}, \tag{6.6}$$

with base case $\delta^{(L,+,g)} = \delta^{(L,-,h)} = \mathbf{1}^\top$ and $\delta^{(L,-,g)} = \delta^{(L,+,h)} = 0$.

The block Jacobians (3.54) give these recursions: the maximum selects $z^{(l+1,+)}$ exactly at active neurons. Adding the recursions eliminates the complementary masks.

Proposition 6.3 (Sensitivity mass conservation, Lean_69). *For every $0 \le l \le L$ and both streams $s \in \{g, h\}$,*

$$\delta^{(l,+,s)} + \delta^{(l,-,s)} = W_{\text{abs}}^{[l]}. \tag{6.7}$$

Proof. Write $S^{(l)} = \delta^{(l,+,s)} + \delta^{(l,-,s)}$. Adding (6.5) and (6.6) gives $S^{(l)} = S^{(l+1)}|W^{(l+1)}|$, since $M^{(l+1,+)} + M^{(l+1,-)} = I$ and $W^{(l+1,+)} + W^{(l+1,-)} = |W^{(l+1)}|$. Backward induction from $S^{(L)} = \mathbf{1}^\top$ yields $S^{(l)} = W_{\text{abs}}^{[l]}$. □

Differentiating split correctness recovers the original gradient.

Corollary 6.4 (Backward invariant, Lean_70). *For every $0 \le l \le L$,*

$$\delta^{(l,+,g)} - \delta^{(l,+,h)} = \delta^{(l)} = -\big(\delta^{(l,-,g)} - \delta^{(l,-,h)}\big), \quad \textit{hence} \quad \big(\delta^{(l,+,g)} - \delta^{(l,-,g)}\big) - \big(\delta^{(l,+,h)} - \delta^{(l,-,h)}\big) = 2\,\delta^{(l)}. \tag{6.8}$$

Proof. Differentiate the correctness identity (6.1) with respect to a^+ and a^-, respectively, and subtract the two resulting identities. □

For the combined gradients (3.57), Corollary 6.4 gives $\delta^{(l,g)} - \delta^{(l,h)} = \delta^{(l)}$: regional blindness means equal combined stream gradients, possibly both zero. Partial sensitivities can differ from $\frac{1}{2}W_{\text{abs}}^{[l]}$, recording routing at active and inactive ReLU gates. Proposition 6.3 guarantees a positive partial sensitivity per head for each feature connected to the output by a nonzero-weight path, without guaranteeing nonzero partial differences, combined gradients or attribution. For example, with $f(x) = \text{ReLU}(\text{ReLU}(x) + \text{ReLU}(-x) - 10)$ and balanced input split, both heads equal 10 locally near $x = 1$, and both combined gradients vanish.

Remark 6.2 (The counterexample, revisited). For (6.3) with balanced split $a^{(0,\pm)} = \pm x/2$, Section 3.2.3 gives $g(x) = x/2 + 1$, $h(x) = x/2$ and $g(x) - h(x) = 1 = f(x)$ for $x > 1$. At $x = 2$ both stream gradients equal $\frac{1}{2}$, while $f'(2) = \frac{1}{2} - \frac{1}{2} = 0$. Thus the heads supply a nonzero signal consistent with $f(2) \neq f(0)$, even where their difference is constant. This example with $x' = 0$ does not establish Definition 6.1 for arbitrary attribution rules or baselines. The following methods turn the pair-valued signal into scalar maps; the shifted combinations of Section 6.1.3 avoid exact cancellation here and assign positive attribution for every $\alpha < \frac{1}{2}$.

Non-negative weights let each neuron only *add* evidence, a structure long associated with interpretability (Ayinde and Zurada, 2018; Lee and Seung, 1999). The split-pair gives an arbitrary pretrained network this structure by doubling its width; the following methods exploit its signal flow.

6.1.1 SplitCAM

CAM methods explain CNN decisions by combining spatial activation maps of a convolutional layer with class-specific weighting coefficients. The original CAM (Zhou et al., 2016) requires a global-average-pooling head; Grad-CAM (Selvaraju et al., 2017) extends it to CNNs without that head requirement by weighting each channel with the spatially pooled gradient of the class logit, and Grad-CAM++ (Chattopadhay et al., 2018) refines the pooling weights. LayerCAM (Jiang et al., 2021) drops the pooling altogether and weights every spatial location individually: for a layer l with channel activation maps $a_c^{(l)}$ and class logit y,

$$\mathrm{LayerCAM}^{(l)} = \mathrm{ReLU}\Big(\sum_c \mathrm{ReLU}\Big(\frac{\partial y}{\partial a_c^{(l)}}\Big) \odot a_c^{(l)}\Big), \tag{6.9}$$

where $\odot$ denotes the elementwise product and the sum runs over the channels. This makes LayerCAM applicable to early layers with high spatial resolution. Inactive neurons contribute no activation-weighted term, and the positive-gradient weighting discards negative gradient evidence.

SplitCAM applies LayerCAM's spatial activation weighting to signed split-stream maps. Raw sensitivities sum *absolute* path products (Proposition 6.3) and become numerically enormous in deep split-pairs, requiring the stabilized sensitivities of Section 3.6.2.

Definition 6.3 (Shifted local sensitivities, `Lean_71`). Fix shift parameters $\alpha^{(l)} \in [0, \frac{1}{2}]$. For $s \in \{g, h\}$ define, backward from layer L, intermediate sensitivities $\delta_{\mathrm{int}}^{(l,\pm,s)}$ by evaluating the recursions (6.5)–(6.6) (respectively the base case of Lemma 6.2 at $l = L$) with the already-shifted sensitivities of the subsequent layer, and set

$$\delta_{\mathrm{shift}}^{(l,\pm,s)} := \delta_{\mathrm{int}}^{(l,\pm,s)} - \alpha^{(l)}\big(\delta_{\mathrm{int}}^{(l,+,s)} + \delta_{\mathrm{int}}^{(l,-,s)}\big), \qquad \delta_{\mathrm{shift}}^{(l,s)} := \frac{\delta_{\mathrm{shift}}^{(l,+,s)} - \delta_{\mathrm{shift}}^{(l,-,s)}}{2}. \tag{6.10}$$

The common shift preserves sensitivity differences. By (3.59), its intermediate bulk is $\delta_{\mathrm{int}}^{(l,+,s)} + \delta_{\mathrm{int}}^{(l,-,s)} = \big[\prod_{i=l+1}^{L}(1 - 2\alpha^{(i)})\big] W_{\mathrm{abs}}^{[l]}$, with empty product 1; the factor $\frac{1}{2}$ matches the balanced input split (3.57). We use $\alpha^{(l)} \equiv \alpha$ unless stated otherwise. Theorem 3.24 gives raw sensitivities at $\alpha = 0$, multiples of the original gradient at $\alpha = \frac{1}{2}$, and controlled layer-dependent damping of the split signal for $0 < \alpha < \frac{1}{2}$.

The SplitCAM attribution.

Definition 6.4 (SplitCAM). For a layer l of the decomposed model, a stream $s \in \{g, h\}$ and a sign $\sigma \in \{+, -\}$, the SplitCAM attribution maps are

$$\mathrm{SplitCAM}^{(l,\sigma,s)} := \sum_c \delta_{\mathrm{shift}}^{(l,\sigma,s)}\big|_c \odot a_c^{(l,\sigma)}, \tag{6.11}$$

$$\mathrm{SplitCAM}^{(l,s)} := \sum_c \delta_{\mathrm{shift}}^{(l,s)}\big|_c \odot a_c^{(l)}, \tag{6.12}$$

where $a_c^{(l,\sigma)}$ denotes the channel-c activation map of the σ-stream, $a_c^{(l)} = a_c^{(l,+)} - a_c^{(l,-)}$ the reconstructed original activation map, and $\cdot|_c$ the restriction of a sensitivity tensor to channel c.

SplitCAM *omits positive-gradient clipping and the outer ReLU*, retaining signed shifted sensitivities. It uses shifted sensitivities rather than raw derivatives: (6.10) changes each map only by multiples of the pattern-independent $W_{\text{abs}}^{[l]}$, keeping the quantities numerically meaningful.

Under Section 3.6, the forward pass scales split activations and restores their invariant against cached original activations. The backward pass applies Definition 6.3 using masks from those activations, preserving (6.8) against the cached original gradient.

Layer choice and resolution. Like LayerCAM, SplitCAM is defined at every layer, and the resulting map is bilinearly upsampled to the input resolution. Empirically the method peaks in the deep convolutional blocks — for VGG-16 (Simonyan and Zisserman, 2015) at layer 26 — where it concentrates on entire classified objects noticeably better than LayerCAM at the same layer; the full layer-ablation study and the quantitative comparison on the Quantus benchmark (Hedström et al., 2023) for VGG-16 and ResNet-18 (He et al., 2016) are reported in Chapter 7.

6.1.2 SplitLRP

LRP (Bach et al., 2015) propagates a relevance budget backward, distributing each neuron's relevance to its inputs in proportion to their contributions; Montavon et al. (2018) give an overview and Kohlbrenner et al. (2020) best practices. The γ-rule (Montavon et al., 2019) emphasizes positive contributions. We use its bias-excluding, ε-stabilized variant: for $z_i^{(l+1)} = \sum_j W_{ij}^{(l+1)} a_j^{(l)} + b_i^{(l+1)}$, define

$$\tilde{z}_{ij}^{(l+1)} \coloneqq a_j^{(l)} \Big(W_{ij}^{(l+1)} + \gamma \, [W_{ij}^{(l+1)}]^+ \Big), \qquad \gamma \geq 0, \tag{6.13}$$

and redistributes the relevances $R_i^{(l+1)}$ onto the inputs as

$$R_j^{(l)} = \sum_i \frac{\tilde{z}_{ij}^{(l+1)}}{\sum_{j'} \tilde{z}_{ij'}^{(l+1)} + \varepsilon \, \operatorname{sign}\big(\sum_{j'} \tilde{z}_{ij'}^{(l+1)}\big)} \, R_i^{(l+1)}, \tag{6.14}$$

with a small stabilizer $\varepsilon > 0$. Setting $\gamma = 0$ gives the corresponding bias-excluding ε-rule. For affine–ReLU blocks, ReLU activations pass relevance through unchanged, and redistribution acts on the affine layers.

Relevance propagation on the split-pair. *SplitLRP* treats both streams as neuron groups in one decomposed network. Each layer carries relevance tensors $R^{(l,+)}$,$R^{(l,-)}$, jointly propagated through the four coupling blocks built from $W^{(l,+)}$,$W^{(l,-)}$. Propagation starts at the class-specific positive g-output $a^{(L,+)}$. All weights are non-negative, so we use (6.14) with $\gamma = 0$, $\varepsilon = 10^{-6}$: the ε-rule needs no preference for positive over negative contributions. Paired Maxout gates (6.4) are transparent in the relevance pass.

The following conventions complete the specification.

- *Forward pass.* The split activations are computed with the same scaling stabilization and forward-invariant correction of Section 3.6 as SplitCAM.
- *Normalization.* The denominator of (6.14) renormalizes relevance layer-wise, preserving the combined budget of $R^{(l,+)}$,$R^{(l,-)}$ at $a^{(L,+)}$ up to the ε-stabilizer (Remark 6.3). This replaces backward shifting (Definition 6.3).

– *Pooling and normalization layers.* Relevance passes through max-pooling by the winner-takes-it-all scheme, assigning the full relevance to the location attaining the maximum; batch-normalization layers are merged into the adjacent linear or convolutional layers beforehand; residual additions split relevance evenly across branches.

Definition 6.5 (Combined SplitLRP map). The SplitLRP attribution at layer l is the pair $\big(R^{(l,+)}, R^{(l,-)}\big)$; the *combined* map is

$$R^{(l,\mathrm{comb})} := R^{(l,+)} - R^{(l,-)}. \tag{6.15}$$

Positive-stream relevance traverses an even number of sign inversions and represents support; negative-stream relevance represents opposition. Equation (6.15) combines them into a signed saliency map.

Remark 6.3 (A probabilistic reading). The original network's ε-rule mixes signed contributions, producing a signed path measure (Chapter 8). In the split-pair, $\tilde{z}_{ij}^{(l+1)}$ multiplies non-negative activations and weights. Up to the stabilizer, its propagation matrices are stochastic: SplitLRP is a Markov chain on the doubled graph, with unsigned relevance measure $\big(R^{(l,+)},R^{(l,-)}\big)$. Doubling the state space absorbs weight signs, as in the stopping game's probabilistic backward flow (Chapter 4).

Remark 6.4 (Sensitivity to learned parameters). The parameter-randomization sanity check of Adebayo et al. (2018) differentiates the three split methods. After reinitializing the last three feedforward layers of VGG-16, every reported SplitCAM and SplitGrad configuration has lower SSIM than every listed baseline, whereas the HOG and Spearman comparisons are mixed. This establishes sensitivity to the tested parameter changes, not faithfulness to the learned decision process. SplitLRP retains high HOG and Spearman similarity despite a decrease in SSIM. This retained structure is consistent with parameter-insensitivity concerns for some relevance-propagation rules (Sixt et al., 2020), but the comparison does not identify the mechanism in SplitLRP. Section 7.4.1 reports the quantitative results and the caveats of Binder et al. (2023) on randomization-based checks.

6.1.3 SplitGrad

SplitGrad uses the stabilized combined gradients of (3.57) directly. The balanced embedding $a^{(0,\pm)} = \pm x/2$ gives the factor $\frac{1}{2}$ in (6.10); Theorem 3.24 shows that the difference of the stabilized sensitivities of the two convex heads recovers the exact original gradient.

Definition 6.6 (SplitGrad). For a layer l and stream $s \in \{g, h\}$, the SplitGrad attribution map is the channel mean of the combined shifted sensitivity,

$$\mathrm{SplitGrad}^{(l,s)} := \frac{1}{C^{(l)}} \sum_{c=1}^{C^{(l)}} \delta_{\mathrm{shift}}^{(l,s)}\big|_c, \tag{6.16}$$

where $C^{(l)}$ is the number of channels of layer l; the default configuration reads the g-stream, with forward and backward stabilization exactly as in Section 6.1.1.

What SplitGrad measures. Theorem 3.24 gives original-gradient terms $\delta/2$ and $-\delta/2$, where $\delta = \partial f/\partial x$, minus a shared, geometrically damped sum of layer-wise *subthreshold probe-gradients* through inactive neurons. The shared sum cancels in the difference, recovering δ. It can itself vanish, so subthreshold contributions need not yield nonzero attribution. In Remark 6.2, however, $\delta = 0$

and the probe term is nonzero, correctly attributing the feature for every $\alpha < \frac{1}{2}$. The factors $\overline{\alpha}_j$ counteract probe-gradient growth toward the input, interpolating between full split information ($\alpha = 0$) and signed halves of the original gradient ($\alpha = \frac{1}{2}$). The sweep in Chapter 7 supports metric- and configuration-dependent choices: the two main-text configurations have localization optima at $\alpha = 0.35$ or 0.4, and lowest Maximum Sensitivity at $\alpha = 0$ (full ablation: Section B.2).

The three methods share one pipeline: a stabilized split forward pass caching the original activations, a stabilized backward pass — relevance-based for SplitLRP (initialized at the class-specific $a^{(L,+)}$ and propagated by the ε-rule), sensitivity-based otherwise (the shifted recursions of (6.10)) — and a method-specific readout, with the resulting map bilinearly upsampled to the input resolution. The network can be split once per model. In the reported implementation, SplitCAM takes approximately 14–29 times as long per image as the corresponding ordinary Grad-CAM and LayerCAM configurations; SplitGrad also takes substantially longer. Their timings are of the same order as the row labeled Guided Grad-CAM, but the released VGG-16 configuration dispatches Guided LayerCAM, limiting identification of that baseline. Section 7.4.2 details this ambiguity and the measurement scope.

6.1.4 Directly Trained Difference-of-Convex Input Neural Networks

To study mechanisms exposed by the DC decomposition of pretrained networks (Sections 6.1.1–6.1.3), we directly train small DC input neural networks (DCINNs) and their monotone relatives on MNIST. Their shared opposing-stream structure provides a controlled setting for examining class-dependent sensitivities on present and candidate missing strokes, with a modest, architecture-dependent accuracy cost.

Architecture and training. Use the ICNN and monotone-network definitions from Section 2.2. An ICNN (Amos et al., 2017) is a convex, monotone network acting on an unconstrained affine first layer (Definition 2.5); constraining that layer too gives monotonicity in the original input (Propositions 2.3 and 2.2).

Definition 6.7 (DCINN and difference-of-monotone models). For a fixed base architecture, a *DCINN* (DIC model) is $f_{\mathrm{DIC}} = g - h$, where g,h are independently parameterized ICNNs, trained jointly by empirical risk minimization on their difference. A *DM model* instead uses two monotone networks. The *inverted-input* variants DICIN and DMIN feed h the complement $\mathbf{1} - x$ for inputs in $[0,1]^d$.

The two non-negative streams learn convex (respectively monotone) functions, making their difference a training-time inductive bias. Related CDiNN architectures address optimal control (Sankaranarayanan and Rengaswamy, 2022), with recurrent extensions for safety-critical model predictive control (Chen et al., 2025); we study interpretability. Feeding a monotone h-stream $\mathbf{1} - x$ makes it accumulate evidence from missing input mass.

Three implementation choices matter in practice.

– *Activations.* Both streams use convex, monotone rank-2 Maxout units (Goodfellow et al., 2013), which consistently outperformed ReLU streams in the difference setting. Their constrained layers use the same non-negative-weight Maxout building blocks as the split-stream construction in Section 3.2.3.

- *Non-negativity.* Constrained weights are parameterized as $p(\tilde{W})$ with free parameters $\tilde{W}$ and a positivity transform p; among softplus, exponential and squaring, $p = \mathrm{ReLU}$ with absolute-value-of-Gaussian initialization gave the best test accuracy. The transform is also applied to biases and to the scale and shift parameters of batch-normalization layers.
- *Regularization.* Since non-negative weights preclude cancellation, activations can grow quickly; a small initialization scale and a weight decay of 10^{-3} keep the streams stable during training.

The empirical study uses two MNIST base architectures: `cnn_maxout`, a CNN with four convolutional layers (32, 64, 128 and 128 channels with batch normalization) followed by three fully connected layers, roughly 1.2 million parameters in total, and `mlp_maxout`, a wide four-layer MLP ($784 \to 800 \to 800 \to 800 \to 10$) with roughly 4.2 million parameters — each in eight variants: unconstrained (None), single-stream constrained (IC, Mon), and difference-of-two models (DIC, DM, DICIN, DMIN, and an unconstrained difference baseline).

Stream sensitivities and candidate counterfactual features. The directly trained streams expose *plain input gradients* without shifting, relevance rules or probe networks. For an inverted-input DIC class logit $f(x) = g(x) - h(\mathbf{1} - x)$, $\nabla g(x)$ measures the sensitivity of $g(x)$, while $(\nabla h)(\mathbf{1} - x)$ measures that of $-h(\mathbf{1} - x)$ to increases in the original input: complement and subtraction cancel the two chain-rule signs. At differentiable inputs, the complete model gradient is

$$\nabla f(x) = \nabla g(x) + (\nabla h)(\mathbf{1} - x).$$

The first affine layer of an ICNN is unconstrained, so convexity of the streams does not make either gradient non-negative in the original input coordinates. A positive entry of one stream gradient can be outweighed by a negative entry of the other. Even a positive entry of the complete gradient only establishes an increase for sufficiently small feasible increases in that coordinate at a differentiable input; it does not establish a necessary feature or a change in the predicted class. Finite counterfactual claims require evaluating the complete model under the proposed intervention. The displayed maps are thus stream-local sensitivities; their concentration on present and candidate missing strokes motivates a factual/counterfactual interpretation to be tested by interventions.

Figure 6.1 overlays both gradients for each class logit of a test "3". The g-gradient best traces strokes for the ground-truth logit; for competing "6", "8" and "9", h-gradients concentrate *between the arches*, where added pixels would geometrically close loops. A second correctly classified "3" (Figure 6.2) shows the same qualitative pattern. The single `cnn_maxout` IC model's gradients are mostly positive on strokes and negative on background, with less class-dependent directional variation. These examples suggest a role for the difference structure without establishing general disentanglement.

Qualitative comparison with a single input-convex model. Figure 6.3 reproduces the source paper's ten-class input-gradient comparison for the single `cnn_maxout` IC model, showing the stroke/background sign pattern described above. Together with the class-dependent DICIN maps (Figures 6.1, 6.2), it motivates a factual/counterfactual reading; finite-intervention effects and separation across all inputs and models require further tests.

Accuracy trade-offs. Architectural constraints can reduce expressivity (Liu et al., 2020; Sill, 1997), but for the difference-of-two construction the accuracy cost is small on the convolutional base model and even negative on the MLP, where every difference-of-two variant outperforms the unconstrained baseline; the full accuracy table across ten seeds and all eight variants is reported in Chapter 7.

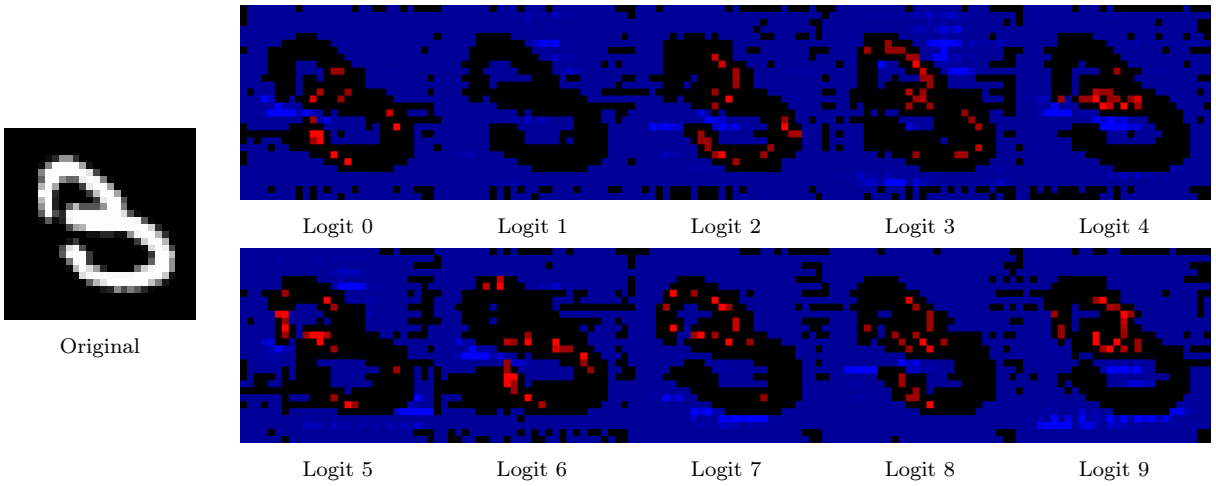


Fig. 6.1: Stream gradients of the inverted-input DIC model (`cnn_maxout` DICIN) on a correctly classified MNIST test digit "3", thresholded and overlaid per class logit. The g-stream gradient (red) highlights sensitivities on present strokes, while the h-stream gradient (blue) highlights sensitivities at candidate missing-stroke locations. These stream-local maps do not by themselves establish the effect of adding pixels on the complete model. Left: the original digit.

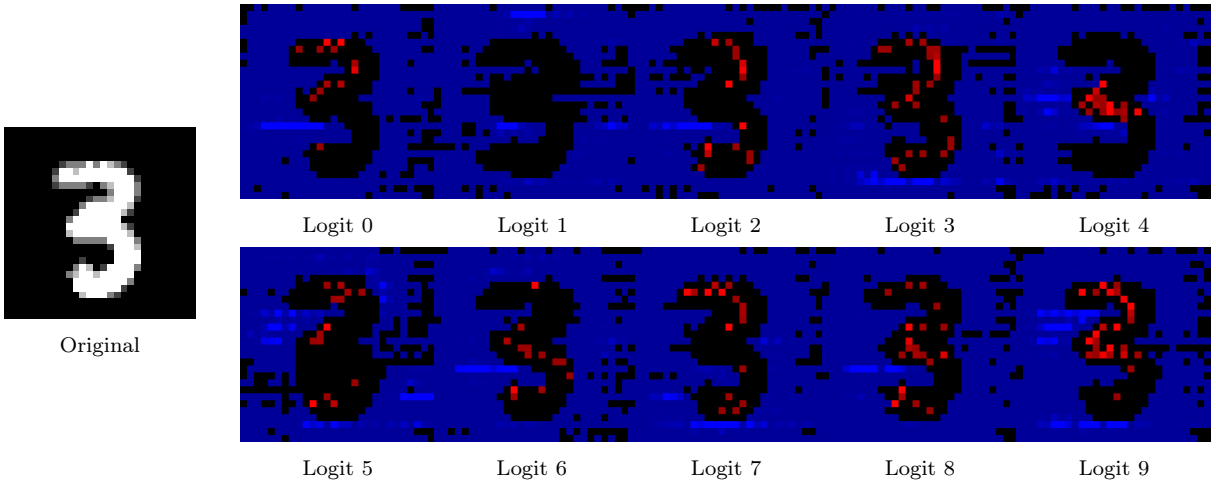


Fig. 6.2: Second correctly classified MNIST test "3", using the model, per-class thresholded overlays and color convention of Figure 6.1. Left: input. These stream-local maps do not by themselves establish the effect of adding pixels on the complete model.

Remark 6.5 (Post-hoc and by-design, reconciled). The MNIST study probes the same opposing-stream mechanism as post hoc decomposition: directly trained differences reach competitive accuracy and show class-dependent sensitivities on present and candidate missing strokes. Establishing faithful counterfactuals or general explanatory-role separation for these models and pretrained VGG-16/ResNet-18 split-pairs requires complete-model interventions and larger-architecture tests (Chapter 9).

6.2 Game-Based Attribution: Modes and Technicalities

The game route attributes by the per-neuron difference of players' discounted equilibrium occupation measures. It recovers gradients in the SG (Theorem 4.5) and the full $\alpha\beta$-LRP family in the RG at $\tau = 1$ (Theorem 4.7). Here we collect tunable *modes* and input-side *technicalities*.

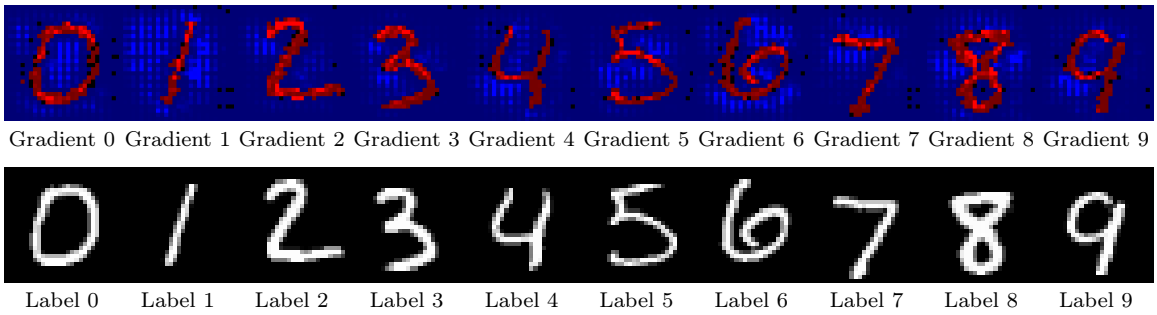


Fig. 6.3: Input-gradient visualizations of the `cnn_maxout` IC model for ten MNIST digits, each with respect to its ground-truth label. Top: saliency maps; bottom: input images. As in the source figure, positive and negative gradient values above threshold 0.2 are colored red and blue, with ranking colorized using exponential decay.

6.2.1 Attribution Modes

Chapter 4 varies equilibrium trajectory laws through routing and stopping temperatures τ,θ, the policy-entropy bonus, risk $\lambda \leq 0$, stream balance (α,β), and LRP stabilization $\epsilon \geq 0$. Section 4.6 gives an exact dictionary from each parameter to its backward rule. These controls yield tunable attribution maps and form the sweeps of Chapter 7.

Here (α,β) denotes the $\alpha\beta$-LRP discount pair, with $\alpha - \beta = 1$ and anchor (2,1) weighting support against opposition; a bare α is this stream weight, unrelated to the backward shift $\alpha \in [0,\frac{1}{2}]$ of Section 6.1. Routing temperature τ recovers $\alpha\beta$-LRP exactly only at $\tau = 1$ (Theorem 4.7); its myopic application preserves the forward pass. It is distinct from the spatial transforms τ_t below.

6.2.2 Technicalities

Input-transform averaging and a detached-operator (DAVE-style) reading of the gradient decomposition improve maps without changing the game. They matter especially for ViTs, whose 16×16 patch tokenization breaks translation equivariance and introduces grid artifacts (Dosovitskiy et al., 2021; Wróbel et al., 2026).

Reynolds-inspired averaging. Following DAVE (Wróbel et al., 2026), compute attributions on N randomly transformed inputs (small rotations, translations, horizontal flips), map them back by τ_t^{-1}, and average. These spatial transforms $\tau_1, \ldots, \tau_N$ are distinct from routing temperature τ. Reynolds-inspired averaging can suppress transform-dependent architectural artifacts while retaining common image structure; N trades computation for smoothness and is ablated in Chapter 7.

Connection to DAVE-style gradient decomposition. The averaging pairs with a *conditioned forward pass* inspired by DAVE (Wróbel et al., 2026). Our game fixes both LayerNorm statistics, the GELU gate and the softmax row; DAVE retains linear mean-centering while fixing the normalization scale. Conservation concerns the game's propagation rule with its bias accounting. The attribution takes the transform-averaged input×gradient form $\frac{1}{N}\sum_t \tau_t^{-1}(W_t) \odot X$, where W_t is the conditioned backward signal on the t-th transformed input. Chapter 5 specifies the conditioned operators used by the game-based maps.

7 Experiments and Evaluation

This chapter evaluates two design choices: exposing split streams to attribution and changing backward routing through game policies. The split-method and game-based campaigns examine localization, faithfulness, robustness, and numerical or computational cost. We retain their checkpoints, sample exclusions, metric conventions, and selection protocols, comparing scores within each campaign. Model-evaluation scores come from the papers; Pascal VOC uncertainty estimates and comparison statistics are recomputed locally using images as independent sampling units.

7.1 Experimental Setup

The split campaign evaluates pretrained VGG-16 and ResNet-18 on a custom 50-class ImageNet-S subset at 224×224 resolution. The source paper reports 50 validation images (one per class) for layer–formula selection and 566 test images. Its released loader selects locally available image–mask pairs without the game campaign's label-agreement filter; these counts alone cannot recover historical sample membership. Explanations target the predicted class, including for misclassified images; localization uses the annotated object mask. Baselines use Captum, Zennit for γ-LRP, and separate Grad-CAM++ and LayerCAM implementations. Each method retains localization-selected and Pixel-Flipping-selected configurations (full settings and results: Section B.1).

The split tables report Pixel Flipping as score-drop AUC ($\mathrm{PF}_5^{\mathrm{drop}}$, $\mathrm{PF}_{20}^{\mathrm{drop}}$; ↑), perturbing 112 scalar input features per step, and Maximum Sensitivity with 25 perturbations. The revised game campaign reports remaining-logit AUC (PF_5, PF_{20}; ↓), with 1,568 scalar features per step, and sensitivity with 50 perturbations on a common 200-image test subset. Both Pixel-Flipping implementations broadcast spatial attribution over color channels, indexing the flattened $C \times H \times W$ input: 150,528 features for RGB images at this resolution. Quantus measures Selectivity, Attribution Localization and Pointing Game. Both campaigns include VGG-16; their ResNet-18 and ResNet-50 scores are not interchangeable. Table abbreviations are AL (Attribution Localization), PG (Pointing Game), Sel (Selectivity), MaxS (Maximum Sensitivity), AvgS (Average Sensitivity), GCAM (Grad-CAM), GCAM++ (Grad-CAM++), LCAM (LayerCAM), IntGrad (Integrated Gradients), and SmGrad (SmoothGrad); Guided GCAM denotes Guided Grad-CAM. Dark blue shading and bold type mark the best values within each caption's comparison; n/r means unreported in the source paper, and n/a means inapplicable.

7.2 Split Methods: VGG-16

Qualitative analysis. Figure 7.1 compares split and baseline saliency maps at their best-performing layers. Mid-layer SplitCAM resembles earlier-layer classical γ-LRP. Against same-layer LayerCAM, it gives more weight across object pixels while emphasizing key features, balancing interpretability and focus (low entropy). For $\alpha \ll 0.5$, SplitGrad yields less focused, finely aligned gradients following image details. We attribute this to Maxout retaining pre-activation information that ReLU discards at inactive neurons. Further examples appear in Section B.1.

Quantitative analysis. Tables B.1 and B.2 give the complete selected VGG-16 and ResNet-18 configurations; *Abs* indicates absolute saliency before Quantus evaluation. Section B.1 specifies layers and parameters.

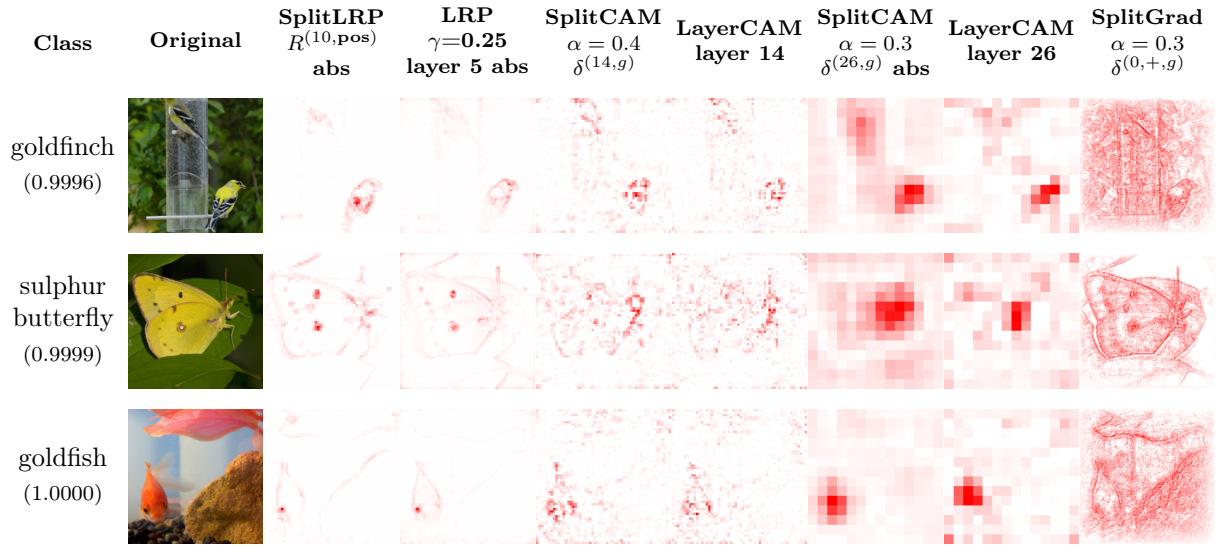


Fig. 7.1: Saliency map comparison for VGG-16 on our custom ImageNet-S test split. Each row shows the predicted class (with confidence), original image, and saliency maps from different methods.

The backpropagation-based methods depend strongly on layer choice (ablations: Section B.2).

VGG-16 gains depend on metric and configuration. Table B.1 gives SplitCAM Pointing Game 0.938 versus Guided Backprop's 0.887, Attribution Localization 0.651 versus Guided Backprop's 0.632, and Selectivity 2.727 versus LayerCAM's 2.792, from different SplitCAM layers. LayerCAM has the highest displayed Pixel Flipping score-drop AUCs, 0.317 and 2.474, versus SplitCAM's 0.300 and 2.404. SplitLRP has the lowest Maximum Sensitivity, 0.282, versus Guided Backprop's 0.375, and higher Pointing Game but lower Attribution Localization and Pixel Flipping scores than the displayed classical γ-LRP configurations.

7.3 Split Methods: ResNet-18

On ResNet-18 (Table B.2), SplitCAM has the lowest reported Selectivity, 3.503, versus 3.517 for the strongest baseline, Guided Backprop: a difference of 0.014. SplitCAM and SplitGrad reach Pointing Game 0.972 versus LayerCAM's 0.971. Attribution Localization is 0.673 for SplitCAM and 0.664 for LayerCAM; Guided Grad-CAM leads the full comparison at 0.742 (Table B.2). Baselines retain the best Pixel Flipping and Maximum Sensitivity scores: the gains are metric-specific.

7.4 Split-Method Sanity and Runtime Checks

7.4.1 Randomization Sanity Check

The model-randomization sanity check (Adebayo et al., 2018) tests dependence on learned parameters versus input patterns surviving model corruption. For each method, maps from trained VGG-16 are compared with maps after Gaussian reinitialization of the last three feedforward layers; parameter-sensitive methods should show a substantial similarity drop. Given the shortcomings of top-down randomization discussed by Binder et al. (2023), this complements the primary trajectory-space diagnostic of Section 8.4. Similarity measures are (i) HOG similarity (Pearson correlation of histograms of oriented gradients), (ii) signed Spearman rank correlation, and (iii) SSIM; lower similarity indicates stronger parameter dependence.

Tab. 7.1: Randomization sanity check for the split methods on VGG-16: mean similarity between attribution maps of the trained model and of a copy whose last three feedforward layers are randomly reinitialized, across the custom test subset of ImageNet-S. Lower similarity indicates stronger dependence on the learned parameters. Abbreviations as in Table B.1.

Method	HOG ↓	Spearman ↓	SSIM ↓
Baselines			
Deconvolution	0.985	0.999	0.481
DeepLIFT	0.365	0.274	0.716
GradientSHAP	0.286	0.081	0.571
Input × Gradient	0.313	0.082	0.621
IntGrad	0.303	0.102	0.584
LCAM (layer 14)	0.688	0.123	0.417
LCAM (layer 2)	0.542	0.114	0.475
Saliency	0.188	0.416	0.353
SplitCAM			
SplitCAM (sc, α 0.4) (12, g)	0.736	0.729	0.202
SplitCAM (sc, α 0.4) (14, g)	0.724	0.194	0.174
SplitCAM (sc, α 0.4, wta) (14, g)	0.724	0.389	0.303
SplitCAM (sc, α 0.4, wta) (26, +, g) abs	0.728	0.342	0.319
SplitCAM (sc, α 0.5, wta) (14, g)	0.728	0.096	0.292
SplitCAM (sc, α 0.5, wta) (26, +, g)	0.804	0.186	0.148
SplitGrad			
SplitGrad (α 0.4) (12, +, g)	0.677	0.711	0.174
SplitGrad (α 0.4) (14, +, g) abs	0.704	0.197	0.091
SplitGrad (α 0.4, wta) (12, +, g)	0.706	0.454	0.174
SplitGrad (α 0.4, wta) (14, +, g) abs	0.721	0.211	0.199
SplitGrad (α 0.5, wta) (12, +, g) abs	0.725	0.238	0.206
SplitGrad (α 0.5, wta) (26, +, g) abs	0.795	0.149	0.306
SplitLRP			
SplitLRP (7, pos)	0.997	1.000	0.471
SplitLRP (19, pos)	0.998	1.000	0.562
SplitLRP (wta) (7, pos)	0.998	1.000	0.456
SplitLRP (wta) (26, pos) abs	0.994	0.999	0.446

In Table 7.1, every reported SplitCAM and SplitGrad configuration has lower SSIM than every listed baseline; HOG and Spearman comparisons are mixed. This shows responses to the tested randomization without isolating stabilization's effect on numerical gradient growth. SplitLRP retains high HOG and Spearman similarity (approximately 0.99 and 1.00), while SSIM falls to 0.45–0.56. Attenuation by the ϵ-LRP rule could explain this behavior; the comparison alone does not identify its cause.

7.4.2 Computational Efficiency

Table 7.2 reports mean attribution times on the custom 50-image ImageNet-S validation subset for ResNet-18 and VGG-16. Splitting times, averaged separately over 20 setups, are 0.033 s and 0.030 s, respectively. SplitCAM and SplitGrad take 0.33–0.44 s per image, versus 0.28–0.31 s for the row

labeled Guided Grad-CAM. SplitCAM takes approximately 14–29 times as long as the corresponding ordinary Grad-CAM and LayerCAM rows; the split methods are much faster than the reported Feature Ablation and Occlusion configurations.

The released evaluator splits the model once per configuration and reuses it across images. The per-image timer covers attribution, helper postprocessing, copying and cleanup, excluding image/model loading, initial device transfer, target prediction and splitting. The helper computes maps for all spatial layer chunks from cached activations and gradients before selecting the output; this extra computation is timed. The shifting parameter α changes the backward rule; its runtime effect was not measured separately.

The historical timing artifacts lack hardware, precision and exact configuration IDs. The released VGG-16 `guided_grad_cam` configuration dispatches Guided LayerCAM, whereas the source table labels it Guided Grad-CAM. We retain the reported label and numbers, with this baseline identity unresolved. These timings describe the source implementation and do not bound other implementations' overhead.

Tab. 7.2: Mean attribution time in seconds per image across the custom 50-image validation subset of ImageNet-S. Model Splitting Time is instead seconds per setup, averaged over 20 procedures and excluded from the attribution times. Timer scope and limits of the historical run metadata are described in Section 7.4.2.

Method	**ResNet-18**	**VGG-16**
Model Splitting Time	0.0331	0.0295
Split Methods		
SplitCAM	0.4368	0.3348
SplitGrad	0.4284	0.3309
SplitLRP	0.5711	0.3204
Classical Methods		
GCAM	0.0216	0.0232
GCAM++	0.0157	0.0155
LCAM	0.0149	0.0124
Guided GCAM	0.3102	0.2808
LRP $\gamma = 0.25$	0.0599	0.0507
DeepLIFT	0.0203	0.0210
IntGrad	0.0070	0.0121
GradientSHAP	0.0084	0.0082
Guided Backprop	0.0253	0.0258
Deconvolution	0.0087	0.0074
Saliency	0.0060	0.0047
Input $\times$ Gradient	0.0055	0.0046
Feature Ablation	231.6631	192.2602
Occlusion	238.2302	201.0051

7.5 Split-Method Stabilization Ablations

7.5.1 Backward-Pass Stabilization of the Split Methods

Two configurations in Table B.3 illustrate the effect of the shift α on SplitGrad and SplitCAM on the custom ImageNet-S test split. As in Section 3.6, $\alpha = 0$ leaves the backward pass unstabilized; $\alpha = 0.5$ gives local sensitivities resembling scaled original-network gradients. Reducing α from 0.5 to 0.4 improves both configurations' Pointing Game and Attribution Localization, while SplitCAM's $\mathrm{PF}_{20}^{\mathrm{drop}}$ falls from 1.607 to 1.585. SplitGrad peaks in $\mathrm{PF}_{20}^{\mathrm{drop}}$ at $\alpha = 0.4$ and localization at $\alpha = 0.35$; SplitCAM's Pointing Game and Attribution Localization peak at 0.4 and 0.35, respectively. Both minimize Maximum Sensitivity at $\alpha = 0$. The sweep supports metric-dependent α choices; numerical stabilization and attribution-rule changes remain confounded. Section B.2 (Table B.3) gives the full grid and compares convex with winner-takes-it-all max-pooling.

The unresolved source-table discrepancy for this SplitCAM configuration is documented in Section B.2.1.

7.5.2 Forward-Pass Stabilization

We assess forward stabilization for framework-selected SplitLRP layers and manually selected SplitCAM layers. The shifting baseline sets positive-neuron activations to half the original activation and negative-neuron activations to its negative half. Without forward stabilization, late VGG-16 SplitCAM activations reach the order of 10^{18}, with numerical errors of order 10^{9} despite per-layer correction. Unstabilized SplitLRP produced nearly white maps with small, random, semantically meaningless artifacts; no quantitative metrics were computed.

Tables B.4 and B.5 report SplitCAM without forward stabilization and with shifting, respectively, using the configurations of Table B.3. Table 7.3 compares SplitLRP shifting and scaling. Scaling improves all reported metrics for both layer-7 configurations. At later layers, shifting gives higher Attribution Localization and Pixel-Flipping score-drop AUC; scaling lowers Maximum Sensitivity. Pointing Game rankings reverse between configurations, supporting a configuration-dependent trade-off.

Tab. 7.3: Scale (sc) vs. shift (sh) forward-pass stabilization for SplitLRP methods on VGG-16.

Method	Type	Sel↓	AL↑	PG↑	$\mathrm{PF}_{5}^{\mathrm{drop}}$↑	$\mathrm{PF}_{20}^{\mathrm{drop}}$↑	MaxS↓
SplitLRP (wta), 7	sc	**3.235**	**-0.036**	**0.836**	**0.296**	**2.291**	**0.866**
	sh	3.619	-0.610	0.698	0.260	2.070	0.914
SplitLRP (wta), 26, abs	sc	**5.168**	0.588	0.871	0.151	1.473	**0.282**
	sh	5.168	**0.628**	**0.877**	**0.161**	**1.478**	0.468
SplitLRP, 7	sc	**3.235**	**-0.036**	**0.836**	**0.296**	**2.291**	**0.866**
	sh	3.619	-0.695	0.698	0.260	2.070	0.914
SplitLRP, 19	sc	5.168	0.588	**0.871**	0.151	1.473	**0.282**
	sh	**4.955**	**0.629**	0.853	**0.197**	**1.692**	0.507

7.6 Game-Based Case Study

Setup. We evaluate RG and SG against standard backward baselines (Section A.3.1) on VGG-16 (Simonyan and Zisserman, 2015), ResNet-50 (He et al., 2016), and ViT-B/16 (Dosovitskiy et al., 2021), using PyTorch (Paszke et al., 2019) and torchvision ImageNet-1K checkpoints (Deng et al., 2009;

Russakovsky et al., 2015)[1]. ImageNet-S (Gao et al., 2023) supplies 50 validation and 565 test images at 224×224; Pascal VOC 2012 (Everingham et al., 2010) tests cross-dataset transfer. The data filters, annotated-class targets and pixel-mask protocol are specified in Section A.2. Quantus (Hedström et al., 2023) measures localization (AL, PG, TK; ↑), faithfulness (PF_5, PF_{20}, Sel; ↓), and robustness (MaxS, AvgS; ↓). Baseline implementations and settings, mode parameters, and metric definitions appear in Sections A.3.1, B.3 and A.2.2.

Protocol. Every tunable method, including baselines and DAVE, uses the same validation search: 2,021 configurations in the declared grids (Table A.1). Independent localization and faithfulness rank-sums select two configurations per method; both are evaluated once on test data. Confidence intervals and Holm-corrected paired comparisons follow Section A.2.4. The RG/RG+Eq anchors additionally represent $\alpha\beta$-LRP exactly (Theorem 4.7). Table 7.4 gives both selections and intervals; the comparison below uses localization-selected rows and retains their original testing family. Reporting faithfulness-selected rows separately preserves the distinction between mask agreement and faithfulness.

Tab. 7.4: ViT-B/16, ImageNet-S test split: all rows with 95% bootstrap confidence intervals (MaxS/AvgS: the fixed 200-image subset). Metrics: Section A.2.2; configuration symbols and row labels: Section B.3. Bold/dark shading marks the best row; ≈/light shading marks no significant difference from it under the full-table comparison (Section A.2.4).

Method	Config	AL↑	PG↑	TK↑	PF_5↓	PF_{20}↓	Sel↓	MaxS↓	AvgS↓
Gradient	fixed	0.411 [0.389,0.432]	0.361 [0.322,0.402]	0.386 [0.359,0.413]	8.45 [8.33,8.56]	8.08 [7.94,8.22]	6.033 [5.905,6.154]	1.290 [1.233,1.351]	0.922 [0.893,0.953]
SmGrad (loc/faith)	N=50,σ/range=0.2	0.517 [0.498,0.536]	0.706 [0.667,0.743]	0.675 [0.654,0.694]	8.10 [7.96,8.24]	7.02 [6.83,7.20]	5.029 [4.894,5.163]	0.597 [0.569,0.630]	0.468 [0.451,0.485]
IntGrad (loc/faith)	N_{steps}=100,black baseline	0.458 [0.437,0.479]	0.513 [0.473,0.554]	0.498 [0.472,0.524]	8.37 [8.24,8.48]	7.93 [7.78,8.08]	5.712 [5.577,5.842]	1.090 [1.041,1.140]	0.825 [0.796,0.855]
LRP-ε (loc)	ε=0.25	0.592 [0.575,0.610]	0.834 [0.804,0.864]	0.746 [0.728,0.764]	7.97 [7.81,8.11]	6.88 [6.69,7.07]	4.859 [4.733,4.983]	0.486 [0.468,0.505]	0.384 [0.372,0.395]
LRP-ε (faith)	ε=0.1	0.582 [0.564,0.599]	0.834 [0.804,0.864]	0.748 [0.730,0.765]	7.84 [7.68,7.99]	6.76 [6.57,6.96]	4.890 [4.763,5.015]	0.623 [0.597,0.650]	0.498 [0.482,0.515]
LRP-γ (loc)	ε=0.1,γ=2	0.644 [0.628,0.660]	0.873 [0.844,0.899]	0.816≈ [0.800,0.832]	7.93 [7.78,8.08]	6.64 [6.44,6.83]	4.451≈ [4.324,4.572]	0.300 [0.287,0.314]	0.218 [0.210,0.226]
LRP-γ (faith)	ε=0.1,γ=0.1	0.614 [0.597,0.630]	0.844 [0.814,0.873]	0.779 [0.763,0.795]	7.73 [7.56,7.89]	6.40 [6.20,6.60]	4.603 [4.478,4.724]	0.394 [0.378,0.410]	0.309 [0.299,0.319]
AttnLRP (loc)	γ_{conv}=0.25, γ_{lin}=0.05, $\varepsilon_{\text{attn}}$=0.25	0.644 [0.628,0.660]	0.855 [0.825,0.883]	0.809 [0.794,0.825]	7.77 [7.61,7.93]	6.43 [6.23,6.62]	4.582 [4.452,4.706]	0.456 [0.430,0.484]	0.342 [0.324,0.364]
AttnLRP (faith)	γ_{conv}=0.25, γ_{lin}=0.05, $\varepsilon_{\text{attn}}$=0.01	0.629 [0.613,0.645]	0.850 [0.819,0.878]	0.798 [0.783,0.814]	**7.64** [7.47,7.81]	**6.12** [5.92,6.32]	**4.441** [4.314,4.563]	0.416 [0.395,0.438]	0.317 [0.304,0.331]
DAVE (loc/faith)	N=50,$\theta_{\max}$=0, $s_{\max}$=0,$t_{\max}$=0.5	0.491 [0.472,0.510]	0.717 [0.678,0.752]	0.612 [0.591,0.633]	8.12 [7.98,8.25]	7.37 [7.19,7.54]	5.386 [5.256,5.514]	0.781 [0.755,0.810]	0.643 [0.626,0.661]
TIBAV	fixed	0.615 [0.597,0.634]	0.878 [0.851,0.904]	0.782 [0.759,0.805]	8.13 [8.01,8.24]	7.60 [7.46,7.74]	4.906 [4.774,5.033]	1.062 [1.001,1.126]	0.512 [0.486,0.540]
GAE	fixed	0.533 [0.514,0.551]	0.743 [0.708,0.779]	0.649 [0.623,0.677]	8.13 [8.01,8.25]	7.62 [7.47,7.76]	5.098 [4.961,5.231]	0.525 [0.494,0.558]	0.368 [0.346,0.391]
AR (loc/faith)	w_r=0.75	0.447 [0.428,0.467]	0.565 [0.522,0.605]	0.537 [0.510,0.564]	8.44 [8.34,8.53]	8.06 [7.94,8.17]	5.254 [5.128,5.374]	0.208 [0.195,0.222]	0.122≈ [0.116,0.128]
LIME	N=1000,80 superpixels,ridge	0.535 [0.515,0.553]	0.897 [0.871,0.922]	0.686 [0.656,0.716]	8.36 [8.25,8.46]	7.68 [7.54,7.82]	5.376 [5.242,5.505]	0.937 [0.905,0.972]	0.668 [0.647,0.689]

continued on next page

[1]Checkpoint documentation: https://docs.pytorch.org/vision/stable/models.html; identifiers: Section A.3.

Method	Config	AL↑	PG↑	TK↑	PF_5↓	PF_{20}↓	Sel↓	MaxS↓	AvgS↓
KernelSHAP	N=2M+2048,80 superpixels	0.565 [0.548,0.583]	**0.961** [0.945,0.977]	0.823≈ [0.799,0.848]	8.29 [8.18,8.40]	7.50 [7.35,7.65]	5.140 [5.012,5.261]	0.897 [0.868,0.928]	0.650 [0.632,0.667]
RG anchor	τ=1	0.509 [0.490,0.528]	0.724 [0.685,0.759]	0.699 [0.678,0.720]	8.19 [8.05,8.31]	6.97 [6.78,7.15]	4.854 [4.732,4.972]	**0.166** [0.161,0.172]	0.120≈ [0.117,0.124]
RG (loc)	τ=0.4,λ_{sm}=−10, σ^2=5,ε=0.1	**0.711** [0.692,0.730]	0.825 [0.793,0.855]	0.788 [0.768,0.808]	8.27 [8.14,8.39]	7.48 [7.31,7.64]	5.109 [4.987,5.226]	0.729 [0.685,0.777]	0.494 [0.471,0.518]
RG (faith)	τ=0.7,λ_{sm}=−5, σ^2=5,ε=0.1	0.549 [0.531,0.567]	0.770 [0.736,0.804]	0.741 [0.721,0.760]	8.11 [7.97,8.24]	6.78 [6.59,6.97]	4.808 [4.686,4.925]	0.257 [0.245,0.268]	0.174 [0.168,0.179]
RG+Eq anchor	τ=1,N=50	0.520 [0.501,0.539]	0.761 [0.726,0.796]	0.752 [0.729,0.775]	8.41 [8.29,8.52]	7.72 [7.56,7.87]	4.864 [4.742,4.981]	0.170≈ [0.165,0.175]	**0.115** [0.112,0.118]
RG+Eq (loc)	τ=0.5,λ_{sm}=−10, σ^2=2,N=50	0.698 [0.680,0.717]	0.855 [0.827,0.883]	**0.836** [0.815,0.857]	8.39 [8.28,8.50]	7.80 [7.65,7.94]	5.010 [4.889,5.127]	0.441 [0.421,0.463]	0.301 [0.290,0.313]
RG+Eq (faith)	τ=0.7,λ_{sm}=−10, σ^2=2,N=50	0.654 [0.636,0.672]	0.812 [0.781,0.844]	0.820 [0.799,0.841]	8.39 [8.28,8.51]	7.71 [7.56,7.86]	4.856 [4.735,4.974]	0.310 [0.297,0.325]	0.199 [0.194,0.206]

ViT-B/16 (Table 7.4). Under the localization-selected protocol, the RG family beats every baseline on attribution localization, including the prior transformer-specific backward methods AttnLRP (Achtibat et al., 2024), our DAVE implementation (Wróbel et al., 2026) (Section A.3.1), *Transformer Interpretability Beyond Attention Visualization* (TIBAV) (Chefer et al., 2021b) and *Generic Attention Explainability* (GAE) (Chefer et al., 2021a): the selected mode reaches AL 0.711 against 0.644 for the best baseline (LRP-γ), and RG+Eq 0.698, both significantly ahead of every baseline row after Holm correction. RG+Eq also has the highest observed TK (0.836), but the Holm-corrected tests do not detect a difference from KernelSHAP at 0.823 or LRP-γ at 0.816. The pointing game is led by KernelSHAP (0.961), whose maps are constant on superpixels: a hit needs only one pixel of the maximal region inside the mask. Scoring a single pixel drawn from that region instead, KernelSHAP falls below RG+Eq, which is then not significantly below the best baseline (LRP-γ; Section B.3.5). Among the localization-selected rows, faithfulness favors the LRP rules: AttnLRP is best on PF_5 and PF_{20} and LRP-γ on Sel (PF_{20} 6.43 versus 7.48 for RG), and the faithfulness-selected RG row narrows but does not close the gap (PF_{20} 6.78). Across both selection objectives, faithfulness-selected AttnLRP has the lowest PF_5, PF_{20} and Selectivity, at 7.64, 6.12 and 4.441, respectively. The two anchors are the most robust rows (MaxS 0.166 and 0.170 versus 0.208 for AR), with no Holm-significant MaxS difference between the two anchors. The localization-selected mode reaches MaxS 0.729: sharper routing trades robustness and faithfulness for localization. ADF adds modest overhead (RG: 0.17 s vs. 0.10 s base), still below sampling baselines like SmoothGrad or IntGrad; Reynolds averaging is the only expensive variant (8.66 s), exceeding even the perturbation method KernelSHAP (6.71 s).

ViT-B/16 attribution runtime. For the localization-selected or fixed configurations in Table 7.4, seconds per image on one GPU were: Gradient 0.02, SmoothGrad 1.12, LRP-γ 0.08, AttnLRP 0.10, TIBAV 0.15, GAE 0.03, KernelSHAP 6.71, RG anchor 0.10, RG 0.17, RG+Eq anchor 4.93, and RG+Eq 8.66. The anchors are the exact $\alpha\beta$-LRP representations of Theorem 4.7; hardware and timing scope are specified in Section A.3.3.

Higher τ spreads attribution into context. Localization-optimal modes concentrate mass on the few patches aligning with the mask. Increasing τ spreads equilibrium routing over more predecessors: the displayed examples retain object focus while extending attribution into context, without establishing which contextual features the model uses. Around the selected ViT-B/16 setting, AL decreases monotonically from 0.723 at τ=0.3 to 0.461 at τ=1; faithfulness is non-monotonic. High τ shows how

localization scores favor concentrated maps within the same game. User-perceived informativeness remains untested.

Tab. 7.5: ViT-B/16, RG: attention ablation on 565 ImageNet-S test images at the anchor and validation-selected settings. Cells: gradient fallback above; composite Value rule (Theorem 5.8, gray) below. Configurations: Section B.3. †: paired bootstrap (McNemar for PG), Holm-corrected over the table, $p < 0.05$.

Center	AL↑	PG↑	TK↑	PF_5↓	PF_{20}↓	Sel↓
anchor (τ=1)	0.504†	0.722	0.694†	8.19	6.98†	4.877†
	0.509	0.724	0.699	8.19	6.97	4.854
loc (τ=0.4,λ_{sm}=−10,σ^2=5,ε=0.1)	0.624†	0.743†	0.672†	8.31	7.50	5.364†
	0.711	0.825	0.788	8.27	7.48	5.109
faith (τ=0.7,λ_{sm}=−5,σ^2=5,ε=0.1)	0.543†	0.729†	0.706†	8.13	6.87†	4.816
	0.549	0.770	0.741	8.11	6.78	4.808

Cross-dataset transfer (Pascal VOC). A natural worry with validation-tuned modes is that the chosen τ, λ_{sm} and σ^2 are overfitted to the ImageNet-S 50-class subset. We test transfer on a per-image disjoint split of Pascal VOC 2012 trainval (2,743 test (image, foreground-class) samples with pixel-accurate segmentation masks): RG and RG+Eq use the *same* mode parameters as Table 7.4 without any VOC-side re-tuning, while every baseline gets a fresh per-method validation grid on the new pool. This tests the portability of the selected parameters across these datasets for the same pretrained model, without identifying whether the underlying structure is model-specific or dataset-specific. The localization lead carries over (AL 0.459 for RG and 0.456 for RG+Eq versus 0.413 for TIBAV, both significant against every baseline row), while TK and PG are led by TIBAV and the perturbation methods, and faithfulness by the LRP rules (Section B.3.3).

VGG-16 and ResNet-50 (Section B.3). On CNNs, where LRP-type rules are strong, localization-selected RG leads AL within that selection: clearly on VGG-16 (0.685 versus 0.613 for LRP-ε) and narrowly on ResNet-50 (0.719 versus 0.711 for LRP-γ). The ResNet-50 difference is not Holm-significant (adjusted $p = 0.420$), so its numerical lead does not establish superiority. Across both objectives, VGG-16's strongest baseline AL is faithfulness-selected LRP-ε at 0.639, below RG's 0.685. Pixel Flipping favors LRP-ε for VGG-16 PF_5 (12.43), LRP-γ for VGG-16 PF_{20} (7.89 versus LRP-ε's 8.17), and LRP-γ for both ResNet-50 metrics; CAM and perturbation methods lead TK and PG. VGG-16's risk-averse SG is most robust (MaxS 0.278 versus 0.314 for GCAM++).

7.6.1 Game-Method Ablations

One-factor mode ablations around validation-selected configurations (Section B.3.4; 565 test images), granularity-controlled Pointing Game (Section B.3.5), and attention-rule comparisons (Table 7.5) expose trade-offs among localization, perturbation faithfulness and robustness. These diagnostics do not affect selection.

Gradient-fallback attention. Conditioned RG routes only through Values, with zero Q, K gradients, as in CP-LRP (Ali et al., 2022) (Section 5.4.6). Gradient fallback uses pure autograd through attention, recovering zennit-style ε-LRP; Value-routing risk λ_{sm} is inactive. Fallback lowers all three localization metrics in every configuration (AL and TK significantly throughout) and worsens every PF_5, PF_{20} and Sel cell, significantly in four of nine.

8 The Trajectory Distribution View

Chapter 6 turns internal signals and backward routing choices into attribution maps; Chapter 7 evaluates them. Their use in interventions also requires examining the underlying calculation. The equilibrium policies of Chapter 4 induce backward-trajectory distributions, enabling comparison before projection onto input pixels. We study their Hellinger distance, its exact per-input-state disagreement map, and complete versus survival-conditioned laws. Spectral analysis diagnoses amplification and target discrimination through the attribution redistribution operator's singular spectrum at a fixed input (Maeß et al., 2026); our experiments compare normalized routing under input noise and parameter randomization.

8.1 Comparison with Normalized Relevance Measures

The NRM framework of Xiong et al. (2026), by Ping Xiong, Thomas Schnake, Grégoire Montavon, Klaus-Robert Müller, and Shinichi Nakajima, studies backward calculations through a normalized signed measure on complete neuron walks. Summing walk relevance through selected neurons or concepts gives their relevance; conditional relevance divides joint by nonzero marginal relevance. With suitable normalized propagation rules, this covers ordinary LRP, concept relevance, and Concept Relevance Propagation (CRP) (Achtibat et al., 2023). Their CRP quantity is joint relevance of an input neuron, intermediate concept neuron, and target output, without an automatic conditional-probability interpretation.

Both views retain network paths before forming attribution maps; their factorization supports sum-product computation and queries across layers. In NRM, individual walk relevance may be negative despite unit total. Our game policies define a non-negative probability law Π on the expanded graph, including player labels and termination. Signed differences of its discounted occupation measures recover gradients and relevance scores (Chapter 4), generally distinct from unweighted marginals. Player expansion thus separates trajectory probability from contribution sign and scale, without identifying Π with a given NRM measure.

The NRM construction normalizes propagation columns under positive-sum assumptions; identifying it with globally normalized original walk relevance additionally requires the constant-sum conditions of Xiong et al. (2026, Section 3.5). Our transitions remain normalized; discounts retain scale, with explicit bias and stabilizer absorption. Hellinger distance applies directly to these probability laws, but generally not to signed relevance measures. Conditioning Π on a concept requires positive probability; signed relevance can instead vanish by cancellation.

Symbolic XAI scores logical feature queries by selecting signed terms from a multi-order prediction decomposition (Schnake et al., 2025). SemanticLens combines component attributions and semantic embeddings to audit expected and spurious visual concepts (Dreyer et al., 2025). These address feature relationships and human interpretation; trajectory probabilities describe normalized routing. We propose querying game trajectories through concepts, retaining attribution signs and discounts, and comparing their laws across inputs or models. Equivalence with a particular NRM/CRP construction remains unproved and would require matching propagation rules, normalization, and absorption conventions.

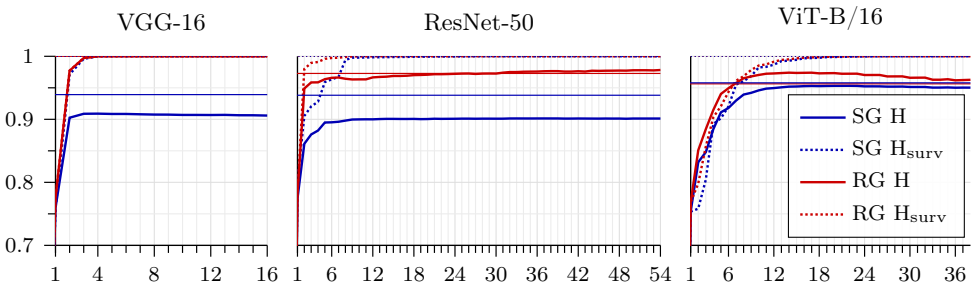


Fig. 8.1: Hellinger trajectory distance under cascading parameter randomization (Adebayo et al., 2018). Hellinger distance H (vertical axis, from 0.7) against the number of cascadingly re-initialized layers, output to input (horizontal axis; H = 0 before the first layer is randomized). Solid: plain H; dotted: H_{surv}, conditioned on survival to an ordinary input-layer state. Thin horizontal reference lines mark the double-random baseline values from Table 8.3 (solid: plain H; dotted: H_{surv}), in the color of the corresponding game.

8.2 Trajectory Sanity Check

The cascading-randomization test of Adebayo et al. (2018) asks whether explanations depend on learned parameters. We compare normalized backward-trajectory laws Π through Hellinger distance, before their model-dependent signed discounted occupation projection into pixel maps. This tests whether randomization changes routing even when the displayed attribution remains stable. Section 8.3 derives an exact edge-linear computation and per-input-state decomposition.

8.2.1 Trajectories change, maps need not

Under cascading randomization (layers cumulatively re-initialized output→input), SG and RG diverge from the first randomized layer and plateau near the double-random reference (Table 8.3, Figure 8.1). Shared short absorbing paths keep plain H below one; removing cemetery mass gives $H_{surv} \approx 1$. Trajectory and map readouts have weak per-image rank association (Spearman $|\rho| \leq 0.19$ between H and pixel-level Gradient or $\alpha\beta$-LRP similarity). Section 8.4 develops the floor mechanism, per-game values, and per-image analysis.

What the map hides. Map-level checks under parameter randomization reproduce Adebayo et al. (2018) and Sixt et al. (2020): some methods retain structure at the cascade endpoint. On ViT-B/16, the RG anchor retains Spearman $|\rho| = 0.454$, highest among all methods, and HOG $r = 0.469$; VGG-16 LRP-ε retains 0.341 and 0.457 (Table 8.2). Localization-selected RG modes are slightly more model sensitive than the anchor on all three models. In Figure 8.1, H reaches the double-random reference while maps retain structure. The map-level failure remains; H detects routing changes obscured by the map. Since discounted occupation projections also vary across models, this discrepancy does not isolate loss under a fixed projection.

8.3 Hellinger Distance and Disagreement Map

We compare two laws on a common layered graph. The geometric mean $\sqrt{PQ}$ preserves trajectory factorization, yielding an exact edge-linear Hellinger pass and a non-negative input-state disagreement map. We first define the processes, then derive the recursion, decomposition, and survival-conditioned form.

8.3.1 Layered Markov Processes with Cemetery

Fix $L \geq 1$, finite ordinary-state sets $\mathscr{S}^{(l)}$, and an absorbing cemetery $\dagger$ at each layer, writing $\mathscr{S}_+^{(l)} = \mathscr{S}^{(l)} \cup \{\dagger\}$. The disjoint union of these layer-tagged sets is the state graph. Backward kernels $T_M^{(l)} : \mathscr{S}_+^{(l)} \to \Delta(\mathscr{S}_+^{(l-1)})$, $M \in \{A,B\}$, satisfy

$$T_M^{(l)}(\dagger \to \dagger) = 1, \qquad T_M^{(l)}(\dagger \to s') = 0 \quad (s' \in \mathscr{S}^{(l-1)}). \tag{8.1}$$

Ordinary states either move to an ordinary predecessor or stop at $\dagger$. Both processes share a deterministic start $s_0 \in \mathscr{S}^{(L)}$; their weights, inputs, or both may differ. For layer-indexed trajectories $\zeta = (\zeta_L, \ldots, \zeta_0) \in \prod_{l=0}^{L} \mathscr{S}_+^{(l)}$,

$$\Pi_M(\zeta) = \mathbf{1}\{\zeta_L = s_0\} \prod_{k=1}^{L} T_M^{(k)}(\zeta_k \to \zeta_{k-1}). \tag{8.2}$$

The prefix law $\Pi_M^{(l)}$ marginalizes the trailing coordinates; $\Pi_M = \Pi_M^{(0)}$. Input-layer states are called pixels, with the full terminal set including the cemetery.

Marginal occupations. The visitation marginals and their backward recursion are

$$m_M^{(l)}(s) = \sum_{\zeta : \zeta_l = s} \Pi_M^{(l)}(\zeta), \qquad m_M^{(L)}(s) = \mathbf{1}\{s = s_0\}, \tag{8.3}$$

$$m_M^{(l-1)}(s') = \sum_{s \in \mathscr{S}_+^{(l)}} m_M^{(l)}(s) T_M^{(l)}(s \to s'). \tag{8.4}$$

This marginal pass gives ordinary terminal probabilities $m_M^{(0)}(s)$ and absorbed mass $m_M^{(0)}(\dagger)$.

8.3.2 How the Two Games Fit This Abstraction

Use the state notation and SPNE policies of Sections 4.3 and 4.4, with the forward theorems 4.2 and 4.6. Diagnostics use $\tau = 1$, hence deterministic activation gates, and $\varepsilon = 0$ for RG. The natural RG includes intermediate linear-routing states. We index gate, branch, and routing substeps as refined layers and read diagnostics at activation boundaries; collapsing them preserves ordinary surviving paths but can lose absorption histories (Remark 8.1).

Stopping Game. Ordinary states are $s_{j,p}^{(l,\mathrm{act})}$ for $l \geq 1$, with start $s_0 = s_{u,+}^{(L,\mathrm{act})}$; input terminals $s_{i,q}^{(0)}$ carry the arriving player. Identifying $p,\kappa \in \{+,-\}$ with signs, for $2 \leq l \leq L$ the natural SG kernel is

$$T_M^{(l)}(s_{j,p}^{(l,\mathrm{act})} \to s_{i,\kappa p}^{(l-1,\mathrm{act})}) = \frac{W_M^{(l,\kappa)}[i,j]}{\gamma_{M,j}^{(l)}} \mathbf{1}\{z_j^{(l)} > 0\}, \qquad \gamma_{M,j}^{(l)} = \sum_i |W_M^{(l)}[i,j]|. \tag{8.5}$$

At $l = 1$, the destination is $s_{i,\kappa p}^{(0)}$. Positive weights retain the player; negative weights switch it. With $0/0 \coloneqq 0$, the cemetery probability is $1 - \mathbf{1}\{z_j^{(l)} > 0\}\mathbf{1}\{\gamma_{M,j}^{(l)} > 0\}$.

Routing Game. The start and activation states are the same; input terminals $s^{(0)}_{i,\kappa}$ instead carry the layer-1 routing-edge stream. Composing the gate and uniform stream choice gives

$$T^{(l)}_M(s^{(l,\mathrm{act})}_{j,p} \to s^{(l-1,\mathrm{act})}_{i,\kappa p}) = \tfrac{1}{2}\pi^{(l)}_{M,\kappa}(i \mid j)\,\mathbf{1}\{z^{(l)}_j > 0\}, \quad 2 \le l \le L, \tag{8.6}$$

$$T^{(1)}_M(s^{(1,\mathrm{act})}_{j,p} \to s^{(0)}_{i,\kappa}) = \tfrac{1}{2}\pi^{(1)}_{M,\kappa}(i \mid j)\,\mathbf{1}\{z^{(1)}_j > 0\}. \tag{8.7}$$

The cemetery receives the residual row mass. A nonempty branch uses the temperature-τ Gibbs policy

$$\pi^{(l)}_{M,\kappa}(i \mid j) \propto [a^{(l-1)}_{M,i} W^{(l,\kappa)}_{M,ij}]^{1/\tau}, \qquad i \in \mathrm{Pred}^{(l)}_j. \tag{8.8}$$

At $l = 1$, replace the bracket by $[x_i W^{(1)}_{M,ij}]^{\kappa}$. An empty stream (all scores $-\infty$) has zero ordinary mass, so its whole $1/2$ share is absorbed even at a live source. For $\varepsilon > 0$, each continuing branch additionally sends $\frac{1}{2}\varepsilon^{1/\tau}/Z^{(l)}_{j,\kappa}$ to the cemetery.

The diagnostic process retains the original gates but omits the bias action and renormalizes each remaining branch. This differs from the bias-inclusive attribution game when bias carries mass: $a = W = b = 1$, $\varepsilon = 0$ gives positive-route probability $1/4$ with bias and $1/2$ without. State elimination within one process does not identify these kernels. Full overlap uses the refined gate/branch/routing process; the displayed act-only kernels suffice for ordinary surviving-path overlap.

8.3.3 Bhattacharyya Coefficient and the Geometric-Mean Kernel

Definitions. For two probability distributions P, Q on a common finite set, the *Bhattacharyya coefficient* (Bhattacharyya, 1943) is the geometric overlap

$$\mathrm{BC}(P,Q) \;:=\; \sum_y \sqrt{P(y)\,Q(y)} \;\in\; [0,1], \tag{8.9}$$

and the *squared Hellinger distance* (Hellinger, 1909) is

$$\mathrm{H}^2(P,Q) \;:=\; \tfrac{1}{2}\sum_y \big(\sqrt{P(y)} - \sqrt{Q(y)}\big)^2 \;=\; 1 - \mathrm{BC}(P,Q) \;\in\; [0,1]. \tag{8.10}$$

The Hellinger distance $\mathrm{H}(P,Q) = \sqrt{1 - \mathrm{BC}(P,Q)}$ is a true metric: $0 \le \mathrm{H} \le 1$, with $\mathrm{H} = 0$ iff $P = Q$ and $\mathrm{H} = 1$ iff P and Q have disjoint supports. Crucially, $\sqrt{0 \cdot Q(y)} = 0$, so mismatched supports cause no infinity—only a smaller overlap.

Properties and alternatives. With half-L^1 total variation $d_{\mathrm{TV}}(P,Q) = \frac{1}{2}\sum_y |P(y) - Q(y)|$ and KL in nats, $\mathrm{H}^2 \le d_{\mathrm{TV}} \le \sqrt{2}\,\mathrm{H}$ and $\mathrm{H}^2 \le \frac{1}{2}\mathrm{KL}$; unnormalized L^1 distance is $2d_{\mathrm{TV}}$. Thus small Hellinger controls TV, and small KL controls Hellinger. An alive/dead mismatch gives zero ordinary overlap; its full row vanishes when the alive state has no cemetery leakage (Lemma 8.4).

KL and Jeffreys obey Markov chain rules but are unbounded and can become infinite when routes open or close. Jensen–Shannon stays bounded, but its mixture $\frac{1}{2}(\Pi_A + \Pi_B)$ is generally non-Markov, preventing this exact local recursion. Wasserstein requires a noncanonical ground cost across layers, signs, and permuted neurons and has no corresponding local recursion. Hellinger's factorization gives an edge-linear pass and a non-negative per-state decomposition. It compares the undiscounted routing law: parameters entering only through discounts d_t, such as path weights (α,β) and SG magnitudes $\gamma^{(l)}_j$, leave H unchanged.

Geometric-mean kernel. For each layer $l \geq 1$ and pair of states $(s, s') \in \mathscr{S}_+^{(l)} \times \mathscr{S}_+^{(l-1)}$, define the *geometric-mean kernel*

$$\widetilde{T}^{(l)}(s, s') \;\coloneqq\; \sqrt{T_A^{(l)}(s \to s')\, T_B^{(l)}(s \to s')}. \tag{8.11}$$

This kernel is non-negative but in general sub-stochastic. Its row sum

$$\widetilde{R}^{(l)}(s) \;\coloneqq\; \sum_{s' \in \mathscr{S}_+^{(l-1)}} \widetilde{T}^{(l)}(s, s') \tag{8.12}$$

satisfies $0 \leq \widetilde{R}^{(l)}(s) \leq 1$ by Cauchy–Schwarz, with equality iff $T_A^{(l)}(s \to \cdot) = T_B^{(l)}(s \to \cdot)$. Cemetery handling is automatic: $T_A^{(l)}(\dagger \to \dagger) = T_B^{(l)}(\dagger \to \dagger) = 1$, so $\widetilde{T}^{(l)}(\dagger, \dagger) = 1$ and $\widetilde{R}^{(l)}(\dagger) = 1$.

Trajectory factorization.

Lemma 8.1 (Trajectory overlap and backward pass, Lean_81, Lean_82, Lean_83). *For every $0 \leq l \leq L$,*

$$\mathrm{BC}(\Pi_A^{(l)}, \Pi_B^{(l)}) = \sum_{(\zeta_L, \ldots, \zeta_l)} \mathbf{1}\{\zeta_L = s_0\} \prod_{k=l+1}^{L} \widetilde{T}^{(k)}(\zeta_k, \zeta_{k-1}). \tag{8.13}$$

Restricting this sum to prefixes ending at s defines $g^{(l)}(s)$, with

$$g^{(L)}(s) = \mathbf{1}\{s = s_0\}, \qquad g^{(l-1)}(s') = \sum_s g^{(l)}(s) \widetilde{T}^{(l)}(s, s'), \qquad \mathrm{BC}^{(l)} = \sum_s g^{(l)}(s). \tag{8.14}$$

As l decreases, $1 = \mathrm{BC}^{(L)} \geq \cdots \geq \mathrm{BC}^{(0)} \geq 0$; equivalently, $\mathrm{H}^2_{(l)} = 1 - \mathrm{BC}^{(l)}$ is non-decreasing. Algorithm 8.1 computes these quantities and every terminal disagreement $\mathrm{H}^2(s)$ exactly in edge-linear time; the latter sum to $\mathrm{H}^2_{(0)}$.

Proof. Substitute (8.2) into (8.9) and distribute the square root over non-negative factors. Grouping by the penultimate state gives the recursion, which Algorithm 8.1 computes by downward induction. The shared start gives $\mathrm{BC}^{(L)} = 1$; $0 \leq \widetilde{R}^{(l)}(s) \leq 1$ gives $\mathrm{BC}^{(l-1)} = \sum_s g^{(l)}(s) \widetilde{R}^{(l)}(s) \leq \sum_s g^{(l)}(s) = \mathrm{BC}^{(l)}$. At layer 0, $g^{(0)}(s)$ sums $\sqrt{\Pi_A \Pi_B}$ over paths ending at s; Theorem 8.2 gives the disagreement and its total. Each pass visits every edge once. □

8.3.4 Hellinger backward pass

The backward pass includes the cemetery; visualization restricts its output to ordinary pixels (Theorem 8.2).

Algorithm 8.1: Hellinger backward pass (with per-terminal-state disagreement map)

Initialize $m_M^{(L)}$ and $g^{(L)}$ at the target output. For $l = L, \ldots, 1$, propagate m_M with its kernel and g by (8.14). At each layer return $\mathrm{BC}^{(l)} = \sum_s g^{(l)}(s)$ and $\mathrm{H}^{(l)} = \sqrt{1 - \mathrm{BC}^{(l)}}$; at every terminal state return (8.15). Restrict to ordinary input states for visualization.

Remark 8.1 (Routing-state elimination and absorption histories, `Lean_84`*).* The refined diagnostic process separates gate, uniform κ-branch, and routing steps: dead activations stop at the gate; empty branches or outside options may absorb later. The Hellinger pass retains these histories, padding the cemetery after absorption and reading at activation boundaries.

Deleting intermediate routing states is bijective on ordinary surviving trajectories: adjacent activation states determine neuron and player, and player retention/switching (or the input stream tag) determines κ. Their path probabilities and Bhattacharyya overlap therefore agree with the act-only process, also after conditioning on survival. Absorbing paths may merge gate and later routing stops. Full-law equality requires recoverable absorption histories, as when no alive branch leaks to the cemetery; generally, finite-fibre Cauchy–Schwarz gives

$$\mathrm{BC}_{\text{refined}} \le \mathrm{BC}_{\text{act-only}}, \qquad \mathrm{H}_{\text{refined}} \ge \mathrm{H}_{\text{act-only}}.$$

For $x = W = 1$, model A with bias 0 continues and absorbs its empty negative branch with probability $1/2$, while model B with bias -2 stops at the gate. Refined supports are disjoint ($\mathrm{BC} = 0$); merging the absorption steps gives act-only overlap $1/\sqrt{2}$.

8.3.5 Per-Pixel Disagreement Map

Theorem 8.2 (Per-pixel decomposition, `Lean_85`**).** *Let* $\mathrm{H}^2 = \mathrm{H}^2(\Pi_A^{(0)}, \Pi_B^{(0)})$. *The cumulative overlap* $g^{(0)}(s)$ *gives a non-negative decomposition over terminal states:*

$$\mathrm{H}^2(s) \coloneqq \tfrac{1}{2}\big(m_A^{(0)}(s) + m_B^{(0)}(s)\big) - g^{(0)}(s) \ge 0, \qquad s \in \mathscr{S}_+^{(0)}, \tag{8.15}$$

$$\mathrm{H}^2 = \sum_{s \in \mathscr{S}_+^{(0)}} \mathrm{H}^2(s), \tag{8.16}$$

$$\mathrm{H}^2_{\mathrm{ord}} \coloneqq \sum_{s \in \mathscr{S}^{(0)}} \mathrm{H}^2(s) = \mathrm{H}^2 - \mathrm{H}^2(\dagger). \tag{8.17}$$

Proof. Group trajectories by their terminal state and expand the square:

$$\mathrm{H}^2(s) = \tfrac{1}{2} \sum_{\zeta : \zeta_0 = s} \big(\sqrt{\Pi_A(\zeta)} - \sqrt{\Pi_B(\zeta)}\big)^2 \ge 0.$$

The definitions of $m_M^{(0)}$ and $g^{(0)}$ give the identity; summing over terminal states and removing $\dagger$ give the two totals. Cauchy–Schwarz and AM–GM also give $g^{(0)}(s) \le \sqrt{m_A^{(0)}(s) m_B^{(0)}(s)} \le \tfrac{1}{2}(m_A^{(0)}(s) + m_B^{(0)}(s))$. The contribution vanishes exactly when the two restricted path measures agree: either both terminal masses are zero, or their positive masses and conditional path laws coincide. □

The disagreement map. The visualized map restricts $\mathrm{H}^2(s)$ to ordinary input states, so its total $\mathrm{H}^2_{\mathrm{ord}}$ excludes the cemetery and is unnormalized. Layer-0 states carry player tags for SG and stream tags for RG; summing them gives

$$\mathrm{H}^2_{\mathrm{pixel}}(j) = \mathrm{H}^2(s_{j,+}^{(0)}) + \mathrm{H}^2(s_{j,-}^{(0)}), \qquad \sum_j \mathrm{H}^2_{\mathrm{pixel}}(j) = \mathrm{H}^2_{\mathrm{ord}}.$$

Cauchy–Schwarz bounds each contribution below by $\frac{1}{2}(m_A^{(0)}(s) + m_B^{(0)}(s)) - \sqrt{m_A^{(0)}(s)m_B^{(0)}(s)}$, the squared-Hellinger contribution of the endpoint marginals. With both terminal masses positive, differing conditional path laws make the trajectory contribution strictly larger. This map therefore retains information lost in endpoint marginals. Survival conditioning below separately normalizes the two path laws; it is not merely rescaling this ordinary-state map.

Conditioning on Survival to the Ordinary Input Layer

Conditioning on reaching an ordinary input state separates surviving routes from cemetery absorption. The conditioned process remains Markov through a Doob h-transform, with the remaining survival probability as h.

Theorem 8.3 (Conditioned-survival laws and Hellinger distance, `Lean_86`**).** *Let* $\mathsf{Surv} = \{\zeta : \zeta_0 \in \mathscr{S}^{(0)}\}$*, equivalently the event of never entering* †*. For* $M \in \{A,B\}$*,* $0 \le l \le L$*, and* $s \in \mathscr{S}_+^{(l)}$*, define survival through the kernel continued from* s*, including unreachable states:*

$$\psi_M^{(l)}(s) = \mathbb{P}_M(\mathsf{Surv} \mid \zeta_l = s), \tag{8.18}$$

$$\psi_M^{(0)}(s) = \mathbf{1}\{s \in \mathscr{S}^{(0)}\}, \tag{8.19}$$

$$\psi_M^{(l)}(s) = \sum_{s' \in \mathscr{S}^{(l-1)}} T_M^{(l)}(s \to s')\psi_M^{(l-1)}(s'), \qquad l \ge 1, \tag{8.20}$$

$$Z_M = \mathbb{P}_M(\mathsf{Surv}) = \psi_M^{(L)}(s_0) = \sum_{s \in \mathscr{S}^{(0)}} m_M^{(0)}(s) = 1 - m_M^{(0)}(\dagger). \tag{8.21}$$

The cemetery has $\psi_M^{(l)}(\dagger) = 0$ *at every layer, so its term may be included in the recursion without effect.*

Assume $Z_A, Z_B > 0$*. On ordinary states with* $\psi_M^{(l)}(s) > 0$*, the Doob kernel is row-stochastic; complete zero-survival rows arbitrarily as probability distributions, since the conditioned process never reaches them. Its law* $\Pi_{M,\mathrm{surv}} = \Pi_M(\cdot \mid \mathsf{Surv})$ *on* Surv *and ordinary-state marginals satisfy*

$$T_{M,\mathrm{surv}}^{(l)}(s \to s') = \frac{T_M^{(l)}(s \to s')\psi_M^{(l-1)}(s')}{\psi_M^{(l)}(s)}, \tag{8.22}$$

$$\Pi_{M,\mathrm{surv}}(\zeta) = \frac{\Pi_M(\zeta)}{Z_M}, \qquad \zeta \in \mathsf{Surv}, \tag{8.23}$$

$$m_{M,\mathrm{surv}}^{(l)}(s) = \frac{m_M^{(l)}(s)\psi_M^{(l)}(s)}{Z_M}. \tag{8.24}$$

Thus (8.2) *holds with the conditioned kernels. Let* $\widetilde{T}_{\mathrm{surv}}^{(l)}$ *be the conditioned kernels' entrywise geometric mean. Write* $\mathrm{BC}_{\mathrm{surv}}$ *for their path-law overlap. On rows with* $\psi_A^{(l)}(s)\psi_B^{(l)}(s) > 0$*,*

$$\widetilde{T}_{\mathrm{surv}}^{(l)}(s,s') = \frac{\widetilde{T}^{(l)}(s,s')\sqrt{\psi_A^{(l-1)}(s')\psi_B^{(l-1)}(s')}}{\sqrt{\psi_A^{(l)}(s)\psi_B^{(l)}(s)}}. \tag{8.25}$$

The chain rule (8.13) applies to these conditioned kernels. Normalization gives

$$\mathrm{BC}^{(0)}_{\mathrm{ord}} = \sum_{s\in\mathscr{S}^{(0)}} g^{(0)}(s) = \sum_{\zeta\in\mathsf{Surv}} \sqrt{\Pi_A(\zeta)\Pi_B(\zeta)}, \tag{8.26}$$

$$\mathrm{BC}_{\mathrm{surv}} = \frac{\mathrm{BC}^{(0)}_{\mathrm{ord}}}{\sqrt{Z_A Z_B}}, \tag{8.27}$$

$$\mathrm{H}^2_{\mathrm{surv}} = \mathrm{H}^2(\Pi_{A,\mathrm{surv}},\Pi_{B,\mathrm{surv}}) = 1 - \frac{\mathrm{BC}^{(0)}_{\mathrm{ord}}}{\sqrt{Z_A Z_B}}. \tag{8.28}$$

Algorithm 8.1 on the conditioned ordinary-state graph gives $g^{(0)}_{\mathrm{surv}}$ *and the non-negative map (8.15) with conditioned marginals and overlap; its sum is* $\mathrm{H}^2_{\mathrm{surv}}$.

Proof. The terminal boundary and one-step conditioning give the survival recursion; absorption makes every cemetery term zero. Summing the terminal marginals gives Z_M. Dividing the recursion by $\psi^{(l)}_M(s) > 0$ proves row-stochasticity, and transitions to zero-survival states have zero conditioned probability. Along any positive-probability surviving path, the Doob factors telescope to $\psi^{(0)}_M(\zeta_0)/\psi^{(L)}_M(s_0) = 1/Z_M$, proving the conditional path law; zero-probability paths contribute zero under either law. Bayes' rule gives the marginals. Taking geometric means gives the conditioned kernel, and Lemma 8.1 gives its overlap pass. Normalizing each surviving path by Z_M extracts $1/\sqrt{Z_A Z_B}$ from the overlap sum; grouping by its terminal state gives $\mathrm{BC}^{(0)}_{\mathrm{ord}}$. Finally, $\mathrm{H}^2 = 1 - \mathrm{BC}$ and Theorem 8.2 give the distance and map. □

Kernel-side conditioning exposes the surviving law and computes its distance and map directly. Post-hoc normalization of the original pass is cheaper for the scalar alone, but can become numerically infeasible in deep networks when either Z_M is too small; conditioned kernels avoid the final division by $\sqrt{Z_A Z_B}$.

8.3.6 Experiments

With the model fixed, we test sensitivity to input perturbations and separation among ImageNet-S classes and from random noise. We use the Stopping-Game kernel (8.5) without risk aversion ($\lambda = 0$; "ADF" in code) and Routing-Game kernel (8.6) at $\tau = 1$. The diagnostic RG omits the bias action and renormalizes the remaining branches, so it may differ from the bias-inclusive attribution game.

Input-Noise Sensitivity

For each architecture and $\sigma \in \{0.005,0.01,0.02,0.03,0.05,0.1,0.2,0.3,0.5\}$, we draw one standard-Gaussian tensor η per image and compute $\mathrm{H}(\Pi_A(x), \Pi_A(x+\sigma\eta))$ for both kernels, targeting the clean input's argmax. Mean H and $\mathrm{H}_{\mathrm{surv}}$ increase with σ for both games on all three architectures (Figure 8.2). They coincide for VGG-16 RG, whose kernel never routes to dead-neuron states. ResNet-50 and ViT-B/16 skip connections reach such states; ViT-B/16's GELU gate also sends some trajectories to the cemetery at every state. Generalised Explanation Faithfulness (GEF) correlates explanation and model-output distortion along perturbation paths (Hedström et al., 2025), anchoring explanation changes to model responses. Our Hellinger curves measure routing sensitivity, without establishing this alignment or explanation faithfulness.

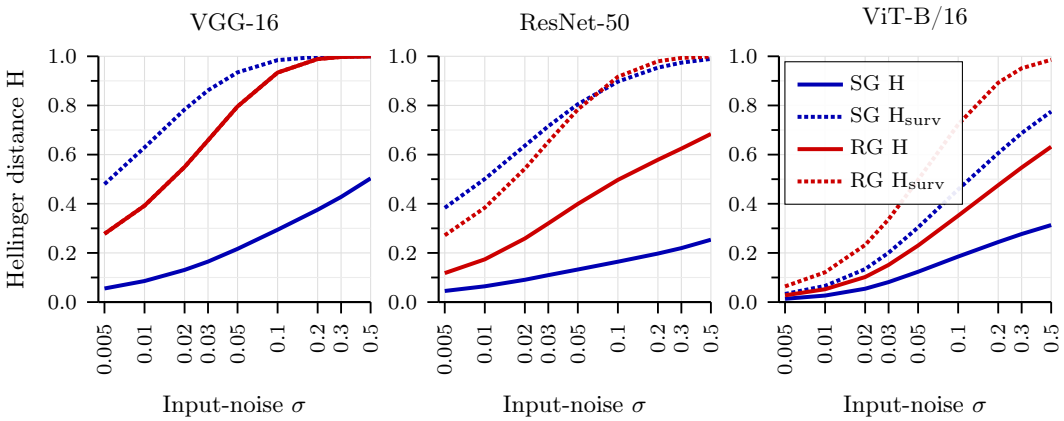


Fig. 8.2: Input-noise Hellinger trajectory distance. Solid: plain H; dotted: H_{surv}, conditioned on survival to an ordinary input-layer state. x-axis is log-scale input-noise σ; same (A,A) model on both sides, only the input differs.

Class Separability and Reference Baselines

Class separation depends on architecture and kernel (Table 8.1):

- On **VGG-16**, SG gives clean-intraclass H = 0.513 below clean-interclass H = 0.607. RG saturates at H = 1 throughout, resolving no class structure.
- On **ResNet-50**, RG separates clean-intraclass H = 0.786 from clean-interclass H = 0.902; SG does not (intraclass 0.272 vs. interclass 0.268).
- On **ViT-B/16**, RG gives intraclass 0.789 vs. interclass 0.882; the SG gap (0.422 ± 0.023 vs. 0.435 ± 0.017) is smaller than the spread over pairs and does not separate classes.

Random-vs-random is lowest in every nonconstant column. Clean-vs-random is highest in each plain-H column except ResNet-50 SG (0.269 vs. intraclass 0.272) and saturated VGG-16 RG, consistent with different routing for Gaussian and in-distribution inputs. Conditioned H_{surv} is at least 0.99 throughout VGG-16, ResNet-50, and ViT-B/16 RG, providing no class-separability signal. ViT-B/16 SG separates random-vs-random (0.852) from clean rows (0.957–0.968), but neither intraclass from interclass (0.963 vs. 0.968) nor clean-vs-random (0.957) from clean pairs.

8.4 Randomization Sanity Checks

We extend Adebayo et al. (2018)'s cascading randomization to trajectory laws, compare attribution maps, and establish double-random and per-image references for Section 8.2.

8.4.1 Cascading Randomization in Trajectory Space

Replacing a weight matrix yields two kernels on the same graph. The Hellinger pass quantifies their change; write $\mathrm{BC}^{(l)}_{\mathrm{ord}} = \sum_{s \in \mathscr{S}^{(l)}} g^{(l)}(s)$ for ordinary overlap.

Architecture	Baseline	Stopping Game		Routing Game	
		H	H_{surv}	H	H_{surv}
VGG-16	clean-intraclass	0.513 ± 0.071	1.000 ± 0.000	1.000 ± 0.000	1.000 ± 0.000
	clean-interclass	0.607 ± 0.032	1.000 ± 0.000	1.000 ± 0.000	1.000 ± 0.000
	clean-vs-random	0.663 ± 0.020	1.000 ± 0.000	1.000 ± 0.000	1.000 ± 0.000
	random-vs-random	0.320 ± 0.019	1.000 ± 0.000	1.000 ± 0.000	1.000 ± 0.000
ResNet-50	clean-intraclass	0.272 ± 0.056	0.999 ± 0.000	0.786 ± 0.044	1.000 ± 0.000
	clean-interclass	0.268 ± 0.045	1.000 ± 0.000	0.902 ± 0.033	1.000 ± 0.000
	clean-vs-random	0.269 ± 0.035	1.000 ± 0.000	0.943 ± 0.010	1.000 ± 0.000
	random-vs-random	0.152 ± 0.008	0.992 ± 0.000	0.739 ± 0.011	0.999 ± 0.000
ViT-B/16	clean-intraclass	0.422 ± 0.023	0.969 ± 0.008	0.789 ± 0.067	1.000 ± 0.000
	clean-interclass	0.435 ± 0.017	0.974 ± 0.005	0.882 ± 0.010	1.000 ± 0.000
	clean-vs-random	0.439 ± 0.014	0.963 ± 0.006	0.891 ± 0.007	1.000 ± 0.000
	random-vs-random	0.377 ± 0.013	0.867 ± 0.013	0.476 ± 0.018	0.997 ± 0.000

Tab. 8.1: Reference baselines for the input-noise sweep, averaged over R pairs drawn from the 565-image ImageNet-S test split (clean rows: $R = 200$ sampled pairs; clean-vs-random: $R = 565$ images paired with a fresh Gaussian each; random-vs-random: $R = 200$ Gaussian pairs); entries are mean $\pm$ standard deviation over pairs. **clean-intraclass**: $\mathrm{H}(\Pi_A(x_1), \Pi_A(x_2))$ with x_1, x_2 from the same ground-truth class; **clean-interclass**: same, with distinct classes; **clean-vs-random**: one side clean, other Gaussian; **random-vs-random**: both sides Gaussian.

Randomization bounds.

Lemma 8.4 (Alive-set mismatch, `Lean_87`). *If dead states send all mass to the cemetery, an alive/dead mismatch has $\sum_{s' \in \mathscr{S}^{(l-1)}} \widetilde{T}^{(l)}(s_i, s') = 0$. Its full geometric-mean row also vanishes when the alive state has no cemetery leakage, as in the nondegenerate Stopping-Game kernel.*

Proof. The dead row is zero on ordinary successors; the alive row, when nonleaking, is zero at the cemetery. Every corresponding geometric mean therefore vanishes. □

Fix a ReLU MLP of hidden widths $n_1, \ldots, n_L$, input x, and target start s_0. Model B replaces $W^{(l^*)}$ by i.i.d. entries from a continuous law symmetric about zero. For each $l \geq l^*$ and neuron i, assume $\mathbb{P}_B(i \in \mathrm{alive}_B^{(l)} \mid \text{kernels above } l) \leq 1 - \delta$, where $\delta \in (0, \frac{1}{2}]$; set $\eta = 1 - \delta \in [\frac{1}{2}, 1)$ and $N = L - l^* + 1$. Kernels below l^* agree; row weights and possibly alive sets change at l^*, with activation changes propagating above it. CNN and Transformer blocks admit the same reduction as single *Layered Markov Process* (LMP) layers when these hypotheses hold.

Theorem 8.5 (Randomization in the Stopping Game, `Lean_88`). $\mathbb{E}_B[\mathrm{BC}_{\mathrm{ord}}^{(0)}(\Pi_A, \Pi_B)] \leq \eta^N$. *For every $z > 0$,*

$$\mathbb{E}_B[\mathrm{H}_{\mathrm{surv}}^2 \, \mathbf{1}\{Z_A Z_B \geq z\}] \geq \mathbb{P}_B(Z_A Z_B \geq z) - \eta^N / \sqrt{z},$$

$$\mathrm{H}^2 = 1 - \mathrm{BC}_{\mathrm{ord}}^{(0)} - g^{(0)}(\dagger),$$

where the indicator-weighted term is zero off its event.

Proof. Dead-in-both mass enters the cemetery; alive mismatches contribute no ordinary overlap (Lemma 8.4); jointly alive rows have mass at most one by Cauchy–Schwarz. Thus $\mathrm{BC}_{\mathrm{ord}}^{(l-1)} \leq \sum_s g^{(l)}(s) \mathbf{1}\{s \in \mathrm{alive}_B^{(l)}\}$. Since $g^{(l)}$ depends only on kernels above l, conditional expectation contracts

ordinary overlap by η. Iterate over N layers from $\mathrm{BC}_{\mathrm{ord}}^{(L)} = 1$; subsequent layers cannot increase it. On $Z_A Z_B \geq z$, apply $\mathrm{H}_{\mathrm{surv}}^2 = 1 - \mathrm{BC}_{\mathrm{ord}}^{(0)}/\sqrt{Z_A Z_B}$ (Theorem 8.3). Splitting full overlap into ordinary and cemetery terms gives the final identity. □

Remark 8.2 (Cemetery floor). For the nonleaking Stopping kernel, shared dead states accumulate permanent cemetery overlap beyond the ordinary-overlap bound:

$$g^{(l-1)}(\dagger) = g^{(l)}(\dagger) + \sum_{s \in \mathrm{dead}_A^{(l)} \cap \mathrm{dead}_B^{(l)}} g^{(l)}(s).$$

Positive accumulated mass keeps plain H below one. A mean-field model with uniform A-dead fraction q and B-alive probability p contracts ordinary overlap by $(1-q)p$ and absorbs fraction $q(1-p)$ per layer, giving $g^{(0)}(\dagger) \to q(1-p)/(1-(1-q)p)$ as $N \to \infty$. The illustration $q = 0.2$, $p = \frac{1}{2}$ gives $\mathrm{H} \approx 0.91$, comparable to 0.94–0.97 (Table 8.3). Conditioning removes this floor: for fixed $z > 0$ with $\mathbb{P}_B(Z_A Z_B \geq z)$ bounded away from zero, $\mathbb{E}_B[\mathrm{H}_{\mathrm{surv}}^2 \mid Z_A Z_B \geq z] \to 1$. Activation-weighted Routing-Game kernels concentrate overlap on large $|aW|$ contributions, rarely shared dead neurons, explaining the observed practical proximity of H and $\mathrm{H}_{\mathrm{surv}}$.

Theorem 8.6 (Randomization in the Routing Game, `Lean_89`). *In the same setup, replace the alive-set assumption by a uniform bound $\sum_{s'} \widetilde{T}^{(l)}(s,s') \leq \bar{b} < 1$, $\bar{b} \geq 0$, on every jointly alive row at $l \geq l^*$, using temperature $\tau > 0$. Then $\mathrm{BC}_{\mathrm{ord}}^{(0)}(\Pi_A, \Pi_B) \leq \bar{b}^N$.*

Proof. The Gibbs policy (8.8) assigns zero mass to dead predecessors because their activations vanish. Thus intermediate overlap is supported on jointly alive states; a non-jointly-alive start sends no ordinary overlap onward. The row bound gives $\mathrm{BC}_{\mathrm{ord}}^{(l-1)} \leq \bar{b}\,\mathrm{BC}_{\mathrm{ord}}^{(l)}$; iterate as above. No additional alive-set factor is needed: successor mismatches already reduce row overlap.

Each act-to-act row factors into the source-alive indicator, uniform κ-branch factor $\frac{1}{2}$, and Gibbs policy proportional to $(a_{M,i}^{(l-1)} W_{M,ij}^{(l,\kappa)})^{1/\tau}$. At l^* the models share predecessor activations but may differ in weights; above it, both factors may differ. Continuous randomization alone does not ensure distinct normalized rows: a sole positive-mass route in each sign branch remains unchanged under sign-preserving weight changes. The uniform $\bar{b} < 1$ bound requires an architecture- and randomization-specific argument or empirical assessment; Cauchy–Schwarz gives strict contraction when completed normalized rows differ. □

The discounted occupation measures at the input layer recover gradient and $\alpha\beta$-LRP maps (Theorems 4.5 and 4.7). The undiscounted endpoint marginal $m_M^{(0)}$ is generally different: Hellinger compares the trajectory routing laws and does not include the path discounts. Under their respective contraction hypotheses, Theorems 8.5–8.6 show that Π_M changes drastically under randomization, while Table 8.2 shows that the resulting attribution maps can remain similar. The two diagnostics therefore detect different aspects of parameter sensitivity: the map-level test of Adebayo et al. (2018) asks whether the explanation delivered to the user depends on the learned parameters, the trajectory-level test whether its normalized routing does. A map can fail the first while its routing passes the second; this discrepancy does not isolate a fixed projection loss, since the discounts can also change across models, and it does not make such a map adequate for model debugging (Adebayo et al., 2018; Sixt et al., 2020).

8.4.2 Heatmap similarity at the cascade endpoint

We apply the cascading randomization test of Adebayo et al. (2018) to Table 8.2's attribution methods. Starting pretrained, we cumulatively re-initialize convolutional and linear layers output-to-input: weights Kaiming normal (He et al., 2015), biases $\mathcal{N}(0, 0.01^2)$. BatchNorm (ResNet-50), LayerNorm, positional embeddings, and class token (ViT-B/16) retain pretrained values. The ViT attribution cascade randomizes decomposed query/key/value projections; the Hellinger cascade (Section 8.2.1) retains packed query/key/value parameters. At each depth, we compare original and randomized models' attributions for every test image. The test asks whether maps change, without requiring monotonic similarity decay. Persistent similarity can reflect input structure or architecture.

Metrics. We report complementary map-similarity measures:

- **Spearman** $\rho(|a|)$: rank correlation of absolute attributions, measuring agreement in pixel importance. Absolute values accommodate differing attribution sign conventions; the correlation itself remains signed.
- **HOG** r: Pearson correlation of Histograms of Oriented Gradients (HOG) descriptors (Dalal and Triggs, 2005), measuring local-gradient-orientation structure.

We omit SSIM. Binder et al. (2023) show that input-dependent structure and activation scales can survive top-down randomization, while low similarity can reflect noise; this noise critique also covers Spearman correlation of absolute attributions. Values average over the 565-image ImageNet-S test split (50 classes). Constant maps receive similarity 0 by convention: at ResNet-50's cascade endpoint, LIME and KernelSHAP outputs no longer depend on superpixels. Table 8.2 reports this endpoint, with all tested convolutional and linear layers re-initialized and the parameter groups above retained.

Discussion. Gradient and Grad-CAM stay closest to zero on the CNNs, as does LayerCAM on VGG-16; on ViT-B/16, Gradient, SmoothGrad and GAE do. These scores record map changes under the tested intervention, not faithfulness. LRP-ε retains the most VGG-16 rank structure (0.341). ResNet-50's LRP rules (LRP-γ 0.254, LRP-ε 0.250) approach Integrated Gradients and LayerCAM, which retain most there. LIME and KernelSHAP retain HOG correlation about 0.86 with low Spearman correlation on VGG-16 and ViT-B/16: input-derived superpixels preserve edge structure. On ResNet-50, endpoint outputs no longer depend on superpixels, yielding constant maps (‡). The RG rows lie between Gradient and LRP-ε on VGG-16, slightly below LRP on ResNet-50, and highest in ViT-B/16 Spearman correlation (0.454 anchor, 0.450 selected mode). Grad-CAM++ on ResNet-50 and TIBAV on ViT-B/16 become anti-correlated, rather than unrelated. Section 8.4.1 complements map similarity with trajectory-distribution divergence.

8.4.3 Double-Random Empirical Baseline

Our cascade-endpoint reference compares two copies of each architecture with independently randomized convolutional and linear layers, retaining the pretrained parameter groups below. This differs from Adebayo et al. (2018)'s trained-versus-randomized comparison.

Tab. 8.2: Parameter randomization sanity check adapted from Adebayo et al. (2018). Each cell reports the average similarity between the attribution of the original pretrained model and the attribution of a copy randomized over the tested convolutional and linear layers (every convolutional and linear layer re-initialized, including the decomposed attention query/key/value projections; normalization layers and the ViT embeddings stay pretrained). Values near zero indicate low map similarity under this parameter intervention; the Spearman correlation is signed, so negative values occur. Bold marks the value closest to zero per metric within each architecture, among maps that are not constant; ‡: the map is constant at the cascade endpoint (similarity 0 by convention). Rows: the reported configuration of every method (Section 7.6).

Method	Spear. $\rho(\lvert a\rvert)$	HOG r
VGG-16		
Gradient	0.013	**0.009**
SmGrad	0.084	0.101
IntGrad	0.207	0.348
DeepLIFT	0.216	0.340
LRP-ε	0.341	0.457
LRP-γ	0.092	0.370
GCAM	**-0.007**	0.069
GCAM++	0.180	0.129
LCAM	-0.011	0.052
LIME	0.257	0.864
KernelSHAP	0.075	0.868
SG (λ=-1,σ^2=5, gate on)	0.289	0.456
RG anchor	0.299	0.535
RG (τ=0.3,ε=0.1)	0.284	0.287
ResNet-50		
Gradient	0.148	0.134
SmGrad	0.233	0.214
IntGrad	0.277	0.424
DeepLIFT	0.224	0.351
LRP-ε	0.250	0.393
LRP-γ	0.254	0.130
GCAM	**0.020**	**0.072**
GCAM++	-0.502	0.151
LCAM	0.266	0.145
LIME‡	0.000	0.000
KernelSHAP‡	0.000	0.000
RG anchor	0.217	0.344
RG (τ=0.9,ε=1)	0.180	0.316
ViT-B/16		
Gradient	**-0.003**	0.041
SmGrad	0.010	**0.038**
IntGrad	0.272	0.362
LRP-ε	0.165	0.300
LRP-γ	0.027	0.234
AttnLRP	0.124	0.176
DAVE	0.178	0.323
TIBAV	-0.175	0.150
GAE	-0.009	0.130
AR	0.120	0.153
LIME	0.248	0.861
KernelSHAP	0.131	0.867
RG anchor	0.454	0.469
RG (τ=0.4,λ_{sm}=-10,σ^2=5,ε=0.1)	0.450	0.310

Spear. $\rho(\lvert a\rvert)$ = Spearman rank correlation of absolute attribution values; HOG r = Pearson correlation of HOG descriptors. RG anchor = the $\alpha\beta$-LRP representation (τ=1, ε=0.5); the second RG row and the SG row are the localization-selected configurations and list the modes that differ (τ = temperature; λ, λ_{sm} = risk aversion; σ^2 = ADF input variance; ε = LRP stabilizer; $\tilde{\Phi}$ = noisy-observation gate).

Protocol. For each architecture, $R = 500$ independent rounds comprise: *(i)* copy the pretrained checkpoint to A_r, B_r and independently re-initialize every `Conv2d/Linear` layer as at the cascade endpoint. Both retain normalization parameters, packed attention query/key/value projections, positional embedding, and class token. We use checkpoints for all architectures: `weights=None` gives ViT an identically zero classification head, making trajectory laws trivially indistinguishable. *(ii)* Draw one image with replacement from the 565-image ImageNet-S test split (336 distinct images among 500 draws with our fixed seed). *(iii)* Compute plain $\mathrm{H}(\Pi_{A_r}, \Pi_{B_r})$ and survival-conditioned $\mathrm{H}_{\mathrm{surv}}$ for both Stopping and Routing Game kernels, targeting the ground-truth class. Table 8.3 reports round-to-round means and standard deviations; variance mixes image selection and random-model-pair realization.

	Stopping Game		Routing Game	
Architecture	H	$\mathrm{H}_{\mathrm{surv}}$	H	$\mathrm{H}_{\mathrm{surv}}$
VGG-16	0.939 ± 0.003	1.000 ± 0.000	1.000 ± 0.000	1.000 ± 0.000
ResNet-50	0.938 ± 0.003	1.000 ± 0.000	0.973 ± 0.002	1.000 ± 0.000
ViT-B/16	0.958 ± 0.002	1.000 ± 0.000	0.956 ± 0.001	1.000 ± 0.000

Tab. 8.3: Double-random Hellinger distances, mean $\pm$ standard deviation over $R = 500$ rounds. Each round pairs independently randomized copies of the architecture on one sampled ImageNet-S test image, retaining the pretrained parameter groups specified above.

Interpretation. Plain H concentrates at 0.94–0.97, consistent with shared dead-mask overlap and the cemetery floor of Remark 8.2; only VGG-16 RG saturates at 1.000. Conditioning removes this floor: $\mathrm{H}_{\mathrm{surv}} \approx 1$ indicates essentially disjoint surviving path laws throughout. Pretrained-versus-randomized distances near this architecture-specific reference are comparable to those between two copies independently randomized over the same layers.

8.4.4 Per-Image Association Between Trajectory and Map Diagnostics

Beyond the aggregate randomization bounds and double-random reference, Table 8.4 asks whether greater trajectory displacement corresponds to a more dissimilar attribution map *per image*, at fixed randomization depth.

Protocol. For each architecture we pick a cascading step at the same network depth in the Hellinger and attribution-sanity runs: the layers down to `features.26` on VGG-16 and down to `laye r4.1.conv3` on ResNet-50 (identical sets of re-initialized layers), and the last encoder block on ViT-B/16, where the Hellinger cascade re-initializes the MLP and the output projection of the block and the attribution cascade additionally its Q, K and V projections. The interventions therefore match on the CNNs; on ViT-B/16, the comparison also reflects this difference in the randomized operators. Figure 8.1 gives mean H and $\mathrm{H}_{\mathrm{surv}}$ at these steps. For each of the 565 test images, we record both distances for each game and Spearman $\rho(|a|)$ and HOG map similarity between pretrained and partially randomized models, using Gradient and the $\alpha\beta$-RG anchor at $\tau = 1$ as the respective game counterparts. Table 8.4 correlates these eight scalars across images by Spearman rank.

Architecture	Hellinger variant	Within Hellinger				Gradient		RG anchor	
		$H_{RG,pl}$	$H_{RG,surv}$	$H_{SG,pl}$	$H_{SG,surv}$	Spear. $\rho(\lvert a\rvert)$	HOG r	Spear. $\rho(\lvert a\rvert)$	HOG r
VGG-16	$H_{RG,plain}$	1.00	+1.00	−0.33	+0.85	+0.07	+0.10	−0.01	+0.04
	$H_{RG,surv}$	+1.00	1.00	−0.33	+0.85	+0.07	+0.10	−0.01	+0.04
	$H_{SG,plain}$	−0.33	−0.33	1.00	−0.30	+0.12	+0.11	+0.00	−0.02
	$H_{SG,surv}$	+0.85	+0.85	−0.30	1.00	+0.01	+0.05	−0.10	−0.10
ResNet-50	$H_{RG,plain}$	1.00	+0.58	−0.08	+0.70	+0.05	−0.01	−0.08	−0.19
	$H_{RG,surv}$	+0.58	1.00	+0.16	+0.89	+0.15	+0.08	+0.05	−0.05
	$H_{SG,plain}$	−0.08	+0.16	1.00	+0.16	−0.04	+0.07	−0.01	+0.02
	$H_{SG,surv}$	+0.70	+0.89	+0.16	1.00	+0.16	+0.07	−0.01	−0.10
ViT-B/16	$H_{RG,plain}$	1.00	+0.96	+0.29	+0.30	+0.02	+0.00	−0.04	−0.08
	$H_{RG,surv}$	+0.96	1.00	+0.25	+0.28	+0.02	+0.02	−0.07	−0.07
	$H_{SG,plain}$	+0.29	+0.25	1.00	+0.99	−0.12	−0.06	−0.03	−0.05
	$H_{SG,surv}$	+0.30	+0.28	+0.99	1.00	−0.12	−0.06	−0.04	−0.05

Tab. 8.4: Per-image Spearman rank correlations across the 565-image ImageNet-S test split at the selected cascading steps of the Hellinger and attribution-sanity pipelines. The randomized layers match on the CNNs; the ViT-B/16 attribution cascade additionally randomizes Q, K, and V. Left block: within-Hellinger cross-correlations of the plain ($H_{\cdot,pl}$) and survival-conditioned ($H_{\cdot,surv}$) distances under the RG and SG kernels (self-entries shown as −; trivially +1). Right block: Hellinger-vs-map cross-correlations.

Readings. The Hellinger-vs-map block of Table 8.4 is uniformly near zero ($|\rho| \leq 0.19$ across all three architectures): trajectory displacement and map drift have weak observed per-image rank association. Statistical independence and nonlinear dependence remain unresolved; ViT-B/16's unequal interventions further limit conclusions about the diagnostics alone.

Within Hellinger, per-image H gives a stable ordering. When little mass reaches the cemetery, plain and conditioned distances rank images almost identically: $\rho = +0.96$ (RG) and +0.99 (SG) on ViT-B/16. VGG-16 RG gives identical values because it never routes to dead-neuron states. For SG on ReLU backbones, the normalizations decouple ($\rho = -0.30$ VGG-16, +0.16 ResNet-50): image-dependent dead masks change the cemetery share, which renormalization by $1/\sqrt{Z_A Z_B}$ removes. The two games' conditioned distances agree strongly on ReLU backbones ($\rho = +0.85$ VGG-16, +0.89 ResNet-50), but weakly on ViT-B/16 (+0.28), where the kernels capture partly different per-image structure.

Within a method, Spearman $\rho(|a|)$ and HOG agree well ($\rho = 0.65$–0.82 from the same per-image joins); Gradient and $\alpha\beta$-RG similarities agree weakly (−0.10 to +0.30, depending on architecture and metric). Alongside the weak trajectory–map association, this shows that the attribution method changes which images a pixel-level sanity check flags.

9 Conclusion and Future Work

9.1 Summary of Contributions

As interpretability guides model correction and intervention, both its internal representation and backward calculation matter. This thesis connects them through *network decomposition*, exposing a trained ReLU network's structure while preserving its predictions.

Chapter 3 defines valid decompositions as coordinated layer-wise stream pairs whose difference recovers the network. The mixing family f^{η}_{mix} interpolates between trivial and fully separated splits; at $\eta = 1$, the classical DC representation $f = g - h$ has convex, monotone streams routing $W^{(l,+)}, W^{(l,-)}$ separately (Zimmermann and Loho, 2026). Theorem 3.12 extends this construction to every activation with integrable curvature — GELU (Hendrycks and Gimpel, 2016), Swish (Ramachandran et al., 2018), Mish, tanh, sigmoid, softplus — canonically selecting minimal curvature; its bounded-variation extension recovers ReLU as the Dirac limit. The stopping split f_{stop} adds early termination. Convex stop/continue regularizers generate f^{Ω}_{reg}, with binary entropy yielding f^{θ}_{soft} and incorporating Dombrowski et al. (2022)'s softplus replacement. Stabilization enables deep split passes; the CNN extension covers convolutions, gated pooling, batch normalization, and residual connections.

Building on Gaubert & Vlassopoulos (2025), Chapter 4 gives the stopping split a two-player zero-sum SG interpretation on doubled states, with weight signs routing between P^+ and P^-. The game framework recovers gradients and $\alpha\beta$-LRP as equilibrium discounted occupation measures; temperature, risk aversion, and stabilizers become principled modifications. In Chapter 8, equilibrium policies induce a probability law Π over complete backward trajectories. It records normalized routing; signs and model-dependent discounts recover attribution. Section 8.1 compares this expanded-state law with NRM's normalized signed relevance on neuron walks (Xiong et al., 2026). Hellinger distance between trajectory laws is computable in one linear-time pass, lifting Adebayo et al. (2018)'s randomization check to trajectory space. Theorem 8.5 proves geometric decay of Bhattacharyya mass with randomized layers, detecting collapse despite similar heatmaps (Sixt et al., 2020).

Chapter 6 derives SplitCAM, SplitLRP, and SplitGrad from established attribution principles on separate positive and negative streams. Difference-of-convex input neural networks (DCINNs; Section 6.1.4), trained directly in split form, show distinct, class-dependent stream sensitivities on MNIST. Chapter 5 gives two Transformer extensions (Vaswani et al., 2017): a constructive DC split retaining full Query, Key, Value, softmax, LayerNorm, and GELU dependence, with convex streams monotone in independent non-negative split inputs (Section 5.3); and the conditioned Value-routing game evaluated on ViT-B/16 (Dosovitskiy et al., 2021).

Chapter 7 evaluates localization, faithfulness, robustness, and cost. Split methods use VGG-16/ResNet-18 on ImageNet-S; games use VGG-16/ResNet-50/ViT-B/16 on ImageNet-S, with ViT-B/16 transfer to Pascal VOC. One adapted $\alpha\beta$-LRP configuration leads all evaluated baselines in ViT localization and transfers this lead without re-tuning. LRP baselines lead faithfulness, while anchors are particularly robust. Declared validation grids, independent localization/faithfulness selections, bootstrap intervals, and Holm-corrected paired tests expose these trade-offs. Hellinger diagnostics distinguish trained from randomized calculations despite similar maps, increase with Gaussian input noise, and separate intra-/inter-class pairs for at least one game per tested architecture.

9.2 Limitations

Computational overhead. Split passes propagate two streams per channel and currently duplicate weight matrices. Reported SplitCAM runtimes are approximately 14–29 times those of corresponding ordinary Grad-CAM/LayerCAM configurations (Section 7.4.2); historical artifacts omit hardware, precision, and exact configuration IDs. Masked views could avoid weight duplication, while retaining both streams; this optimization remains unimplemented and unbenchmarked here.

Stabilization hyperparameters. Stabilization is essential at depth (Section 3.6), introducing thresholds and shifting schemes; games add temperatures τ, θ and risk aversion λ. Theory constrains these choices without determining them. Optimal settings, including SplitCAM/SplitLRP layers, depend on the metric.

Transformer structure and numerical stability. The exact DC theorem covers fixed finite inference graphs of the stated operations, positive LayerNorm ε, and nonempty masked-softmax rows. Its real-arithmetic guarantee coexists with severe cancellation in the numerical prototype; a practical stable implementation remains open. The evaluated Value-routing game conditions on attention, LayerNorm statistics, and GELU gates, omitting the full Q/K derivative. It specifies a backward rule without establishing a unique explanatory game. ViT-B/16 results concern this conditioned game alone.

Benchmark scope. Evidence concerns only the models, datasets, and metrics above. Human-mask localization measures spatial agreement, leaving decision faithfulness unresolved; sharper maps may discard decision-relevant evidence outside the object. Medical-image attributions, for example, highlighted a microscope artifact's boundary rather than its full annotated area (Pahde et al., 2025). Perturbation-based faithfulness rankings can depend on occlusion strategy (Blücher et al., 2024); neither metric family establishes general causal explanations. Transfer beyond vision and downstream task gains remain untested. Trajectory diagnostics were validated through sanity checks and controlled perturbations; dedicated calculation-level benchmarks do not yet exist.

Stream interpretation. The small-model MNIST examples are qualitative. In inverted-input DCINNs, the full logit gradient sums both plotted sensitivities, so one stream can suggest a direction outweighed by the other, despite convexity when the first affine layer is unconstrained. Concentration on present and candidate missing strokes warrants complete-model interventions; it certifies neither finite counterfactuals nor necessary features.

9.3 Broader Impact

This methodological study of existing image classifiers introduces no deployed system, dataset of people, or high-risk generative model. By exposing different calculations behind similar maps, trajectory comparisons support diagnostically accountable interpretability and careful scientific/engineering audits.

Sharper or more stable maps can create unwarranted confidence in brittle, biased, or dataset-dependent classifiers, particularly in safety-critical or fairness-sensitive settings. Hellinger distance measures internal paths; a small distance establishes neither social validity nor fairness, robustness, or deployment

suitability. Results from public benchmark classifiers and ImageNet-derived data support attribution analysis, without causal guarantees or certification for consequential use.

9.4 Future Research Directions

Concept queries on the expanded state graph. NRM neuron-set queries (Xiong et al., 2026) suggest game-trajectory events for cross-layer concept analysis (Section 8.1), through their probabilities and separate signed discounted occupation measures. Correspondence with NRM/CRP requires matching propagation factors, normalization, and absorption conventions; discounts cannot replace probability normalization. Hellinger comparisons conditioned on positive-probability concept events could supply calculation-level benchmarks (Section 9.2). This extension remains proposed.

Regularized decompositions for smooth activations. Joining canonical smooth DC–monotone splits with f^{Ω}_{reg} (Section 3.5) remains open. GELU and Swish cannot be identified directly with a conjugate activation

$$\Omega^*(z) = \sup_{\pi \in [0,1]} \{\pi z - \Omega(\pi)\}$$

which is convex, non-decreasing, and 1-Lipschitz, properties they lack globally. Possible approaches combine their exact splits with games on non-negative streams or formulate local games conditional on realized activation patterns. Transferring trajectory diagnostics requires corresponding recovery and backward-occupation theorems.

DCINNs at scale. Counterfactual evaluation requires complete-model interventions and could use the fidelity, understandability, and sufficiency criteria of visual counterfactual explainers (Bender et al., 2026). Scaling raises further questions: stable difference-of-monotone training at ImageNet scale, acceleration through DC-programming (Le Thi and Pham Dinh, 2018) or ICNN ideas (Amos et al., 2017), and capacity's effect on the accuracy–interpretability trade-off. Positive results would strengthen split architectures as an interpretable design principle.

Transformers, LLMs, and efficient split passes. The theorem covers fixed causal masks with nonempty allowed-key sets. Autoregressive LLMs applications require stable stream evaluation, efficient key–value caching, and robustness tests; trajectory diagnostics additionally require a specified backward rule and recovery guarantees. Non-negative split weights invite hardware-level exploitation. Layer-dependent attribution quality motivates principled layer selection or multi-layer aggregation, treating layers as intermediate marginals of Π.

The same trained weights support convex monotone parts, games between cooperating adversaries, and backward-trajectory laws. These perspectives provide mathematical access to computations for inspection, comparison, and modification. Their use in downstream model correction, training, or steering remains to be evaluated.

Appendix A

Experimental Protocol and Reproducibility

This appendix specifies the evaluation and reproduction protocols. Appendix B contains the complete results, validation selections, ablations and qualitative examples.

A.1 Split-Method Evaluation

The campaigns retain the distinct attribution targets described in Section 7.1. In the split campaign, misclassified images therefore pair the predicted-class attribution with another class's annotated mask.

A.1.1 Artifact Availability and Reproduction Scope

The Hidden Monotonicity tables retain their published-source provenance. The code checkout lacks original per-image outputs, the complete historical evaluation environment, and six advertised MNIST multi-seed training configurations. The runnable notebook-derived example cannot reconstruct those missing settings, so the available artifacts cannot establish exact reproduction of the historical tables.

Deterministic CPU fixtures exercise double-precision reconstruction, input derivatives, attribution, timing, and Pixel Flipping with an independently evaluated initial target score. They establish the tested implementation contracts; agreement with the original ImageNet results remains unverified without a rerun. The historical training runner used the test loader for epoch validation. The current runner reserves a deterministic 10% of training data for validation and evaluates the test set after training, recording partition indices and seed. This does not establish the checkpoint-selection procedure behind the historical MNIST figures.

A.2 Evaluation Protocol

A.2.1 Data and splits

The game-method campaign uses the released ImageNet-S50 validation annotations (Gao et al., 2023), which contain 752 images from 50 classes. We exclude 136 images whose annotation class differs from the official ILSVRC2012 label, leaving 616 eligible images. With seed 42, one image per class forms the **50-image validation split**, used only for configuration selection; the remaining 566 form the test pool. Images whose object mask lies entirely outside the 224×224 center crop are excluded after the split, because the mask-based metrics are undefined for them. This removes one test image and leaves **565 test images**. The explanation target is the dataset class, including when the model misclassifies the image. This population and target convention belong to the game-method protocol and need not coincide with the split-method campaign. The Pascal VOC 2012 transfer split is described in Section B.3.3.

A.2.2 Metric definitions

Let $x \in \mathbb{R}^{C\times H\times W}$ be an image, let y be its target class, and let $a \in \mathbb{R}^{H\times W}$ be the scalar attribution map produced by an explanation method after channel reduction. Write $d \coloneqq HW$ and let $\mathrm{vec}(a) \in \mathbb{R}^d$ denote the flattened map. For any binary segmentation mask $m \in \{0,1\}^{H\times W}$, write $\mathrm{vec}(m) \in \{0,1\}^d$. When a metric is configured with `abs=true` and `normalise=true`, the evaluator applies the max-abs normalization (not min–max)

$$\psi(a) = \frac{|\mathrm{vec}(a)|}{\max_i |\mathrm{vec}(a)_i|}, \tag{A.1}$$

with the convention that the all-zero map remains zero. We denote the preprocessed attribution vector by $\bar{a}$. All metrics below use $\bar{a}$; faithfulness metrics use raw logits (no softmax) to avoid saturation effects, and perturbations use the Quantus per-sample mean replacement baseline.

Attribution localization (AL). We adapt Kohlbrenner et al. (2020)'s positive-relevance ratio within ground-truth boxes to absolute mass, pixel masks, and a stabilized denominator:

$$\mathrm{AL}(a,m) = \frac{\sum_{i=1}^{d} \bar{a}_i m_i}{\sum_{i=1}^{d} \bar{a}_i + 10^{-9}}. \tag{A.2}$$

Pointing game (PG). We use the pointing-game criterion of Zhang et al. (2018a); our configuration applies `abs=true` without normalization. Let $M(a) \coloneqq \{i \in \{1,\dots,d\} : \bar{a}_i = \max_j \bar{a}_j\}$ contain all maximizers. Then

$$\mathrm{PG}(a,m) = \mathbf{1}\big\{M(a) \cap \{i : m_i = 1\} \neq \varnothing\big\}. \tag{A.3}$$

A tie scores a hit if any maximizer lies inside the mask. PG is binary per image–class record; its ImageNet-S test mean is a multiple of $1/565$.

Top-k intersection (TK). Following Theiner et al. (2022), with $k = 1000$ and the permutation π that sorts $\bar{a}$ in descending order:

$$\mathrm{TK}(a,m) = \frac{1}{1000} \sum_{r=1}^{1000} m_{\pi_r}. \tag{A.4}$$

Pixel-flipping AUC ($\mathbf{PF}_5$, $\mathbf{PF}_{20}$). Pixel flipping (Bach et al., 2015; Samek et al., 2017) removes the most relevant features first and tracks the class score. The evaluator uses `pixel_flipping_abs` with `features_in_step = 1568` and `perturb_baseline = mean`. Unlike the spatial localization metrics above, pixel flipping operates on $D \coloneqq CHW$ scalar input features. The spatial attribution is broadcast over the color channels; let $\widetilde{\pi}$ sort the flattened broadcast attribution in descending order. For $T \coloneqq \lceil D/1568 \rceil$, the input $x^{(t)}$ is obtained from $x^{(t-1)}$, starting at $x^{(0)} = x$, by replacing features $\widetilde{\pi}_{(t-1)\cdot 1568+1}, \dots, \widetilde{\pi}_{\min\{t\cdot 1568, D\}}$ with the scalar mean of the current flattened input, as implemented by Quantus. The evaluator computes

$$f_t \coloneqq F(x^{(t)})_y, \qquad t = 1,\dots,T, \tag{A.5}$$

and PF_p is the trapezoidal area over the first $p\%$ of this flipping curve, i.e. over its first K_p points with the abscissa rescaled to [0,1]: $\mathrm{PF}_p = \mathrm{trapz}\big(\{(t-1)/(K_p-1), f_t\}_{t=1}^{K_p}\big)$, $K_p = \max(2, \lceil pT/100 \rceil)$. For $C = 3$ and $H = W = 224$, $D = 150{,}528$ and $T = 96$ steps, giving $K_5 = 5$ and $K_{20} = 20$. The curve contains only post-perturbation scores; the original input score is not an additional point. Lower is better.

Selectivity (Sel). Following Montavon et al. (2018) and Quantus (Hedström et al., 2023), sum $\bar{a}$ over square 39×39 patches (side $\lfloor\sqrt{1568}\rfloor$). Replace non-overlapping patches by the per-sample mean in decreasing attribution order, recording the target logit after each removal. Sel is the trapezoidal area under this curve against the fraction of removed patches. Lower is better.

Max and average sensitivity (MaxS, AvgS). Following Yeh et al. (2019)'s max-sensitivity and its mean analogue, regenerate attributions for both metrics on the same $N_s = 50$ inputs $x + \xi_k$, with uniform noise $\xi_k \sim \mathscr{U}[-r, r]^{C\times H\times W}$, $r = 0.2$. Report the maximum (MaxS) or mean (AvgS) over k of $\|\psi(a(x+\xi_k)) - \psi(a(x))\|_F / \|\psi(a(x))\|_F$; lower values indicate greater stability to input noise. Per-image seeds give every row identical perturbations. Both metrics use the same fixed random subset of 200 test images (seed 0) across rows and models; all other metrics use the full test split.

A.2.3 Selection

Every tunable method, including baselines, selects two configurations on validation: `loc` minimizes rank-sum over $\{\mathrm{AL}, \mathrm{PG}, \mathrm{TK}\}$, and `faith` over $\{\mathrm{PF}_5, \mathrm{PF}_{20}, \mathrm{Sel}\}$. Ranks are within-method and respect metric directions. Ties favor the default (the anchor for RG), then fewer non-default parameters, then grid order. Admission requires non-zero maps and defined metrics on every validation image. Before ranking, we exclude 5 of the 40 VGG-16 SG configurations, all with strong risk aversion and deterministic gates, whose maps vanish on some images; all others qualify. MaxS and AvgS are excluded from selection because each evaluation re-explains every image $2N_s$ times. The main discussion uses `loc`; Section B.3 reports both selections. Localization suits modes controlling routing focus, has external mask ground truth, and has precedent for comparing LRP rules using bounding boxes (Kohlbrenner et al., 2020). Perturbation metrics evaluate off-distribution inputs. Reporting both selections preserves the distinction and trade-off between localization and faithfulness.

A.2.4 Statistics

The game-method test tables report 95% percentile bootstrap confidence intervals (Efron and Tibshirani, 1993) using $B = 10{,}000$ resamples and seed 0. On ImageNet-S, the sampling units are images, with the same resampled image indices used across method rows. MaxS and AvgS intervals resample the fixed 200-image subset of Section A.2.2. Continuous metrics use a two-sided paired bootstrap test of the mean per-image difference: the bootstrap distribution is centered at zero, and p is the fraction of centered resampled differences at least as large in absolute value as the observed difference. Binary PG uses McNemar's test (McNemar, 1947) with continuity correction.

On Pascal VOC, the estimand remains the mean over image–class records, but the independent sampling units are the $G = 2{,}293$ original test images. Each bootstrap replicate samples G images with replacement and retains every class record of each sampled image. Its mean is the sum over these records divided by their replicate-specific count. Thus, classes from the same image remain together, while images with more evaluated classes retain their contribution to the reported record-weighted mean. Paired comparisons use common finite records and common image draws. Every VOC metric,

including binary PG, uses the corresponding two-sided, null-centered paired cluster-bootstrap test. For observed mean difference $\widehat{\Delta}$ and bootstrap differences Δ_b^*, its finite-resample p-value is

$$p = \frac{1 + \sum_{b=1}^{B} \mathbf{1}\{|\Delta_b^* - \widehat{\Delta}| \geq |\widehat{\Delta}|\}}{B+1}. \tag{A.6}$$

For the original localization-selected comparison and the full test tables, Holm correction (Holm, 1979) is applied to all unordered pairs of eligible rows within each metric and reported table, before extracting comparisons with the observed best row or declaring a row ahead of every baseline. The observed winner therefore does not determine the comparison family. A difference is called significant at adjusted $p < 0.05$; otherwise, no difference was detected at this threshold. This does not establish practical equivalence. These intervals and tests condition on the fixed checkpoints, selected configurations and realized attribution evaluations; they do not quantify training or configuration-selection variability. The split-method tables report the source point estimates without corresponding confidence intervals.

A.3 Implementation and Evaluation Details

Backbone and dataset provenance. All game-based experiments use the pretrained checkpoints shipped with the evaluation release of torchvision: VGG-16 uses `VGG16_Weights.IMAGENET1K_V1` (https://download.pytorch.org/models/vgg16-397923af.pth; recipe https://github.com/pytorch/vision/tree/main/references/classification#alexnet-and-vgg), ResNet-50 uses `ResNet50_Weights.IMAGENET1K_V2` (https://download.pytorch.org/models/resnet50-11ad3fa6.pth; recipe https://github.com/pytorch/vision/issues/3995#issuecomment-1013906621), and ViT-B/16 uses `ViT_B_16_Weights.IMAGENET1K_V1` (https://download.pytorch.org/models/vit_b_16-c867db91.pth; recipe https://github.com/pytorch/vision/tree/main/references/classification#vit_b_16). These ImageNet-1K classifiers were trained on ImageNet / ILSVRC (Deng et al., 2009; Russakovsky et al., 2015) and scored with Quantus (Hedström et al., 2023).

Licenses. ImageNet images (Deng et al., 2009) are used under the ImageNet terms of access (non-commercial research and educational use)[1]. The ImageNet-S mask release (Gao et al., 2023) does not state a separate mask license[2]; its images are ImageNet images and fall under the same terms. The Pascal VOC 2012 images are subject to the Flickr terms of use, as stated by the dataset providers[3]. The checkpoints are distributed with torchvision and were trained on ImageNet-1K. Libraries: PyTorch and torchvision (BSD-3-Clause), Quantus (GPL-3.0), zennit (GPL-3.0/LGPL-3.0), lxt, captum and scikit-image (BSD-3-Clause), pytorch-grad-cam (MIT); the reference code of TIBAV and GAE (MIT) is included with its license notice.

A.3.1 Methods

Baselines use lxt, zennit, captum, pytorch-grad-cam, and the authors' TIBAV/GAE code, with reference settings and the declared grids (Achtibat et al., 2024; Anders et al., 2026; Chefer et al., 2021a,b; Gildenblat and contributors, 2021; Kokhlikyan et al., 2020).

[1] https://www.image-net.org/download-images.php.
[2] Authors' release: https://github.com/LUSSeg/ImageNet-S.
[3] https://www.robots.ox.ac.uk/~vgg/projects/pascal/VOC/voc2012/, Database Rights.

Gradient-based block. **Gradient** is raw input-gradient saliency (Simonyan et al., 2014): the maximum absolute gradient over color channels. **SmGrad** averages noisy gradient maps (SmoothGrad) (Smilkov et al., 2017). **IntGrad** uses Integrated Gradients (Sundararajan et al., 2017) with a black-image baseline. **DeepLIFT** propagates activation differences from a reference input using the Rescale rule (Shrikumar et al., 2017) (CNNs only). **DAVE** is the distribution-aware ViT gradient decomposition of Wróbel et al. (2026); our implementation additionally freezes the LayerNorm mean, whereas published DAVE differentiates through mean-centering. **SG**, our SG, belongs here because its occupation measure recovers the gradient (Theorem 4.5).

LRP / relevance-propagation block. **LRP-ε** is stabilized Layer-wise Relevance Propagation (Bach et al., 2015); **LRP-γ** amplifies positive weights (Montavon et al., 2019). **AttnLRP** propagates transformer relevance through softmax, the bilinear attention product and the Value path (Achtibat et al., 2024). **RG** is our RG; **RG+Eq** adds equivariant Reynolds averaging, mainly to suppress ViT tokenization-grid artifacts. **TIBAV** (Chefer et al., 2021b), **GAE** (Chefer et al., 2021a) and Attention Rollout (**AR**) are transformer attention-map baselines.

CAM block. **GCAM**, **GCAM++** and **LCAM** denote Grad-CAM (Selvaraju et al., 2017), Grad-CAM++ (Chattopadhay et al., 2018) and LayerCAM (Jiang et al., 2021), respectively. LayerCAM weights class-activation evidence per location. All three use the CNNs' last convolutional block and bilinear upsampling.

Perturbation block. **LIME** is Local Interpretable Model-agnostic Explanations (Ribeiro et al., 2016); **KernelSHAP** estimates SHAP values by model-agnostic weighted regression (Lundberg and Lee, 2017). Both perturb SLIC superpixels (Achanta et al., 2012) and fit local surrogates, distinguishing them from the backward-propagation methods.

A.3.2 Declared validation grids

Table A.1 lists every method's grid. Functionally duplicate rows (e.g. different σ^2 when $\lambda = \lambda_{\text{sm}} = 0$) are evaluated once, yielding 2,021 configurations. On ViT-B/16, activation-side λ is fixed to 0, keeping GELU gates conditioned (Section 5.4). At attention–Value products, RG applies $\alpha\beta$-LRP-ε to the composite attention-product/Value-projection map (Theorem 5.8; Section 5.4.6). Baselines without grids use their reference implementations' settings.

Tab. A.1: Per-method validation grids (all architectures unless stated). Symbols: Section B.3.

Method	Grid
Gradient	single config
SmGrad	N=50, noise level $\sigma/(x_{\max}-x_{\min}) \in \{0.1, 0.15, 0.2\}$, mean aggregation (3 configs)
IntGrad	black-image baseline, $N_{\text{steps}} \in \{20, 50, 100, 300\}$ (4 configs)
DeepLIFT	Rescale rule, single config (CNNs only)
DAVE	$N \in \{25, 50\} \times \theta_{\max} \in \{0, 10, 20\} \times s_{\max} \in \{0, 0.05, 0.1\} \times t_{\max} \in \{0, 0.25, 0.5\}$, horizontal flips with probability 0.5, averaged effective map multiplied by the input (54 configs; ViT-B/16 only)
SG	$\lambda \in \{-5, -2, -1, -0.3, 0\} \times \sigma^2 \in \{1, 2, 5, 10\}\times$ noisy gate $\tilde{\Phi} \in \{\text{off}, \text{on}\}$ (40 configs; VGG-16 only)
LRP-ε	$\varepsilon \in \{0.01, 0.05, 0.1, 0.25, 0.5, 1, 2\}$ (7 configs)
LRP-γ	$(\varepsilon, \gamma) \in \{0.1, 0.25, 0.5\} \times \{0.1, 0.25, 0.5, 1, 2\}$; ViT-B/16: γ on all Linear layers (15 configs); CNNs: both zennit's EpsilonGammaBox composite ($z^{\mathscr{B}}$ input layer, γ on convolutions, ε on dense layers) and γ on all layers (30 configs)
AttnLRP	γ-composite of Achtibat et al. (2024): reference setting $(\gamma_{\text{conv}}, \gamma_{\text{lin}}, \varepsilon_{\text{attn}})=(0.25, 0.05, 10^{-6})$ + univariate sweeps $\gamma_{\text{lin}} \in \{0.01, 0.025, 0.1, 0.25\}$, $\gamma_{\text{conv}} \in \{0.1, 0.5, 1\}$, $\varepsilon_{\text{attn}} \in \{0.01, 0.05, 0.25\}$ (11 configs; ViT-B/16 only)
TIBAV	single config (ViT-B/16 only)
GAE	single config (ViT-B/16 only)
AR	residual weight $w_r \in \{0, 0.25, 0.5, 0.75\}$ (4 configs; ViT-B/16 only)
RG	$\tau \in \{0.3, 0.4, \ldots, 1.0\}$, $(\alpha,\beta)=(2,1)$, $\sigma^2 \in \{1, 2, 5, 10\}$, $\varepsilon \in \{0.1, 0.25, 0.5, 1\}$; ViT-B/16: λ=0, $\lambda_{\text{sm}} \in \{-50, -10, -5, -2, -1, 0\}$, composite Value rule of Theorem 5.8 (672 configs); CNNs: $\lambda \in \{-5, -2, -1, -0.3, 0\}$ (544 configs)
RG+Eq	$\tau \in \{0.3, 0.4, \ldots, 1.0\} \times \lambda_{\text{sm}} \in \{0, -10\}$, σ^2=2, ε=0.5, N=50, composite Value rule (16 configs; ViT-B/16 only)
GCAM, GCAM++, LCAM	last convolutional block (VGG-16: `features.29`, ResNet-50: `layer4`), single config (CNNs only)
LIME	80 SLIC superpixels, N=1000, ridge surrogate, exponential kernel of width 0.25 on the cosine distance of the binary superpixel vectors, removed superpixels filled with their mean color; fixed reference row
KernelSHAP	80 SLIC superpixels, N=2M+2048 for M superpixels, removed superpixels filled with the image mean; fixed reference row
Anchor	$\alpha\beta$-LRP anchor $(\alpha,\beta,\varepsilon,\tau)=(2,1,0.5,1)$ of RG and RG+Eq, reported as the theorem-driven reference row (not selected)

A.3.3 Compute resources

Evaluation is inference-only; repeated backward passes across validation/test grids dominate its cost, including equivariant-Reynolds and noise averaging. Runtime was measured on one NVIDIA A100-SXM4-40GB at batch size 1. Sweeps used several GPUs of that type, with up to two processes per GPU. The full pipeline (validation, test, robustness, Pascal VOC, ablations and randomization diagnostics) required about 450 GPU-hours.

A.3.4 Code and correctness tests

The code (https://github.com/JayPiZimmermann/Playing_The_Network_Backward_Official_Repository) tests the thesis's backward rules. In float64, `tests/test_attention_ab_recovery.py` compares RG at $\tau = 1$ with an independent $\alpha\beta$-LRP-ε implementation of $X \mapsto A(XW_v^\top + b_v)$ (Theorem 5.8, (5.21)). Single-head and `nn.MultiheadAttention`-layout blocks use signed inputs, Value bias and $\varepsilon > 0$. The test also distinguishes the sequential variant of Section 5.4.1, unavailable in the release (`av_rule` accepts only `'merged'`). `tests/test_hellinger_merged_attention.py` enumerates every backward trajectory of a small attention block under two models at $\tau \in \{0.5,1,2\}$, checking the Hellinger pass's per-input, Bhattacharyya and cemetery masses (Section 8.3). A pre-norm residual `nn.MultiheadAttention` block must match the distance obtained with its attention matrix supplied as a constant.

A.4 Formal Verification and Mathematical Audit

The thesis repository contains TeX sources, pinned links to both official code repositories, and a Lean 4 project with mathlib pinned to 4.35.0. Checked statements have explicit assumptions. Formalization remains partial: a related lemma cannot certify an entire theorem or paper.

Numbered `Lean_1`, `Lean_2`, ... links in theorem and lemma headings follow the planar-features paper's convention. Each targets the corresponding checked declaration or components, distinguished in `formalization/coverage_map.json`, which records exact assumptions and remaining parts by source label. Selected equation numbers link to checked identities or definitions. Following arXiv:2610.01728, links pin the Git commit and declaration line, keeping targets stable under default-branch edits.

Both papers share a finite, varying-width Lean MLP definition: an identity output or an affine layer followed by a smaller network, evaluated with ordinary or paired activations. Local difference preservation suffices algebraically for paired activations; induction proves complete-network recovery. This definition supports ReLU, mixing and Softplus, with a separate affine readout for ungated target logits.

The Transformer development defines signed projections, actual masked Query–Key products, coupled softmax, attention–Value products, input-dependent LayerNorm, residual additions and the feedforward subnetwork. Finite block lists allow arbitrary depth, including zero, an optional final LayerNorm and signed affine readout. Compilation to an explicit expression language equals the ordinary evaluator. Recursive two-stream evaluation proves Theorem 5.5: exact reconstruction and convex, monotone, non-negative components on non-negative doubled inputs. The positive/negative raw-input lift gives global convexity, without asserting raw-input monotonicity. The proofs derive structural properties from the closed-form softmax and normalization constructions, with positive ε and nonempty allowed-key sets. They use exact real arithmetic; Section A.4.1 diagnoses direct evaluation's limits.

A forward trace records actual gates. Each sensitivity row maps an input perturbation linearly to its scalar output change; paired rows describe partial sensitivities to both input streams. Their half-difference is the derivative along the balanced lift $(x/2, -x/2)$. At strict gate comparisons, the recursive Jacobian equals the complete network's Fréchet derivative.

The layer-wise shifted backward algorithm equals its closed formula for arbitrary widths and shifts. Source terms are derivatives of explicit Q-networks with the original prefix, one minimum and a

bias-free linear absolute-weight tail. Checked identities cover shift-coefficient suffix products, sensitivity mass, separate cross-head partials and original-gradient reconstruction. Shifted algorithmic sensitivities remain distinct from derivatives through input-dependent forward re-centering; classical derivatives at gate ties are not asserted.

The game developments define finite sign-labeled paths, probabilities, discounts and occupation sums. Under the stated policies, discounted stopping occupations recover the actual MLP gradient; routing occupations recover the full $\alpha\beta$ recurrence from network contributions. The smooth case includes the full Softplus derivative. Attention uses a fixed matrix and the complete composite Value projection, including its bias; backward recovery alone asserts neither Query/Key differentiation nor global equilibrium. Generic finite layered-game saddle bounds and Bellman uniqueness are checked separately for actual randomized policies.

The trajectory module checks finite path normalization, dynamic-programming identities, geometric-mean overlap, terminal-state disagreement and positive survival conditioning. Hellinger probability laws remain distinct from discounted occupation measures. The regularization module checks the finite Gibbs variational maximum, attaining policy, binary stop/continue optimizer and actual Softplus derivatives.

The audit exposed a missing 1/2 in the four-partial backward invariant, a shift-product index error, an omitted weight matrix, and overclaims about differentiability, multiplicative convexity, re-centering and nonzero combined gradients. Corrections and counterexamples are in reports/mathematical-corrections.md; nonzero raw sensitivity mass need not yield a nonzero balanced head gradient.

The source inventory lists statements outside the checked scope. Proofs and corrections certify neither Python implementations nor experiments. Independently, the table adapter regenerates thirteen outputs from the revised game's published summaries and checks them against the thesis. Diagnostic and hidden-monotonicity tables lacking raw summaries retain source provenance. Reproduce mathematical checks with `lake build` in `formalization/` and its explicit axiom audit; the README gives complete commands.

A.4.1 Numerical Diagnostic of the Transformer DC Construction

The deterministic `scripts/transformer_dc_demo.py` tests Section 5.3's construction on a three-token, width-three Transformer: one causal head, two post-residual LayerNorms, a width-four exact Gaussian-GELU MLP, mean pooling and two signed logits. An independent ordinary PyTorch computation uses identical parameters. The diagnostic runs on CPU in float64, without recentering, clipping or reference-activation substitution.

With random seed 23, the sweep combines $\varepsilon \in \{1, 10^{-2}, 10^{-5}\}$, weight scales $\{0.01, 0.1, 0.5\}$, and input scales $\{0.1, 1\}$. Two additional cases add common input-stream offsets of 1 and 10 to the $\varepsilon = 1$, weight-scale-0.01, input-scale-0.1 case. All twenty cases, including failures, are retained in `reports/evidence/transformer-dc/numeric.json`, together with the environment, source hash, per-stage reconstruction error, and stream magnitudes. Run `python` scripts/transformer_dc_demo.py `-output RESULT.json`.

The benign case reconstructs logits within 2.78×10^{-17}; fifteen of twenty cases exceed 10^{-8} final maximum absolute error. Every tested $\varepsilon = 10^{-5}$ case loses output accuracy, with streams reaching 5.95×10^{111}. Separate primitive and zero-variance LayerNorm checks pass. These finite checks diagnose implementation, without proving convexity or numerical stability. Catastrophic cancellation affects floating-point evaluation of even this small construction; a stable practical implementation remains open.

Appendix B

Extended Experimental Results

The tables and figures below give the complete results for both campaigns. Protocols, metric definitions, validation grids, statistical tests and implementation details appear in Appendix A.

B.1 Full Evaluation Results for the Split Methods

This section reports all methods evaluated in the split-method setup of Section 7.1.

B.1.1 VGG-16 Qualitative Saliency Map Comparison

Figure B.1 extends Figure 7.1. SplitCAM at layer 26 again covers classified objects more fully than LayerCAM at that layer. SplitGrad at layer 0 detects too many input edges to explain the decision usefully, especially amid background details such as the rubble in the wood rabbit image. This agrees with the performance peaks of both methods in deeper layers.

B.1.2 Quantitative Analysis on VGG-16

Table B.1 contains the complete VGG-16 comparison discussed in Section 7.2. Selected rows can differ in layer and absolute-value convention; the controlled α sweeps below isolate these choices more directly.

Tab. B.1: Comprehensive interpretability evaluation results (VGG-16; representative configurations, best per method). *wta* denotes winner-takes-it-all max-pooling, and *sc* forward scaling (Section 3.6); layer brackets follow the split-method upper indices of Chapter 6. For each method configuration the best metric values across the considered layer configurations are highlighted in bold with dark blue shading. PF^{drop} is score-drop AUC; n/r denotes a result not reported in the paper, and n/a an inapplicable entry.

Method	Layer	Abs	Sel↓	AL↑	PG↑	PF_5^{drop}↑	PF_{20}^{drop}↑	MaxS↓
SplitCAM Methods								
SplitCAM (sc, α 0.4)	(12, g)	n	4.720	**0.651**	**0.912**	0.213	1.794	**0.389**
SplitCAM (sc, α 0.4)	(14, g)	n	**2.727**	0.563	0.836	**0.300**	**2.404**	1.012
SplitCAM (sc, α 0.4, wta)	(14, g)	n	**2.761**	0.543	0.827	**0.303**	**2.414**	1.056
SplitCAM (sc, α 0.4, wta)	(26, +, g)	y	4.711	**0.635**	**0.938**	0.165	1.590	**0.456**
SplitCAM (sc, α 0.5, wta)	(14, g)	n	**2.766**	0.536	0.823	**0.303**	**2.416**	1.056
SplitCAM (sc, α 0.5, wta)	(26, +, g)	n	4.608	**0.607**	**0.903**	0.185	1.684	**0.566**
SplitGrad Methods								
SplitGrad (α 0.4)	(12, +, g)	n	5.633	**0.613**	**0.873**	0.148	1.461	**0.364**
SplitGrad (α 0.4)	(14, +, g)	y	**3.429**	0.533	0.820	**0.235**	**2.119**	0.851
SplitGrad (α 0.4, wta)	(12, +, g)	n	4.735	**0.635**	**0.883**	0.186	1.756	0.651
SplitGrad (α 0.4, wta)	(14, +, g)	y	**3.287**	0.540	0.818	**0.249**	**2.186**	0.857
SplitGrad (α 0.5, wta)	(12, +, g)	y	**3.379**	**0.551**	0.859	**0.261**	**2.220**	0.869
SplitGrad (α 0.5, wta)	(26, +, g)	y	4.948	0.543	**0.899**	0.162	1.527	**0.617**

Continued on next page

Tab. B.1 – continued from previous page

Method	Layer	Abs	Sel↓	AL↑	PG↑	PF_5^{drop}↑	PF_{20}^{drop}↑	MaxS↓
SplitLRP Methods								
SplitLRP (sc)	(7, pos)	n	**3.062**	0.092	**0.857**	**0.296**	**2.290**	0.865
SplitLRP (sc)	(19, pos)	n	4.393	**0.600**	0.823	0.166	1.656	**0.291**
SplitLRP (sc, wta)	(7, pos)	n	**3.235**	-0.036	0.836	**0.296**	**2.291**	0.866
SplitLRP (sc, wta)	(26, pos)	y	5.168	**0.588**	**0.871**	0.151	1.473	**0.282**
Classical Methods (No Layer Optimization)								
Deconvolution	0	y	5.435	0.468	0.629	0.204	1.715	0.956
DeepLIFT	0	y	4.447	0.551	0.726	0.281	1.945	1.053
Feature Ablation	0	y	6.029	0.479	0.624	0.123	1.250	0.859
GradientSHAP	0	y	4.808	0.552	0.783	0.261	1.862	1.142
IntGrad	0	y	4.838	0.554	0.797	0.262	1.851	1.069
Occlusion	0	y	5.214	0.499	0.716	0.130	1.217	0.699
GCAM++	30	n	5.955	0.564	0.827	0.130	1.245	0.401
Classical Methods (Layer Optimization)								
Guided Backprop	2	n	**3.276**	**0.632**	0.883	**0.281**	**2.302**	0.571
Guided Backprop	5	n	3.448	0.614	**0.887**	0.240	2.091	**0.375**
Guided GCAM	0	n	**4.301**	**-0.053**	**0.857**	0.161	**1.459**	1.047
Guided GCAM	26	n	5.130	-2.635	0.843	**0.179**	1.415	**0.947**
GCAM	0	n	6.006	**0.356**	**0.638**	0.060	0.899	1.123
GCAM	14	n	**5.555**	0.306	0.580	**0.136**	**1.525**	**0.903**
LCAM	14	n/a	2.792	0.576	0.798	**0.317**	**2.474**	**2.656**
LCAM	2	n/a	**3.128**	**0.580**	**0.855**	0.300	2.191	5.508
LRP $\gamma = 0.25$	0	n	**3.189**	**0.615**	**0.680**	0.218	1.918	0.519
LRP $\gamma = 0.25$	14	n	3.593	0.577	0.611	**0.279**	**2.255**	**0.491**
Baseline Methods								
Baseline: Binary Mask	0	y	6.011	1.000	1.000	0.091	1.054	n/r
Baseline: Original Image	0	y	8.770	-1.699	0.572	0.061	0.619	n/r
Random Baseline	0	y	3.570	0.425	0.458	0.203	1.585	n/r

B.1.3 Quantitative Analysis on ResNet-18

Table B.2 reports all ResNet-18 split configurations, baselines and classical methods.

Tab. B.2: Comprehensive interpretability evaluation results (ResNet-18; representative configurations, best per method). Abbreviations as in Table B.1; triples $x.y.z$ identify the internal convolutional attribution layer. For each method configuration the best metric values across the considered layer configurations are highlighted in bold with dark blue shading. PF^{drop} is score-drop AUC; n/r denotes a result not reported in the paper, and n/a an inapplicable entry.

Method	Layer	Abs	Sel↓	AL↑	PG↑	PF_5^{drop}↑	PF_{20}^{drop}↑	MaxS↓
SplitCAM Methods								
SplitCAM (sc, α 0.3, wta)	(1.0.2, g)	n	**3.503**	0.487	0.747	**0.168**	**1.395**	1.277
SplitCAM (sc, α 0.3, wta)	(4.1.2, g)	n	5.654	**0.673**	**0.972**	0.081	0.880	**0.233**
SplitCAM (sc, α 0.5, wta)	(2.0.1, g)	n	**3.682**	-1.370	0.887	**0.158**	**1.447**	1.078
SplitCAM (sc, α 0.5, wta)	(4.1.1, +, g)	y	6.526	**0.611**	**0.956**	0.066	0.737	**0.187**

Continued on next page

Tab. B.2 – continued from previous page

Method	Layer	Abs	Sel↓	AL↑	PG↑	PF_5^{drop}↑	PF_{20}^{drop}↑	MaxS↓
SplitGrad Methods								
SplitGrad (α 0.3, wta)	(1.0.2, g)	y	**4.449**	0.513	0.777	**0.149**	**1.266**	0.813
SplitGrad (α 0.3, wta)	(4.1.2, g)	y	5.663	**0.649**	**0.972**	0.082	0.882	**0.219**
SplitGrad (α 0.5, wta)	(3.1.1, g)	y	**3.588**	0.559	0.880	**0.130**	**1.366**	0.859
SplitGrad (α 0.5, wta)	(4.1.2, g)	y	5.690	**0.654**	**0.972**	0.083	0.886	**0.230**
SplitLRP Methods								
SplitLRP (sc, wta)	(conv1, pos)	y	**6.313**	**0.506**	**0.587**	**0.056**	**0.697**	0.755
SplitLRP (sc, wta)	(1.0.2, pos)	y	6.785	0.483	0.567	0.050	0.686	**0.696**
Classical Methods (No Layer Optimization)								
Deconvolution	conv1	y	5.474	0.478	0.666	0.151	1.177	0.618
GradientSHAP	conv1	y	5.269	0.527	0.779	0.166	1.227	1.074
GCAM++	4.1.2	n	6.054	0.556	0.945	0.077	0.838	0.140
IntGrad	conv1	y	5.265	0.528	0.777	0.167	1.226	0.994
Feature Ablation	conv1	n	4.655	0.136	0.684	0.188	1.388	1.077
Occlusion	conv1	n	5.300	0.179	0.726	0.130	1.030	0.785
Deconvolution	conv1	n	3.934	0.484	0.666	0.147	1.203	0.751
Classical Methods (Layer Optimization)								
GCAM	2.0.1	y	**4.003**	0.450	0.678	0.052	**1.024**	0.738
GCAM	4.1.2	n	6.070	**0.554**	**0.945**	**0.076**	0.829	**0.131**
Guided GCAM	4.0.2	n	4.929	**0.742**	0.866	0.119	1.008	0.687
Guided GCAM	4.1.2	n	**3.843**	0.717	**0.883**	**0.129**	**1.096**	**0.612**
LCAM	2.0.2	n	**3.690**	0.549	0.865	**0.144**	**1.428**	0.913
LCAM	4.1.1	n	5.661	**0.664**	**0.971**	0.083	0.890	**0.227**
LRP $\gamma = 0.25$	conv1	y	**3.550**	**0.656**	0.852	**0.213**	**1.737**	0.521
LRP $\gamma = 0.25$	4.1.1	n	5.649	0.630	**0.965**	0.081	0.877	**0.200**
Guided Backprop	conv1	n	**3.517**	**0.621**	0.867	**0.212**	**1.757**	0.459
Guided Backprop	3.1.2	n	3.874	0.582	**0.938**	0.121	1.307	**0.213**
Baseline Methods								
Baseline: Original Image	conv1	y	8.351	-1.699	0.572	0.049	0.435	n/r
Baseline: Binary Mask	conv1	y	5.964	1.000	1.000	0.067	0.739	n/r
Random Baseline	n/a	y	4.071	0.425	0.440	0.147	1.067	n/r

B.2 Stabilization Ablations for the Split Methods

These ablations examine forward/backward stabilization and the max-pooling implementation (Section 3.6).

B.2.1 Backward-Pass Stabilization

Table B.3 gives the full α grid underlying Section 7.5.1. Layer-14 configurations show smaller or adverse changes in some metrics. Convex and winner-takes-it-all max-pooling differ little near $\alpha = 0.5$ and more at lower values; their ranking depends on configuration and metric.

Unresolved source-table discrepancy. For SplitCAM with scaling, $\alpha = 0.4$, winner-takes-it-all pooling and absolute-value configuration (26,+,g), Table B.1 reports $PF_{20}^{drop} = 1.590$ and MaxS= 0.456,

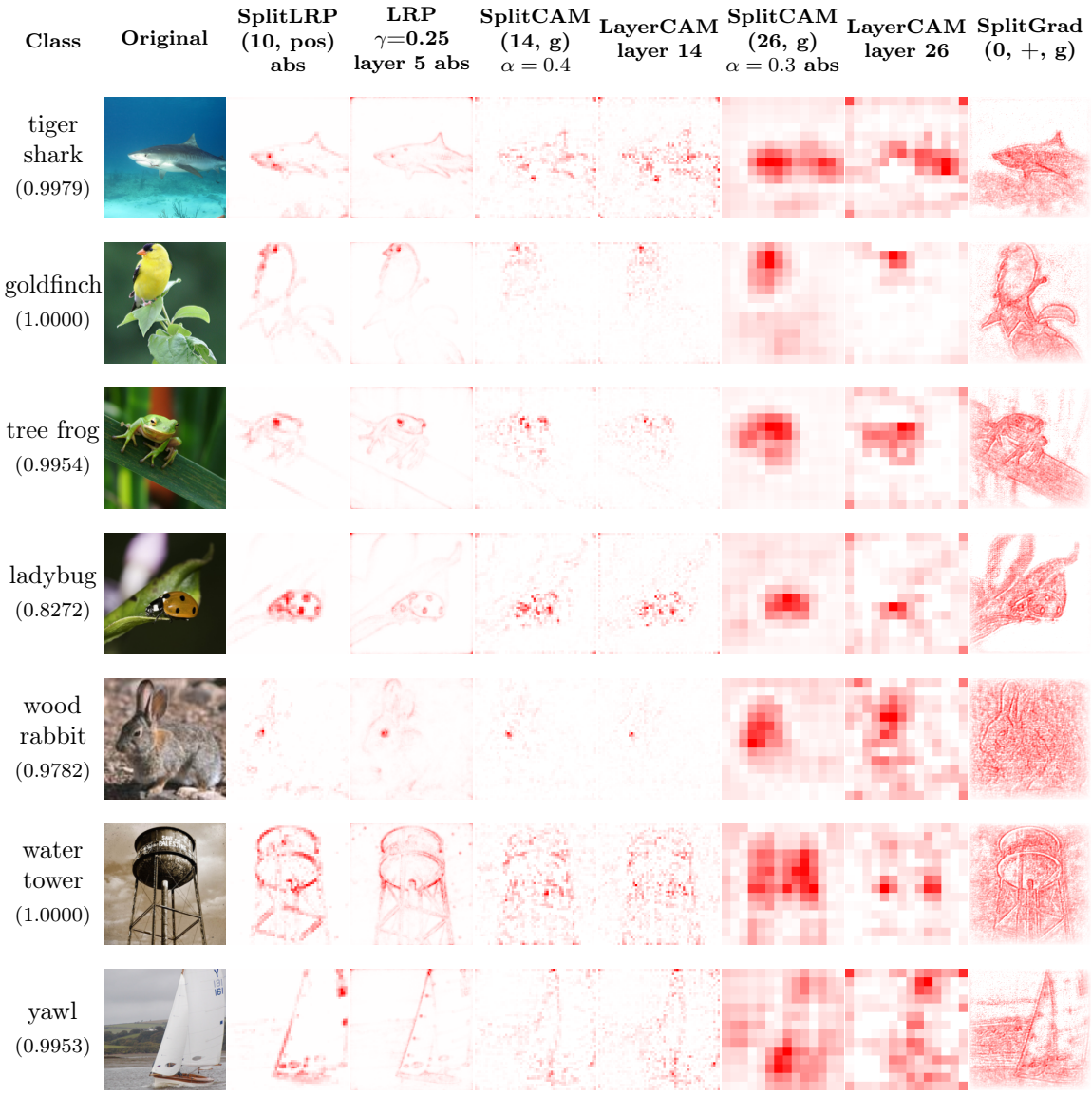


Fig. B.1: Saliency map comparison for VGG-16 on images from the custom ImageNet-S test subset. Each row shows the predicted class (with confidence), original image, and saliency maps from different methods. Methods include SplitLRP, classical LRP, SplitCAM variants, LayerCAM, and SplitGrad.

versus 1.5849 and 0.4573 in Table B.3. These source-table values remain unchanged: the per-image results needed to resolve their discrepancy are unavailable, leaving its cause unknown.

B.2.2 Forward-stabilization grids

Tables B.4 and B.5 give the full unstabilized and shift-stabilized SplitCAM grids discussed in Section 7.5.2.

B.3 Additional Results

One pipeline generates the validation, test and VOC tables from the declared grids and selection rules of Section A.2.

Tab. B.3: Alpha ablation as performed in Section 7.5 across four SplitGrad and SplitCAM configurations of layers 14 and 26, respectively. The notation is similar to Table B.1.

Method	Metric	α 0.50	α 0.45	α 0.40	α 0.35	α 0.30	α 0.00
	PG↑	0.8975	0.9064	0.9470	**0.9558**	0.8887	0.4011
SplitGrad (wta)	AL↑	0.5434	0.5476	0.6051	**0.6169**	0.5346	0.4486
(26, g), abs	MaxS↓	0.6160	0.6071	0.4399	0.2382	0.3344	**0.0283**
	PF_{20}^{drop} ↑	1.5270	1.5337	**1.5808**	1.4476	1.3171	1.2471
	PG↑	0.8975	0.9064	**0.9470**	0.9099	0.2827	0.6413
SplitGrad	AL↑	0.5434	0.5476	**0.6037**	0.5216	0.4245	0.4563
(26, g), abs	MaxS↓	0.6171	0.6102	0.4426	0.3864	0.0622	**0.0079**
	PF_{20}^{drop} ↑	1.5272	1.5333	**1.5876**	1.3702	0.2431	0.5755
	PG↑	0.8198	0.8198	0.8198	**0.8322**	0.8127	0.7898
SplitGrad (wta)	AL↑	**0.5403**	0.5403	0.5402	0.5006	0.4936	0.4857
(14, g), abs	MaxS↓	0.8579	0.8597	0.8584	0.0875	0.0677	**0.0481**
	PF_{20}^{drop} ↑	2.1887	**2.1894**	2.1869	1.8740	1.8304	1.7684
	PG↑	0.8198	**0.8198**	0.8180	0.7951	0.8004	0.7968
SplitGrad	AL↑	0.5403	**0.5403**	0.5330	0.5054	0.4926	0.4847
(14, g), abs	MaxS↓	0.8618	0.8584	0.8478	0.1103	0.0662	**0.0301**
	PF_{20}^{drop} ↑	**2.1894**	2.1891	2.1185	1.3425	1.2959	1.1927
	PG↑	0.8675	0.8675	**0.9382**	0.9329	0.6131	0.8551
SplitCAM (sc, wta)	AL↑	0.5501	0.5535	0.6347	**0.6627**	0.4033	0.5133
(26, +, g), abs	MaxS↓	0.6906	0.6820	0.4573	0.2645	0.2511	**0.1196**
	PF_{20}^{drop} ↑	1.6067	**1.6085**	1.5849	1.4989	0.8534	1.5071
	PG↑	0.8940	0.8905	**0.9346**	0.6466	0.8251	0.8604
SplitCAM (sc)	AL↑	0.5595	0.5636	**0.6363**	0.4015	0.4977	0.5114
(26, +, g), abs	MaxS↓	0.6738	0.6640	0.4520	0.3729	0.1102	**0.1062**
	PF_{20}^{drop} ↑	1.5902	**1.5978**	1.5906	0.9076	1.2784	1.3605
	PG↑	0.8251	0.8251	**0.8269**	0.7880	0.7809	0.7703
SplitCAM (sc, wta)	AL↑	0.5363	0.5363	**0.5435**	0.5338	0.5286	0.5225
(14, g)	MaxS↓	1.0566	1.0569	1.0489	**0.1620**	0.1625	0.1630
	PF_{20}^{drop} ↑	2.4128	2.4130	**2.4135**	1.9014	1.8613	1.8731
	PG↑	0.8251	0.8251	**0.8428**	0.7756	0.7686	0.7756
SplitCAM (sc)	AL↑	0.5363	0.5363	**0.5626**	0.5363	0.5271	0.5208
(14, g)	MaxS↓	1.0552	1.0593	1.0136	0.1629	0.1585	**0.1555**
	PF_{20}^{drop} ↑	2.4171	**2.4172**	2.4065	1.5032	1.5082	1.5139

Configuration symbols. Parameter grids and baseline recipes appear in Table A.1. For RG, RG+Eq and SG, τ is temperature; λ and λ_{sm} are activation-side and attention-logit risk aversion; σ^2 is ADF input variance; ε is the LRP stabilizer; (α,β) are stream weights; $\tilde{\Phi}$ is the noisy-observation gate; and N counts transformed inputs averaged by RG+Eq. For SmoothGrad, N counts noisy samples and σ/range is noise standard deviation relative to input range; N_{steps} counts Integrated Gradients steps. DAVE averages N inputs with rotations up to θ_{max} degrees, shifts up to s_{max} of image extent, and variance-preserving Gaussian-noise interpolation weight up to t_{max}. AttnLRP uses γ-rule parameters γ_{conv}, γ_{lin} for convolutional and linear layers, and attention stabilizer ε_{attn}. For

Tab. B.4: Alpha ablation for scale normalization across four SplitCAM configurations of layers 14 and 26. Here, in the forward pass *no* stabilization is applied. Notation and structure are similar to Table B.3. Light blue entries marked with * indicate better performance of the method compared to the default variant with scale stabilization in the forward pass.

Method	Metric	α 0.50	α 0.45	α 0.40	α 0.35	α 0.30	α 0.00
	PG↑	0.7686	0.7686	0.8632	**0.8712**	0.3441	0.5453
SplitCAM (sc, wta)	AL↑	0.5374	0.5412	0.5943	**0.6118**	0.3595	0.4569
(26, +, g), abs	MaxS↓	0.6578*	0.6507*	0.4841	0.2873	0.1918*	**0.0135***
	PF_{20}^{drop} ↑	1.2744	1.2859	**1.3209**	1.1792	0.4430	0.6527
	PG↑	0.7565	0.7565	**0.8451**	0.3783	0.4507	0.4970
SplitCAM (sc)	AL↑	0.5346	0.5384	**0.5897**	0.3663	0.4420	0.4545
(26, +, g), abs	MaxS↓	0.6604*	0.6512*	0.4985	0.3077*	0.0279*	**0.0043***
	PF_{20}^{drop} ↑	1.2680	1.2856	**1.3295**	0.5912	0.4906	0.5563
	PG↑	**0.8491***	0.8491*	0.8491*	0.7867	0.7847*	0.7686
SplitCAM (sc, wta)	AL↑	0.5324	0.5321	**0.5406**	0.5306	0.5261	0.5200
(14, g)	MaxS↓	1.0834	1.0814	1.0885	**0.2271**	0.2309	0.2384
	PF_{20}^{drop} ↑	2.4454*	2.4482*	**2.4486***	1.9588*	1.9193*	1.9349*
	PG↑	0.7746	0.7767	**0.7847**	0.5855	0.5594	0.5573
SplitCAM (sc)	AL↑	0.4003	0.4144	**0.5517**	0.5300	0.5209	0.5137
(14, g)	MaxS↓	1.5733	1.5627	1.5443	**1.2661**	1.3939	1.4272
	PF_{20}^{drop} ↑	2.1298	**2.1384**	2.1371	1.6669*	1.6793*	1.6740*

Tab. B.5: Alpha ablation for scaling normalization across four SplitCAM configurations of layers 14 and 26. Here, in the forward pass shift normalization is applied. Notation and structure are similar to Table B.3. Light blue entries marked with * indicate better performance of the method compared to the default variant with scale stabilization in the forward pass.

Method	Metric	α 0.50	α 0.45	α 0.40	α 0.35	α 0.30	α 0.00
	PG↑	0.8330	0.8149	0.7968	**0.9235**	0.6036	0.8592*
SplitCAM (sc, wta)	AL↑	0.5242	0.5203	0.5147	**0.6549**	0.3968	0.5093
(26, +, g), abs	MaxS↓	0.6180*	0.6182*	0.6509	0.3104	0.2791	**0.1665**
	PF_{20}^{drop} ↑	**1.6804***	1.6646*	1.5065	1.4962	0.8896*	1.5405*
	PG↑	0.8330	0.8149	0.7988	0.5654	0.8431*	**0.8652***
SplitCAM (sc)	AL↑	**0.5242**	0.5203	0.5076	0.3736	0.4949	0.5094
(26, +, g), abs	MaxS↓	0.6129*	0.6168*	0.6494	0.3820	0.1686	**0.1664**
	PF_{20}^{drop} ↑	**1.6804***	1.6646*	1.5105	0.8714	1.4289*	1.4765*
	PG↑	**0.8310***	0.8310*	0.8310*	0.7887*	0.7827*	0.7726*
SplitCAM (sc, wta)	AL↑	0.5312	0.5312	**0.5376**	0.5315	0.5264	0.5203
(14, g)	MaxS↓	1.0562*	1.0521*	1.0649	**0.1633**	0.1641	0.1642
	PF_{20}^{drop} ↑	**2.4412***	2.4412*	2.4400*	1.9306*	1.8953*	1.9059*
	PG↑	0.8310*	0.8310*	**0.8431***	0.7746	0.7706*	0.7807*
SplitCAM (sc)	AL↑	0.5311	0.5312	**0.5580**	0.5337	0.5245	0.5183
(14, g)	MaxS↓	1.0554	1.0568*	1.0198	0.1641	0.1587	**0.1570**
	PF_{20}^{drop} ↑	**2.4412***	2.4411*	2.4318*	1.5187*	1.5222*	1.5329*

LIME and KernelSHAP, N counts perturbations. Parenthesized row labels give the selection criterion (`loc`, `faith`); fixed methods have none.

B.3.1 Validation selections

Table B.6 gives both rank-sums for each validation choice on the 50-image split. The full test tables identify the selected configurations.

Tab. B.6: ViT-B/16, VGG-16 and ResNet-50: validation rank-sums within each method, lower is better. Each pair reports localization and faithfulness rank-sums for the row selected by the named criterion (Section A.2.3); anchors are fixed reference configurations.

Method	Selection	ViT-B/16		VGG-16		ResNet-50	
		n	$(r_{\text{loc}},r_{\text{faith}})$	n	$(r_{\text{loc}},r_{\text{faith}})$	n	$(r_{\text{loc}},r_{\text{faith}})$
SmGrad	loc	3	(3.5,3)	3	(3,3)	3	(4,5)
	faith		(3.5,3)		(3,3)		(8,4)
IntGrad	loc	4	(3.5,4)	4	(6,3)	4	(3,6)
	faith		(3.5,4)		(6,3)		(7.5,6)
LRP-ε	loc	7	(5.5,7)	7	(6,9)	7	(5.5,9)
	faith		(6.5,3)		(6,4)		(8,4)
LRP-γ	loc	15	(6,25)	30	(13,43)	30	(12.5,33)
	faith		(32,12)		(36,5)		(36,22.5)
AttnLRP	loc	11	(3,29)	—	—	—	—
	faith		(14,7)		—		—
DAVE	loc/faith	54	(14,6)	—	—	—	—
AR	loc/faith	4	(3.5,3)	—	—	—	—
SG	loc	—	—	40	(37,10)	—	—
	faith		—		(49.5,6)		—
RG	loc	672	(47,788)	544	(78,194)	544	(19,224)
	faith		(1156,22)		(173.5,5)		(466,38)
	anchor		(1420,344)		(339,60)		(212.5,140)
RG+Eq	loc	16	(6.5,29)	—	—	—	—
	faith		(14,11)		—		—
	anchor		(47,31)		—		—

n: declared grid size (Table A.1); ranks use admitted configurations (Section A.2.3). Configurations are identified by method and selection in Tables 7.4, B.7 and B.8. Fixed methods have one configuration and n/a rank-sums. A dash marks an architecture on which the method was not evaluated.

B.3.2 Test results with confidence intervals

Table 7.4 and Tables B.7–B.8 report all 565-image test rows: `loc/faith` selections, anchors and fixed configurations. Confidence intervals and highlighting follow Table 7.4; ViT-B/16 runtimes appear in Section 7.6.

Tab. B.7: VGG-16, ImageNet-S test split: all rows with 95% bootstrap confidence intervals, as in Table 7.4.

Method	Config	AL↑	PG↑	TK↑	PF_5↓	PF_{20}↓	Sel↓	MaxS↓	AvgS↓
Gradient	fixed	0.522 [0.501,0.542]	0.738 [0.701,0.773]	0.688 [0.665,0.711]	14.47 [13.95,14.99]	10.96 [10.55,11.37]	6.470 [6.212,6.731]	1.212 [1.162,1.269]	0.947 [0.917,0.980]
SmGrad (loc/faith)	N=50,σ/range=0.1	0.533 [0.515,0.552]	0.782 [0.747,0.816]	0.723 [0.705,0.741]	14.12 [13.60,14.65]	9.24 [8.85,9.63]	5.768 [5.526,6.007]	0.641 [0.625,0.659]	0.523 [0.515,0.531]
IntGrad (loc/faith)	N_{steps}=20,black baseline	0.550 [0.530,0.571]	0.770 [0.735,0.804]	0.706 [0.683,0.730]	15.39 [14.86,15.95]	11.89 [11.46,12.32]	6.438 [6.174,6.694]	1.132 [1.089,1.179]	0.917 [0.890,0.945]
DeepLIFT	Rescale	0.542 [0.523,0.562]	0.701 [0.664,0.738]	0.654 [0.631,0.677]	13.05 [12.59,13.52]	9.32 [8.96,9.69]	6.049 [5.806,6.291]	1.193 [1.145,1.249]	0.948 [0.919,0.985]
LRP-ε (loc)	ε=0.5	0.613 [0.595,0.632]	0.832 [0.800,0.862]	0.745 [0.725,0.764]	12.63 [12.18,13.10]	8.61 [8.26,8.96]	5.685 [5.458,5.908]	1.173 [1.127,1.223]	0.918 [0.893,0.947]
LRP-ε (faith)	ε=1	0.639 [0.621,0.657]	0.809 [0.777,0.841]	0.744 [0.725,0.762]	**12.43** [11.98,12.89]	8.17 [7.83,8.50]	5.489 [5.270,5.704]	1.100 [1.059,1.143]	0.840 [0.820,0.861]
LRP-γ (loc)	ε=0.5,γ=0.25, $z^{\mathscr{B}}/\gamma/\varepsilon$ composite	0.601 [0.584,0.617]	0.692 [0.655,0.729]	0.704 [0.687,0.721]	14.83 [14.28,15.39]	10.07 [9.68,10.46]	**5.010** [4.799,5.219]	0.515 [0.489,0.542]	0.368 [0.353,0.384]
LRP-γ (faith)	ε=0.25,γ=0.1	0.615 [0.598,0.633]	0.793 [0.759,0.827]	0.698 [0.681,0.716]	12.66 [12.18,13.13]	**7.89** [7.56,8.23]	5.078≈ [4.867,5.285]	0.702 [0.675,0.733]	0.561 [0.543,0.580]
GCAM	`features.29`	0.557 [0.538,0.576]	0.830 [0.798,0.860]	**0.821** [0.798,0.844]	16.36 [15.79,16.93]	12.47 [12.01,12.93]	5.640 [5.407,5.870]	0.430 [0.412,0.449]	0.334 [0.320,0.349]
GCAM++	`features.29`	0.551 [0.533,0.569]	0.823 [0.791,0.855]	0.814≈ [0.792,0.835]	16.68 [16.09,17.27]	12.87 [12.40,13.35]	5.700 [5.462,5.938]	0.314 [0.301,0.327]	0.243 [0.233,0.253]
LCAM	`features.29`	0.544 [0.526,0.562]	0.720 [0.683,0.756]	0.747 [0.725,0.768]	15.98 [15.42,16.54]	11.48 [11.05,11.91]	5.341 [5.113,5.565]	0.434 [0.416,0.452]	0.330 [0.319,0.341]
LIME	N=1000,80 superpixels,ridge	0.603 [0.585,0.622]	**0.972** [0.958,0.984]	0.814≈ [0.790,0.837]	16.66 [16.09,17.25]	12.62 [12.17,13.06]	5.785 [5.551,6.015]	0.848 [0.816,0.882]	0.621 [0.599,0.644]
KernelSHAP	N=2M+2048,80 superpixels	0.559 [0.541,0.577]	0.954≈ [0.936,0.970]	0.820≈ [0.795,0.844]	16.68 [16.10,17.27]	12.82 [12.37,13.26]	6.140 [5.912,6.366]	0.994 [0.955,1.035]	0.736 [0.712,0.761]
SG (loc)	λ=−1,σ^2=5,gate on	0.597 [0.577,0.616]	0.770 [0.735,0.804]	0.770 [0.747,0.793]	17.28 [16.69,17.87]	13.32 [12.85,13.79]	6.217 [5.969,6.459]	**0.278** [0.267,0.290]	**0.196** [0.188,0.204]
SG (faith)	λ=−0.3,σ^2=1,gate off	0.598 [0.577,0.618]	0.821 [0.789,0.851]	0.788≈ [0.764,0.812]	17.08 [16.50,17.67]	12.98 [12.53,13.44]	6.278 [6.025,6.524]	0.543 [0.516,0.571]	0.423 [0.401,0.446]
RG anchor	τ=1	0.619 [0.601,0.637]	0.759 [0.724,0.795]	0.691 [0.673,0.709]	14.27 [13.74,14.80]	9.49 [9.10,9.89]	5.179 [4.968,5.388]	0.551 [0.526,0.578]	0.398 [0.385,0.412]
RG (loc)	τ=0.3,ε=0.1	**0.685** [0.665,0.705]	0.736 [0.699,0.772]	0.673 [0.654,0.692]	14.48 [13.95,15.02]	9.64 [9.25,10.03]	5.632 [5.411,5.845]	1.349 [1.273,1.430]	0.879 [0.842,0.918]
RG (faith)	τ=0.8,ε=0.1	0.640 [0.623,0.658]	0.712 [0.673,0.749]	0.688 [0.670,0.706]	14.05 [13.53,14.58]	9.07 [8.70,9.46]	5.117 [4.906,5.325]	0.655 [0.621,0.690]	0.466 [0.448,0.485]

Tab. B.8: ResNet-50, ImageNet-S test split: all rows with 95% bootstrap confidence intervals, as in Table 7.4.

Method	Config	AL↑	PG↑	TK↑	PF_5↓	PF_{20}↓	Sel↓	MaxS↓	AvgS↓
Gradient	fixed	0.481 [0.459,0.504]	0.556 [0.515,0.596]	0.568 [0.540,0.595]	6.33 [6.22,6.44]	5.91 [5.78,6.04]	5.027 [4.900,5.151]	1.448 [1.365,1.535]	1.072 [1.019,1.130]
SmGrad (loc)	N=50, σ/range=0.15	0.549 [0.530,0.569]	0.789 [0.756,0.821]	0.738 [0.719,0.755]	6.05 [5.93,6.17]	5.17 [5.02,5.31]	4.429 [4.303,4.554]	0.735 [0.718,0.753]	0.600 [0.593,0.609]
SmGrad (faith)	N=50,σ/range=0.1	0.544 [0.525,0.564]	0.773 [0.740,0.807]	0.735 [0.715,0.753]	6.06 [5.94,6.19]	5.19 [5.04,5.34]	4.489 [4.361,4.612]	0.726 [0.706,0.748]	0.586 [0.577,0.597]
IntGrad (loc)	N_{steps}=100,black baseline	0.510 [0.488,0.532]	0.635 [0.596,0.676]	0.600 [0.574,0.626]	6.27 [6.16,6.38]	5.83 [5.69,5.96]	4.949 [4.820,5.072]	1.203 [1.152,1.257]	0.947 [0.917,0.979]
IntGrad (faith)	N_{steps}=50,black baseline	0.507 [0.484,0.529]	0.604 [0.563,0.642]	0.595 [0.568,0.622]	6.28 [6.16,6.39]	5.82 [5.68,5.95]	4.952 [4.822,5.075]	1.207 [1.157,1.261]	0.951 [0.922,0.983]

continued on next page

Method	Config	AL↑	PG↑	TK↑	PF_5↓	PF_{20}↓	Sel↓	MaxS↓	AvgS↓
DeepLIFT	Rescale	0.543 [0.523,0.564]	0.727 [0.690,0.765]	0.653 [0.629,0.677]	5.97 [5.85,6.10]	5.29 [5.15,5.43]	4.515 [4.390,4.637]	1.273 [1.203,1.351]	1.000 [0.955,1.052]
LRP-ε (loc)	ε=0.5	0.603 [0.584,0.622]	0.775 [0.740,0.809]	0.714 [0.694,0.734]	5.94 [5.81,6.06]	5.14 [4.99,5.28]	4.318 [4.200,4.433]	1.246 [1.203,1.292]	0.982 [0.960,1.006]
LRP-ε (faith)	ε=2	0.637 [0.618,0.656]	0.699 [0.662,0.736]	0.683 [0.663,0.702]	5.94 [5.81,6.06]	4.91 [4.76,5.05]	4.280 [4.164,4.394]	1.262 [1.224,1.301]	0.967 [0.948,0.986]
LRP-γ (loc)	ε=0.5,γ=2,$z^{\mathscr{B}}/\gamma/\varepsilon$ composite	0.711≈ [0.695,0.726]	0.842 [0.812,0.873]	0.822≈ [0.807,0.836]	5.69≈ [5.55,5.83]	4.53 [4.38,4.68]	3.935≈ [3.814,4.053]	0.633 [0.601,0.666]	0.486 [0.464,0.507]
LRP-γ (faith)	ε=0.5,γ=0.5, $z^{\mathscr{B}}/\gamma/\varepsilon$ composite	0.700 [0.684,0.716]	0.834 [0.802,0.864]	0.796 [0.781,0.812]	**5.65** [5.51,5.79]	**4.43** [4.28,4.59]	**3.931** [3.809,4.050]	0.745 [0.714,0.777]	0.584 [0.562,0.605]
GCAM	`layer4`	0.664 [0.645,0.683]	0.873 [0.844,0.899]	0.850≈ [0.830,0.870]	6.42 [6.32,6.52]	6.15 [6.03,6.26]	4.344 [4.216,4.469]	0.292 [0.274,0.312]	0.207 [0.195,0.220]
GCAM++	`layer4`	0.627 [0.607,0.646]	0.834 [0.804,0.864]	0.836 [0.815,0.857]	6.43 [6.34,6.52]	6.18 [6.07,6.29]	4.483 [4.356,4.609]	0.294 [0.278,0.310]	0.207 [0.197,0.219]
LCAM	`layer4`	0.649 [0.630,0.668]	0.869 [0.841,0.897]	**0.851** [0.831,0.871]	6.44 [6.34,6.53]	6.18 [6.07,6.29]	4.437 [4.311,4.559]	**0.274** [0.260,0.288]	**0.194** [0.184,0.204]
LIME	N=1000,80 superpixels,ridge	0.553 [0.533,0.572]	0.926≈ [0.903,0.947]	0.716 [0.687,0.744]	6.37 [6.26,6.46]	5.99 [5.88,6.11]	4.759 [4.630,4.881]	0.916 [0.891,0.941]	0.672 [0.655,0.690]
KernelSHAP	N=2M+2048,80 superpixels	0.558 [0.540,0.575]	**0.965** [0.949,0.979]	0.818≈ [0.793,0.841]	6.26 [6.16,6.36]	5.81 [5.69,5.92]	4.727 [4.601,4.849]	0.924 [0.896,0.954]	0.688 [0.670,0.708]
RG anchor	τ=1	0.693 [0.676,0.710]	0.752 [0.717,0.788]	0.756 [0.738,0.774]	5.84 [5.70,5.97]	4.71 [4.56,4.86]	4.017 [3.898,4.133]	0.708 [0.651,0.775]	0.508 [0.481,0.541]
RG (loc)	τ=0.9,ε=1	**0.719** [0.701,0.737]	0.763 [0.727,0.796]	0.760 [0.741,0.778]	5.85 [5.72,5.98]	4.70 [4.55,4.85]	4.047 [3.929,4.163]	0.798 [0.749,0.855]	0.560 [0.535,0.586]
RG (faith)	τ=1,λ=−1,σ^2=1, ε=0.25	0.674 [0.656,0.691]	0.743 [0.708,0.779]	0.738 [0.719,0.756]	5.88 [5.74,6.01]	4.74 [4.59,4.89]	4.030 [3.911,4.144]	0.685 [0.629,0.755]	0.489 [0.460,0.525]

B.3.3 Cross-Dataset Evaluation: Pascal VOC 2012

Table B.9 gives all rows underlying the transfer comparison in Section 7.6.

Dataset. We use Pascal VOC 2012 (Everingham et al., 2010) segmentation annotations[1]: the trainval pool of 2,913 images with pixel-accurate `SegmentationClass` masks across 20 foreground classes plus background. Each image is expanded to one (image, foreground-class) sample per distinct VOC class label present in its mask, yielding 3,422 total samples after dropping the `person` class (no faithful ImageNet-1K target exists for it) and discarding sliver masks whose pixel area on the *original* (pre-resize) annotation is below 64 pixels. Each (image, class) sample carries the same input image and an explanation target equal to the most semantically central ImageNet-1K index for that VOC class (e.g. `horse` → n02389026 sorrel; `tvmonitor` → n03782006 monitor); the curated mapping is part of the released code.

Per-image disjoint val/test split. The trainval pool is partitioned randomly with seed 42 at the *image* level (not the (image, class) level) into a 1:5 val/test split: the trainval stems are shuffled deterministically and the first 1/6 is assigned to val. All (image, class) samples for a given image stay in the same partition, preventing the same image from appearing in both tuning and test sets. As on ImageNet-S, samples whose object mask lies entirely outside the 224×224 center crop are excluded after the split (18 val and 105 test samples). The validation split contains 460 images and 556 image–class samples; the test split contains 2,293 images and 2,743 samples.

[1] Release statistics: https://www.robots.ox.ac.uk/~vgg/projects/pascal/VOC/voc2012/dbstats.html.

Pixel masks for localization. *Attribution Localization* (AL), *Pointing Game* (PG), and Top-k Intersection use each image–class sample's binary VOC2012 segmentation mask, rewarding only foreground pixels rather than background within a bounding box. NEAREST resizing and 224×224 center-cropping match the input pipeline (`Resize(256)` → `CenterCrop(224)`); Quantus (Hedström et al., 2023) receives this mask unchanged alongside the attribution map.

Tuning protocol. Every ViT-B/16 baseline configuration in Table A.1 is evaluated on the 556-sample VOC validation pool; the `loc` and `faith` criteria of Section A.2.3 select rows for the 2,743-sample test pool. RG and RG+Eq transfer their ImageNet-S anchors and both selected configurations unchanged. To save compute, VOC evaluates only localization and faithfulness, omitting MaxS, AvgS and runtime. Per-metric tests (Section A.2.4) count significant wins, losses and undetected differences; the last category does not establish equivalence.

Tab. B.9: ViT-B/16 on Pascal VOC 2012: all 2,743 test image–class records from 2,293 images, with record-weighted means and 95% image-cluster bootstrap confidence intervals. The transfer and baseline-selection protocol is specified above; only localization and faithfulness were evaluated. Metrics: Table 7.4; configuration symbols and row labels: Section B.3.

Method	Config	AL↑	PG↑	TK↑	PF_5↓	PF_{20}↓	Sel↓
Gradient	fixed	0.295 [0.286,0.304]	0.360 [0.342,0.378]	0.355 [0.343,0.368]	3.45 [3.33,3.56]	3.17 [3.05,3.28]	2.316 [2.234,2.397]
SmGrad (loc)	N=50,σ/range=0.1	0.344 [0.334,0.354]	0.557 [0.538,0.576]	0.511 [0.498,0.523]	3.25 [3.13,3.36]	2.70 [2.60,2.81]	1.940 [1.871,2.011]
SmGrad (faith)	N=50, σ/range=0.15	0.347 [0.337,0.356]	0.555 [0.536,0.574]	0.511 [0.499,0.523]	3.24 [3.12,3.35]	2.67 [2.57,2.77]	1.907 [1.840,1.977]
IntGrad (loc)	N_{steps}=300,black baseline	0.292 [0.283,0.302]	0.388 [0.369,0.406]	0.358 [0.346,0.370]	3.43 [3.32,3.55]	3.21 [3.10,3.32]	2.278 [2.198,2.356]
IntGrad (faith)	N_{steps}=100,black baseline	0.292 [0.283,0.302]	0.389 [0.371,0.407]	0.358 [0.346,0.370]	3.44 [3.32,3.55]	3.21 [3.10,3.32]	2.279 [2.199,2.358]
LRP-ε (loc/faith)	ε=0.1	0.378 [0.368,0.388]	0.604 [0.585,0.623]	0.518 [0.506,0.531]	3.07 [2.96,3.18]	2.63 [2.54,2.73]	1.923 [1.856,1.990]
LRP-γ (loc/faith)	ε=0.1,γ=1	0.403 [0.393,0.414]	0.591 [0.572,0.610]	0.545 [0.532,0.558]	2.99 [2.88,3.10]	2.37 [2.27,2.46]	1.694≈ [1.634,1.756]
AttnLRP (loc/faith)	γ_{conv}=0.25, γ_{lin}=0.05, $\varepsilon_{\text{attn}}$=0.05	0.404 [0.393,0.414]	0.602 [0.583,0.620]	0.542 [0.529,0.554]	**2.91** [2.80,3.01]	**2.31** [2.22,2.40]	**1.690** [1.630,1.751]
DAVE (loc)	N=50,$\theta_{\max}$=0, $s_{\max}$=0,$t_{\max}$=0.25	0.334 [0.324,0.343]	0.547 [0.527,0.566]	0.456 [0.444,0.468]	3.37 [3.26,3.49]	3.01 [2.91,3.12]	2.172 [2.097,2.247]
DAVE (faith)	N=50,$\theta_{\max}$=0, $s_{\max}$=0,$t_{\max}$=0.5	0.332 [0.322,0.342]	0.535 [0.515,0.554]	0.454 [0.443,0.466]	3.37 [3.26,3.48]	2.98 [2.88,3.09]	2.153 [2.079,2.227]
TIBAV	fixed	0.413 [0.402,0.424]	0.729 [0.713,0.747]	**0.575** [0.561,0.590]	3.04 [2.93,3.15]	2.67 [2.57,2.77]	1.731 [1.665,1.798]
GAE	fixed	0.355 [0.345,0.365]	0.712 [0.695,0.729]	0.548 [0.535,0.562]	3.00 [2.89,3.11]	2.63 [2.52,2.74]	1.754 [1.685,1.824]
AR (loc/faith)	w_r=0.75	0.272 [0.263,0.281]	0.471 [0.452,0.490]	0.411 [0.397,0.425]	3.57 [3.45,3.68]	3.41 [3.30,3.51]	2.146 [2.075,2.216]
LIME	N=1000,80 superpixels,ridge	0.351 [0.341,0.361]	0.785≈ [0.768,0.800]	0.523 [0.508,0.538]	3.45 [3.33,3.56]	3.21 [3.10,3.32]	2.121 [2.051,2.194]
KernelSHAP	N=2M+2048,80 superpixels	0.341 [0.332,0.351]	**0.793** [0.777,0.809]	0.561≈ [0.546,0.576]	3.36 [3.25,3.48]	3.03 [2.93,3.14]	2.077 [2.009,2.147]
RG anchor	τ=1	0.323 [0.313,0.333]	0.517 [0.498,0.536]	0.489 [0.475,0.502]	3.30 [3.19,3.41]	2.67 [2.58,2.77]	1.980 [1.914,2.047]
RG (loc)	τ=0.4,λ_{sm}=−10, σ^2=5,ε=0.1	**0.459** [0.447,0.472]	0.562 [0.543,0.582]	0.512 [0.498,0.527]	3.44 [3.33,3.56]	3.06 [2.96,3.17]	2.152 [2.083,2.222]

continued on next page

Method	Config	AL↑	PG↑	TK↑	PF_5↓	PF_{20}↓	Sel↓
RG (faith)	τ=0.7,λ_{sm}=−5, σ^2=5,ε=0.1	0.351 [0.341,0.361]	0.538 [0.519,0.557]	0.508 [0.495,0.522]	3.27 [3.16,3.38]	2.63 [2.53,2.72]	1.965 [1.899,2.032]
RG+Eq anchor	τ=1,N=50	0.329 [0.319,0.339]	0.574 [0.554,0.593]	0.534 [0.519,0.549]	3.43 [3.31,3.54]	3.06 [2.96,3.17]	1.988 [1.922,2.056]
RG+Eq (loc)	τ=0.5,λ_{sm}=−10, σ^2=2,N=50	0.456 [0.443,0.468]	0.599 [0.579,0.619]	0.571≈ [0.555,0.586]	3.45 [3.34,3.57]	3.14 [3.03,3.25]	2.085 [2.016,2.154]
RG+Eq (faith)	τ=0.7,λ_{sm}=−10, σ^2=2,N=50	0.424 [0.412,0.436]	0.591 [0.572,0.610]	0.562≈ [0.547,0.578]	3.44 [3.32,3.55]	3.08 [2.98,3.19]	2.004 [1.938,2.071]

B.3.4 One-factor ablations

Tables B.10–B.12 vary one parameter over its declared grid around each validation-selected RG, RG+Eq and SG configuration (each selected center shown once in bold), fixing the others. The notes retain every selected factor value. Localization and faithfulness are evaluated on the 565-image test split without feeding back into selection. On ViT-B/16, "+Eq" applies RG+Eq at the selected RG parameters. Validation-inadmissible configurations (Section A.2.3) are excluded.

Tab. B.10: ViT-B/16, ImageNet-S test split: one-factor ablations around the selected configurations. Center: the selected configuration (criterion in parentheses); Factor and Value give the varied parameter and its value, with other parameters fixed. Symbols: Section B.3.

Selected centers (one bold row each): RG (faith): τ=0.7, λ_{sm}=−5, σ^2=5, ε=0.1; RG (loc): τ=0.4, λ_{sm}=−10, σ^2=5, ε=0.1; RG+Eq (faith): τ=0.7, λ_{sm}=−10; RG+Eq (loc): τ=0.5, λ_{sm}=−10.

Center	Factor	Value	AL↑	PG↑	TK↑	PF_5↓	PF_{20}↓	Sel↓
RG (faith)	τ	0.3	0.705	0.802	0.731	8.34	7.60	5.195
		0.4	0.688	0.827	0.773	8.26	7.41	5.095
		0.5	0.665	0.821	0.793	8.22	7.20	4.956
		0.6	0.613	0.814	0.774	8.13	6.88	4.820
		0.7	0.549	0.770	0.741	8.11	6.78	4.808
		0.8	0.507	0.742	0.712	8.14	6.85	4.881
		0.9	0.482	0.697	0.683	8.21	7.02	4.983
		1	0.467	0.648	0.648	8.29	7.25	5.115
RG (faith)	λ_{sm}	-50	0.467	0.556	0.541	8.51	8.13	5.928
		-10	0.556	0.777	0.763	8.16	6.97	4.876
		-2	0.547	0.743	0.720	8.11	6.81	4.793
		-1	0.546	0.733	0.713	8.11	6.83	4.794
		0	0.546	0.727	0.706	8.13	6.87	4.804
RG (faith)	σ^2	1	0.558	0.738	0.703	8.16	6.90	4.790
		2	0.555	0.743	0.719	8.13	6.81	4.783
		10	0.545	0.765	0.749	8.12	6.82	4.833
RG (faith)	ε	0.25	0.595	0.804	0.778	8.16	6.95	4.846
		0.5	0.621	0.809	0.792	8.24	7.21	4.944
		1	0.630	0.812	0.794	8.29	7.43	5.051
RG (faith)	+Eq (N=50)	on	0.557	0.800	0.791	8.38	7.69	4.816
RG (loc)	τ	0.3	0.723	0.816	0.746	8.33	7.64	5.201
		0.4	0.711	0.825	0.788	8.27	7.48	5.109
		0.5	0.691	0.830	0.812	8.23	7.31	4.969

Center	Factor	Value	AL↑	PG↑	TK↑	PF_5↓	PF_{20}↓	Sel↓
		0.6	0.632	0.827	0.801	8.16	7.00	4.847
		0.7	0.556	0.777	0.763	8.16	6.97	4.876
		0.8	0.506	0.743	0.715	8.22	7.13	5.013
		0.9	0.477	0.683	0.673	8.29	7.28	5.143
		1	0.461	0.628	0.629	8.36	7.42	5.247
RG (loc)	λ_{sm}	-50	0.520	0.543	0.528	8.52	7.99	5.745
		-5	0.688	0.827	0.773	8.26	7.41	5.095
		-2	0.676	0.827	0.764	8.26	7.39	5.086
		-1	0.673	0.818	0.761	8.26	7.38	5.080
		0	0.671	0.818	0.758	8.26	7.38	5.076
RG (loc)	σ^2	1	0.707	0.823	0.769	8.23	7.35	4.986
		2	0.712	0.823	0.780	8.24	7.39	5.032
		10	0.705	0.837	0.787	8.31	7.58	5.168
RG (loc)	ε	0.25	0.709	0.842	0.780	8.29	7.55	5.155
		0.5	0.708	0.828	0.776	8.30	7.57	5.171
		1	0.708	0.823	0.773	8.31	7.59	5.181
RG (loc)	+Eq (N=50)	on	0.711	0.846	0.842	8.38	7.82	5.077
RG+Eq (faith)	τ	0.3	0.721	0.805	0.786	8.37	7.75	5.129
		0.4	0.712	0.832	0.822	8.39	7.81	5.049
		0.5	0.698	0.855	0.836	8.39	7.80	5.010
		0.6	0.681	0.846	0.833	8.39	7.77	4.943
		0.7	0.654	0.812	0.820	8.39	7.71	4.856
		0.8	0.610	0.805	0.803	8.38	7.70	4.814
		0.9	0.567	0.796	0.787	8.41	7.71	4.825
		1	0.534	0.777	0.772	8.42	7.74	4.876
RG+Eq (faith)	λ_{sm}	0	0.613	0.814	0.815	8.37	7.73	4.914
RG+Eq (loc)	τ	0.3	0.721	0.805	0.786	8.37	7.75	5.129
		0.4	0.712	0.832	0.822	8.39	7.81	5.049
		0.5	0.698	0.855	0.836	8.39	7.80	5.010
		0.6	0.681	0.846	0.833	8.39	7.77	4.943
		0.7	0.654	0.812	0.820	8.39	7.71	4.856
		0.8	0.610	0.805	0.803	8.38	7.70	4.814
		0.9	0.567	0.796	0.787	8.41	7.71	4.825
		1	0.534	0.777	0.772	8.42	7.74	4.876
RG+Eq (loc)	λ_{sm}	0	0.652	0.848	0.828	8.40	7.85	5.073

Tab. B.11: VGG-16, ImageNet-S test split: one-factor ablations around the selected configurations, as in Table B.10.

Selected centers (one bold row each): RG (faith): τ=0.8, λ=0, ε=0.1; RG (loc): τ=0.3, λ=0, ε=0.1; SG (faith): λ=−0.3, σ^2=1, gate $\tilde{\Phi}$ off; SG (loc): λ=−1, σ^2=5, gate $\tilde{\Phi}$ on.

Center	Factor	Value	AL↑	PG↑	TK↑	PF_5↓	PF_{20}↓	Sel↓
RG (faith)	τ	0.3	0.685	0.736	0.673	14.48	9.64	5.632
		0.4	0.645	0.657	0.641	14.40	9.36	5.574

Center	Factor	Value	AL↑	PG↑	TK↑	PF_5↓	PF_{20}↓	Sel↓
		0.5	0.626	0.653	0.628	14.27	9.14	5.451
		0.6	0.633	0.651	0.646	14.07	9.00	5.258
		0.7	0.643	0.671	0.674	14.03	8.98	5.142
		0.8	0.640	0.712	0.688	14.05	9.07	5.117
		0.9	0.630	0.742	0.693	14.05	9.17	5.126
		1	0.618	0.749	0.695	14.06	9.25	5.137
RG (faith)	λ	-5	0.518	0.458	0.536	17.60	13.82	7.362
		-2	0.623	0.604	0.631	15.90	11.58	5.852
		-1	0.636	0.676	0.661	15.03	10.35	5.404
		-0.3	0.640	0.694	0.676	14.54	9.70	5.231
RG (faith)	ε	0.25	0.639	0.692	0.684	14.18	9.23	5.146
		0.5	0.638	0.690	0.680	14.34	9.46	5.183
		1	0.639	0.690	0.676	14.63	9.79	5.249
RG (loc)	τ	**0.3**	0.685	0.736	0.673	14.48	9.64	5.632
		0.4	0.645	0.657	0.641	14.40	9.36	5.574
		0.5	0.626	0.653	0.628	14.27	9.14	5.451
		0.6	0.633	0.651	0.646	14.07	9.00	5.258
		0.7	0.643	0.671	0.674	14.03	8.98	5.142
		0.8	0.640	0.712	0.688	14.05	9.07	5.117
		0.9	0.630	0.742	0.693	14.05	9.17	5.126
		1	0.618	0.749	0.695	14.06	9.25	5.137
RG (loc)	λ	-5	0.561	0.543	0.555	17.05	13.21	6.898
		-2	0.657	0.696	0.634	15.31	10.80	5.885
		-1	0.674	0.735	0.656	14.85	10.13	5.731
		-0.3	0.680	0.727	0.666	14.66	9.87	5.675
RG (loc)	ε	0.25	0.691	0.747	0.673	14.66	9.91	5.659
		0.5	0.697	0.742	0.672	14.87	10.19	5.705
		1	0.702	0.742	0.671	15.13	10.55	5.767
SG (faith)	λ	-2	0.614	0.710	0.636	18.17	14.83	6.941
		-1	0.609	0.750	0.697	17.91	13.87	6.517
		-0.3	0.598	0.821	0.788	17.08	12.98	6.278
		0	0.547	0.781	0.764	17.63	14.35	6.789
SG (faith)	σ^2	2	0.601	0.811	0.765	17.30	13.07	6.323
		5	0.604	0.798	0.731	17.66	13.50	6.417
		10	0.607	0.763	0.706	17.86	13.77	6.500
SG (faith)	gate $\tilde{\Phi}$	on	0.568	0.823	0.805	17.30	13.74	6.428
SG (loc)	λ	-5	0.568	0.550	0.558	18.17	14.31	7.509
		-2	0.620	0.703	0.681	17.84	13.63	6.404
		-1	0.597	0.770	0.770	17.28	13.32	6.217
		-0.3	0.566	0.830	0.809	17.33	13.75	6.452
		0	0.554	0.827	0.802	17.55	14.16	6.618
SG (loc)	σ^2	1	0.596	0.793	0.766	17.32	13.35	6.246
		2	0.597	0.786	0.766	17.31	13.34	6.232
		10	0.595	0.772	0.776	17.25	13.30	6.206
SG (loc)	gate $\tilde{\Phi}$	off	0.614	0.696	0.628	18.19	15.00	7.050

Tab. B.12: ResNet-50, ImageNet-S test split: one-factor ablations around the selected configurations, as in Table B.10.

Selected centers (one bold row each): RG (faith): τ=1, λ=−1, σ^2=1, ε=0.25; RG (loc): τ=0.9, λ=0, ε=1.

Center	Factor	Value	AL↑	PG↑	TK↑	PF_5↓	PF_{20}↓	Sel↓
RG (faith)	τ	0.3	0.688	0.727	0.684	6.10	5.10	4.357
		0.4	0.676	0.712	0.692	6.08	5.03	4.340
		0.5	0.677	0.720	0.706	6.06	5.00	4.308
		0.6	0.673	0.704	0.711	5.99	4.96	4.270
		0.7	0.667	0.680	0.707	5.95	4.89	4.219
		0.8	0.665	0.688	0.711	5.89	4.85	4.134
		0.9	0.674	0.720	0.728	5.88	4.79	4.075
		1	0.674	0.743	0.738	5.88	4.74	4.030
RG (faith)	λ	-5	0.667	0.740	0.730	5.90	4.81	4.034
		-2	0.672	0.740	0.736	5.89	4.75	4.034
		-0.3	0.674	0.747	0.739	5.87	4.75	4.026
		0	0.675	0.747	0.740	5.86	4.74	4.023
RG (faith)	σ^2	2	0.673	0.742	0.737	5.87	4.74	4.032
		5	0.671	0.736	0.736	5.88	4.75	4.031
		10	0.670	0.747	0.734	5.89	4.76	4.031
RG (faith)	ε	0.1	0.658	0.745	0.722	5.90	4.79	4.049
		0.5	0.692	0.747	0.754	5.86	4.72	4.023
		1	0.712	0.766	0.761	5.85	4.72	4.038
RG (loc)	τ	0.3	0.733	0.761	0.719	6.03	5.04	4.272
		0.4	0.717	0.758	0.718	6.02	4.96	4.243
		0.5	0.709	0.740	0.720	5.98	4.92	4.216
		0.6	0.710	0.733	0.728	5.93	4.87	4.172
		0.7	0.716	0.745	0.741	5.91	4.80	4.125
		0.8	0.720	0.761	0.752	5.87	4.72	4.078
		0.9	0.719	0.763	0.760	5.85	4.70	4.047
		1	0.713	0.770	0.763	5.85	4.70	4.035
RG (loc)	λ	-5	0.708	0.765	0.745	5.89	4.80	4.069
		-2	0.716	0.773	0.755	5.87	4.74	4.055
		-1	0.718	0.777	0.757	5.86	4.72	4.054
		-0.3	0.719	0.768	0.758	5.86	4.72	4.050
RG (loc)	ε	0.1	0.656	0.712	0.712	5.89	4.84	4.106
		0.25	0.674	0.717	0.730	5.87	4.77	4.067
		0.5	0.696	0.736	0.749	5.84	4.70	4.042

B.3.5 Pointing game at matched granularity

Quantus Pointing Game counts a hit if any maximum-attribution pixel lies in the mask. A piecewise-constant maximum spans hundreds to thousands of pixels for superpixel and 14×14 attention maps, versus usually one for pixel-level maps. Table B.13 reports the median maximum-set size, the expected hit rate of a uniformly drawn maximum pixel (PG_{1px}), and Pointing Game after average-pooling all maps to a common 16×16-pixel grid (PG_{16px}). Single-pixel sampling removes most of LIME's and

KernelSHAP's advantage. At common granularity, leaders move closer: LRP-γ and AttnLRP lead on ViT-B/16; localization-selected SG and faithfulness-selected RG are not significantly below Grad-CAM++, the best VGG-16 baseline; Grad-CAM, LayerCAM and LRP-γ lead on ResNet-50 (McNemar tests against the best baseline).

B.3.6 Qualitative comparison

Figure B.2 uses the ViT-B/16 `loc`-selected configurations of Table 7.4, with fixed Gradient and TIBAV configurations. Both games use the composite attention–Value rule of Theorem 5.8, treating the conditioned block as one map $X \mapsto O$. Every map explains the ImageNet-S label; all examples are classified correctly except the kuvasz (predicted: English setter). The first image is from test, the other five from the 50-image validation split used for selection. Maps precede the absolute value and max-abs normalization of Section A.2.2; TIBAV's 14×14 map uses nearest-neighbor upsampling, as in evaluation.

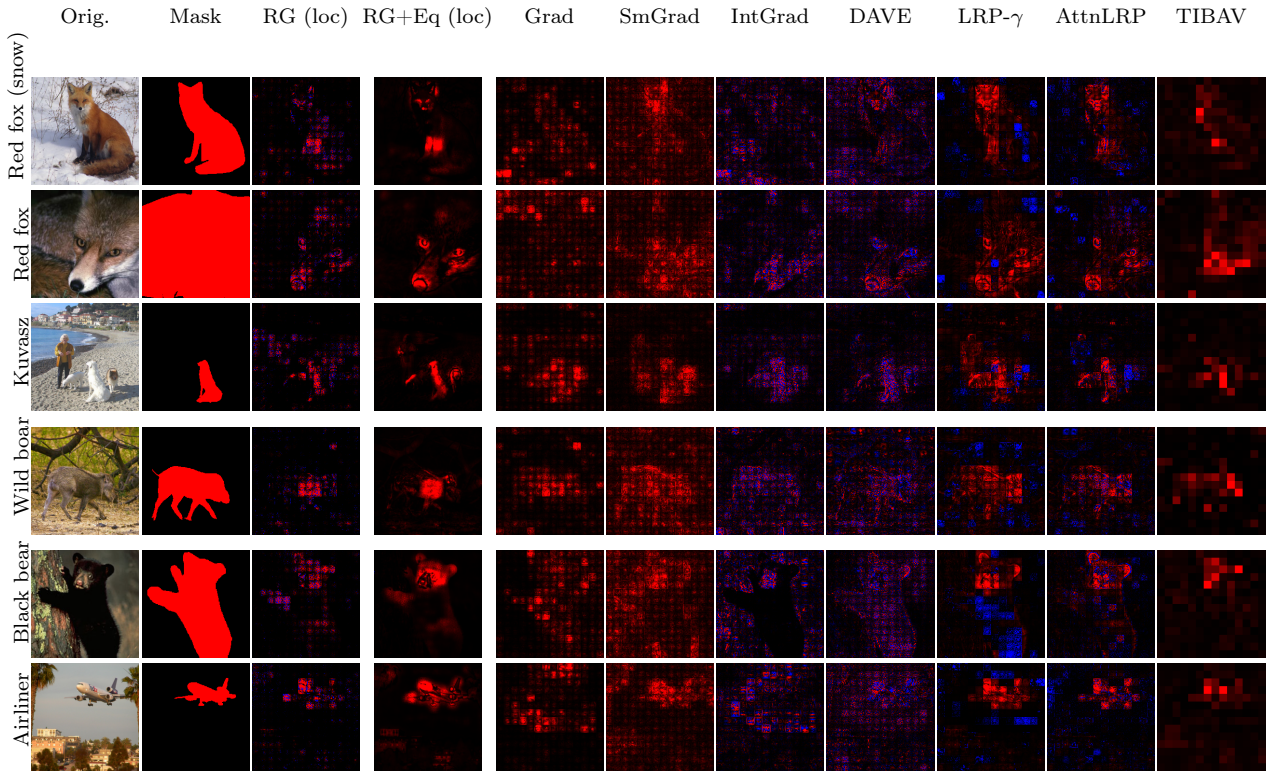


Fig. B.2: Six ImageNet-S examples, ViT-B/16, at the configurations specified above. Red and blue show positive and negative attribution. Each sign is divided by its image-wide 99th percentile and clipped, saturating the top 1% so concentrated maps remain legible; every method uses this display transform.

Tab. B.13: Pointing game under the Quantus tie rule (PG, as in the main tables) and at matched granularity, ImageNet-S test split. max. set: median number of pixels attaining the map maximum; PG_{1px}: expected hit rate of one pixel drawn uniformly from that set; PG_{16px}: PG after average-pooling the map to 16×16-pixel cells. Rows: the `loc`-selected or fixed configuration of every method, plus the `faith` rows of our games. **Bold**: best per model and column.

Model	Method	PG	max. set (px)	PG_{1px}	PG_{16px}
ViT-B/16	Gradient	0.361	1	0.361	0.382
	SmGrad	0.706	1	0.706	0.835
	IntGrad	0.513	1	0.513	0.543
	LRP-ε	0.834	1	0.834	0.904
	LRP-γ	0.873	1	**0.873**	**0.968**
	AttnLRP	0.855	1	0.855	0.926
	DAVE	0.717	1	0.717	0.781
	TIBAV	0.878	256	0.784	0.878
	GAE	0.743	256	0.670	0.743
	AR	0.565	256	0.516	0.565
	LIME	0.897	1104	0.693	0.786
	KernelSHAP	**0.961**	1031	0.833	0.903
	RG (faith)	0.770	1	0.770	0.899
	RG	0.825	1	0.825	0.888
	RG+Eq (faith)	0.812	1	0.812	0.919
	RG+Eq	0.855	1	0.855	0.919
VGG-16	Gradient	0.738	1	0.738	0.812
	SmGrad	0.782	1	0.782	0.915
	IntGrad	0.770	1	0.770	0.860
	DeepLIFT	0.701	1	0.701	0.784
	LRP-ε	0.832	1	**0.832**	0.906
	LRP-γ	0.692	1	0.692	0.908
	GCAM	0.830	1	0.823	0.920
	GCAM++	0.823	1	0.815	**0.924**
	LCAM	0.720	1	0.715	0.834
	LIME	**0.972**	1103	0.826	0.899
	KernelSHAP	0.954	1011	0.826	0.894
	SG (faith)	0.821	1	0.821	0.883
	SG	0.770	1	0.770	0.908
	RG (faith)	0.712	1	0.712	0.915
	RG	0.736	1	0.736	0.873
ResNet-50	Gradient	0.556	1	0.556	0.630
	SmGrad	0.789	1	0.789	0.919
	IntGrad	0.635	1	0.635	0.743
	DeepLIFT	0.727	1	0.727	0.793
	LRP-ε	0.775	1	0.775	0.880
	LRP-γ	0.842	1	0.842	0.958
	GCAM	0.873	1	**0.862**	**0.959**
	GCAM++	0.834	1	0.827	0.945
	LCAM	0.869	1	0.860	0.958
	LIME	0.926	1058	0.729	0.844
	KernelSHAP	**0.965**	1014	0.825	0.912
	RG (faith)	0.743	1	0.743	0.903
	RG	0.763	1	0.763	0.912

Appendix C

Publication Provenance and Tool Use

This thesis integrates two coauthored works with the thesis author as first author: *Hidden Monotonicity*, a CVPR 2026 Highlight (Zimmermann and Loho, 2026), and the revised *Playing the Network Backward* preprint, arXiv:2605.06212v2 (Zimmermann et al., 2026), without a claim of AISTATS acceptance. It reuses their model-evaluation outputs and point estimates, recomputing VOC intervals and paired comparisons by whole-image resampling (Section A.2.4). A shared decomposition framework adds common definitions, mathematical extensions, connections and Lean formalization.

C.1 Author's Contribution

For both papers, I developed the theory and experiments, ran the experiments, and wrote all appendices and most text outside the introductions, collaborating with the coauthors listed in Section 1.1.

The following disclosures distinguish original-paper preparation from later thesis work.

C.2 Disclosure in the Revised Game Paper

The revised game paper discloses the following for its preparation:

> In this work, we used generative AI tools for implementing standard baseline methods and evaluation code, for producing plots and tables of results, for drafting parts of the text, and for editing (grammar, spelling, word choice, and reformatting). We have not used generative AI tools to generate research ideas, to formulate the games, or to produce the theorems and their proofs. All AI-assisted code was reviewed and tested by the authors, including unit tests that check the game-theoretic occupation measures against reference gradient and $\alpha\beta$-LRP implementations; all AI-assisted text was checked by the authors for correctness, and every reference was verified manually. We take responsibility for the final content of this work, including text, claims, and artifacts produced with the aid of generative AI.

C.3 AI Assistance in the Thesis Revision

The November 2026 thesis revision used OpenAI's Codex for mathematical exploration, proof development, Lean implementation and checking, code review and repair, statistical reaggregation, and drafting and editing. Mathematical assistance developed and checked convex, monotone stream extensions. The quoted historical statement about theorem and proof development concerns the original paper, excluding this later thesis work.

The formalization repository records reviewed theorem types, proof-checking receipts and coverage limits. Lean certifies the stated propositions; source correspondence requires mathematical review, and kernel checks do not certify experiments.

List of References

Abnar, S., W. Zuidema (2020): Quantifying Attention Flow in Transformers. *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, 4190–4197. DOI: 10.18653/v1/2020.acl-main.385. URL: http://dx.doi.org/10.18653/v1/2020.acl-main.385 (cit. on pp. 6, 37, 54).

Achanta, R., A. Shaji, K. Smith, A. Lucchi, P. Fua, S. Süsstrunk (2012): SLIC Superpixels Compared to State-of-the-Art Superpixel Methods. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 34 (11), 2274–2282. DOI: 10.1109/TPAMI.2012.120. URL: https://doi.org/10.1109/TPAMI.2012.120 (cit. on p. 105).

Achtibat, R., M. Dreyer, I. Eisenbraun, S. Bosse, T. Wiegand, W. Samek, S. Lapuschkin (2023): From Attribution Maps to Human-Understandable Explanations through Concept Relevance Propagation. *Nature Machine Intelligence* 5 (9), 1006–1019. DOI: 10.1038/s42256-023-00711-8. URL: https://www.nature.com/articles/s42256-023-00711-8 (cit. on pp. 54, 83).

Achtibat, R., S. M. V. Hatefi, M. Dreyer, A. Jain, T. Wiegand, S. Lapuschkin, W. Samek (2024): AttnLRP: Attention-Aware Layer-Wise Relevance Propagation for Transformers. *Proceedings of the 41st International Conference on Machine Learning*. Vol. 235. Proceedings of Machine Learning Research. PMLR, 135–168. URL: https://proceedings.mlr.press/v235/achtibat24a.html (cit. on pp. 6, 37, 38, 54, 61, 81, 104–106).

Adebayo, J., J. Gilmer, M. Muelly, I. Goodfellow, M. Hardt, B. Kim (2018): Sanity Checks for Saliency Maps. *Advances in Neural Information Processing Systems*. Vol. 31. URL: https://proceedings.neurips.cc/paper/2018/hash/294a8ed24b1ad22ec2e7efea049b8737-Abstract.html (cit. on pp. 1, 5, 70, 76, 84, 91, 93–95, 98).

Ali, A., T. Schnake, O. Eberle, G. Montavon, K.-R. Müller, L. Wolf (2022): XAI for Transformers: Better Explanations through Conservative Propagation. *Proceedings of the 39th International Conference on Machine Learning (ICML)*. Vol. 162. Proceedings of Machine Learning Research. PMLR, 435–451. URL: https://proceedings.mlr.press/v162/ali22a.html (cit. on pp. 6, 37, 38, 54, 56, 60, 61, 64, 82).

Alvarez-Melis, D., T. S. Jaakkola (2018): On the Robustness of Interpretability Methods. *ICML 2018 Workshop on Human Interpretability in Machine Learning*, 66–71. DOI: 10.48550/arXiv.1806.08049. arXiv: 1806.08049 [cs.LG]. URL: https://arxiv.org/abs/1806.08049 (cit. on p. 5).

Amos, B., L. Xu, J. Z. Kolter (2017): Input Convex Neural Networks. *Proceedings of the 34th International Conference on Machine Learning*. Ed. by D. Precup, Y. W. Teh. Vol. 70. Proceedings of Machine Learning Research. PMLR, 146–155. URL: https://proceedings.mlr.press/v70/amos17b.html (cit. on pp. 2, 5, 9, 18, 20, 36, 71, 100).

Ancona, M., E. Ceolini, C. Öztireli, M. Gross (2018): Towards better understanding of gradient-based attribution methods for Deep Neural Networks. *International Conference on Learning Representations*. URL: https://openreview.net/forum?id=Sy21R9JAW (cit. on p. 37).

Anders, C. J., D. Neumann, W. Samek, K.-R. Müller, S. Lapuschkin (2026): Software for dataset-wide XAI: From local explanations to global insights with Zennit, CoRelAy, and ViRelAy. *PLOS One* 21 (1), e0336683. DOI: 10.1371/journal.pone.0336683. arXiv: 2106.13200. URL: https://journals.plos.org/plosone/article?id=10.1371/journal.pone.0336683 (cit. on pp. 38, 47, 104).

Anders, C. J., L. Weber, D. Neumann, W. Samek, K.-R. Müller, S. Lapuschkin (2022): Finding and removing Clever Hans: Using explanation methods to debug and improve deep models. *Information Fusion* 77, 261–295. DOI: 10.1016/j.inffus.2021.07.015. URL: https://www.sciencedirect.com/science/article/pii/S1566253521001573 (cit. on p. 1).

Arora, R., A. Basu, P. Mianjy, A. Mukherjee (2018): Understanding Deep Neural Networks with Rectified Linear Units. *International Conference on Learning Representations*. arXiv: 1611.01491 [cs.LG]. URL: https://openreview.net/forum?id=B1J_rgWRW (cit. on pp. 5, 8, 26, 27, 39).

Arras, L., B. Puri, P. Kahardipraja, S. Lapuschkin, W. Samek (2025): *A Close Look at Decomposition-based XAI-Methods for Transformer Language Models*. Preprint, version 1. DOI: 10.48550/arXiv.2502.15886. arXiv: 2502.15886v1 [cs.CL]. URL: https://arxiv.org/abs/2502.15886v1 (cit. on p. 64).

Auer, P., N. Cesa-Bianchi, P. Fischer (2002): Finite-time Analysis of the Multiarmed Bandit Problem. *Machine Learning* 47 (2-3), 235–256. ISSN: 1573-0565. DOI: 10.1023/a:1013689704352. URL: http://dx.doi.org/10.1023/A:1013689704352 (cit. on p. 50).

Awasthi, P., A. Mao, M. Mohri, Y. Zhong (2026): DC-programming for neural network optimizations. *Journal of Global Optimization* 95. Published online 2 January 2024, 5–21. DOI: 10.1007/s10898-023-01344-2. URL: https://link.springer.com/article/10.1007/s10898-023-01344-2 (cit. on pp. 5, 19).

Ayinde, B. O., J. M. Zurada (2018): Deep Learning of Constrained Autoencoders for Enhanced Understanding of Data. *IEEE Transactions on Neural Networks and Learning Systems* 29 (9), 3969–3979. DOI: 10.1109/TNNLS.2017.2747861. URL: https://doi.org/10.1109/TNNLS.2017.2747861 (cit. on pp. 2, 67).

Bach, F. (2017): Breaking the Curse of Dimensionality with Convex Neural Networks. *Journal of Machine Learning Research* 18 (19), 1–53. URL: http://jmlr.org/papers/v18/14-546.html (cit. on p. 5).

Bach, S., A. Binder, G. Montavon, F. Klauschen, K.-R. Müller, W. Samek (2015): On Pixel-Wise Explanations for Non-Linear Classifier Decisions by Layer-Wise Relevance Propagation. *PLOS ONE* 10 (7), e0130140. DOI: 10.1371/journal.pone.0130140. URL: https://doi.org/10.1371/journal.pone.0130140 (cit. on pp. 2, 5, 37, 38, 47, 54, 65, 69, 102, 105).

Balestriero, R., R. G. Baraniuk (2021): Mad Max: Affine Spline Insights Into Deep Learning. *Proceedings of the IEEE* 109 (5), 704–727. DOI: 10.1109/JPROC.2020.3042100. URL: https://doi.org/10.1109/JPROC.2020.3042100 (cit. on p. 5).

Bender, S., J. Herrmann, K.-R. Müller, G. Montavon (2026): Towards desiderata-driven design of visual counterfactual explainers. *Pattern Recognition* 174, 112811. DOI: 10.1016/j.patcog.2025.112811. URL: https://www.sciencedirect.com/science/article/pii/S0031320325014748 (cit. on p. 100).

Berend, J., R. Achtibat, D. Schäffer, A. Binder, W. Samek, S. Lapuschkin, M. Dreyer (2026): *ResLRP: The Role of Residual Cancellation in Attribution Instability in Vision Transformers*. Preprint, version 1. DOI: 10.48550/arXiv.2609.17152. arXiv: 2609.17152v1 [cs.CV]. URL: https://arxiv.org/abs/2609.17152v1 (cit. on p. 54).

Bhatt, U., A. Weller, J. M. F. Moura (2020): Evaluating and Aggregating Feature-based Model Explanations. *Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence*. International Joint Conferences on Artificial Intelligence Organization, 3016–3022. DOI: 10.24963/ijcai.2020/417. URL: https://www.ijcai.org/proceedings/2020/417 (cit. on p. 5).

Bhattacharyya, A. K. (1943): On a Measure of Divergence between Two Statistical Populations Defined by Their Probability Distributions. *Bulletin of the Calcutta Mathematical Society* 35, 99–109. URL: https://dspace.bcu-iasi.ro/handle/123456789/43660 (cit. on p. 86).

Binder, A., G. Montavon, S. Lapuschkin, K.-R. Müller, W. Samek (2016): Layer-wise Relevance Propagation for Neural Networks with Local Renormalization Layers. *International Conference on Artificial Neural Networks (ICANN)*. Vol. 9887. Lecture Notes in Computer Science. Springer, 63–71. DOI: 10.1007/978-3-319-44781-0_8. URL: https://link.springer.com/chapter/10.1007/978-3-319-44781-0_8 (cit. on p. 37).

Binder, A., L. Weber, S. Lapuschkin, G. Montavon, K.-R. Müller, W. Samek (2023): Shortcomings of Top-Down Randomization-Based Sanity Checks for Evaluations of Deep Neural Network Explanations. *2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, 16143–16152. DOI: 10.1109/cvpr52729.2023.01549. URL: http://dx.doi.org/10.1109/CVPR52729.2023.01549 (cit. on pp. 70, 76, 94).

Blücher, S., J. Vielhaben, N. Strodthoff (2024): Decoupling Pixel Flipping and Occlusion Strategy for Consistent XAI Benchmarks. *Transactions on Machine Learning Research*. ISSN: 2835-8856. arXiv: 2401.06654. URL: https://openreview.net/forum?id=bIiLXdtUVM (cit. on p. 99).

Boyd, S., L. Vandenberghe (2004): *Convex Optimization*. Cambridge University Press. ISBN: 9780511804441. DOI: 10.1017/cbo9780511804441. URL: http://dx.doi.org/10.1017/CBO9780511804441 (cit. on pp. 14, 44, 61).

Brandenburg, M.-C., M. Grillo, C. Hertrich (2025): Decomposition Polyhedra of Piecewise Linear Functions. *International Conference on Learning Representations*. Ed. by Y. Yue, A. Garg, N. Peng, F. Sha, R. Yu. Vol. 2025, 62808–62837. URL: https://proceedings.iclr.cc/paper_files/paper/2025/hash/9d66f74820f11ce037fb5f711ab9acd4-Abstract-Conference.html (cit. on p. 5).

Calafiore, G. C., S. Gaubert, C. Possieri (2020): A Universal Approximation Result for Difference of Log-Sum-Exp Neural Networks. *IEEE Trans. Neural Networks Learn. Syst.* 31 (12), 5603–5612. DOI: 10.1109/TNNLS.2020.2975051. URL: https://doi.org/10.1109/TNNLS.2020.2975051 (cit. on p. 5).

Chattopadhay, A., A. Sarkar, P. Howlader, V. N. Balasubramanian (2018): Grad-CAM++: Generalized Gradient-Based Visual Explanations for Deep Convolutional Networks. *2018 IEEE Winter Conference on Applications of Computer Vision (WACV)*. IEEE, 839–847. DOI: 10.1109/WACV.2018.00097. URL: https://doi.org/10.1109/WACV.2018.00097 (cit. on pp. 5, 68, 105).

Chefer, H., S. Gur, L. Wolf (2021a): Generic Attention-Model Explainability for Interpreting Bi-Modal and Encoder-Decoder Transformers. *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 397–406. DOI: 10.1109/ICCV48922.2021.00045. URL: https://openaccess.thecvf.com/content/ICCV2021/html/Chefer_Generic_Attention-Model_Explainability_for_Interpreting_Bi-Modal_and_Encoder-Decoder_Transformers_ICCV_2021_paper.html (cit. on pp. 6, 54, 81, 104, 105).

Chefer, H., S. Gur, L. Wolf (2021b): Transformer Interpretability Beyond Attention Visualization. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 782–791. DOI: 10.1109/CVPR46437.2021.00084. URL: https://openaccess.thecvf.com/content/CVPR2021/html/Chefer_Transformer_Interpretability_Beyond_Attention_Visualization_CVPR_2021_paper.html (cit. on pp. 6, 37, 54, 81, 104, 105).

Chen, H., Y. Wang, W. Lin, Z. Ding (2025): Recurrent Convex Difference Neural Networks for Safety-Critical Model Predictive Control. *IEEE Robotics and Automation Letters* 10 (6), 6400–6407. DOI: 10.1109/LRA.2025.3568315. URL: https://ieeexplore.ieee.org/document/10993304/ (cit. on pp. 5, 71).

Chormai, P., J. Herrmann, K.-R. Müller, G. Montavon (2024): Disentangled Explanations of Neural Network Predictions by Finding Relevant Subspaces. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 46 (11), 7283–7299. DOI: 10.1109/TPAMI.2024.3388275. URL: https://doi.org/10.1109/TPAMI.2024.3388275 (cit. on p. 65).

Clevert, D., T. Unterthiner, S. Hochreiter (2016): Fast and Accurate Deep Network Learning by Exponential Linear Units (ELUs). *4th International Conference on Learning Representations, ICLR 2016, San Juan, Puerto Rico, May 2-4, 2016, Conference Track Proceedings*. Ed. by Y. Bengio, Y. LeCun. URL: http://arxiv.org/abs/1511.07289 (cit. on p. 36).

Dalal, N., B. Triggs (2005): Histograms of Oriented Gradients for Human Detection. *2005 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR'05)*. Vol. 1. IEEE, 886–893. DOI: 10.1109/CVPR.2005.177. URL: https://doi.org/10.1109/CVPR.2005.177 (cit. on p. 94).

Daniels, H., M. Velikova (2010): Monotone and partially monotone neural networks. *Trans. Neur. Netw.* 21 (6), 906–917. ISSN: 1045-9227. DOI: 10.1109/TNN.2010.2044803. URL: https://doi.org/10.1109/TNN.2010.2044803 (cit. on pp. 5, 9).

Darcet, T., M. Oquab, J. Mairal, P. Bojanowski (2024): Vision Transformers Need Registers. *ICLR*. OpenReview.net. URL: https://proceedings.iclr.cc/paper_files/paper/2024/hash/0b408293619f725fd30162af057e531a-Abstract-Conference.html (cit. on pp. 6, 54).

Deng, J., W. Dong, R. Socher, L.-J. Li, K. Li, L. Fei-Fei (2009): ImageNet: A Large-Scale Hierarchical Image Database. *2009 IEEE Conference on Computer Vision and Pattern Recognition*. IEEE, 248–255. DOI: 10.1109/CVPR.2009.5206848. URL: https://doi.org/10.1109/CVPR.2009.5206848 (cit. on pp. 79, 104).

Dombrowski, A.-K., M. Alber, C. J. Anders, M. Ackermann, K.-R. Müller, P. Kessel (2019): Explanations Can Be Manipulated and Geometry Is to Blame. *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 32. URL: https://papers.nips.cc/paper/2019/hash/bb836c01cdc9120a9c984c525e4b1a4a-Abstract.html (cit. on p. 5).

Dombrowski, A.-K., C. J. Anders, K.-R. Müller, P. Kessel (2022): Towards robust explanations for deep neural networks. *Pattern Recognition* 121, 108194. ISSN: 0031-3203. DOI: 10.1016/j.patcog.2021.108194. URL: http://dx.doi.org/10.1016/j.patcog.2021.108194 (cit. on pp. 6, 37, 53, 98).

Doshi-Velez, F., B. Kim (2017): *Towards A Rigorous Science of Interpretable Machine Learning*. DOI: 10.48550/arXiv.1702.08608. arXiv: 1702.08608. URL: https://arxiv.org/abs/1702.08608 (cit. on p. 1).

Dosovitskiy, A., L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, N. Houlsby (2021): An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. *ICLR*. OpenReview.net. URL: https://openreview.net/forum?id=YicbFdNTTy (cit. on pp. 54, 74, 79, 98).

Dreyer, M., J. Berend, T. Labarta, J. Vielhaben, T. Wiegand, S. Lapuschkin, W. Samek (2025): Mechanistic understanding and validation of large AI models with SemanticLens. *Nature Machine Intelligence* 7 (9), 1572–1585. DOI: 10.1038/s42256-025-01084-w. URL: https://www.nature.com/articles/s42256-025-01084-w (cit. on p. 83).

Efron, B., R. J. Tibshirani (1993): *An Introduction to the Bootstrap*. Vol. 57. Monographs on Statistics and Applied Probability. Chapman & Hall. ISBN: 9780412042317. DOI: 10.1007/978-1-4899-4541-9. URL: https://www.routledge.com/An-Introduction-to-the-Bootstrap/Efron-Tibshirani/p/book/9780429246593 (cit. on p. 103).

Everingham, M., L. Van Gool, C. K. I. Williams, J. Winn, A. Zisserman (2010): The PASCAL Visual Object Classes (VOC) Challenge. *International Journal of Computer Vision* 88 (2), 303–338. DOI: 10.1007/s11263-009-0275-4. URL: https://doi.org/10.1007/s11263-009-0275-4 (cit. on pp. 80, 117).

Eykholt, K., I. Evtimov, E. Fernandes, B. Li, A. Rahmati, C. Xiao, A. Prakash, T. Kohno, D. Song (2018): Robust Physical-World Attacks on Deep Learning Visual Classification. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 1625–1634. DOI: 10.1109/CVPR.2018.00175. URL: https://openaccess.thecvf.com/content_cvpr_2018/html/Eykholt_Robust_Physical-World_Attacks_CVPR_2018_paper.html (cit. on p. 1).

Fei, Y., Z. Yang, Y. Chen, Z. Wang, Q. Xie (2020): Risk-Sensitive Reinforcement Learning: Near-Optimal Risk-Sample Tradeoff in Regret. *NeurIPS*. URL: https://papers.neurips.cc/paper_files/paper/2020/hash/fdc42b6b0ee16a2f866281508ef56730-Abstract.html (cit. on pp. 6, 50).

Fel, T., M. Ducoffe, D. Vigouroux, R. Cadène, M. Capelle, C. Nicodème, T. Serre (2023): Don't Lie to Me! Robust and Efficient Explainability With Verified Perturbation Analysis. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 16153–16163. DOI: 10.1109/CVPR52729.2023.01550. URL: https://openaccess.thecvf.com/content/CVPR2023/html/Fel_Dont_Lie_to_Me_Robust_and_Efficient_Explainability_With_Verified_CVPR_2023_paper.html (cit. on pp. 5, 65).

Fong, R. C., A. Vedaldi (2017): Interpretable Explanations of Black Boxes by Meaningful Perturbation. *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 3429–3437. DOI: 10.1109/ICCV.2017.371. URL: https://openaccess.thecvf.com/content_iccv_2017/html/Fong_Interpretable_Explanations_of_ICCV_2017_paper.html (cit. on pp. 5, 65).

Gao, S., Z.-Y. Li, M.-H. Yang, M.-M. Cheng, J. Han, P. Torr (2023): Large-Scale Unsupervised Semantic Segmentation. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 45 (6), 7457–7476. DOI: 10.1109/TPAMI.2022.3218275. arXiv: 2106.03149 [cs.CV]. URL: https://doi.org/10.1109/TPAMI.2022.3218275 (cit. on pp. 4, 80, 101, 104).

Gast, J., S. Roth (2018): Lightweight Probabilistic Deep Networks. *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition*. IEEE, 3369–3378. DOI: 10.1109/cvpr.2018.00355. URL: http://dx.doi.org/10.1109/cvpr.2018.00355 (cit. on p. 50).

Gaubert, S., Y. Vlassopoulos (2025): *Relu and softplus neural nets as zero-sum turn-based games*. DOI: 10.48550/arXiv.2512.20582. arXiv: 2512.20582 [cs.LG]. URL: https://arxiv.org/abs/2512.20582 (cit. on pp. 3, 6, 12–14, 37, 40, 52, 98).

Geirhos, R., J.-H. Jacobsen, C. Michaelis, R. Zemel, W. Brendel, M. Bethge, F. A. Wichmann (2020): Shortcut learning in deep neural networks. *Nature Machine Intelligence* 2 (11), 665–673. DOI: 10.1038/s42256-020-00257-z. URL: https://www.nature.com/articles/s42256-020-00257-z (cit. on p. 1).

Geist, M., B. Scherrer, O. Pietquin (2019): A Theory of Regularized Markov Decision Processes. *Proceedings of the 36th International Conference on Machine Learning*. Ed. by K. Chaudhuri, R. Salakhutdinov. Vol. 97. Proceedings of Machine Learning Research. PMLR, 2160–2169. URL: https://proceedings.mlr.press/v97/geist19a.html (cit. on p. 44).

Ghorbani, A., A. Abid, J. Zou (2019): Interpretation of Neural Networks Is Fragile. *Proceedings of the AAAI Conference on Artificial Intelligence* 33 (01), 3681–3688. ISSN: 2159-5399. DOI: 10.1609/aaai.v33i01.33013681. URL: http://dx.doi.org/10.1609/aaai.v33i01.33013681 (cit. on pp. 1, 5).

Gildenblat, J., contributors (2021): *PyTorch library for CAM methods*. https://github.com/jacobgil/pytorch-grad-cam, 2021. URL: https://github.com/jacobgil/pytorch-grad-cam (cit. on p. 104).

Goodfellow, I., D. Warde-Farley, M. Mirza, A. Courville, Y. Bengio (2013): Maxout Networks. *Proceedings of the 30th International Conference on Machine Learning*. Ed. by S. Dasgupta, D. McAllester. Vol. 28. Proceedings of Machine Learning Research 3. Atlanta, Georgia, USA: PMLR, 1319–1327. URL: https://proceedings.mlr.press/v28/goodfellow13.html (cit. on pp. 10, 19, 55, 71).

Griewank, A., A. Walther (2020): Polyhedral DC Decomposition and DCA Optimization of Piecewise Linear Functions. *Algorithms* 13 (7), 166. DOI: 10.3390/a13070166. URL: https://www.mdpi.com/1999-4893/13/7/166 (cit. on p. 5).

Gu, J., Y. Yang, V. Tresp (2019): Understanding Individual Decisions of CNNs via Contrastive Backpropagation. *Computer Vision – ACCV 2018, Revised Selected Papers, Part III*. Vol. 11363. Lecture Notes in Computer Science. Springer, 119–134. DOI: 10.1007/978-3-030-20893-6_8. URL: https://link.springer.com/chapter/10.1007/978-3-030-20893-6_8 (cit. on p. 37).

Guillemot, M., C. Heusèle, R. Korichi, S. Schnebert, L. Chen (2020): Breaking Batch Normalization for better explainability of Deep Neural Networks through Layer-wise Relevance Propagation. *Conférence sur*

l'Apprentissage Automatique (CAp). Accepted paper published online; conference event cancelled. arXiv: 2002.11018 [cs.LG]. URL: https://cap-rfiap2020.sciencesconf.org/data/pages/CAp2020_paper_1.pdf (cit. on p. 35).

Gupta, M., A. Cotter, J. Pfeifer, K. Voevodski, K. Canini, A. Mangylov, W. Moczydlowski, A. van Esbroeck (2016): Monotonic Calibrated Interpolated Look-Up Tables. *Journal of Machine Learning Research* 17 (109), 1–47. URL: http://jmlr.org/papers/v17/15-243.html (cit. on p. 5).

Hanna, M., S. Pezzelle, Y. Belinkov (2024): Have Faith in Faithfulness: Going Beyond Circuit Overlap When Finding Model Mechanisms. *First Conference on Language Modeling*. arXiv: 2403.17806 [cs.LG]. URL: https://openreview.net/forum?id=TZ0CCGDcuT (cit. on pp. 6, 54).

He, K., X. Zhang, S. Ren, J. Sun (2015): Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification. *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*. IEEE, 1026–1034. DOI: 10.1109/ICCV.2015.123. URL: https://openaccess.thecvf.com/content_iccv_2015/html/He_Delving_Deep_into_ICCV_2015_paper.html (cit. on p. 94).

He, K., X. Zhang, S. Ren, J. Sun (2016): Deep Residual Learning for Image Recognition. *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, 770–778. DOI: 10.1109/cvpr.2016.90. URL: http://dx.doi.org/10.1109/CVPR.2016.90 (cit. on pp. 34, 69, 79).

Hedström, A., P. L. Bommer, T. F. Burns, S. Lapuschkin, W. Samek, M. M.-C. Höhne (2025): Evaluating Interpretable Methods via Geometric Alignment of Functional Distortions. *Transactions on Machine Learning Research*. ISSN: 2835-8856. URL: https://openreview.net/forum?id=ukLxqA8zXj (cit. on p. 90).

Hedström, A., L. Weber, D. Krakowczyk, D. Bareeva, F. Motzkus, W. Samek, S. Lapuschkin, M. M.-C. Höhne (2023): Quantus: An Explainable AI Toolkit for Responsible Evaluation of Neural Network Explanations and Beyond. *Journal of Machine Learning Research* 24 (34), 1–11. URL: https://jmlr.org/papers/v24/22-0142.html (cit. on pp. 1, 5, 69, 80, 103, 104, 118).

Hellinger, E. (1909): Neue Begründung der Theorie quadratischer Formen von unendlichvielen Veränderlichen. *Journal für die reine und angewandte Mathematik* 136, 210–271. DOI: 10.1515/crll.1909.136.210. URL: https://doi.org/10.1515/crll.1909.136.210 (cit. on p. 86).

Hendrycks, D., K. Gimpel (2016): Gaussian Error Linear Units (GELUs). *arXiv preprint arXiv:1606.08415*. DOI: 10.48550/arXiv.1606.08415. URL: https://arxiv.org/abs/1606.08415 (cit. on pp. 21, 31, 51, 98).

Hertrich, C., G. Loho (2026): Neural Networks and (Virtual) Extended Formulations. *SIAM Journal on Mathematics of Data Science* 8 (4), 1350–1369. DOI: 10.1137/25M180130X. arXiv: 2411.03006 [math.CO]. URL: https://epubs.siam.org/doi/10.1137/25M180130X (cit. on p. 56).

Holm, S. (1979): A Simple Sequentially Rejective Multiple Test Procedure. *Scandinavian Journal of Statistics* 6 (2), 65–70. URL: https://www.jstor.org/stable/4615733 (cit. on p. 104).

Huchette, J., G. Muñoz, T. Serra, C. Tsay (2026): When Deep Learning Meets Polyhedral Theory: A Survey. *INFORMS Journal on Computing*. Articles in Advance. DOI: 10.1287/ijoc.2024.0902. arXiv: 2305.00241 [math.OC]. URL: https://pubsonline.informs.org/doi/10.1287/ijoc.2024.0902 (cit. on pp. 5, 8).

Hui, L. Y. W., A. Binder (2019): BatchNorm Decomposition for Deep Neural Network Interpretation. *Advances in Computational Intelligence – IWANN 2019*. Vol. 11507. Lecture Notes in Computer Science. Springer, 280–291. DOI: 10.1007/978-3-030-20518-8_24. URL: https://link.springer.com/chapter/10.1007/978-3-030-20518-8_24 (cit. on p. 35).

Jiang, P.-T., C.-B. Zhang, Q. Hou, M.-M. Cheng, Y. Wei (2021): LayerCAM: Exploring Hierarchical Class Activation Maps for Localization. *IEEE Transactions on Image Processing* 30, 5875–5888. DOI: 10.1109/TIP.2021.3089943. URL: https://doi.org/10.1109/TIP.2021.3089943 (cit. on pp. 5, 68, 105).

Kahardipraja, P., R. Achtibat, T. Wiegand, W. Samek, S. Lapuschkin (2025): The Atlas of In-Context Learning: How Attention Heads Shape In-Context Retrieval Augmentation. *NeurIPS*. DOI: 10.52202/085713-3940. URL: https://proceedings.nips.cc/paper_files/paper/2025/hash/ab49f5e61eadbe979373c133232a96a0-Abstract-Conference.html (cit. on pp. 1, 6, 54).

Kauffmann, J., K.-R. Müller, G. Montavon (2020a): Towards explaining anomalies: A deep Taylor decomposition of one-class models. *Pattern Recognition* 101, 107198. ISSN: 0031-3203. DOI: 10.1016/j.patcog.2020.107198. URL: https://www.sciencedirect.com/science/article/pii/S0031320320300054 (cit. on p. 1).

Kauffmann, J., L. Ruff, G. Montavon, K.-R. Müller (2020b): *The Clever Hans Effect in Anomaly Detection*. DOI: 10.48550/arXiv.2006.10609. arXiv: 2006.10609 [cs.LG]. URL: https://arxiv.org/abs/2006.10609 (cit. on p. 1).

Kazda, K., X. Li (2024): A linear programming approach to difference-of-convex piecewise linear approximation. *European Journal of Operational Research* 312 (2), 493–511. ISSN: 0377-2217. DOI: 10.1016/j.ejor.2023.07.026. URL: https://www.sciencedirect.com/science/article/pii/S0377221723005647 (cit. on p. 5).

Kindermans, P.-J., K. T. Schütt, M. Alber, K.-R. Müller, D. Erhan, B. Kim, S. Dähne (2018): Learning How to Explain Neural Networks: PatternNet and PatternAttribution. *International Conference on Learning Representations (ICLR)*. URL: https://openreview.net/forum?id=Hkn7CBaTW (cit. on pp. 1, 5).

Kohlbrenner, M., A. Bauer, S. Nakajima, A. Binder, W. Samek, S. Lapuschkin (2020): Towards Best Practice in Explaining Neural Network Decisions with LRP. *2020 International Joint Conference on Neural Networks (IJCNN)*. IEEE, 1–7. DOI: 10.1109/IJCNN48605.2020.9206975. URL: https://doi.org/10.1109/IJCNN48605.2020.9206975 (cit. on pp. 37, 69, 102, 103).

Kokhlikyan, N., V. Miglani, M. Martin, E. Wang, B. Alsallakh, J. Reynolds, A. Melnikov, N. Kliushkina, C. Araya, S. Yan, O. Reblitz-Richardson (2020): *Captum: A unified and generic model interpretability library for PyTorch*. DOI: 10.48550/arXiv.2009.07896. arXiv: 2009.07896 [cs.LG]. URL: https://arxiv.org/abs/2009.07896 (cit. on p. 104).

Kolek, S., R. Windesheim, H. Andrade-Loarca, G. Kutyniok, R. Levie (2023): Explaining Image Classifiers With Multiscale Directional Image Representation. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 18600–18609. DOI: 10.1109/CVPR52729.2023.01784. URL: https://openaccess.thecvf.com/content/CVPR2023/html/Kolek_Explaining_Image_Classifiers_With_Multiscale_Directional_Image_Representation_CVPR_2023_paper.html (cit. on pp. 5, 65).

Komorowski, P., E. Golimblevskaia, R. Achtibat, T. Wiegand, S. Lapuschkin, W. Samek (2026): Attribution-Guided Decoding. *The Fourteenth International Conference on Learning Representations*. arXiv: 2509.26307 [cs.LG]. URL: https://proceedings.iclr.cc/paper_files/paper/2026/hash/da30215ee52c1daaaaddada8137cfd0b-Abstract-Conference.html (cit. on pp. 1, 6, 54).

Koutschan, C., B. Moser, A. Ponomarchuk, J. Schicho (2025): Representing piecewise linear functions by functions with small arity. *Applicable Algebra in Engineering, Communication and Computing* 36 (4), 595–610. DOI: 10.1007/s00200-023-00627-1. URL: https://doi.org/10.1007/s00200-023-00627-1 (cit. on p. 5).

Kramár, J., T. Lieberum, R. Shah, N. Nanda (2024): *AtP*: An efficient and scalable method for localizing LLM behaviour to components*. DOI: 10.48550/arXiv.2403.00745. arXiv: 2403.00745 [cs.LG]. URL: https://arxiv.org/abs/2403.00745 (cit. on pp. 6, 54).

Kripfganz, A., R. Schulze (1987): Piecewise affine functions as a difference of two convex functions. *Optimization* 18 (1), 23–29. DOI: 10.1080/02331938708843210. URL: https://www.tandfonline.com/doi/abs/10.1080/02331938708843210 (cit. on pp. 5, 8, 26).

Lapuschkin, S., S. Wäldchen, A. Binder, G. Montavon, W. Samek, K.-R. Müller (2019): Unmasking Clever Hans predictors and assessing what machines really learn. *Nature Communications* 10 (1), 1096. DOI: 10.1038/s41467-019-08987-4. URL: https://doi.org/10.1038/s41467-019-08987-4 (cit. on p. 1).

Le Thi, H. A., T. Pham Dinh (2018): DC programming and DCA: thirty years of developments. *Mathematical Programming* 169 (1), 5–68. DOI: 10.1007/s10107-018-1235-y. URL: https://link.springer.com/article/10.1007/s10107-018-1235-y (cit. on pp. 5, 100).

Lee, D. D., H. S. Seung (1999): Learning the Parts of Objects by Non-Negative Matrix Factorization. *Nature* 401 (6755), 788–791. DOI: 10.1038/44565. URL: https://doi.org/10.1038/44565 (cit. on pp. 2, 67).

Lipton, Z. C. (2018): The Mythos of Model Interpretability. *Communications of the ACM* 61 (10), 36–43. DOI: 10.1145/3233231. URL: https://arxiv.org/abs/1606.03490 (cit. on p. 1).

Liu, X., X. Han, N. Zhang, Q. Liu (2020): Certified Monotonic Neural Networks. *Advances in Neural Information Processing Systems*. Vol. 33. Curran Associates, Inc., 15427–15438. URL: https://proceedings.neurips.cc/paper_files/paper/2020/hash/b139aeda1c2914e3b579aafd3ceeb1bd-Abstract.html (cit. on pp. 5, 9, 72).

Lundberg, S. M., S.-I. Lee (2017): A Unified Approach to Interpreting Model Predictions. *Advances in Neural Information Processing Systems*. Ed. by I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, R. Garnett. Vol. 30. Curran Associates, Inc. URL: https://proceedings.neurips.cc/paper_files/paper/2017/hash/8a20a8621978632d76c43dfd28b67767-Abstract.html (cit. on pp. 5, 6, 37, 65, 105).

Maeß, J., G. Montavon, S. Nakajima, K.-R. Müller, T. Schnake (2026): Uncovering the Structure of Explanation Quality with Spectral Analysis. *Explainable Artificial Intelligence*. Vol. 2578. Communications in Computer and Information Science. Springer, 289–309. DOI: 10.1007/978-3-032-08327-2_14. URL: https://link.springer.com/chapter/10.1007/978-3-032-08327-2_14 (cit. on p. 83).

McNemar, Q. (1947): Note on the Sampling Error of the Difference Between Correlated Proportions or Percentages. *Psychometrika* 12 (2), 153–157. DOI: 10.1007/BF02295996. URL: https://doi.org/10.1007/BF02295996 (cit. on p. 103).

Melzer, D. (1986): On the expressibility of piecewise-linear continuous functions as the difference of two piecewise-linear convex functions. In: *Quasidifferential Calculus*. Ed. by V. F. Demyanov, L. C. W. Dixon. Vol. 29. Mathematical Programming Studies. Berlin, Heidelberg: Springer, 118–134. DOI: 10.1007/BFb0121142. URL: https://link.springer.com/chapter/10.1007/BFb0121142 (cit. on pp. 5, 8, 26).

Montavon, G., A. Binder, S. Lapuschkin, W. Samek, K.-R. Müller (2019): Layer-Wise Relevance Propagation: An Overview. In: *Explainable AI: Interpreting, Explaining and Visualizing Deep Learning*. Ed. by W. Samek, G. Montavon, A. Vedaldi, L. K. Hansen, K.-R. Müller. Cham: Springer International Publishing, 193–209. ISBN: 978-3-030-28954-6. DOI: 10.1007/978-3-030-28954-6_10. URL: https://doi.org/10.1007/978-3-030-28954-6_10 (cit. on pp. 69, 105).

Montavon, G., S. Lapuschkin, A. Binder, W. Samek, K.-R. Müller (2017): Explaining nonlinear classification decisions with deep Taylor decomposition. *Pattern Recognition* 65, 211–222. DOI: 10.1016/j.patcog.2016.11.008. URL: https://www.sciencedirect.com/science/article/pii/S0031320316303582 (cit. on pp. 5, 37, 38, 65).

Montavon, G., W. Samek, K.-R. Müller (2018): Methods for interpreting and understanding deep neural networks. *Digital Signal Processing* 73, 1–15. DOI: 10.1016/j.dsp.2017.10.011. URL: https://www.sciencedirect.com/science/article/pii/S1051200417302385 (cit. on pp. 5, 69, 103).

Murdoch, W. J., C. Singh, K. Kumbier, R. Abbasi-Asl, B. Yu (2019): Definitions, methods, and applications in interpretable machine learning. *Proceedings of the National Academy of Sciences* 116 (44), 22071–22080. DOI: 10.1073/pnas.1900654116. URL: https://doi.org/10.1073/pnas.1900654116 (cit. on p. 1).

Muttenthaler, L., K. Greff, F. Born, B. Spitzer, S. Kornblith, M. C. Mozer, K.-R. Müller, T. Unterthiner, A. K. Lampinen (2025): Aligning machine and human visual representations across abstraction levels. *Nature* 647 (8089), 349–355. DOI: 10.1038/s41586-025-09631-6. URL: https://doi.org/10.1038/s41586-025-09631-6 (cit. on p. 1).

Nam, W., S. Gur, J. Choi, L. Wolf, S. Lee (2020): Relative Attributing Propagation: Interpreting the Comparative Contributions of Individual Units in Deep Neural Networks. *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 34. 3. AAAI Press, 2501–2508. DOI: 10.1609/aaai.v34i03.5632. URL: https://ojs.aaai.org/index.php/AAAI/article/view/5632 (cit. on p. 37).

Nguyen, A.-P., D. L. Moreno, N. Le-Bel, M. Rodríguez Martínez (2023): MonoNet: enhancing interpretability in neural networks via monotonic features. *Bioinformatics Advances* 3 (1), vbad016. ISSN: 2635-0041. DOI: 10.1093/bioadv/vbad016. eprint: https://academic.oup.com/bioinformaticsadvances/article-pdf/3/1/vbad016/50156829/vbad016.pdf. URL: https://doi.org/10.1093/bioadv/vbad016 (cit. on p. 5).

Nie, W., Y. Zhang, A. B. Patel (2018): A Theoretical Explanation for Perplexing Behaviors of Backpropagation-based Visualizations. *Proceedings of the 35th International Conference on Machine Learning*. Vol. 80. Proceedings of Machine Learning Research. PMLR, 3809–3818. URL: https://proceedings.mlr.press/v80/nie18a.html (cit. on pp. 1, 5).

Osborne, M. J., A. Rubinstein (1994): *A Course in Game Theory*. MIT Press. ISBN: 9780262650403. URL: https://mitpress.mit.edu/9780262150415/a-course-in-game-theory/ (cit. on p. 39).

Pahde, F., M. Dreyer, W. Samek, S. Lapuschkin (2023): Reveal to Revise: An Explainable AI Life Cycle for Iterative Bias Correction of Deep Models. *Medical Image Computing and Computer Assisted Intervention – MICCAI 2023*. Vol. 14221. Lecture Notes in Computer Science. Springer Nature Switzerland, 596–606. DOI: 10.1007/978-3-031-43895-0_56. URL: https://doi.org/10.1007/978-3-031-43895-0_56 (cit. on pp. 1, 6).

Pahde, F., T. Wiegand, S. Lapuschkin, W. Samek (2025): Ensuring medical AI safety: interpretability-driven detection and mitigation of spurious model behavior and associated data. *Machine Learning* 114 (9), 206. DOI: 10.1007/s10994-025-06834-w. URL: https://link.springer.com/article/10.1007/s10994-025-06834-w (cit. on p. 99).

Paszke, A., S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Köpf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, S. Chintala (2019): PyTorch: an imperative style, high-performance deep learning library. *Proceedings of the 33rd International Conference on Neural Information Processing Systems*. Red Hook, NY, USA: Curran Associates Inc. URL: https://proceedings.neurips.cc/paper_files/paper/2019/hash/bdbca288fee7f92f2bfa9f7012727740-Abstract.html (cit. on pp. 34, 79).

Phuong, M., C. H. Lampert (2020): Functional vs. parametric equivalence of ReLU networks. *ICLR*. OpenReview.net. URL: https://iclr.cc/virtual_2020/poster_Bylx-TNKvH.html (cit. on p. 5).

Puterman, M. L. (1994): *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. John Wiley & Sons. DOI: 10.1002/9780470316887. URL: https://doi.org/10.1002/9780470316887 (cit. on pp. 10, 11).

Ramachandran, P., B. Zoph, Q. V. Le (2018): Searching for Activation Functions. *International Conference on Learning Representations (ICLR), Workshop Track*. Preprint first posted in 2017. arXiv: 1710.05941 [cs.NE]. URL: https://openreview.net/forum?id=Hkuq2EkPf (cit. on pp. 21, 36, 98).

Ribeiro, M. T., S. Singh, C. Guestrin (2016): "Why Should I Trust You?": Explaining the Predictions of Any Classifier. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. KDD '16. San Francisco, California, USA: Association for Computing Machinery, 1135–1144. ISBN: 9781450342322. DOI: 10.1145/2939672.2939778. URL: https://doi.org/10.1145/2939672.2939778 (cit. on pp. 5, 65, 105).

Ross, A. S., F. Doshi-Velez (2018): Improving the Adversarial Robustness and Interpretability of Deep Neural Networks by Regularizing Their Input Gradients. *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 32. 1. DOI: 10.1609/aaai.v32i1.11504. URL: https://arxiv.org/abs/1711.09404 (cit. on p. 1).

Ross, A. S., M. C. Hughes, F. Doshi-Velez (2017): Right for the Right Reasons: Training Differentiable Models by Constraining their Explanations. *Proceedings of the Twenty-Sixth International Joint Conference on Artificial Intelligence*, 2662–2670. DOI: 10.24963/ijcai.2017/371. URL: https://ijcai.org/proceedings/2017/371 (cit. on p. 1).

Rudin, C. (2019): Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead. *Nature Machine Intelligence* 1 (5), 206–215. DOI: 10.1038/s42256-019-0048-x. URL: https://doi.org/10.1038/s42256-019-0048-x (cit. on p. 2).

Runje, D., S. M. Shankaranarayana (2023): Constrained Monotonic Neural Networks. *Proceedings of the 40th International Conference on Machine Learning*. Vol. 202. Proceedings of Machine Learning Research. PMLR, 29338–29353. URL: https://proceedings.mlr.press/v202/runje23a.html (cit. on pp. 5, 9, 10).

Russakovsky, O., J. Deng, H. Su, J. Krause, S. Satheesh, S. Ma, Z. Huang, A. Karpathy, A. Khosla, M. Bernstein, A. C. Berg, L. Fei-Fei (2015): ImageNet Large Scale Visual Recognition Challenge. *International Journal of Computer Vision* 115 (3), 211–252. ISSN: 1573-1405. DOI: 10.1007/s11263-015-0816-y. URL: http://dx.doi.org/10.1007/s11263-015-0816-y (cit. on pp. 79, 104).

Samek, W., A. Binder, G. Montavon, S. Lapuschkin, K.-R. Müller (2017): Evaluating the Visualization of What a Deep Neural Network Has Learned. *IEEE Transactions on Neural Networks and Learning Systems* 28 (11), 2660–2673. DOI: 10.1109/TNNLS.2016.2599820. URL: https://doi.org/10.1109/TNNLS.2016.2599820 (cit. on p. 102).

Sankaranarayanan, P., R. Rengaswamy (2022): CDiNN – Convex difference neural networks. *Neurocomput.* 495 (C), 153–168. ISSN: 0925-2312. DOI: 10.1016/j.neucom.2022.01.024. URL: https://doi.org/10.1016/j.neucom.2022.01.024 (cit. on pp. 5, 71).

Schlüter, N., M. Schulze Darup (2020): Novel Convex Decomposition of Piecewise Affine Functions. *21st IFAC World Congress*. Late-breaking results contribution; reprinted as arXiv:2108.03950 in 2021. arXiv: 2108.03950 [eess.SY]. URL: https://arxiv.org/abs/2108.03950 (cit. on p. 5).

Schnake, T., F. Rezaei Jafari, J. Lederer, P. Xiong, S. Nakajima, S. Gugler, G. Montavon, K.-R. Müller (2025): Towards symbolic XAI — explanation through human understandable logical relationships between features. *Information Fusion* 118, 102923. DOI: 10.1016/j.inffus.2024.102923. URL: https://doi.org/10.1016/j.inffus.2024.102923 (cit. on p. 83).

Selvaraju, R. R., M. Cogswell, A. Das, R. Vedantam, D. Parikh, D. Batra (2017): Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization. *2017 IEEE International Conference on Computer Vision (ICCV)*, 618–626. DOI: 10.1109/ICCV.2017.74. URL: https://openaccess.thecvf.com/content_iccv_2017/html/Selvaraju_Grad-CAM_Visual_Explanations_ICCV_2017_paper.html (cit. on pp. 5, 65, 68, 105).

Shapley, L. S. (1953): Stochastic Games. *Proceedings of the National Academy of Sciences* 39 (10), 1095–1100. DOI: 10.1073/pnas.39.10.1095. URL: https://doi.org/10.1073/pnas.39.10.1095 (cit. on p. 12).

Shrikumar, A., P. Greenside, A. Kundaje (2017): Learning Important Features Through Propagating Activation Differences. *Proceedings of the 34th International Conference on Machine Learning*. Ed. by D. Precup, Y. W. Teh. Vol. 70. Proceedings of Machine Learning Research. PMLR, 3145–3153. URL: https://proceedings.mlr.press/v70/shrikumar17a.html (cit. on pp. 5, 65, 105).

Siahkamari, A., A. Gangrade, B. Kulis, V. Saligrama (2020): Piecewise Linear Regression via a Difference of Convex Functions. *Proceedings of the 37th International Conference on Machine Learning*. Ed. by H. D. III, A. Singh. Vol. 119. Proceedings of Machine Learning Research. PMLR, 8895–8904. URL: https://proceedings.mlr.press/v119/siahkamari20a.html (cit. on pp. 5, 19).

Sill, J. (1997): Monotonic Networks. *Advances in Neural Information Processing Systems*. Vol. 10. MIT Press, 661–667. URL: https://papers.nips.cc/paper_files/paper/1997/file/83adc9225e4deb67d7ce42d58fe5157c-Paper.pdf (cit. on pp. 2, 5, 9, 72).

Simonyan, K., A. Vedaldi, A. Zisserman (2014): Deep Inside Convolutional Networks: Visualising Image Classification Models and Saliency Maps. *2nd International Conference on Learning Representations, ICLR 2014, Banff, AB, Canada, April 14-16, 2014, Workshop Track Proceedings*. Ed. by Y. Bengio, Y. LeCun. URL: http://arxiv.org/abs/1312.6034 (cit. on pp. 5, 65, 105).

Simonyan, K., A. Zisserman (2015): Very Deep Convolutional Networks for Large-Scale Image Recognition. *3rd International Conference on Learning Representations, ICLR 2015, San Diego, CA, USA, May 7-9, 2015, Conference Track Proceedings*. Ed. by Y. Bengio, Y. LeCun. URL: http://arxiv.org/abs/1409.1556 (cit. on pp. 34, 69, 79).

Sivaprasad, S., A. Singh, N. Manwani, V. Gandhi (2021): The Curious Case of Convex Neural Networks. *Machine Learning and Knowledge Discovery in Databases. Research Track*. Ed. by N. Oliver, F. Pérez-Cruz, S. Kramer, J. Read, J. A. Lozano. Cham: Springer International Publishing, 738–754. ISBN: 978-3-030-86486-6. DOI: 10.1007/978-3-030-86486-6_45. URL: https://link.springer.com/chapter/10.1007/978-3-030-86486-6_45 (cit. on p. 5).

Sivaraman, A., G. Farnadi, T. Millstein, G. V. den Broeck (2020): Counterexample-Guided Learning of Monotonic Neural Networks. *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*. Method/tool: COMET. URL: https://proceedings.neurips.cc/paper/2020/hash/8ab70731b1553f17c11a3bbc87e0b605-Abstract.html (cit. on p. 5).

Sixt, L., M. Granz, T. Landgraf (2020): When Explanations Lie: Why Many Modified BP Attributions Fail. *ICML*. Vol. 119. Proceedings of Machine Learning Research. PMLR, 9046–9057. URL: https://proceedings.mlr.press/v119/sixt20a.html (cit. on pp. 1, 5, 70, 84, 93, 98).

Smilkov, D., N. Thorat, B. Kim, F. Viégas, M. Wattenberg (2017): *SmoothGrad: removing noise by adding noise*. DOI: 10.48550/arXiv.1706.03825. arXiv: 1706.03825 [cs.LG]. URL: https://arxiv.org/abs/1706.03825 (cit. on pp. 5, 65, 105).

Springenberg, J. T., A. Dosovitskiy, T. Brox, M. Riedmiller (2015): Striving for Simplicity: The All Convolutional Net. *International Conference on Learning Representations (Workshop)*. DOI: 10.48550/arXiv.1412.6806. URL: https://arxiv.org/abs/1412.6806 (cit. on pp. 5, 65).

Sriperumbudur, B. K., G. R. G. Lanckriet (2009): On the Convergence of the Concave-Convex Procedure. *Advances in Neural Information Processing Systems (NeurIPS)*. URL: https://papers.nips.cc/paper/3646-on-the-convergence-of-the-concave-convex-procedure (cit. on p. 5).

Sundararajan, M., A. Taly, Q. Yan (2017): Axiomatic Attribution for Deep Networks. *Proceedings of the 34th International Conference on Machine Learning*. Ed. by D. Precup, Y. W. Teh. Vol. 70. Proceedings of Machine Learning Research. PMLR, 3319–3328. URL: https://proceedings.mlr.press/v70/sundararajan17a.html (cit. on pp. 5, 65, 66, 105).

Sutton, R. S., A. G. Barto (2018): *Reinforcement Learning: An Introduction*. 2nd ed. MIT Press. URL: https://mitpress.mit.edu/9780262039246/reinforcement-learning/ (cit. on pp. 10, 11).

Syed, A., C. Rager, A. Conmy (2024): Attribution Patching Outperforms Automated Circuit Discovery. *Proceedings of the 7th BlackboxNLP Workshop: Analyzing and Interpreting Neural Networks for NLP*. Association for Computational Linguistics, 407–416. DOI: 10.18653/v1/2024.blackboxnlp-1.25. URL: http://dx.doi.org/10.18653/v1/2024.blackboxnlp-1.25 (cit. on pp. 6, 54).

Tao, P. D., L. T. H. An (1997): Convex Analysis Approach to D.C. Programming: Theory, Algorithms and Applications. *Acta Mathematica Vietnamica* 22 (1), 289–355. URL: https://math.ac.vn/uploads/files/9701289.pdf (cit. on p. 5).

Theiner, J., E. Müller-Budack, R. Ewerth (2022): Interpretable Semantic Photo Geolocation. *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, 1474–1484. DOI: 10.1109/WACV51458.2022.00154. URL: https://openaccess.thecvf.com/content/WACV2022/html/Theiner_Interpretable_Semantic_Photo_Geolocation_WACV_2022_paper.html (cit. on p. 102).

Vaswani, A., N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, I. Polosukhin (2017): Attention Is All You Need. *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 30, 5998–

6008. URL: https://proceedings.neurips.cc/paper_files/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html (cit. on pp. 54, 98).

Voita, E., D. Talbot, F. Moiseev, R. Sennrich, I. Titov (2019): Analyzing Multi-Head Self-Attention: Specialized Heads Do the Heavy Lifting, the Rest Can Be Pruned. *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*, 5797–5808. DOI: 10.18653/v1/P19-1580. URL: https://aclanthology.org/P19-1580/ (cit. on pp. 6, 54).

Vukadin, D., P. Afrić, M. Šilić, G. Delač (2024): Advancing Attribution-Based Neural Network Explainability through Relative Absolute Magnitude Layer-Wise Relevance Propagation and Multi-Component Evaluation. *ACM Transactions on Intelligent Systems and Technology* 15 (3), 1–30. ISSN: 2157-6912. DOI: 10.1145/3649458. URL: http://dx.doi.org/10.1145/3649458 (cit. on pp. 37, 54).

Wang, S., M. Gupta (2020): Deontological Ethics By Monotonicity Shape Constraints. *Proceedings of the Twenty Third International Conference on Artificial Intelligence and Statistics*. Ed. by S. Chiappa, R. Calandra. Vol. 108. Proceedings of Machine Learning Research. PMLR, 2043–2054. URL: https://proceedings.mlr.press/v108/wang20e.html (cit. on p. 2).

Wróbel, A., S. Gairola, J. Tabor, B. Schiele, B. M. Zieliński, D. D. Rymarczyk (2026): DAVE: Distribution-Aware Attribution via ViT Gradient Decomposition. *Proceedings of the 43rd International Conference on Machine Learning*. Vol. 306. Proceedings of Machine Learning Research. PMLR, 135468–135511. arXiv: 2602.06613 [cs.CV]. URL: https://proceedings.mlr.press/v306/wrobel26a.html (cit. on pp. 6, 37, 38, 54, 56, 60, 61, 64, 74, 81, 105).

Xiong, P., T. Schnake, G. Montavon, K.-R. Müller, S. Nakajima (2026): Normalized Relevance Measure as a Unifying Framework to Explain Neural Network Latent Structures. *arXiv preprint arXiv:2606.00557*. DOI: 10.48550/arXiv.2606.00557. arXiv: 2606.00557 [cs.LG]. URL: https://arxiv.org/abs/2606.00557 (cit. on pp. 6, 37, 83, 98, 100).

Yeh, C.-K., C.-Y. Hsieh, A. S. Suggala, D. I. Inouye, P. Ravikumar (2019): On the (In)fidelity and Sensitivity of Explanations. *Advances in Neural Information Processing Systems*. Vol. 32. Curran Associates, Inc., 10965–10976. URL: https://proceedings.neurips.cc/paper/2019/hash/a7471fdc77b3435276507cc8f2dc2569-Abstract.html (cit. on pp. 5, 103).

Zalgaller, V. A. (2000): Representation of functions of several variables by differences of convex functions. *Journal of Mathematical Sciences* 100, 2209–2227. DOI: 10.1007/s10958-000-0006-4. URL: https://link.springer.com/article/10.1007/s10958-000-0006-4 (cit. on p. 5).

Zech, J. R., M. A. Badgeley, M. Liu, A. B. Costa, J. J. Titano, E. K. Oermann (2018): Variable generalization performance of a deep learning model to detect pneumonia in chest radiographs: A cross-sectional study. *PLoS Medicine* 15 (11), e1002683. DOI: 10.1371/journal.pmed.1002683. URL: https://doi.org/10.1371/journal.pmed.1002683 (cit. on p. 1).

Zeiler, M. D., R. Fergus (2014): Visualizing and Understanding Convolutional Networks. *Computer Vision – ECCV 2014*. Ed. by D. Fleet, T. Pajdla, B. Schiele, T. Tuytelaars. Cham: Springer International Publishing, 818–833. ISBN: 978-3-319-10590-1. DOI: 10.1007/978-3-319-10590-1_53. URL: https://link.springer.com/chapter/10.1007/978-3-319-10590-1_53 (cit. on pp. 5, 65).

Zhang, J., S. A. Bargal, Z. Lin, J. Brandt, X. Shen, S. Sclaroff (2018a): Top-Down Neural Attention by Excitation Backprop. *International Journal of Computer Vision* 126 (10), 1084–1102. DOI: 10.1007/s11263-017-1059-x. URL: https://doi.org/10.1007/s11263-017-1059-x (cit. on pp. 5, 6, 37, 38, 102).

Zhang, L., G. Naitzat, L.-H. Lim (2018b): Tropical Geometry of Deep Neural Networks. *Proceedings of the 35th International Conference on Machine Learning*. Ed. by J. Dy, A. Krause. Vol. 80. Proceedings of Machine Learning Research. PMLR, 5824–5832. URL: https://proceedings.mlr.press/v80/zhang18i.html (cit. on pp. 5, 20).

Zhou, B., A. Khosla, A. Lapedriza, A. Oliva, A. Torralba (2016): Learning Deep Features for Discriminative Localization. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2921–2929. DOI: 10.1109/CVPR.2016.319. URL: https://openaccess.thecvf.com/content_cvpr_2016/html/Zhou_Learning_Deep_Features_CVPR_2016_paper.html (cit. on pp. 65, 68).

Zimmermann, J. P., J. Berend, G. Loho, S. Lapuschkin, W. Samek (2026): *Playing the Network Backward: A Game-Theoretic Attribution Framework*. Preprint, version 2. DOI: 10.48550/arXiv.2605.06212. arXiv: 2605.06212v2 [cs.LG]. URL: https://arxiv.org/abs/2605.06212v2 (cit. on pp. 3, 4, 125).

Zimmermann, J. P., G. Loho (2026): Hidden Monotonicity: Explaining Deep Neural Networks via their DC Decomposition. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. Highlight, 24107–24117. arXiv: 2601.07700 [cs.CV]. URL: https://openaccess.thecvf.com/conte

nt/CVPR2026/html/Zimmermann_Hidden_Monotonicity_Explaining_Deep_Neural_Networks_via_their_DC_Decomposition_CVPR_2026_paper.html (cit. on pp. 2, 4, 20, 26, 27, 31, 37, 39, 54, 65, 98, 125).